


Grado en Ingeniería Electrónica de Comunicaciones

Curso 2026-2027

# Procesamiento de Señales

## Apuntes y ejercicios resueltos

Jesús Chacón Sombría

Cada tema reúne sus apuntes y, al final, la hoja de ejercicios con las soluciones. Las transparencias, los documentos sueltos de cada tema y el código de los ejemplos están en `https://github.com/UCM-237/PdS`.

# Índice general


**1 Introducción: señales y sistemas** **4**

1.1 Señales y sistemas . . . 4
1.2 Clasificación de las señales . . . 5
1.3 Señales básicas . . . 9
1.4 Operaciones con señales . . . 13
1.5 Convolución . . . 15
1.6 Representación en frecuencia . . . 16
1.7 Propiedades de los sistemas . . . 23
1.8 Sistemas LTI . . . 26
1.9 Sistemas LTI continuos y discretos . . . 28
1.10 Contenido en frecuencia de señales reales . . . 31
Resumen del tema . . . 33
Bibliografía . . . 34
1.11 Ejercicios . . . 34

**2 Muestreo y reconstrucción de señales** **43**

2.1 Del mundo analógico al digital . . . 43
2.2 Muestreo periódico y aliasing . . . 44
2.3 El teorema del muestreo . . . 47
2.4 Reconstrucción . . . 50
2.5 Cuantificación y codificación . . . 56
2.6 Muestreo de señales discretas . . . 60
Resumen del tema . . . 64
Bibliografía . . . 65
2.7 Ejercicios . . . 65

**3 Transformada discreta de Fourier** **73**

3.1 De la DTFT a la DFT . . . 73
3.2 La transformada discreta de Fourier . . . 75
3.3 Propiedades de la DFT . . . 78
3.4 Análisis espectral con la DFT . . . 82
3.5 FFT y convolución rápida . . . 87
Resumen del tema . . . 90
Bibliografía . . . 91
3.6 Ejercicios . . . 91

**4 Filtros** **97**

4.1 Filtros ideales, especificaciones y fase . . . 97
4.2 Filtros continuos . . . 101
4.3 Filtros IIR . . . 107
4.4 Filtros FIR . . . 111

4.5 Filtros especiales . . . 117
Bibliografía . . . 120
4.6 Ejercicios . . . 121

**5 Aplicaciones** **128**
5.1 Transformada de Fourier de tiempo corto . . . 128
5.2 Wavelets . . . 134
5.3 Procesamiento de imágenes . . . 141
Bibliografía . . . 149
5.4 Ejercicios . . . 149

**6 Señales aleatorias** **156**
6.1 Variables aleatorias . . . 156
6.2 Varias variables aleatorias . . . 164
6.3 Señales aleatorias . . . 168
6.4 Análisis espectral de señales aleatorias . . . 175
Bibliografía . . . 181
6.5 Ejercicios . . . 181

# 1 Introducción: señales y sistemas

**Cómo usar estos apuntes**

Estos apuntes desarrollan el contenido de las cuatro presentaciones del tema: las secciones 1.1–1.3 corresponden a la parte 1 (tipos de señales y señales básicas), las secciones 1.4–1.6 a la parte 2 (operaciones y análisis en frecuencia), las secciones 1.7–1.9 a la parte 3 (sistemas) y la sección 1.10 a la parte 4 (señales reales). Los ejemplos de código están en `teoría/T1/codigo/`, en Python y en Matlab, y se pueden ejecutar tal cual. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

## 1.1 Señales y sistemas

El procesamiento de señales estudia cómo representar, analizar y transformar la información que transportan las magnitudes físicas. Este primer tema fija la terminología y repasa las herramientas matemáticas que usaremos durante todo el curso: los tipos de señales, las operaciones que se hacen con ellas, su representación en frecuencia y las propiedades de los sistemas que las transforman. El resultado de aprendizaje que persigue, según la guía docente, es *relacionar señales y sistemas de tiempo continuo con señales y sistemas de tiempo discreto, en el dominio del tiempo y de la frecuencia*. Por eso cada concepto se presenta en sus dos versiones, continua y discreta, señalando dónde se parecen y, sobre todo, dónde se diferencian.

**Definición 1.1.1: Señal**

Una *señal* es una función de una o varias variables independientes (tiempo, espacio, …) que transporta información sobre la evolución de una magnitud física. Ejemplos: la presión acústica que capta un micrófono, la tensión en un electrodo de ECG, la intensidad luminosa de cada punto de una imagen o la temperatura que mide un sensor.

**Definición 1.1.2: Sistema**

Un *sistema* es una entidad que, excitada por una señal de entrada, produce una señal de salida. Modela la transformación que un dispositivo físico (un circuito, un canal de comunicaciones, un sensor) o un algoritmo (un filtro digital, un ecualizador) hace sobre una señal.

### 1.1.1 Notación

A lo largo del curso seguiremos estos convenios, que son los de la bibliografía básica:

- Las señales se escriben en minúscula: $x$, $y$, $h$, … Una señal de tiempo continuo se escribe $x(t)$, con $t \in \mathbb{R}$; una de tiempo discreto, o *secuencia*, se escribe $x[n]$, con $n \in \mathbb{Z}$. **Paréntesis para tiempo continuo y corchetes para tiempo discreto**: nunca escribiremos $x(n)$ para una secuencia, porque la diferencia entre ambos tipos de señal es justamente lo que el curso estudia.

- Un sistema se representa con un operador $\mathcal{T}$: $y(t) = \mathcal{T}\{x(t)\}$ o $y[n] = \mathcal{T}\{x[n]\}$.
- La frecuencia de una señal continua se escribe $\omega$ (en rad/s) o $f$ (en Hz), con $\omega = 2\pi f$. La de una secuencia se escribe $\Omega$ (en rad/muestra) o $F$ (en ciclos/muestra), con $\Omega = 2\pi F$. Cuando una secuencia se obtiene muestreando con periodo $T_s$, ambas se relacionan por $\Omega = \omega T_s$ (Tema 2).
- Las transformadas se escriben en mayúscula: $X(j\omega)$ (Fourier continua), $X(e^{j\Omega})$ (Fourier discreta), $X(s)$ (Laplace), $X(z)$ (Z) y $X[k]$ (DFT, Tema 3).

### 1.1.2 Procesado analógico y procesado digital

Las señales del mundo físico son casi siempre analógicas. Para procesarlas con un computador, un DSP o un microcontrolador se sigue la cadena de la figura 1.1: un filtro *antialiasing* limita el ancho de banda, el conversor A/D muestrea y cuantifica, el procesador aplica el algoritmo y, si hace falta devolver una señal analógica, el conversor D/A y el filtro de reconstrucción completan el camino. El Tema 2 estudia el muestreo y la cuantificación; los temas siguientes, el bloque central.

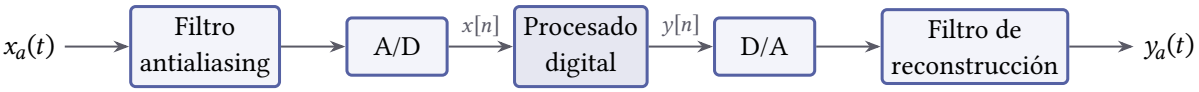



**Figura 1.1.** Cadena de procesado digital de una señal analógica.

El procesado digital tiene ventajas claras: es preciso y repetible (no hay deriva de componentes con la temperatura o el envejecimiento), es flexible (cambiar el algoritmo es cambiar el programa) y permite operaciones muy difíciles en analógico, como filtros de fase lineal, la FFT o los filtros adaptativos. A cambio, el ancho de banda queda limitado por la frecuencia de muestreo, aparecen errores de cuantificación y de precisión finita, y el cálculo introduce retardo y consume energía.

## 1.2 Clasificación de las señales

### 1.2.1 Dimensión y número de canales

Una señal es *unidimensional* si depende de una sola variable independiente (normalmente el tiempo) y *multidimensional* si depende de varias: una imagen en escala de grises es una función $x(u, v)$ de dos coordenadas espaciales y un vídeo añade el tiempo, $x(u, v, t)$. Por otro lado, una señal es *escalar* si en cada punto toma un solo valor y *vectorial* (o multicanal) si toma varios: un acelerómetro de tres ejes da $[x_1(t)\ x_2(t)\ x_3(t)]$ y una imagen en color, tres planos $[x_R\ x_G\ x_B](u, v)$. En este curso trabajaremos sobre todo con señales escalares unidimensionales, aunque casi todo se generaliza.

### 1.2.2 Variable independiente y amplitud

Tanto la variable independiente como la amplitud pueden ser continuas o discretas, lo que da cuatro combinaciones (figura 1.2):

| | amplitud continua | amplitud discreta |
|---|---|---|
| tiempo continuo | analógica | cuantificada |
| tiempo discreto | muestreada | **digital** |

Una amplitud discreta toma valores en un conjunto finito o numerable, por ejemplo los múltiplos $k\Delta$ de un escalón de cuantificación. Un computador solo puede representar señales *digitales*, discretas en tiempo y en amplitud. En la mayor parte del curso ignoraremos la cuantificación y trataremos las secuencias como si su amplitud fuera continua; el Tema 2 cuantifica el error que esto supone.

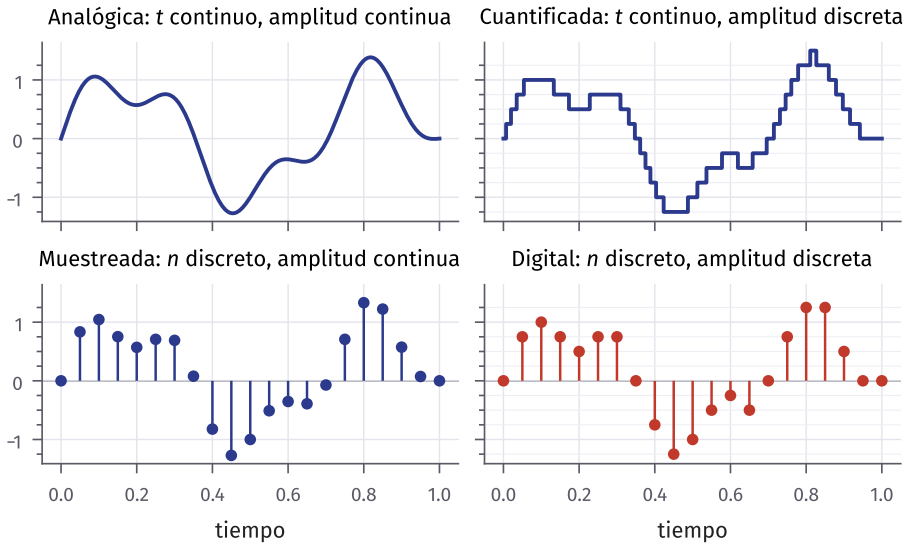



**Figura 1.2.** Una misma señal analógica, cuantificada, muestreada y digital.

### 1.2.3 Deterministas y aleatorias

Una señal es *determinista* si su valor en cada instante está fijado por una expresión, una tabla o una regla, de modo que se puede predecir. Es *aleatoria* o *estocástica* si su valor en cada instante es una variable aleatoria, y entonces solo se puede describir estadísticamente: con su densidad de probabilidad, su media, su autocorrelación o su densidad espectral de potencia. El ruido de un sensor, la voz o el tráfico de una red se modelan como señales aleatorias; el Tema 6 las estudia con detalle.

### 1.2.4 Periódicas y aperiódicas

**Definición 1.2.1: Señal periódica**

Una señal continua es periódica si existe $T > 0$ tal que $x(t+T) = x(t)$ para todo $t$. Una secuencia es periódica si existe un entero $N > 0$ tal que $x[n+N] = x[n]$ para todo $n$. El menor periodo positivo es el *periodo fundamental*, $T_0$ o $N_0$.

Si $T$ es un periodo, también lo es $lT$ para todo entero $l \neq 0$: por eso hablamos del periodo *fundamental*. Fíjate en que el periodo de una secuencia tiene que ser entero; esta restricción, que parece inocente, hace que muchas sinusoides discretas no sean periódicas (apartado 1.3.5).

La suma de dos señales periódicas continuas, de periodos $T_1$ y $T_2$, es periódica si y solo si el cociente $T_1/T_2$ es racional, $T_1/T_2 = p/q$ con $p, q$ enteros primos entre sí. El periodo de la suma es entonces $T_0 = qT_1 = pT_2$, el mínimo común múltiplo.

**Ejemplo 1.2.2: Suma de sinusoides**

$x(t) = \cos(2t) + \cos(3t)$ tiene periodos $T_1 = \pi$ y $T_2 = 2\pi/3$. El cociente $T_1/T_2 = 3/2$ es racional, así que $x$ es periódica con $T_0 = 2T_1 = 3T_2 = 2\pi$. En cambio, $\cos t + \cos(\pi t)$ tiene periodos $2\pi$ y 2, cuyo cociente $\pi$ es irracional: la suma no es periódica, aunque cada término lo sea.

### 1.2.5 Pares e impares

Una señal es *par* si es simétrica respecto al eje vertical, $x(t) = x(-t)$, e *impar* si es simétrica respecto al origen, $x(t) = -x(-t)$; en discreto, igual con $x[n]$. Una señal impar vale necesariamente cero en el origen. El producto de dos señales pares o de dos impares es par, y el de una par por una impar es impar. Además, la integral de una señal impar en un intervalo simétrico es nula, lo que simplifica muchos cálculos de series y transformadas de Fourier.

**Propiedad 1.2.3: Descomposición par–impar**

Toda señal se escribe de forma única como suma de una parte par y una impar, $x = x_p + x_i$, con

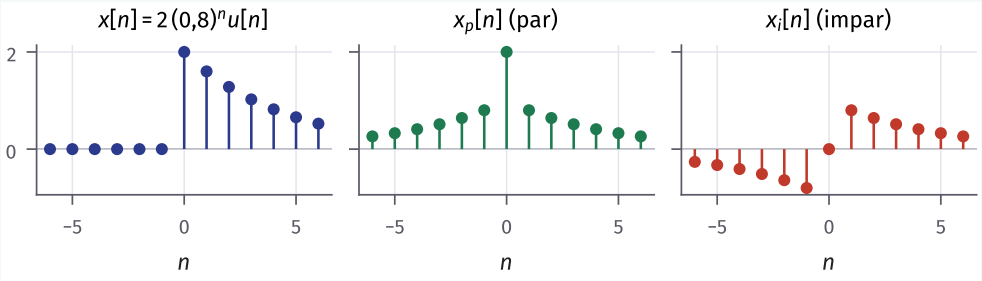
$$x_p(t) = \frac{x(t) + x(-t)}{2}, \qquad x_i(t) = \frac{x(t) - x(-t)}{2}.$$

Basta comprobar que $x_p$ es par, que $x_i$ es impar y que su suma es $x$. La unicidad se debe a que la única señal que es a la vez par e impar es la nula.

**Ejemplo 1.2.4: Descomposición de una exponencial causal**

Sea $x[n] = 2\,(0{,}8)^n u[n]$. Para $n > 0$, $x[-n] = 0$, de modo que $x_p[n] = x_i[n] = (0{,}8)^n$; para $n < 0$ ocurre lo contrario y $x_p[n] = (0{,}8)^{-n}$, $x_i[n] = -(0{,}8)^{-n}$; y en $n = 0$, $x_p[0] = 2$, $x_i[0] = 0$. En resumen:

$$x_p[n] = \begin{cases} 2 & n = 0 \\ (0{,}8)^{|n|} & n \neq 0 \end{cases}, \qquad x_i[n] = \operatorname{sgn}[n]\,(0{,}8)^{|n|}.$$




### 1.2.6 Causales, anticausales y no causales

Una señal es *causal* si vale cero para $t < 0$ (o $n < 0$), *anticausal* si vale cero para $t > 0$ (o $n > 0$) y *no causal* si no es causal. El nombre procede de los sistemas: la respuesta al impulso de un sistema causal, que es el único que se puede implementar en tiempo real, es una señal causal (apartado 1.8). Multiplicar una señal por el escalón $u(t)$ la hace causal.

### 1.2.7 Señales de energía y de potencia

**Definición 1.2.5: Energía y potencia media**

$$E_x = \int_{-\infty}^{\infty} |x(t)|^2 \, dt, \qquad P_x = \lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} |x(t)|^2 \, dt,$$

$$E_x = \sum_{n=-\infty}^{\infty} |x[n]|^2, \qquad P_x = \lim_{N\to\infty} \frac{1}{2N+1} \sum_{n=-N}^{N} |x[n]|^2.$$

Si $x$ fuera la tensión sobre una resistencia de $1\,\Omega$, $E_x$ sería la energía disipada y $P_x$ la potencia media; en general las usamos como medidas del «tamaño» de la señal, sin unidades físicas. Una señal es *de energía* si $0 < E_x < \infty$, y entonces su potencia es nula, porque la energía acumulada en $[-T, T]$ se mantiene acotada mientras el denominador $2T$ crece. Una señal es *de potencia* si $0 < P_x < \infty$, y entonces su energía es infinita. Las señales limitadas en el tiempo o que decaen son de energía; las periódicas, el escalón o un ruido estacionario son de potencia. Algunas señales, como la rampa, no son de ninguno de los dos tipos.

Para una señal periódica basta promediar en un periodo:

$$P_x = \frac{1}{T_0} \int_{T_0} |x(t)|^2 \, dt, \qquad P_x = \frac{1}{N_0} \sum_{n=0}^{N_0-1} |x[n]|^2.$$

**Ejemplo 1.2.6: Tres cálculos**

- $x[n] = a^n u[n]$ con $|a| < 1$: por la serie geométrica, $E_x = \sum_{n\geq 0} |a|^{2n} = 1/(1-|a|^2)$. Es una señal de energía; con $a = 0{,}9$, $E_x = 5{,}263$.
- $x(t) = A\cos(\omega_0 t + \phi)$: como $\cos^2\theta = \frac{1}{2}(1+\cos 2\theta)$ y el coseno integra cero en un periodo, $P_x = A^2/2$. Es de potencia; con $A = 3$, $P_x = 4{,}5$.
- $u(t)$: su energía es infinita y $P = \lim_{T\to\infty} \frac{1}{2T}\int_0^T dt = \frac{1}{2}$.

**En código**

Aproximamos la suma infinita y el límite con sumas finitas y comparamos con los valores teóricos del ejemplo anterior.

**Python**

```python
import numpy as np

n = np.arange(0, 1000)
x = 0.9**n                    # 0.9^n u[n]
E = np.sum(np.abs(x)**2)      # 1/(1-0.81)

N = 10_000
m = np.arange(-N, N + 1)
y = 3 * np.cos(0.1 * np.pi * m)
P = np.sum(np.abs(y)**2) / (2*N + 1)  # 9/2
print(f"E = {E:.3f}, P = {P:.3f}")
```

**Matlab**

```matlab
n = 0:999;
x = 0.9.^n;                   % 0.9^n u[n]
E = sum(abs(x).^2);           % 1/(1-0.81)

N = 10000;
m = -N:N;
y = 3*cos(0.1*pi*m);
P = sum(abs(y).^2) / (2*N + 1);   % 9/2
fprintf('E = %.3f, P = %.3f\n', E, P)
```

### 1.2.8 Señales complejas

Una señal compleja se describe con dos señales reales, ya sea en forma cartesiana (parte real e imaginaria) o polar (módulo y fase):

$$x = \mathrm{Re}\{x\} + j\,\mathrm{Im}\{x\} = |x|\,e^{j\arg x}, \qquad |x| = \sqrt{\mathrm{Re}\{x\}^2 + \mathrm{Im}\{x\}^2}, \qquad \arg x = \mathrm{atan2}\big(\mathrm{Im}\{x\}, \mathrm{Re}\{x\}\big).$$

La función atan2 tiene en cuenta el cuadrante; arctan(Im / Re) solo da el ángulo correcto cuando $\mathrm{Re}\{x\} > 0$. En Python y Matlab se usa `np.angle` y `angle`. Las señales complejas aparecen en las series y transformadas de Fourier y, en comunicaciones, para representar a la vez amplitud y fase de una señal en banda base (componentes I y Q).

Con $x = a + jb$ y $^*$ el conjugado, se definen las siguientes simetrías:

| Señal | Condición | Consecuencia |
|---|---|---|
| Real | $x(t) = x^*(t)$ | $b(t) = 0$ |
| Imaginaria pura | $x(t) = -x^*(t)$ | $a(t) = 0$ |
| Hermítica (conjugada simétrica) | $x(t) = x^*(-t)$ | $a$ par, $b$ impar |
| Antihermítica | $x(t) = -x^*(-t)$ | $a$ impar, $b$ par |

La importancia de la simetría hermítica es que la transformada de Fourier de una señal *real* es hermítica, $X(-j\omega) = X^*(j\omega)$: su módulo es par y su fase impar, y basta representar las frecuencias positivas.

Ejercicios 1–4. *Herramienta interactiva:* `T1-par-impar-energia`.

## 1.3 Señales básicas

### 1.3.1 Impulso

El impulso continuo o *delta de Dirac* es el límite de una familia de pulsos de área unidad cada vez más estrechos y altos, por ejemplo $\delta(t) = \lim_{\varepsilon\to 0} \frac{1}{\varepsilon}\Pi(t/\varepsilon)$. No es una función ordinaria, sino una *distribución*: se define por su efecto dentro de una integral.

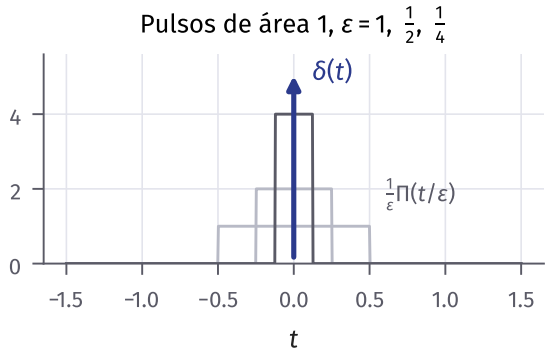
**Propiedad 1.3.1: Delta de Dirac**

- Selección: $\int_{-\infty}^{\infty} x(t)\,\delta(t - t_0)\,\mathrm{d}t = x(t_0)$ si $x$ es continua en $t_0$.
- Producto: $x(t)\,\delta(t - t_0) = x(t_0)\,\delta(t - t_0)$.
- Escalado: $\delta(at) = \delta(t)/|a|$ para $a \neq 0$. En particular, $\delta$ es par.
- Elemento neutro de la convolución: $(x * \delta)(t) = x(t)$, y $x(t) * \delta(t - t_0) = x(t - t_0)$.

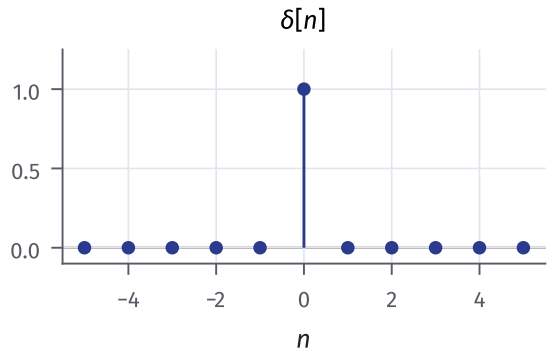
El impulso discreto o *delta de Kronecker*, $\delta[n] = 1$ si $n = 0$ y 0 en otro caso, es en cambio una secuencia ordinaria, sin problemas de definición. Toda secuencia se puede escribir como una suma de impulsos desplazados y ponderados,

$$x[n] = \sum_{k=-\infty}^{\infty} x[k]\,\delta[n - k], \tag{1.1}$$

que es la base para obtener la respuesta de un sistema LTI en la apartado 1.8.




**(a)** Delta de Dirac como límite de pulsos.




**(b)** Delta de Kronecker.

**Figura 1.3.** Impulsos continuo y discreto.

### 1.3.2 Escalón y rampa

$$u(t) = \begin{cases} 1 & t > 0 \\ 0 & t < 0 \end{cases} \qquad u[n] = \begin{cases} 1 & n \geq 0 \\ 0 & n < 0 \end{cases} \qquad r(t) = t\,u(t), \qquad r[n] = n\,u[n].$$

El valor de $u(0)$ no afecta a ninguna integral; según el contexto se toma 1 o $\frac{1}{2}$. Las tres señales básicas se relacionan por integración y derivación (en continuo) o por acumulación y diferencia (en discreto):

$$u(t) = \int_{-\infty}^{t} \delta(\tau)\,\mathrm{d}\tau, \quad \delta(t) = \frac{\mathrm{d}u(t)}{\mathrm{d}t}, \qquad r(t) = \int_{-\infty}^{t} u(\tau)\,\mathrm{d}\tau, \quad u(t) = \frac{\mathrm{d}r(t)}{\mathrm{d}t},$$

$$u[n] = \sum_{k=-\infty}^{n} \delta[k], \quad \delta[n] = u[n] - u[n-1], \qquad r[n] = \sum_{k=-\infty}^{n-1} u[k], \quad u[n] = r[n+1] - r[n].$$

Observa el desfase de una muestra en la rampa discreta: $\sum_{k \leq n} u[k] = n+1$ para $n \geq 0$, es decir, $r[n+1]$. Es un detalle típico del paso de continuo a discreto: la diferencia hacia atrás y la acumulación son inversas exactas, pero no reproducen todas las fórmulas continuas sin ajustar un índice.

### 1.3.3 Pulsos rectangular y triangular

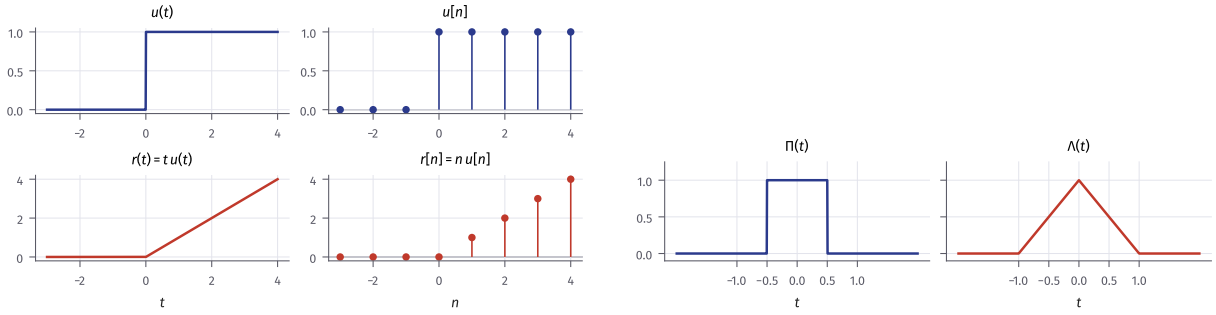
$$\Pi(t) = \begin{cases} 1 & |t| \leq \frac{1}{2} \\ 0 & |t| > \frac{1}{2} \end{cases} \qquad \Lambda(t) = \begin{cases} 1 - |t| & |t| \leq 1 \\ 0 & |t| > 1 \end{cases}$$

$\Pi(t/T)$ es un pulso de anchura $T$ centrado en el origen. El pulso triangular es la convolución de dos rectangulares, $\Lambda = \Pi * \Pi$, lo que hace muy sencilla su transformada de Fourier (apartado 1.6). En discreto se usa sobre todo el pulso causal de $L$ muestras, $p_L[n] = u[n] - u[n-L]$.

### 1.3.4 Exponenciales complejas continuas

La exponencial compleja $x(t) = A\mathrm{e}^{j\omega_0 t}$ es la señal más importante del curso. Por la fórmula de Euler,

$$\mathrm{e}^{j\theta} = \cos\theta + j\sin\theta, \qquad \cos\theta = \frac{\mathrm{e}^{j\theta} + \mathrm{e}^{-j\theta}}{2}, \qquad \sin\theta = \frac{\mathrm{e}^{j\theta} - \mathrm{e}^{-j\theta}}{2j},$$




**(a)** Escalón y rampa.

**(b)** Pulsos rectangular y triangular.

**Figura 1.4.** Señales básicas continuas y discretas.

su parte real e imaginaria son sinusoides. Es periódica para todo $\omega_0 \neq 0$, con $T_0 = 2\pi/|\omega_0|$, porque $e^{j\omega_0(t+T_0)} = e^{j\omega_0 t}e^{\pm j2\pi} = e^{j\omega_0 t}$. Dos frecuencias distintas dan siempre señales distintas, y cuanto mayor es $|\omega_0|$ más rápido oscila la señal.

La exponencial general $x(t) = Ce^{at}$, con $C = |C|e^{j\phi}$ y $a = r + j\omega_0$, es

$$x(t) = |C|\, e^{rt}\, e^{j(\omega_0 t+\phi)},$$

una sinusoide compleja con envolvente exponencial decreciente ($r < 0$), constante ($r = 0$) o creciente ($r > 0$). Si $a$ es real se reduce a la exponencial real. Estas señales son las respuestas naturales de los sistemas descritos por ecuaciones diferenciales lineales, y el exponente $a$ es un *polo* del sistema (apartado 1.9).

### 1.3.5 Exponenciales complejas discretas

La exponencial compleja discreta $x[n] = Ae^{j\Omega_0 n}$ se parece a la continua, pero tiene dos diferencias fundamentales que conviene entender bien.

**Primera diferencia: no siempre es periódica**

$x[n+N] = x[n]$ exige $e^{j\Omega_0 N} = 1$, es decir, $\Omega_0 N = 2\pi k$ para algún entero $k$. Por tanto, $e^{j\Omega_0 n}$ es periódica si y solo si

$$F_0 = \frac{\Omega_0}{2\pi} = \frac{k}{N} \quad \text{es racional.}$$

Si la fracción $k/N$ es irreducible, el periodo fundamental es $N$.

**Ejemplo 1.3.2: Periodicidad de sinusoides discretas**

- $\cos(\pi n/4)$: $F_0 = 1/8$, periodo $N = 8$.
- $\cos(3\pi n/8)$: $F_0 = 3/16$, periodo $N = 16$. El periodo de la sinusoide continua $\cos(3\pi t/8)$ sería $16/3$, que no es entero: la secuencia necesita 16 muestras, tres periodos de la continua, para repetirse.
- $\cos(0{,}4\,n)$: $F_0 = 0{,}2/\pi$ es irracional y la secuencia *no es periódica*, aunque $\cos(0{,}4\,t)$ sí lo sea.

**Segunda diferencia: frecuencias que difieren en $2\pi$ son la misma**

$$e^{j(\Omega_0+2\pi k)n} = e^{j\Omega_0 n}\, e^{j2\pi kn} = e^{j\Omega_0 n} \qquad \forall k \in \mathbb{Z},$$

porque $kn$ es entero. Todas las frecuencias distintas caben en un intervalo de longitud $2\pi$, que se suele tomar $[0, 2\pi)$ o $[-\pi, \pi)$.

En consecuencia, la oscilación más rápida posible es la de $\Omega = \pi$, $e^{j\pi n} = (-1)^n$, que cambia de signo en cada muestra. Las frecuencias bajas están cerca de $\Omega = 0$ *y también* cerca de $\Omega = 2\pi$. Cuando se muestrea una sinusoide continua de frecuencia $\omega$ con periodo $T_s$, la secuencia resultante tiene $\Omega = \omega T_s$; si $\omega T_s > \pi$, la secuencia coincide con la de una frecuencia más baja. Este es el origen del *aliasing*, que estudia el Tema 2.

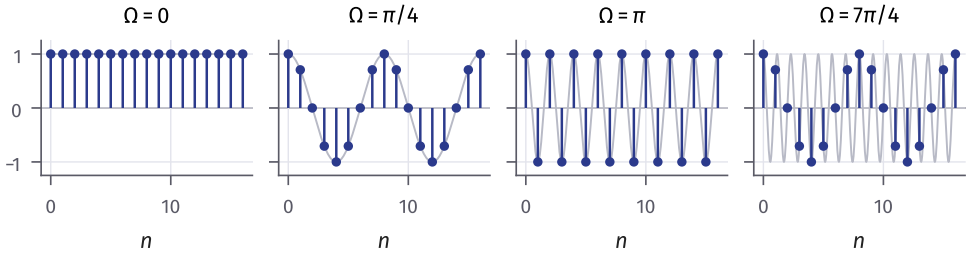



**Figura 1.5.** $\cos(\Omega_0 n)$ para varias frecuencias: la oscilación es más rápida al acercarse a $\Omega_0 = \pi$ y vuelve a ser lenta cerca de $2\pi$.

**En código**

Las tres secuencias son la misma aunque sus frecuencias sean $\pi/4$, $2\pi - \pi/4$ y $\pi/4 + 2\pi$ (la segunda, por ser un coseno, que es par):

**Python**

```python
import numpy as np

n = np.arange(0, 32)
x1 = np.cos(np.pi/4 * n)        # π/4
x2 = np.cos(7*np.pi/4 * n)      # π2 - π/4
x3 = np.cos(9*np.pi/4 * n)      # π/4 + π2
print(np.max(np.abs(x1 - x2)))   # ≈ 0
print(np.max(np.abs(x1 - x3)))   # ≈ 0
```

**Matlab**

```matlab
n = 0:31;
x1 = cos(pi/4 * n);             % π/4
x2 = cos(7*pi/4 * n);           % π2 - π/4
x3 = cos(9*pi/4 * n);           % π/4 + π2
disp(max(abs(x1 - x2)))         % ≈ 0
disp(max(abs(x1 - x3)))         % ≈ 0
```

La exponencial discreta general es $x[n] = C\alpha^n$, con $C = |C|e^{j\phi}$ y $\alpha = |\alpha|e^{j\Omega_0}$:

$$x[n] = |C|\,|\alpha|^n\, e^{j(\Omega_0 n + \phi)}.$$

Su envolvente decrece si $|\alpha| < 1$, es constante si $|\alpha| = 1$ y crece si $|\alpha| > 1$. Si $\alpha$ es real y negativo, la secuencia alterna de signo, algo que no tiene equivalente en la exponencial real continua. Son las respuestas naturales de las ecuaciones en diferencias, y $\alpha$ es un polo en el plano $z$.

### 1.3.6 Otras señales: Sa, sinc, signo y chirp

$$\mathrm{Sa}(t) = \frac{\sin t}{t}, \qquad \mathrm{sinc}(t) = \frac{\sin(\pi t)}{\pi t}, \qquad \mathrm{sgn}(t) = \begin{cases} 1 & t > 0 \\ 0 & t = 0 \\ -1 & t < 0 \end{cases}$$

con $\mathrm{Sa}(0) = \mathrm{sinc}(0) = 1$, de modo que $\mathrm{sinc}(t) = \mathrm{Sa}(\pi t)$. La función sinc se anula en todos los enteros no nulos; es la transformada de Fourier del pulso rectangular y el interpolador ideal del Tema 2. Tanto `np.sinc` como `sinc` de Matlab calculan $\sin(\pi t)/(\pi t)$.

Las ondas cuadrada y triangular son periódicas y solo contienen armónicos impares (apartado 1.6). El *chirp* lineal, $x(t) = \cos\left(2\pi(f_0 t + \frac{\beta}{2}t^2)\right)$, tiene una frecuencia instantánea $f_i(t) = \frac{1}{2\pi}\frac{\mathrm{d}\theta}{\mathrm{d}t} = f_0 + \beta t$ que crece linealmente; se usa en radar, sonar y para medir respuestas en frecuencia.

**En código**

Para representar secuencias basta un vector de índices y comparaciones lógicas:

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(-10, 11)          # eje temporal
delta = (n == 0).astype(float)  # δ[n]
u = (n >= 0).astype(float)      # u[n]
r = n * u                       # r[n] = n u[n]
x = 0.8**n * u                  # 0.8^n u[n]

plt.stem(n, x)
plt.xlabel("n"); plt.ylabel("x[n]")
plt.show()
```

**Matlab**

```matlab
n = -10:10;                % eje temporal
delta = double(n == 0);    % δ[n]
u = double(n >= 0);        % u[n]
r = n .* u;                % r[n] = n u[n]
x = 0.8.^n .* u;           % 0.8^n u[n]

stem(n, x)
xlabel('n'); ylabel('x[n]')
```

Ejercicios 1, 2 y 9. *Herramientas interactivas:* `T1-exponencial-continua`, `T1-exponencial-discreta` y `T1-continua-discreta`.

## 1.4 Operaciones con señales

### 1.4.1 Transformaciones de la variable independiente

Estas operaciones cambian el eje de tiempos sin alterar los valores que toma la señal: $y(t) = x\big(f(t)\big)$. Cuando $f$ es afín se reducen a tres operaciones básicas.

- **Desplazamiento**: $y(t) = x(t - t_0)$. Si $t_0 > 0$ es un *retardo*: la señal se desplaza hacia la derecha, porque $y$ toma en $t = t_0$ el valor que $x$ tenía en $t = 0$. Si $t_0 < 0$ es un *adelanto*. En discreto, $y[n] = x[n - n_0]$ con $n_0$ entero.
- **Inversión**: $y(t) = x(-t)$, reflejo respecto al origen.
- **Escalado**: $y(t) = x(\alpha t)$ con $\alpha > 0$ comprime la señal si $\alpha > 1$ y la expande si $\alpha < 1$. Es reversible: $x(t) = y(t/\alpha)$.

En discreto el escalado no es reversible en general, porque el índice tiene que seguir siendo entero. El *diezmado* $y[n] = x[Mn]$, con $M$ entero positivo, se queda con una de cada $M$ muestras y descarta las demás: se pierde información. La *expansión* por un factor $L$ define $y[n] = x[n/L]$ cuando $n$ es múltiplo de $L$ y $y[n] = 0$ en otro caso; las muestras intermedias se rellenan después mediante interpolación (Tema 2).

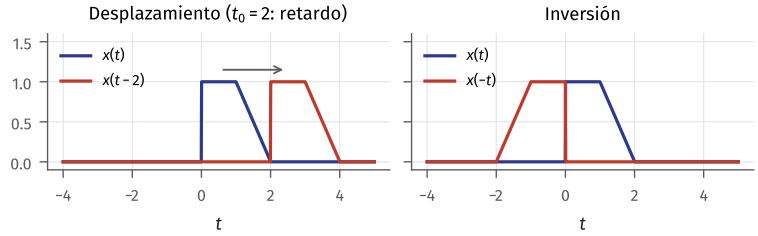



**Figura 1.6.** Desplazamiento e inversión de una señal continua.

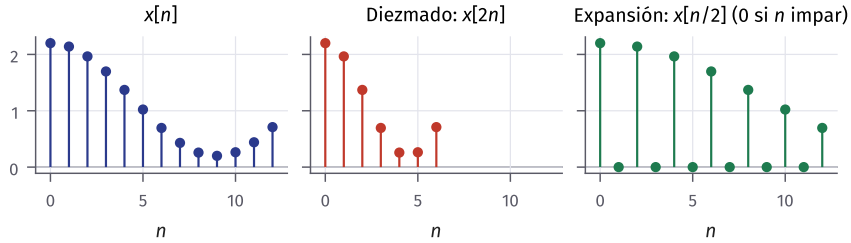
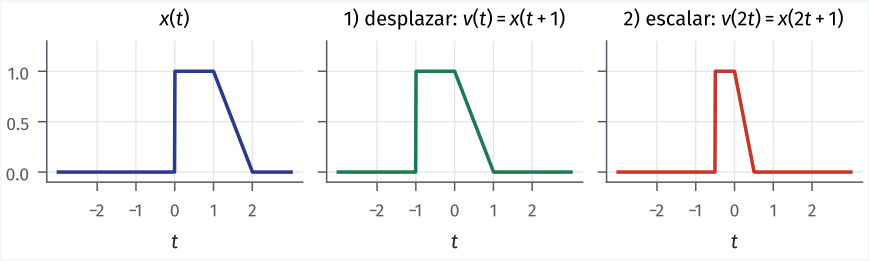



**Figura 1.7.** Diezmado por $M = 2$ y expansión por $L = 2$ de una secuencia.

**Transformaciones combinadas.** Para dibujar $y(t) = x(\alpha t + \beta)$ hay un procedimiento que no falla: primero desplazar, $v(t) = x(t + \beta)$, y después escalar (e invertir si $\alpha < 0$), $y(t) = v(\alpha t) = x(\alpha t + \beta)$. Si se escala primero, el desplazamiento que hay que aplicar después es $\beta/\alpha$, porque $x(\alpha t + \beta) = x\big(\alpha(t + \beta/\alpha)\big)$. Una comprobación rápida consiste en seguir un punto: el valor que $x$ tenía en $t = 0$ aparece en $y$ cuando $\alpha t + \beta = 0$.

**Ejemplo 1.4.1:** $y(t) = x(2t + 1)$

Primero $v(t) = x(t + 1)$, que adelanta $x$ una unidad; después $y(t) = v(2t)$, que comprime $v$ a la mitad. El punto $t = 0$ de $x$ aparece en $y$ en $2t + 1 = 0$, es decir, en $t = -\frac{1}{2}$. Si $x$ es no nula en $[-1, 1]$, $y$ es no nula en $[-1, 0]$.




### 1.4.2 Operaciones sobre la amplitud

Las operaciones punto a punto se aplican igual a señales continuas y discretas: suma, producto (que aparece en la modulación y el enventanado), escalado de amplitud y cociente (donde el denominador no se anule). Además, para señales continuas se definen la derivada y la integral acumulada, y para secuencias sus análogos discretos:

$$\text{diferencia: } y[n] = x[n] - x[n-1], \qquad \text{acumulación: } y[n] = \sum_{k=-\infty}^{n} x[k].$$

La diferencia es la versión discreta de la derivada y la acumulación, la de la integral: por ejemplo $u[n] - u[n-1] = \delta[n]$ y $\sum_{k \le n} \delta[k] = u[n]$, igual que $u'(t) = \delta(t)$.

**Producto y convolución**

El producto $x_1(t)\,x_2(t)$ y la convolución $(x_1 * x_2)(t)$ son operaciones muy distintas. En estos apuntes el producto se escribe por yuxtaposición o con un punto, y el asterisco $*$ se reserva siempre para la convolución.

*Herramienta interactiva:* `T1-variable-independiente`.

## 1.5 Convolución

**Definición 1.5.1: Convolución**

$$(x * h)(t) = \int_{-\infty}^{\infty} x(\tau)\,h(t-\tau)\,\mathrm{d}\tau, \qquad (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]\,h[n-k].$$

La convolución es la operación central de los sistemas LTI: en la apartado 1.8 veremos que la salida de cualquier sistema LTI es la convolución de la entrada con su respuesta al impulso. Para calcularla a mano conviene seguir el procedimiento gráfico, para cada valor de $t$:

1. reflejar $h(\tau)$ para obtener $h(-\tau)$;
2. desplazarla hasta $h(t-\tau)$;
3. multiplicar por $x(\tau)$;
4. integrar (o sumar) el producto en $\tau$.

A medida que $t$ avanza, la versión reflejada de $h$ se desliza sobre $x$ y conviene dividir el eje $t$ en intervalos según cómo se solapen las dos señales.

**Propiedad 1.5.2: Convolución**

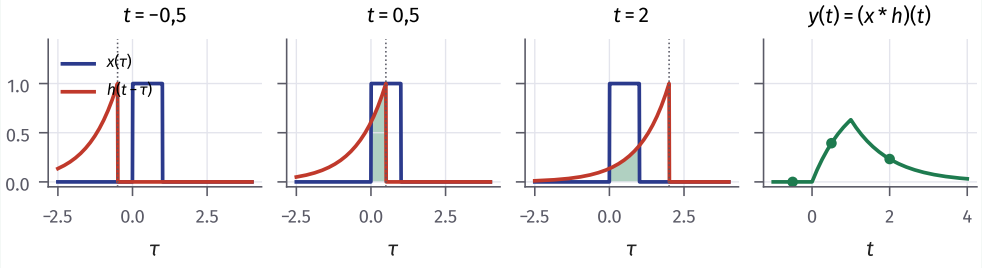
- Conmutativa: $x * h = h * x$.
- Asociativa: $(x * h_1) * h_2 = x * (h_1 * h_2)$.
- Distributiva respecto a la suma: $x * (h_1 + h_2) = x * h_1 + x * h_2$.
- Elemento neutro y desplazamiento: $x * \delta = x$, $x(t) * \delta(t - t_0) = x(t - t_0)$.
- Con el escalón: $(x * u)(t) = \int_{-\infty}^{t} x(\tau)\,\mathrm{d}\tau$, $(x * u)[n] = \sum_{k \le n} x[k]$.
- Duración: si $x$ es no nula en $[a, b]$ y $h$ en $[c, d]$, $x * h$ es no nula como mucho en $[a + c,\ b + d]$.

**Ejemplo 1.5.3: Convolución continua de un pulso y una exponencial**

Sean $x(t) = u(t) - u(t-1)$ y $h(t) = \mathrm{e}^{-t}u(t)$. Usamos $y(t) = \int x(\tau)\,h(t-\tau)\,\mathrm{d}\tau$, con $x(\tau) = 1$ en $[0, 1]$ y $h(t-\tau) = \mathrm{e}^{-(t-\tau)}$ para $\tau \le t$.

- Si $t < 0$, las señales no se solapan y $y(t) = 0$.
- Si $0 \le t \le 1$, el solape es $[0, t]$: $y(t) = \int_0^t \mathrm{e}^{-(t-\tau)}\,\mathrm{d}\tau = \mathrm{e}^{-t}(\mathrm{e}^{t} - 1) = 1 - \mathrm{e}^{-t}$.
- Si $t > 1$, el pulso queda entero dentro: $y(t) = \int_0^1 \mathrm{e}^{-(t-\tau)}\,\mathrm{d}\tau = (\mathrm{e} - 1)\,\mathrm{e}^{-t}$.

La salida es continua en $t = 1$, donde ambas expresiones valen $1 - e^{-1} \approx 0{,}632$.




**Ejemplo 1.5.4: Convolución discreta**

Sean $x[n] = \{\underline{1}, 2, 1\}$ y $h[n] = \{\underline{1}, 1, 1, 1\}$, donde el subrayado marca $n = 0$. Por la descomposición en impulsos, $x * h = x[0]h[n] + x[1]h[n-1] + x[2]h[n-2]$, y basta sumar copias desplazadas de $h$:

| $n$ | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $x[0]h[n]$ | 1 | 1 | 1 | 1 | | |
| $x[1]h[n-1]$ | | 2 | 2 | 2 | 2 | |
| $x[2]h[n-2]$ | | | 1 | 1 | 1 | 1 |
| $y[n]$ | 1 | 3 | 4 | 4 | 3 | 1 |

Si $x$ tiene $N_x$ muestras y $h$ tiene $N_h$, la convolución tiene $N_x + N_h - 1$; si empiezan en $n_x$ y $n_h$, el resultado empieza en $n_x + n_h$.

**En código**

`np.convolve` y `conv` trabajan con vectores sin índice: el índice inicial del resultado hay que llevarlo aparte (es la suma de los índices iniciales).

**Python**

```python
import numpy as np

x = np.array([1, 2, 1])      # n = 0..2
h = np.array([1, 1, 1, 1])   # n = 0..3
y = np.convolve(x, h)        # n = 0..5
print(y)                     # [1 3 4 4 3 1]
print(len(y) == len(x) + len(h) - 1)
```

**Matlab**

```matlab
x = [1 2 1];                 % n = 0..2
h = [1 1 1 1];               % n = 0..3
y = conv(x, h);              % n = 0..5
disp(y)                      % 1 3 4 4 3 1
disp(length(y) == length(x) + length(h) - 1)
```

Ejercicios 5 y 6. *Herramientas interactivas:* `T1-convolucion-continua` y `T1-convolucion-discreta`.

## 1.6 Representación en frecuencia

### 1.6.1 La idea: descomponer en exponenciales complejas

Las exponenciales complejas $e^{j\omega t}$ y $e^{j\Omega n}$ son *autofunciones* de los sistemas LTI: atraviesan el sistema sin cambiar de forma, solo multiplicadas por un número complejo (apartado 1.8.3). Si una señal se escribe como suma o integral de exponenciales complejas, la respuesta del sistema se obtiene sumando las respuestas a cada una. Según la señal sea continua o discreta, periódica o aperiódica, se usa una de cuatro representaciones:

| | Tiempo continuo | Tiempo discreto |
|---|---|---|
| Periódica | Serie de Fourier: $c_k$ | Serie de Fourier discreta (DFS): $c_k$ |
| Aperiódica | Transformada de Fourier (CTFT): $X(j\omega)$ | Transformada de Fourier (DTFT): $X(\mathrm{e}^{j\Omega})$ |

Hay una dualidad que conviene retener: la periodicidad en un dominio se corresponde con un espectro discreto (de líneas) en el otro, y el tiempo discreto se corresponde con un espectro periódico.

### 1.6.2 Serie de Fourier de señales continuas periódicas

Sea $x(t)$ periódica de periodo $T_0$ y frecuencia fundamental $\omega_0 = 2\pi/T_0$. Su serie de Fourier exponencial es

$$x(t) = \sum_{k=-\infty}^{\infty} c_k\, \mathrm{e}^{jk\omega_0 t}, \qquad c_k = \frac{1}{T_0}\int_{T_0} x(t)\, \mathrm{e}^{-jk\omega_0 t}\, \mathrm{d}t.$$

La fórmula de análisis se obtiene multiplicando la de síntesis por $\mathrm{e}^{-jm\omega_0 t}$ e integrando en un periodo: todas las exponenciales con $k \neq m$ integran cero (son ortogonales) y solo queda $T_0 c_m$. El coeficiente $c_0$ es el valor medio de la señal y los pares $|c_k|$, $\arg c_k$ forman su espectro de líneas.

Si $x$ es real, $c_{-k} = c_k^*$ y la serie se puede escribir en forma trigonométrica:

$$x(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty} \big(a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t)\big),$$

$$a_k = \frac{2}{T_0}\int_{T_0} x(t)\cos(k\omega_0 t)\,\mathrm{d}t, \quad b_k = \frac{2}{T_0}\int_{T_0} x(t)\sin(k\omega_0 t)\,\mathrm{d}t,$$

con $c_k = \frac{1}{2}(a_k - jb_k)$. Si $x$ es par, todos los $b_k$ se anulan; si es impar, se anulan los $a_k$.

**Propiedad 1.6.1: Parseval para series de Fourier**

$$P_x = \frac{1}{T_0}\int_{T_0} |x(t)|^2\,\mathrm{d}t = \sum_{k=-\infty}^{\infty} |c_k|^2.$$

La potencia de la señal es la suma de las potencias de sus armónicos.

**Ejemplo 1.6.2: Onda cuadrada**

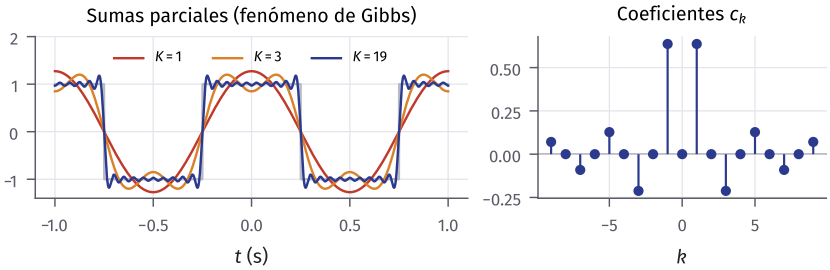
Sea la onda cuadrada par de amplitud $\pm 1$ y ciclo de trabajo del 50 %: $x(t) = 1$ para $|t| < T_0/4$ y $x(t) = -1$ para $T_0/4 < |t| < T_0/2$. Su valor medio es $c_0 = 0$. Escribiendo $x = 2p - 1$, donde $p$ es un tren de pulsos de altura 1 y anchura $T_0/2$, para $k \neq 0$:

$$c_k = \frac{2}{T_0}\int_{-T_0/4}^{T_0/4} \mathrm{e}^{-jk\omega_0 t}\,\mathrm{d}t = \frac{2}{T_0}\cdot\frac{2\sin(k\omega_0 T_0/4)}{k\omega_0} = \frac{\sin(\pi k/2)}{\pi k/2} = \operatorname{sinc}(k/2).$$

Los coeficientes pares son nulos y los impares valen $\pm 2/(\pi k)$, de modo que

$$x(t) = \frac{4}{\pi}\Big(\cos\omega_0 t - \tfrac{1}{3}\cos 3\omega_0 t + \tfrac{1}{5}\cos 5\omega_0 t - \cdots\Big).$$

Comprobación con Parseval: $P_x = 1$ y $\sum_k |c_k|^2 = 2\sum_{k\ \text{impar}} \frac{4}{\pi^2 k^2} = \frac{8}{\pi^2}\cdot\frac{\pi^2}{8} = 1$.




Cerca de las discontinuidades, las sumas parciales sobrepasan el salto en torno a un 9 % por mucho que crezca el número de armónicos: es el *fenómeno de Gibbs*. El pico se estrecha, pero su altura no disminuye.

**En código**

Suma parcial de la serie de la onda cuadrada con $K$ armónicos. Prueba con $K = 1, 3, 19, 99$.

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt

T = 1.0                          # periodo
t = np.linspace(-T, T, 2000)
K = 19                           # armónicos
xK = np.zeros_like(t)
for k in range(1, K + 1):
    ck = np.sinc(k / 2)          # c_k,
    cuadrada par
    xK += 2*ck*np.cos(2*np.pi*k*t/T)
plt.plot(t, xK); plt.xlabel("t (s)")
plt.show()
```

**Matlab**

```matlab
T = 1;                           % periodo
t = linspace(-T, T, 2000);
K = 19;                          % armónicos
xK = zeros(size(t));
for k = 1:K
    ck = sin(pi*k/2)/(pi*k/2); % c_k,
    cuadrada par
    xK = xK + 2*ck*cos(2*pi*k*t/T);
end
plot(t, xK); xlabel('t (s)')
```

### 1.6.3 Serie de Fourier discreta

Si $x[n]$ es periódica de periodo $N$, solo hay $N$ exponenciales $\mathrm{e}^{j\frac{2\pi}{N}kn}$ distintas (por la segunda diferencia de la apartado 1.3.5), de modo que la serie es una suma finita:

$$x[n] = \sum_{k=0}^{N-1} c_k \, \mathrm{e}^{j\frac{2\pi}{N}kn}, \qquad c_k = \frac{1}{N}\sum_{n=0}^{N-1} x[n]\,\mathrm{e}^{-j\frac{2\pi}{N}kn}, \qquad c_{k+N} = c_k.$$

No hay problemas de convergencia ni fenómeno de Gibbs: son $N$ ecuaciones con $N$ incógnitas. Para $x$ real, la forma trigonométrica es

$$x[n] = \frac{a_0}{2} + \sum_{k=1}^{L}\left(a_k \cos\left(\tfrac{2\pi}{N}kn\right) + b_k \sin\left(\tfrac{2\pi}{N}kn\right)\right)\left[\,+\,\tfrac{a_{N/2}}{2}(-1)^n\right],$$

con $a_k = \frac{2}{N}\sum_{n=0}^{N-1} x[n]\cos\left(\frac{2\pi}{N}kn\right)$, $b_k = \frac{2}{N}\sum_{n=0}^{N-1} x[n]\sin\left(\frac{2\pi}{N}kn\right)$ y $L = (N-1)/2$ si $N$ es impar. Si $N$ es par, $L = N/2 - 1$ y hay que añadir el término entre corchetes, que corresponde a la frecuencia $\Omega = \pi$. La DFT de un periodo, que estudia el Tema 3, es $X[k] = Nc_k$.

### 1.6.4 Transformada de Fourier en tiempo continuo

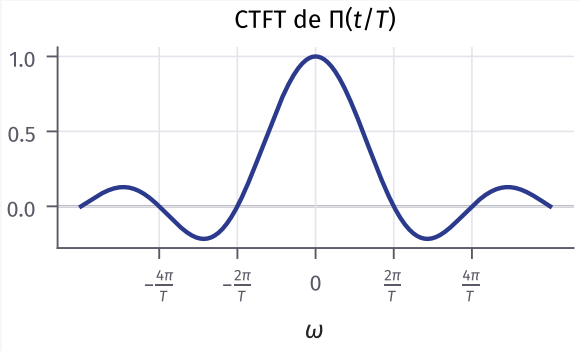
Cuando el periodo $T_0$ de una señal crece sin límite, las líneas del espectro se acercan ($\omega_0 \to 0$) y la serie se convierte en una integral. Así se obtiene la transformada de Fourier:

$$X(j\omega) = \int_{-\infty}^{\infty} x(t)\, e^{-j\omega t}\, dt, \qquad x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(j\omega)\, e^{j\omega t}\, d\omega.$$

Existe si $x$ es absolutamente integrable, $\int |x|\, dt < \infty$; admitiendo deltas en frecuencia, se extiende a señales de potencia como las sinusoides o el escalón. Si $x$ es real, $X(-j\omega) = X^*(j\omega)$.

**Ejemplo 1.6.3: Dos transformadas básicas**

- $x(t) = e^{-at}u(t)$, $a > 0$: $X(j\omega) = \int_0^{\infty} e^{-(a+j\omega)t}\, dt = \dfrac{1}{a + j\omega}$. El módulo, $1/\sqrt{a^2 + \omega^2}$, es máximo en $\omega = 0$ y decae: es una señal «de baja frecuencia».
- $x(t) = \Pi(t/T)$: $X(j\omega) = \int_{-T/2}^{T/2} e^{-j\omega t}\, dt = \dfrac{2\sin(\omega T/2)}{\omega} = T\,\mathrm{sinc}\left(\dfrac{\omega T}{2\pi}\right)$. Los ceros están en $\omega = 2\pi k/T$: cuanto más corto es el pulso, más ancho es su espectro.




| $x(t)$ | $X(j\omega)$ | $x(t)$ | $X(j\omega)$ |
|---|---|---|---|
| $\delta(t)$ | $1$ | $e^{-at}u(t)$, $a > 0$ | $\dfrac{1}{a + j\omega}$ |
| $1$ | $2\pi\,\delta(\omega)$ | $\Pi(t/T)$ | $T\,\mathrm{sinc}\left(\frac{\omega T}{2\pi}\right)$ |
| $e^{j\omega_0 t}$ | $2\pi\,\delta(\omega - \omega_0)$ | $\Lambda(t)$ | $\mathrm{sinc}^2\left(\frac{\omega}{2\pi}\right)$ |
| $\cos(\omega_0 t)$ | $\pi[\delta(\omega - \omega_0) + \delta(\omega + \omega_0)]$ | $\mathrm{sgn}(t)$ | $\dfrac{2}{j\omega}$ |

### 1.6.5 Transformada de Fourier en tiempo discreto

$$X(e^{j\Omega}) = \sum_{n=-\infty}^{\infty} x[n]\, e^{-j\Omega n}, \qquad x[n] = \frac{1}{2\pi}\int_{2\pi} X(e^{j\Omega})\, e^{j\Omega n}\, d\Omega.$$

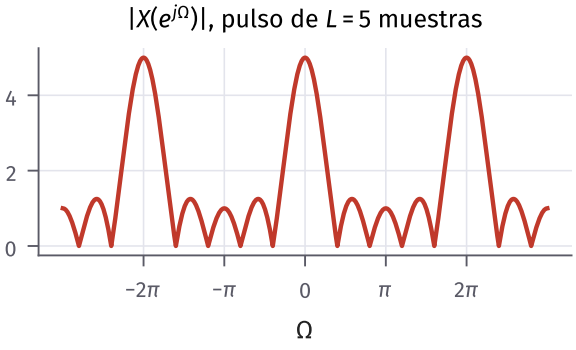
La DTFT existe si $\sum|x[n]| < \infty$. Su propiedad más característica es que es *periódica en $\Omega$ con periodo $2\pi$*, consecuencia directa de que $e^{-j(\Omega+2\pi)n} = e^{-j\Omega n}$. Basta, por tanto, representarla en $[-\pi, \pi)$ o en $[0, 2\pi)$. Numéricamente se evalúa con `freqz`.

**Ejemplo 1.6.4: Exponencial causal y pulso**

- $x[n] = a^n u[n]$, $|a| < 1$: $X(\mathrm{e}^{j\Omega}) = \sum_{n\geq 0} (a\mathrm{e}^{-j\Omega})^n = \dfrac{1}{1 - a\mathrm{e}^{-j\Omega}}$.
- $x[n] = u[n] - u[n-L]$: suma geométrica finita,

$$X(\mathrm{e}^{j\Omega}) = \sum_{n=0}^{L-1} \mathrm{e}^{-j\Omega n} = \frac{1 - \mathrm{e}^{-j\Omega L}}{1 - \mathrm{e}^{-j\Omega}} = \mathrm{e}^{-j\Omega\frac{L-1}{2}} \frac{\sin(\Omega L/2)}{\sin(\Omega/2)}.$$

Es el equivalente discreto del sinc: el cociente de senos se llama a veces *núcleo de Dirichlet*. Sus ceros están en $\Omega = 2\pi k/L$, $k \neq 0, L, 2L, \ldots$




**En código**

`freqz` evalúa $\sum_n x[n]\mathrm{e}^{-j\Omega n}$ en una rejilla de frecuencias; con la opción `whole` cubre $[0, 2\pi)$ y deja ver la periodicidad.

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

x = np.ones(5)                  # pulso, L = 5
w, X = signal.freqz(x, 1, worN=1024,
                    whole=True)   # [0, π2)
plt.plot(w, np.abs(X))
plt.xlabel("Ω (rad/muestra)")
plt.show()
```

**Matlab**

```matlab
x = ones(1, 5);                 % pulso, L = 5
[X, w] = freqz(x, 1, 1024, ...
               'whole');        % [0, π2)
plot(w, abs(X))
xlabel('\Omega (rad/muestra)')
```

### 1.6.6 Propiedades de la transformada de Fourier

| Propiedad | CTFT | DTFT |
|---|---|---|
| Linealidad | $ax_1 + bx_2 \overset{\mathscr{F}}{\longleftrightarrow} aX_1 + bX_2$ | ídem |
| Desplazamiento | $x(t-t_0) \overset{\mathscr{F}}{\longleftrightarrow} \mathrm{e}^{-j\omega t_0}X(j\omega)$ | $x[n-n_0] \overset{\mathscr{F}}{\longleftrightarrow} \mathrm{e}^{-j\Omega n_0}X(\mathrm{e}^{j\Omega})$ |
| Modulación | $\mathrm{e}^{j\omega_0 t}x(t) \overset{\mathscr{F}}{\longleftrightarrow} X(j(\omega-\omega_0))$ | $\mathrm{e}^{j\Omega_0 n}x[n] \overset{\mathscr{F}}{\longleftrightarrow} X(\mathrm{e}^{j(\Omega-\Omega_0)})$ |
| Escalado | $x(at) \overset{\mathscr{F}}{\longleftrightarrow} \frac{1}{\lvert a\rvert}X\left(j\frac{\omega}{a}\right)$ | — |
| Derivada / diferencia | $\frac{\mathrm{d}x}{\mathrm{d}t} \overset{\mathscr{F}}{\longleftrightarrow} j\omega\, X(j\omega)$ | $x[n]-x[n-1] \overset{\mathscr{F}}{\longleftrightarrow} (1-\mathrm{e}^{-j\Omega})X(\mathrm{e}^{j\Omega})$ |
| Convolución | $x * h \overset{\mathscr{F}}{\longleftrightarrow} X(j\omega)H(j\omega)$ | $x * h \overset{\mathscr{F}}{\longleftrightarrow} X(\mathrm{e}^{j\Omega})H(\mathrm{e}^{j\Omega})$ |
| Producto | $x\,y \overset{\mathscr{F}}{\longleftrightarrow} \frac{1}{2\pi}(X * Y)(j\omega)$ | $x\,y \overset{\mathscr{F}}{\longleftrightarrow} \frac{1}{2\pi}\int_{2\pi} X(\mathrm{e}^{j\theta})Y(\mathrm{e}^{j(\Omega-\theta)})\,\mathrm{d}\theta$ |
| Parseval | $\int\lvert x\rvert^2\,\mathrm{d}t = \frac{1}{2\pi}\int\lvert X\rvert^2\,\mathrm{d}\omega$ | $\sum\lvert x\rvert^2 = \frac{1}{2\pi}\int_{2\pi}\lvert X\rvert^2\,\mathrm{d}\Omega$ |

Tres de ellas merecen un comentario. La propiedad de *desplazamiento* dice que retrasar una señal no cambia el módulo de su espectro, solo añade una fase lineal en la frecuencia. La de *modulación* dice que multiplicar por una exponencial desplaza el espectro: es la base de la modulación en comunicaciones. Y la de *convolución* es la clave del curso: *filtrar es multiplicar espectros*. Un sistema LTI con respuesta en frecuencia $H$ deja pasar las frecuencias donde $|H|$ es grande y atenúa aquellas donde es pequeño.

La DTFT no tiene propiedad de escalado: diezmar o expandir una secuencia cambia su espectro de una forma más complicada, que se estudia en el Tema 2 al hablar de cambio de frecuencia de muestreo.

### 1.6.7 Transformada de Laplace

La transformada de Fourier no existe para señales que crecen, como $\mathrm{e}^{t}u(t)$, ni es la herramienta más cómoda para resolver ecuaciones diferenciales. La transformada de Laplace la generaliza sustituyendo $j\omega$ por una variable compleja $s = \sigma + j\omega$:

$$X(s) = \mathscr{L}\{x(t)\} = \int_{-\infty}^{\infty} x(t)\,\mathrm{e}^{-st}\,\mathrm{d}t.$$

Equivale a la transformada de Fourier de $x(t)\mathrm{e}^{-\sigma t}$: el factor $\mathrm{e}^{-\sigma t}$ hace converger la integral para algunos valores de $\sigma$. El conjunto de valores de $s$ en que converge es la *región de convergencia* (ROC), que es siempre una franja vertical $\sigma_1 < \mathrm{Re}\{s\} < \sigma_2$ y que no contiene polos.

**La ROC es parte de la transformada**

$$\mathrm{e}^{-at}u(t) \overset{\mathscr{L}}{\longleftrightarrow} \frac{1}{s+a},\ \mathrm{Re}\{s\} > -a, \qquad -\mathrm{e}^{-at}u(-t) \overset{\mathscr{L}}{\longleftrightarrow} \frac{1}{s+a},\ \mathrm{Re}\{s\} < -a.$$

La misma expresión algebraica corresponde a dos señales distintas, una causal y otra anticausal. Sin la ROC, la transformada no determina la señal.

Para la primera, $\int_0^{\infty} \mathrm{e}^{-(s+a)t}\,\mathrm{d}t$ converge solo si $\mathrm{Re}\{s+a\} > 0$. Para la segunda, $-\int_{-\infty}^{0} \mathrm{e}^{-(s+a)t}\,\mathrm{d}t$ converge solo si $\mathrm{Re}\{s+a\} < 0$. En general, una señal causal tiene la ROC a la derecha de su polo más a la derecha, una anticausal a la izquierda del polo más a la izquierda, y una bilateral, en una franja entre polos.

Si la ROC contiene el eje imaginario, la transformada de Fourier existe y se obtiene evaluando en él: $X(j\omega) = X(s)|_{s=j\omega}$. La versión *unilateral*, $\int_{0^-}^{\infty}$, coincide con la bilateral para señales causales y tiene la

ventaja de incorporar las condiciones iniciales en la propiedad de la derivada, por lo que es la que se usa para resolver ecuaciones diferenciales con condiciones iniciales no nulas.

| $x(t)$ | $X(s)$ | ROC | Propiedad | |
|---|---|---|---|---|
| $\delta(t)$ | $1$ | todo $\mathbb{C}$ | Linealidad | $ax_1 + bx_2 \overset{\mathscr{L}}{\longleftrightarrow} aX_1 + bX_2$ |
| $u(t)$ | $1/s$ | $\mathrm{Re}\{s\} > 0$ | Desplazamiento | $x(t-t_0) \overset{\mathscr{L}}{\longleftrightarrow} \mathrm{e}^{-st_0}X(s)$ |
| $t\,u(t)$ | $1/s^2$ | $\mathrm{Re}\{s\} > 0$ | Desplaz. en $s$ | $\mathrm{e}^{s_0 t}x(t) \overset{\mathscr{L}}{\longleftrightarrow} X(s-s_0)$ |
| $\mathrm{e}^{-at}u(t)$ | $\dfrac{1}{s+a}$ | $\mathrm{Re}\{s\} > -a$ | Derivada | $\dfrac{\mathrm{d}x}{\mathrm{d}t} \overset{\mathscr{L}}{\longleftrightarrow} sX(s)$ |
| $\cos(\omega_0 t)u(t)$ | $\dfrac{s}{s^2+\omega_0^2}$ | $\mathrm{Re}\{s\} > 0$ | (unilateral) | $\dfrac{\mathrm{d}x}{\mathrm{d}t} \overset{\mathscr{L}}{\longleftrightarrow} sX(s) - x(0^-)$ |
| $\sin(\omega_0 t)u(t)$ | $\dfrac{\omega_0}{s^2+\omega_0^2}$ | $\mathrm{Re}\{s\} > 0$ | Integral | $\int_{-\infty}^{t} x\,\mathrm{d}\tau \overset{\mathscr{L}}{\longleftrightarrow} X(s)/s$ |
| $\mathrm{e}^{-at}\cos(\omega_0 t)u(t)$ | $\dfrac{s+a}{(s+a)^2+\omega_0^2}$ | $\mathrm{Re}\{s\} > -a$ | Convolución | $x * h \overset{\mathscr{L}}{\longleftrightarrow} X(s)H(s)$ |

### 1.6.8 Transformada Z

La transformada Z es el equivalente discreto de la de Laplace:

$$X(z) = \mathscr{Z}\{x[n]\} = \sum_{n=-\infty}^{\infty} x[n]\, z^{-n}, \qquad z = re^{j\Omega} \in \mathbb{C}.$$

Con $z = re^{j\Omega}$ es la DTFT de $x[n]r^{-n}$. Su ROC es una corona $r_1 < |z| < r_2$ que no contiene polos. Igual que en Laplace, la ROC forma parte de la transformada:

$$a^n u[n] \overset{\mathscr{Z}}{\longleftrightarrow} \frac{1}{1-az^{-1}}, \quad |z| > |a|, \qquad -a^n u[-n-1] \overset{\mathscr{Z}}{\longleftrightarrow} \frac{1}{1-az^{-1}}, \quad |z| < |a|.$$

Una secuencia causal tiene la ROC fuera de un círculo, $|z| > \max|p_i|$; una anticausal, dentro de un círculo; y una bilateral, en una corona entre polos.

**Ejemplo 1.6.5: Una secuencia bilateral**

$x[n] = (0{,}5)^n u[n] - 2^n u[-n-1]$. Cada término aporta su transformada con su ROC: $|z| > 0{,}5$ para el primero y $|z| < 2$ para el segundo. La ROC de la suma es la intersección, $0{,}5 < |z| < 2$, y

$$X(z) = \frac{1}{1-0{,}5z^{-1}} + \frac{1}{1-2z^{-1}}.$$

Como la ROC contiene la circunferencia unidad, la DTFT existe y vale $X(\mathrm{e}^{j\Omega})$. Si el segundo término fuera $3^n u[n]$, las ROC ($|z| > 0{,}5$ y $|z| > 3$) también se cortarían, pero la circunferencia unidad quedaría fuera y la DTFT no existiría.

Si la ROC contiene la circunferencia unidad, la DTFT existe y $X(\mathrm{e}^{j\Omega}) = X(z)|_{z=\mathrm{e}^{j\Omega}}$. La versión unilateral, $\sum_{n\geq 0}$, incorpora las condiciones iniciales a la propiedad de desplazamiento.

| $x[n]$ | $X(z)$ | ROC | Propiedad | |
|---|---|---|---|---|
| $\delta[n]$ | $1$ | todo $\mathbb{C}$ | Linealidad | $ax_1 + bx_2 \overset{\mathcal{Z}}{\longleftrightarrow} aX_1 + bX_2$ |
| $u[n]$ | $\dfrac{1}{1-z^{-1}}$ | $\lvert z\rvert > 1$ | Desplazamiento | $x[n-n_0] \overset{\mathcal{Z}}{\longleftrightarrow} z^{-n_0}X(z)$ |
| $a^n u[n]$ | $\dfrac{1}{1-az^{-1}}$ | $\lvert z\rvert > \lvert a\rvert$ | (unilateral) | $x[n-1] \overset{\mathcal{Z}}{\longleftrightarrow} z^{-1}X(z) + x[-1]$ |
| $n\,a^n u[n]$ | $\dfrac{az^{-1}}{(1-az^{-1})^2}$ | $\lvert z\rvert > \lvert a\rvert$ | (unilateral) | $x[n+1] \overset{\mathcal{Z}}{\longleftrightarrow} zX(z) - z\,x[0]$ |
| $\cos(\Omega_0 n)u[n]$ | $\dfrac{1-\cos\Omega_0\, z^{-1}}{1-2\cos\Omega_0\, z^{-1}+z^{-2}}$ | $\lvert z\rvert > 1$ | Escalado en $z$ | $a^n x[n] \overset{\mathcal{Z}}{\longleftrightarrow} X(z/a)$ |
| $\sin(\Omega_0 n)u[n]$ | $\dfrac{\sin\Omega_0\, z^{-1}}{1-2\cos\Omega_0\, z^{-1}+z^{-2}}$ | $\lvert z\rvert > 1$ | Convolución | $x * h \overset{\mathcal{Z}}{\longleftrightarrow} X(z)H(z)$ |

### 1.6.9 Relación entre las transformadas

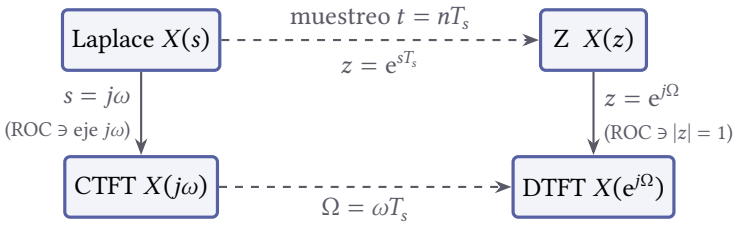



**Figura 1.8.** Relación entre las transformadas continuas y discretas.

Laplace y Z se usan para resolver ecuaciones diferenciales y en diferencias y para estudiar la función de transferencia, los polos, los ceros y la estabilidad de los sistemas LTI. Fourier se usa para describir el contenido en frecuencia de las señales y la respuesta en frecuencia de los sistemas estables. La relación $z = e^{sT_s}$ entre los planos $s$ y $z$, que aparece al muestrear, transforma el semiplano izquierdo en el interior de la circunferencia unidad y el eje $j\omega$ en la propia circunferencia; se estudia en el Tema 2 y se usa en el diseño de filtros IIR.

Ejercicios 7 y 8. *Herramientas interactivas:* `T1-serie-fourier`, `T1-propiedades-fourier` y `T1-region-convergencia`.

## 1.7 Propiedades de los sistemas

### 1.7.1 Clasificación general

Según el número de entradas y salidas, un sistema es SISO (una entrada y una salida), MIMO (varias de cada) o de alguno de los tipos intermedios SIMO y MISO. Un sistema sin entradas que evoluciona desde sus condiciones iniciales es *autónomo*. Según el tipo de señal, es continuo, discreto o híbrido (los conversores A/D y D/A). Y es determinista o estocástico según intervengan o no procesos aleatorios en la relación entre la entrada y la salida.

### 1.7.2 Memoria, causalidad, estabilidad e invertibilidad

**Definición 1.7.1: Memoria**

Un sistema es *sin memoria* (estático) si la salida en cada instante depende solo de la entrada en ese mismo instante. En otro caso es *con memoria* (dinámico).

Una resistencia, $v(t) = R\,i(t)$, o el sistema $y[n] = x^2[n]$ no tienen memoria. Un condensador, $v(t) = \frac{1}{C}\int_{-\infty}^{t} i(\tau)\,\mathrm{d}\tau$, o el sistema $y[n] = x[n] + x[n-1]$ sí la tienen. Los sistemas continuos con memoria se describen con ecuaciones diferenciales y los discretos, con ecuaciones en diferencias.

**Definición 1.7.2: Causalidad**

Un sistema es *causal* si la salida en cada instante $t_0$ solo depende de la entrada en instantes $t \le t_0$.

Un sistema no causal usa valores futuros de la entrada y no se puede implementar en tiempo real, pero sí sobre datos grabados: el procesado de audio en posproducción o el de imágenes, donde la variable no es el tiempo, usan a menudo sistemas no causales. La media $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$ es causal; la media centrada $y[n] = \frac{1}{3}(x[n+1] + x[n] + x[n-1])$ no lo es, aunque basta retrasar la salida una muestra para hacerla causal. Ese retardo es el precio de «usar el futuro».

**Definición 1.7.3: Estabilidad BIBO**

Un sistema es estable en sentido BIBO (*bounded input, bounded output*) si toda entrada acotada produce una salida acotada: $|x| \le B_x < \infty$ implica $|y| \le B_y < \infty$.

El integrador $y(t) = \int_{-\infty}^{t} x(\tau)\,\mathrm{d}\tau$ no es estable: la entrada $x = u(t)$ está acotada, pero la salida es la rampa $r(t)$, que no lo está. Para demostrar que un sistema es inestable basta una entrada acotada con salida no acotada; para demostrar que es estable hay que acotar la salida para *cualquier* entrada acotada.

**Definición 1.7.4: Invertibilidad**

Un sistema es *invertible* si entradas distintas producen salidas distintas. Existe entonces un sistema inverso $\mathcal{T}^{-1}$ con $\mathcal{T}^{-1}\{\mathcal{T}\{x\}\} = x$.

$y = 2x$ es invertible, con inverso $x = y/2$; $y = x^2$ no lo es, porque $x$ y $-x$ producen la misma salida. El acumulador y la diferencia son inversos uno del otro: $\sum_{k\le n}(x[k] - x[k-1]) = x[n]$ si $x[n] \to 0$ cuando $n \to -\infty$.

### 1.7.3 Linealidad e invarianza temporal

**Definición 1.7.5: Linealidad**

Un sistema es lineal si cumple el principio de superposición:

$$\mathcal{T}\{\alpha x_1 + \beta x_2\} = \alpha\,\mathcal{T}\{x_1\} + \beta\,\mathcal{T}\{x_2\} \qquad \forall x_1, x_2,\ \forall \alpha, \beta.$$

Equivale a la aditividad ($\mathcal{T}\{x_1 + x_2\} = \mathcal{T}\{x_1\} + \mathcal{T}\{x_2\}$) más la homogeneidad ($\mathcal{T}\{\alpha x\} = \alpha\mathcal{T}\{x\}$).

Tomando $\alpha = 0$ se ve que un sistema lineal responde a la entrada nula con salida nula. Por eso $y = x + 1$ no es lineal, aunque su gráfica sea una recta (se dice que es *afín* o *incrementalmente lineal*). Los sistemas reales son no lineales (saturaciones, zonas muertas, histéresis), pero casi siempre se pueden linealizar en torno a un punto de funcionamiento para pequeñas variaciones.

**Definición 1.7.6: Invarianza temporal**

Un sistema es invariante en el tiempo si un desplazamiento de la entrada produce el mismo desplazamiento de la salida: si $\mathcal{T}\{x(t)\} = y(t)$, entonces $\mathcal{T}\{x(t-t_0)\} = y(t-t_0)$ para todo $t_0$. En discreto, igual con $n_0$ entero.

Para comprobarlo se calcula la respuesta $y_d$ a la entrada desplazada $x_d[n] = x[n-n_0]$ y se compara con la salida desplazada $y[n-n_0]$. Un sistema que es a la vez lineal e invariante en el tiempo se llama **LTI**.

**Ejemplo 1.7.7: Cómo se comprueban las propiedades**

- $y[n] = n\,x[n]$. Es lineal: $n(\alpha x_1 + \beta x_2) = \alpha n x_1 + \beta n x_2$. No es invariante: la respuesta a $x[n-n_0]$ es $n\,x[n-n_0]$, pero $y[n-n_0] = (n-n_0)\,x[n-n_0]$. No es estable: con $x[n] = 1$ la salida $y[n] = n$ no está acotada. No es invertible, porque $x[0]$ se pierde.
- $y(t) = x(2t)$. Es lineal. No es causal: $y(1) = x(2)$ depende de una entrada futura. No es invariante: la respuesta a $x(t-t_0)$ es $x(2t-t_0)$, mientras que $y(t-t_0) = x(2t-2t_0)$.
- $y[n] = x[n] - x[n-1]$. Es LTI, causal y estable, pero no invertible: cualquier constante produce salida nula.

| Sistema | Memoria | Causal | Estable | Lineal | Invariante | Invertible |
|---|---|---|---|---|---|---|
| $y(t) = x^2(t)$ | no | sí | sí | no | sí | no |
| $y(t) = x(t) + 1$ | no | sí | sí | no | sí | sí |
| $y[n] = n\,x[n]$ | no | sí | no | sí | no | no |
| $y(t) = x(2t)$ | sí | no | sí | sí | no | sí |
| $y[n] = x[n] - x[n-1]$ | sí | sí | sí | sí | sí | no |
| $y(t) = \int_{-\infty}^{t} x(\tau)\,d\tau$ | sí | sí | no | sí | sí | sí |
| $y[n] = x[-n]$ | sí | no | sí | sí | no | sí |

**En código**

Probar con señales aleatorias no demuestra que un sistema sea lineal, pero un solo contraejemplo basta para descartarlo. Cambia la función `T` para estudiar otros sistemas.

**Python**

```python
import numpy as np

def T(x):                    # sistema a estudiar
    return x**2

rng = np.random.default_rng(0)
x1, x2 = rng.standard_normal((2, 50))
a, b = 2.0, -3.0
izq = T(a*x1 + b*x2)
der = a*T(x1) + b*T(x2)
print("¿Lineal?", np.allclose(izq, der))
```

**Matlab**

```matlab
T = @(x) x.^2;               % sistema a estudiar

x1 = randn(1, 50); x2 = randn(1, 50);
a = 2; b = -3;
izq = T(a*x1 + b*x2);
der = a*T(x1) + b*T(x2);
fprintf('¿Lineal? %d\n', ...
        max(abs(izq - der)) < 1e-10)
```

### 1.7.4 Interconexión de sistemas

Los sistemas se combinan en serie (cascada), en paralelo o con realimentación. Para sistemas LTI, con $H$ la transformada de Laplace, Z o Fourier de la respuesta al impulso:

- en serie, $h = h_1 * h_2$ y $H = H_1 H_2$; por la propiedad conmutativa, el orden no importa;
- en paralelo, $h = h_1 + h_2$ y $H = H_1 + H_2$;
- con realimentación negativa, $Y = G(X - KY)$, de donde $H = \dfrac{G}{1 + GK}$.

La conmutatividad de la cascada solo vale para sistemas LTI: por ejemplo, elevar al cuadrado y después retrasar no es lo mismo que retrasar y después multiplicar por $n$.

Ejercicio 10. *Herramienta interactiva:* `T1-propiedades-sistemas`.

## 1.8 Sistemas LTI

### 1.8.1 Respuesta al impulso y convolución

**Definición 1.8.1: Respuesta al impulso**

La respuesta al impulso de un sistema es su salida cuando la entrada es un impulso: $h(t) = \mathcal{T}\{\delta(t)\}$, $h[n] = \mathcal{T}\{\delta[n]\}$.

Para un sistema LTI discreto, partimos de la descomposición en impulsos (1.1) y aplicamos primero la linealidad y después la invarianza:

$$y[n] = \mathcal{T}\Big\{\sum_k x[k]\,\delta[n-k]\Big\} \overset{\text{lineal}}{=} \sum_k x[k]\,\mathcal{T}\{\delta[n-k]\} \overset{\text{invariante}}{=} \sum_k x[k]\,h[n-k] = (x * h)[n].$$

En continuo se llega a lo mismo a partir de $x(t) = \int x(\tau)\,\delta(t-\tau)\,\mathrm{d}\tau$: $y(t) = (x * h)(t)$.

**Consecuencia**

Un sistema LTI queda *completamente caracterizado* por su respuesta al impulso: conocida $h$, la respuesta a cualquier entrada es $y = x * h$.

### 1.8.2 Propiedades del sistema a partir de $h$

Como $h$ caracteriza el sistema, todas sus propiedades se pueden leer en ella:

| Propiedad | Continuo | Discreto |
|---|---|---|
| Sin memoria | $h(t) = K\,\delta(t)$ | $h[n] = K\,\delta[n]$ |
| Causal | $h(t) = 0 \ \ \forall t < 0$ | $h[n] = 0 \ \ \forall n < 0$ |
| Estable (BIBO) | $\int_{-\infty}^{\infty} \lvert h(t)\rvert\,\mathrm{d}t < \infty$ | $\sum_{n=-\infty}^{\infty} \lvert h[n]\rvert < \infty$ |
| Invertible | existe $h_{\text{inv}}$ con $h * h_{\text{inv}} = \delta$ | ídem |

La condición de causalidad se entiende mirando la convolución: $y[n] = \sum_k h[k]\,x[n-k]$ solo usa valores pasados y presentes de $x$ si $h[k] = 0$ para $k < 0$. La de estabilidad es suficiente porque, si $|x| \le B$,

$$|y[n]| \le \sum_k |h[k]|\,|x[n-k]| \le B \sum_k |h[k]|,$$

y es necesaria porque, si $\sum|h[k]| = \infty$, la entrada acotada $x[n] = \operatorname{sgn}(h[-n])$ produce $y[0] = \sum|h[k]| = \infty$.

La respuesta al escalón, $s(t) = \mathcal{T}\{u(t)\}$, también caracteriza el sistema: como $u$ es la integral de $\delta$, $s(t) = \int_{-\infty}^{t} h(\tau)\,d\tau$ y $h(t) = s'(t)$. En discreto, $s[n] = \sum_{k\le n} h[k]$ y $h[n] = s[n] - s[n-1]$. En la práctica es más fácil medir la respuesta al escalón que la respuesta al impulso.

### 1.8.3 Autofunciones y respuesta en frecuencia

Si la entrada a un sistema LTI continuo es $x(t) = e^{st}$, con $s$ complejo, usando la forma conmutada de la convolución:

$$y(t) = \int_{-\infty}^{\infty} h(\tau)\,e^{s(t-\tau)}\,d\tau = e^{st}\int_{-\infty}^{\infty} h(\tau)\,e^{-s\tau}\,d\tau = H(s)\,e^{st}.$$

La exponencial sale del sistema multiplicada por la constante $H(s)$, que es precisamente la transformada de Laplace de $h$: las exponenciales complejas son *autofunciones* de los sistemas LTI y $H(s)$ es el autovalor correspondiente. En discreto, con $x[n] = z^n$, se obtiene $y[n] = H(z)\,z^n$ con $H(z) = \sum_k h[k]\,z^{-k}$.

Con $s = j\omega$ o $z = e^{j\Omega}$ se obtiene la *respuesta en frecuencia* del sistema, $H(j\omega)$ o $H(e^{j\Omega})$, que dice cómo trata el sistema cada frecuencia:

- $e^{j\omega_0 t} \to H(j\omega_0)\,e^{j\omega_0 t}$: la salida tiene la misma frecuencia, su amplitud se multiplica por $|H(j\omega_0)|$ y su fase se desplaza $\arg H(j\omega_0)$.
- Si $h$ es real, $H(-j\omega) = H^*(j\omega)$ y, sumando las respuestas a las dos exponenciales del coseno,

$$A\cos(\omega_0 t + \phi) \;\to\; A\,|H(j\omega_0)|\cos\big(\omega_0 t + \phi + \arg H(j\omega_0)\big).$$

- Para una señal periódica, cada armónico se multiplica por la respuesta en su frecuencia: $y(t) = \sum_k c_k\,H(jk\omega_0)\,e^{jk\omega_0 t}$.
- Para una señal cualquiera, por la propiedad de convolución, $Y(j\omega) = H(j\omega)\,X(j\omega)$ y $Y(e^{j\Omega}) = H(e^{j\Omega})\,X(e^{j\Omega})$.

Estas expresiones suponen que la entrada existe desde $t = -\infty$. Si la entrada empieza en $t = 0$ y el sistema es estable, dan la *respuesta en régimen permanente*, la que queda cuando se extingue el transitorio.

**Ejemplo 1.8.2: Respuesta de un sistema discreto a una sinusoide**

El sistema $y[n] = 0{,}8\,y[n-1] + 0{,}2\,x[n]$ tiene, como veremos en la apartado 1.9, $H(e^{j\Omega}) = \dfrac{0{,}2}{1 - 0{,}8e^{-j\Omega}}$. Para la entrada $x[n] = \cos(\pi n/2)$:

$$H(e^{j\pi/2}) = \frac{0{,}2}{1 + 0{,}8j}, \qquad |H| = \frac{0{,}2}{\sqrt{1{,}64}} \approx 0{,}156, \qquad \arg H = -\arctan 0{,}8 \approx -0{,}675 \text{ rad},$$

y la salida en régimen permanente es $y[n] \approx 0{,}156\cos(\pi n/2 - 0{,}675)$. En $\Omega = 0$ la ganancia es 1 y en $\Omega = \pi$ es $0{,}2/1{,}8 \approx 0{,}111$: el sistema es un paso bajo.

## 1.9 Sistemas LTI continuos y discretos

### 1.9.1 Sistemas continuos: ecuación diferencial y función de transferencia

Los sistemas LTI continuos de orden finito, como los circuitos con resistencias, condensadores y bobinas, se describen con una ecuación diferencial lineal de coeficientes constantes. Aplicando la transformada de Laplace con condiciones iniciales nulas, cada derivada se convierte en un producto por $s$:

$$\sum_{k=0}^{N} a_k \frac{\mathrm{d}^k y(t)}{\mathrm{d}t^k} = \sum_{k=0}^{M} b_k \frac{\mathrm{d}^k x(t)}{\mathrm{d}t^k} \quad \xrightarrow{\mathscr{L}} \quad H(s) = \frac{Y(s)}{X(s)} = \frac{\sum_{k=0}^{M} b_k\, s^k}{\sum_{k=0}^{N} a_k\, s^k}.$$

$H(s) = \mathscr{L}\{h(t)\}$ es la *función de transferencia*, un cociente de polinomios. Las raíces del numerador son los *ceros* y las del denominador, los *polos*. El sistema es dinámico si $N \geq 1$ y es realizable sin derivadores ideales si $M \leq N$ (función de transferencia propia).

**Estabilidad de un sistema continuo causal**

Un sistema causal con $H(s)$ racional es estable si y solo si todos sus polos están en el semiplano izquierdo: $\mathrm{Re}\{p_i\} < 0$. Equivale a que la ROC, que por ser causal está a la derecha del polo más a la derecha, contenga el eje $j\omega$.

Para entender por qué, descomponemos $H(s)$ en fracciones simples. Cada polo simple $p$ aporta a $h(t)$ un término, o *modo*, proporcional a $\mathrm{e}^{pt}u(t)$, y un par de polos complejos conjugados $\sigma \pm j\omega_0$ aporta $\mathrm{e}^{\sigma t}\cos(\omega_0 t + \phi)u(t)$. Si $\mathrm{Re}\{p\} < 0$ el modo decae, y tanto más rápido cuanto más a la izquierda esté el polo; si $\mathrm{Re}\{p\} > 0$ crece. Si $\mathrm{Re}\{p\} = 0$ el modo es una oscilación mantenida: el sistema es marginalmente estable, pero *no* es BIBO estable, porque una entrada sinusoidal de esa misma frecuencia produce una salida que crece sin límite (resonancia).

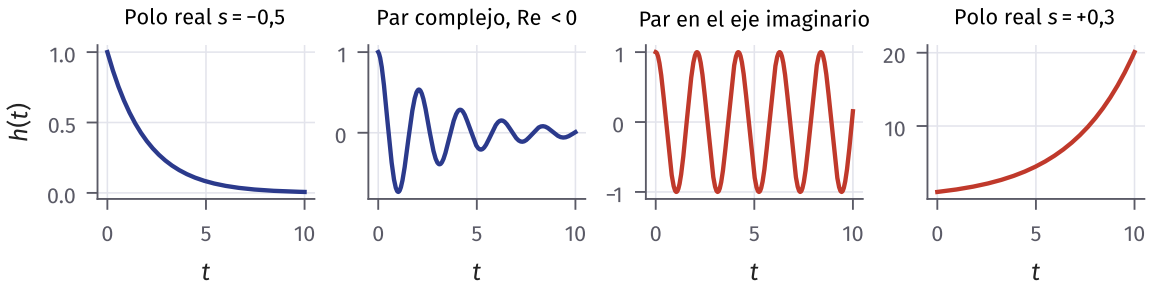



**Figura 1.9.** Posición de los polos en el plano $s$ y forma del modo correspondiente.

**Ejemplo 1.9.1: Circuito RC**

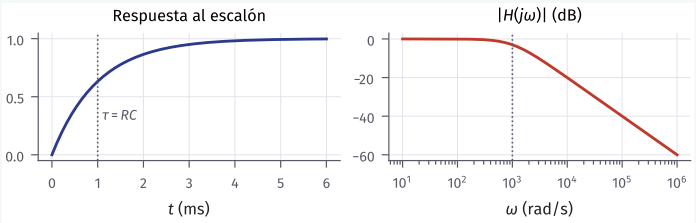
En un circuito RC paso bajo, con la entrada $x$ aplicada a la serie y la salida $y$ tomada en el condensador, $RC\,\frac{\mathrm{d}y}{\mathrm{d}t} + y = x$. Transformando,

$$H(s) = \frac{1}{1 + RCs},$$

con un único polo en $s = -1/RC$: el sistema es causal y estable. Su respuesta al impulso es $h(t) = \frac{1}{RC}\mathrm{e}^{-t/RC}u(t)$ y su respuesta al escalón, $s(t) = \left(1 - \mathrm{e}^{-t/RC}\right)u(t)$, que alcanza el 63 % del valor final en $t = RC$. La respuesta en frecuencia,

$$|H(j\omega)| = \frac{1}{\sqrt{1 + (\omega RC)^2}},$$

es la de un paso bajo con frecuencia de corte $\omega_c = 1/RC$, donde la ganancia cae 3 dB. Con $R = 1\,\text{k}\Omega$ y $C = 1\,\mu\text{F}$, $RC = 1\,\text{ms}$, el polo está en $-1000$ y la frecuencia de corte es $1000\,\text{rad/s}$, unos $159\,\text{Hz}$.




**En código**

**Python**

```python
from scipy import signal

RC = 1e-3                          # 1 Ωk, 1 μF
H = signal.TransferFunction([1], [RC, 1])
print("polos:", H.poles)           # [-1000.]
t, s = signal.step(H)              # escalón
W, mag, fase = signal.bode(H)      # dB, grados
```

**Matlab**

```matlab
RC = 1e-3;                         % 1 Ωk, 1 μF
H = tf(1, [RC 1]);                 % 1/(RC s + 1)
disp(pole(H))                      % -1000
[s, t] = step(H);                  % escalón
[mag, fase, W] = bode(H);
```

### 1.9.2 Sistemas discretos: ecuación en diferencias y función de transferencia

Los sistemas LTI discretos de orden finito se describen con una ecuación en diferencias. La forma estándar, que es la que usan `filter` en Matlab y `scipy.signal.lfilter` en Python, normaliza $a_0 = 1$:

$$\sum_{k=0}^{N} a_k\, y[n-k] = \sum_{k=0}^{M} b_k\, x[n-k] \quad \xrightarrow{\mathcal{Z}} \quad H(z) = \frac{\sum_{k=0}^{M} b_k\, z^{-k}}{1 + \sum_{k=1}^{N} a_k\, z^{-k}}.$$

A diferencia de la ecuación diferencial, la ecuación en diferencias es directamente un algoritmo. Despejando $y[n]$,

$$y[n] = \sum_{k=0}^{M} b_k\, x[n-k] - \sum_{k=1}^{N} a_k\, y[n-k],$$

cada muestra de la salida se calcula a partir de las entradas actual y anteriores y de las salidas anteriores: es un cálculo *recursivo*. Si la ecuación se escribe con adelantos, $\sum a_k\, y[n+k] = \sum b_k\, x[n+k]$, se llega a la misma $H(z)$ en potencias positivas de $z$; basta multiplicar numerador y denominador por $z^{-N}$ para pasar de una forma a otra.

**Estabilidad de un sistema discreto causal**

Un sistema causal con $H(z)$ racional es estable si y solo si todos sus polos están dentro de la circunferencia unidad: $|p_i| < 1$. Equivale a que la ROC, $|z| > \max|p_i|$, contenga la circunferencia unidad.

El razonamiento es el mismo que en continuo: cada polo $p$ aporta a $h[n]$ un modo $p^n u[n]$, que decae si $|p| < 1$, se mantiene si $|p| = 1$ y crece si $|p| > 1$. Un par $re^{\pm j\Omega_0}$ aporta $r^n \cos(\Omega_0 n + \phi)$, y un polo real negativo da una respuesta que alterna de signo.

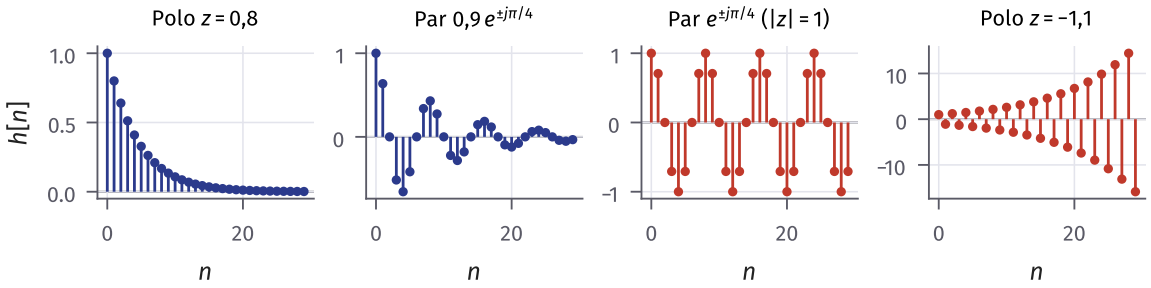



**Figura 1.10.** Posición de los polos en el plano $z$ y forma del modo correspondiente.

**Ejemplo 1.9.2: Suavizado exponencial**

El sistema $y[n] = 0{,}8\,y[n-1] + 0{,}2\,x[n]$ tiene, en forma estándar, $b = [0{,}2]$ y $a = [1, -0{,}8]$, y por tanto

$$H(z) = \frac{0{,}2}{1 - 0{,}8z^{-1}}, \qquad h[n] = 0{,}2\,(0{,}8)^n u[n].$$

Su único polo, $z = 0{,}8$, está dentro de la circunferencia unidad: es estable. Lo confirma la condición sobre $h$: $\sum_n |h[n]| = 0{,}2/(1 - 0{,}8) = 1$. La ganancia en continua es $H(1) = 1$, de modo que una entrada constante sale sin cambios, y la respuesta en frecuencia es la del ejemplo 1.8.2.

**En código**

Convenio de `filter`/`lfilter`: `b` son los coeficientes de $x[n-k]$ y `a` los de $y[n-k]$, con `a[0]=1`.

**Python**

```python
import numpy as np
from scipy import signal

# y[n] = 0.8 y[n-1] + 0.2 x[n]
b, a = [0.2], [1, -0.8]
d = np.zeros(30); d[0] = 1      # δ[n]
h = signal.lfilter(b, a, d)     # 0.2·0.8^n
polos = np.roots(a)             # [0.8]
print("¿Estable?", np.all(np.abs(polos) < 1))
w, H = signal.freqz(b, a)       # [0, π)
```

**Matlab**

```matlab
% y[n] = 0.8 y[n-1] + 0.2 x[n]
b = 0.2; a = [1 -0.8];
d = [1 zeros(1, 29)];           % δ[n]
h = filter(b, a, d);            % 0.2·0.8^n
polos = roots(a);               % 0.8
fprintf('¿Estable? %d\n', all(abs(polos) < 1))

[H, w] = freqz(b, a);           % [0, π)
```

### 1.9.3 Sistemas FIR e IIR

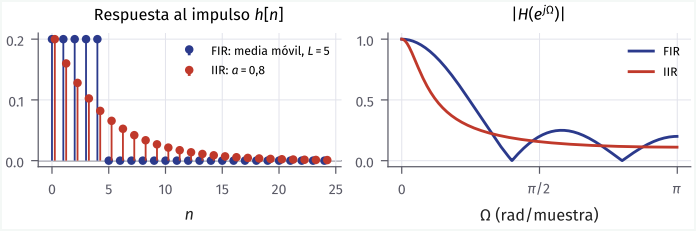
Si todos los $a_k$ con $k \geq 1$ son nulos, el sistema no es recursivo:

$$y[n] = \sum_{k=0}^{M} b_k\,x[n-k], \qquad h[n] = \begin{cases} b_n & 0 \leq n \leq M \\ 0 & \text{en otro caso.} \end{cases}$$

Su respuesta al impulso tiene $M + 1$ muestras: es un sistema de *respuesta al impulso finita* (FIR). Todos sus polos están en $z = 0$ y, como $\sum|h[n]|$ es una suma finita, es *siempre estable*. Además, si sus coeficientes son simétricos, tiene fase lineal (Tema 4). Si algún $a_k$ con $k \geq 1$ es distinto de cero, el sistema es recursivo y, en general, su respuesta al impulso tiene duración infinita: es un sistema IIR. Un IIR puede ser inestable, por lo que hay que comprobar sus polos, pero a cambio consigue la misma selectividad que un FIR con muchos menos coeficientes.

**Ejemplo 1.9.3: Media móvil y suavizado exponencial**

La media móvil de cinco muestras, $y[n] = \frac{1}{5}\sum_{k=0}^{4} x[n-k]$, es un FIR con $h[n] = \frac{1}{5}$ para $0 \le n \le 4$. El suavizado exponencial del ejemplo 1.9.2 es un IIR con un solo coeficiente de realimentación. Los dos suavizan la señal, pero el IIR «recuerda» todo el pasado con un peso que decae exponencialmente.




**Diagramas de bloques.** Con sumadores, multiplicadores por constantes y retardos de una muestra ($z^{-1}$) se implementa cualquier ecuación en diferencias, y eso es lo que ejecuta un DSP o un microcontrolador. Un FIR es una línea de retardos cuyas salidas se ponderan con los $b_k$ y se suman; un IIR añade un lazo que realimenta las salidas anteriores ponderadas con $-a_k$ (figura 1.11).

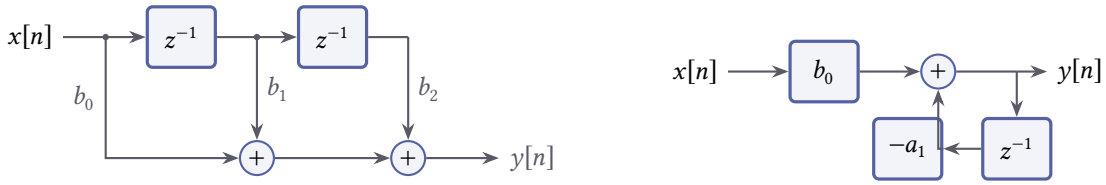



**Figura 1.11.** Diagramas de bloques de un FIR de orden 2, $y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2]$, y de un IIR de primer orden, $y[n] = b_0x[n] - a_1y[n-1]$.

**¿Hay sistemas FIR en tiempo continuo?** Un sistema continuo descrito por una ecuación diferencial de orden finito tiene $H(s)$ racional y su $h(t)$ es una suma de modos $e^{p_i t}u(t)$, de duración infinita (más, si $M \ge N$, términos con $\delta$ y sus derivadas). Existen sistemas continuos con respuesta al impulso de duración finita, como el integrador de ventana

$$y(t) = \int_{t-T}^{t} x(\tau)\,d\tau, \qquad h(t) = u(t) - u(t-T), \qquad H(s) = \frac{1 - e^{-sT}}{s},$$

pero su función de transferencia no es racional y no se puede construir con un número finito de resistencias, condensadores y bobinas; solo se aproxima. En discreto, en cambio, un FIR es trivial: basta una línea de retardos. Es una de las ventajas del procesado digital, porque los FIR, siempre estables y con fase lineal posible, solo son prácticos en tiempo discreto.

Ejercicios 11–16. *Herramientas interactivas:* `T1-respuesta-lti-continua` y `T1-respuesta-lti-discreta`.

## 1.10 Contenido en frecuencia de señales reales

Cada tipo de señal ocupa una banda de frecuencias característica, y cada fuente de ruido o interferencia también. Conocer las dos bandas decide casi todo el diseño de un sistema de procesado: la frecuencia de muestreo mínima ($f_s > 2f_{\max}$, Tema 2), el filtro que separa la señal del ruido (Tema 4) y la resolución

en frecuencia que hace falta para analizarla (Tema 3). La figura 1.12 resume, en escala logarítmica, las bandas de las aplicaciones que se comentan a continuación, desde las centésimas de hercio de la sismología hasta los cientos de terahercios de la fibra óptica.

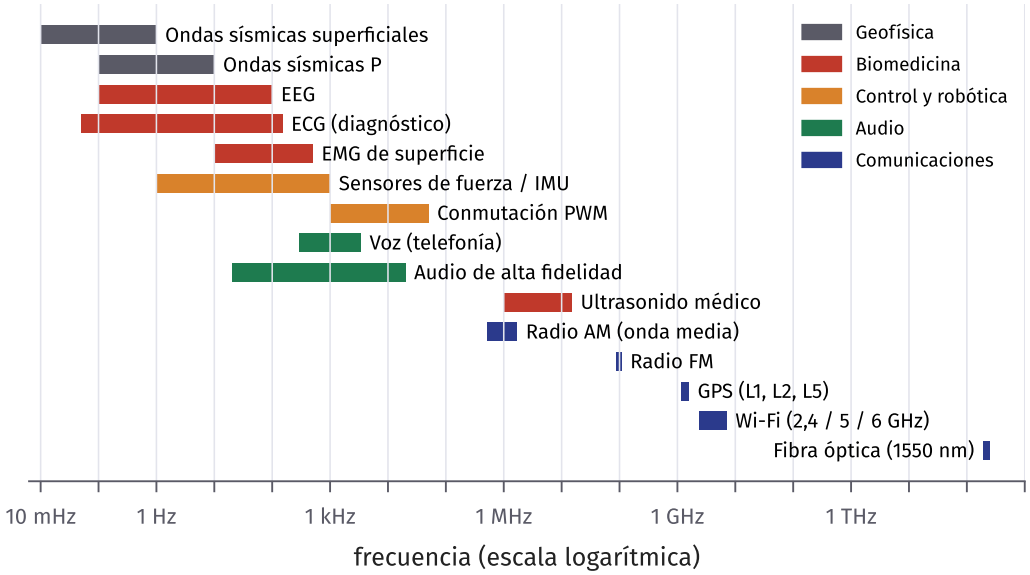



**Figura 1.12.** Bandas de frecuencia de algunas aplicaciones (escala logarítmica).

**Comunicaciones.** La voz en telefonía ocupa de 300 Hz a 3,4 kHz; la radio AM de onda media, de 526,5 kHz a 1606,5 kHz; la FM, de 87,5 MHz a 108 MHz; el GPS usa la banda L1 (1575,42 MHz), además de L2 y L5; el Wi-Fi, las bandas de 2,4, 5 y 6 GHz; y la fibra óptica, luz de 1550 nm, unos 193 THz. El ruido térmico es blanco: se reparte por igual en toda la banda y su potencia crece con el ancho de banda del receptor ($kTB$). Hay además interferencias electromagnéticas de otros equipos y desvanecimientos por multitrayecto, variaciones lentas de la amplitud que van de hercios a kilohercios según la velocidad del receptor. Las señales de radio se procesan casi siempre en banda base, después de desplazarlas en frecuencia con la propiedad de modulación de la transformada de Fourier.

**Audio y música.** La banda audible va de 20 Hz a 20 kHz y se suele dividir en subgraves (20–60 Hz), graves (60–250 Hz), medios (250 Hz–2 kHz) y agudos (2–20 kHz). Las frecuencias de muestreo habituales son 8 kHz en telefonía, 44,1 kHz en el CD, 48 kHz en vídeo y 96 o 192 kHz en estudio. El ruido de cuantificación es aproximadamente blanco, repartido en toda la banda $[0, f_s/2]$, con potencia $\Delta^2/12$ (Tema 2). La red eléctrica induce un zumbido de 50 Hz en Europa (60 Hz en América) y sus armónicos; el ruido ambiental es de banda ancha y los clics y chasquidos son impulsivos. El proyecto «Audio con ruido» trabaja con una grabación que contiene los cuatro tipos de ruido: el zumbido se elimina con filtros de muesca o en peine, el soplido de banda ancha con un paso bajo o un filtro de Wiener, los clics con predicción lineal y un silbido de frecuencia variable con la STFT.

**Señales biomédicas.** El ECG ocupa de 0,05 Hz a 150 Hz en diagnóstico (de 0,5 Hz a 40 Hz en monitorización); el EEG, de 0,5 Hz a 100 Hz, con los ritmos delta (0,5–4 Hz), theta (4–8 Hz), alfa (8–13 Hz), beta (13–30 Hz) y gamma (más de 30 Hz); el EMG de superficie, de 10 Hz a 500 Hz; el fonocardiograma, de 20 Hz a 1 kHz, aproximadamente; y la ecografía usa ultrasonidos de 2 a 15 MHz. Las perturbaciones típicas son la red eléctrica (50/60 Hz), la deriva de la línea base por la respiración (por debajo de 0,5 Hz), los artefactos de movimiento (por debajo de 10 Hz) y la actividad muscular superpuesta al ECG (por

encima de 20 Hz). Como se ve, las bandas de la señal y del ruido se solapan, y eso es lo que hace difícil el problema. El proyecto «ECG» parte de registros de holter muestreados a 360 Hz con deriva, red, ruido muscular y artefactos: hay que limpiarlos sin deformar el segmento ST, detectar los latidos con el algoritmo de Pan-Tompkins y analizar la variabilidad del ritmo.

**Control y robótica.** Las señales de posición, velocidad, fuerza o de una IMU tienen su dinámica útil por debajo de 0,1–1 kHz. Una cámara a 30–60 fotogramas por segundo solo puede representar variaciones de hasta 15–30 Hz, la mitad de su frecuencia de muestreo. En un LiDAR la luz es óptica (905 o 1550 nm), pero lo que marca el muestreo de la escena es la *tasa de barrido*, de 10 Hz a 1 kHz. La conmutación PWM de los motores está entre 1 y 50 kHz, y los buses de comunicación van de 125 kbit/s a 1 Mbit/s en CAN o a 100 Mbit/s en EtherCAT. El ruido incluye la deriva térmica y el ruido $1/f$ de los sensores (de baja frecuencia), el ruido térmico y el de cuantificación (blancos) y las vibraciones y resonancias mecánicas (de 10 Hz a 10 kHz). El proyecto «Registro de carrera» usa sesiones reales con velocidad GPS, frecuencia cardiaca y altitud, muestreadas de forma irregular: hay que remuestrearlas, filtrarlas, modelar el ruido y detectar los cambios de ritmo.

**Geofísica y sismología.** Las ondas sísmicas P tienen frecuencias de 0,1 a 10 Hz y las S, de 0,1 a 3 Hz; las ondas superficiales (Rayleigh y Love), que son las que más dañan las estructuras, de 0,01 a 1 Hz. Los microtremores están entre 0,1 y 1 Hz y las vibraciones de estructuras entre 0,1 y 100 Hz, donde interesan sobre todo sus resonancias. El ruido procede de la actividad humana (tráfico, maquinaria: de 1 a 10 Hz y más), de los microsismos del oleaje (0,1–0,3 Hz) y del viento y la presión atmosférica (por debajo de 1 Hz). Con señales de tan baja frecuencia, lo que limita la resolución espectral no es la frecuencia de muestreo sino la duración del registro: $\Delta f \approx 1/T_{\text{obs}}$ (Tema 3).

**Imagen.** En una imagen la variable independiente es espacial y la frecuencia se mide en ciclos por píxel o por milímetro. Las bajas frecuencias corresponden a las zonas uniformes y las altas, a los bordes, las texturas y el ruido. Los ruidos típicos son el gaussiano del sensor (de banda ancha), el de sal y pimienta (impulsivo) y los patrones periódicos, que aparecen como picos aislados en el espectro bidimensional. Los proyectos «Imagen con ruido» y «Códigos de barras» restauran imágenes con estos defectos y con desenfoque, y leen códigos EAN-13 tratando cada fila de la imagen como una señal unidimensional muestreada.

**En resumen**

El ruido térmico y el de cuantificación son de banda ancha; la interferencia de la red eléctrica es de banda estrecha (50/60 Hz y armónicos); las derivas son de muy baja frecuencia. La banda de la señal fija la frecuencia de muestreo y el tipo de filtro. Los proyectos de evaluación del curso trabajan sobre estas situaciones.

## Resumen del tema

- Señal y sistema; notación $x(t)$ frente a $x[n]$, $\omega$ frente a $\Omega$.
- Clasificación de señales: analógica, muestreada, digital; determinista o aleatoria; periódica (con su periodo fundamental); par o impar; causal; de energía o de potencia; real, compleja o hermítica.
- Señales básicas: $\delta$, $u$, $r$, $\Pi$, $\Lambda$, sinc, sgn y exponenciales. La exponencial discreta $e^{j\Omega_0 n}$ solo es periódica si $\Omega_0/2\pi$ es racional, y $\Omega_0$ y $\Omega_0 + 2\pi$ dan la misma secuencia.

- Operaciones sobre la variable independiente (desplazar primero, escalar después) y sobre la amplitud; diferencia y acumulación como análogos de la derivada y la integral.
- Convolución: definición, cálculo gráfico, propiedades y longitud del resultado.
- Series de Fourier continua y discreta, CTFT y DTFT (periódica de periodo $2\pi$). Convolución en el tiempo $\leftrightarrow$ producto en frecuencia.
- Laplace y Z bilaterales, siempre con su región de convergencia.
- Propiedades de los sistemas: memoria, causalidad, estabilidad BIBO, invertibilidad, linealidad e invarianza temporal.
- Un sistema LTI queda caracterizado por $h$: $y = x * h$; causalidad y estabilidad se leen en $h$ y en los polos.
- Las exponenciales son autofunciones: $\mathrm{e}^{j\omega t} \to H(j\omega)\,\mathrm{e}^{j\omega t}$.
- Ecuaciones diferenciales y en diferencias, función de transferencia, polos, ceros y estabilidad; sistemas FIR e IIR.

## Bibliografía


- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulos 1 y 2. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.
- A. V. Oppenheim y A. S. Willsky, *Señales y sistemas*, 2.ª ed. Prentice Hall, 1997. Capítulos 1 a 5, 9 y 10. Es la referencia clásica para todo el tema.
- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos 1 a 4, para la parte discreta.
- V. K. Ingle y J. G. Proakis, *Digital Signal Processing Using MATLAB*, 3.ª ed. Cengage Learning, 2012. Capítulos 2 a 4, con ejemplos en Matlab.


## 1.11 Ejercicios

Los ejercicios 1–4 y 9 corresponden a la parte 1 del tema (tipos de señales y señales básicas), los ejercicios 5–8 a la parte 2 (operaciones y análisis en frecuencia) y los ejercicios 10–16 a la parte 3 (sistemas). En todos ellos $x(t)$ y $x[n]$ son la entrada, $u$ es siempre el escalón unitario y, salvo que se diga lo contrario, los sistemas definidos por $H(s)$ o $H(z)$ son causales. Muchos resultados se pueden comprobar numéricamente con los ejemplos de `teoría/T1/codigo/`.

### Señales

**Ejercicio 1** (clasificación) Clasifica las siguientes señales según su dimensión, su número de canales y si la variable independiente y la amplitud son continuas o discretas.

a) $x(t) = \begin{pmatrix} 4t \\ e^{j2t} \end{pmatrix}$

b) $x[n] = 4\,e^{j\frac{3}{5}n}$

c) $x[n] = 3\,e^{j\frac{2}{\pi}n}$

d) $x[n] = \left\lfloor \sin\left(\frac{2\pi n}{20}\right) \right\rfloor$

e) $x(t, r) = \sin(\omega t - \kappa r)$, donde $r$ es una posición en metros

**Solución**

a) Unidimensional (depende solo de $t$), vectorial de dos canales, con tiempo y amplitud continuos: analógica. El segundo canal es complejo.

b) Unidimensional, escalar y compleja; tiempo discreto y amplitud continua: es una secuencia muestreada (no cuantificada).

c) Igual que la anterior.

d) Unidimensional y escalar, con tiempo discreto y amplitud discreta: solo toma los valores $\{-1, 0, 1\}$ (vale 1 cuando el seno vale exactamente 1, en $n = 5 + 20l$). Es una señal digital.

e) Bidimensional (tiempo y espacio), escalar y real, con ambas variables y la amplitud continuas. Es una onda que viaja en el sentido de $r$ creciente a velocidad $\omega/\kappa$.

**Ejercicio 2** (periodicidad y paridad) Clasifica las siguientes señales según su periodicidad y su paridad. Para las periódicas, indica su periodo fundamental.

a) $x(t) = 4\,e^{j\frac{3}{5}t}$

b) $x(t) = 3t$

c) $x(t) = 3t^2$

d) $x[n] = \sin(3n)$

e) $x[n] = \cos\left(\frac{\pi n}{20}\right)$

f) $x[n] = \cos(3n - 4)\big(u[n] - u[n - N_0 - 1]\big)$, con $N_0 > 0$ entero

**Solución**

a) Periódica con $T_0 = 2\pi/(3/5) = 10\pi/3$. No es par ni impar, pero es hermítica: $x^*(-t) = x(t)$, con parte real $4\cos(3t/5)$ par e imaginaria $4\sin(3t/5)$ impar.

b) Aperiódica e impar.

c) Aperiódica y par.

d) $F_0 = 3/(2\pi)$ es irracional: *no* es periódica, aunque $\sin(3t)$ sí lo sea. Es impar.

e) $F_0 = 1/40$: periódica con $N_0 = 40$. Es par.

f) Tiene duración finita (solo es no nula en $0 \le n \le N_0$), así que no es periódica. No es par ni impar: $x[-1] = 0$ pero $x[1] = \cos(-1) \neq 0$, y $x[0] = \cos(-4) \neq 0$.

**Ejercicio 3** (energía y potencia) Calcula la energía y la potencia media de las siguientes señales e indica si son de energía, de potencia o de ninguno de los dos tipos.

a) $x(t) = t$

b) $x(t) = u(t)$

c) $x(t) = e^{-j4t}$

d) $x(t) = 4t$ para $0 \le t \le 10$ y 0 en otro caso

**Solución**

a) $E = \infty$ y $\frac{1}{2T}\int_{-T}^{T} t^2\,\mathrm{d}t = T^2/3 \to \infty$: $P = \infty$. No es de energía ni de potencia.

b) $E = \infty$ y $P = \lim \frac{1}{2T}\int_{0}^{T} \mathrm{d}t = \frac{1}{2}$: señal de potencia.

c) $|x(t)| = 1$, así que $E = \infty$ y $P = 1$: señal de potencia.

d) $E = \int_0^{10} 16t^2\,\mathrm{d}t = 16000/3 \approx 5333$ y $P = 0$: señal de energía.

**Ejercicio 4** (señales complejas) Indica si las siguientes señales son reales o complejas y, en este caso, si son hermíticas.

a) $x(t) = \mathrm{e}^{-3t}$

b) $x(t) = \mathrm{e}^{j2\pi t}$

c) $x[n] = \mathrm{e}^{j\pi n}$

d) $x(t) = \cos(4t) + j\sin(5t)$

e) $x(t) = \mathrm{e}^{j\pi}\,\mathrm{e}^{-j3t}$

**Solución**

a) Real.

b) Compleja y hermítica: $x^*(-t) = \left(\mathrm{e}^{-j2\pi t}\right)^* = \mathrm{e}^{j2\pi t} = x(t)$.

c) Real, aunque no lo parezca: $\mathrm{e}^{j\pi n} = \cos(\pi n) = (-1)^n$, porque $\sin(\pi n) = 0$ para todo entero $n$.

d) Compleja. Su parte real, $\cos(4t)$, es par y su parte imaginaria, $\sin(5t)$, es impar: es hermítica.

e) Compleja: $\mathrm{e}^{j\pi} = -1$ y $x(t) = -\mathrm{e}^{-j3t}$. Es hermítica: $x^*(-t) = -\left(\mathrm{e}^{j3t}\right)^* = -\mathrm{e}^{-j3t} = x(t)$.

## Operaciones y análisis en frecuencia

**Ejercicio 5** (operaciones) Sean $x_1(t) = \sin t$, $x_2(t) = \cos\left(\frac{2\pi}{5}t\right)$, $x_3[n] = \{\underline{0}, 3, 1, 1\}$ (el subrayado marca $n = 0$) y $x_4[n] = \mathrm{e}^{j4n}$. Obtén:

a) el producto $x_1(t)\,x_2(t)$ y la suma $x_1(t) + x_2(t)$; ¿es periódica la suma?

b) la convolución $(x_1 * x_2)(t)$;

c) $x_3[n]/x_4[n]$ y $x_3[n] - x_4[n]$;

d) la convolución $(x_3 * x_4)[n]$;

e) $x_3[n-2]$, $x_3[-n]$, el diezmado $x_3[2n]$ y la expansión por $L = 2$ de $x_3$.

**Solución**

a) $x_1x_2 = \sin t\cos\left(\frac{2\pi}{5}t\right) = \frac{1}{2}\left[\sin\left((1+\frac{2\pi}{5})t\right) + \sin\left((1-\frac{2\pi}{5})t\right)\right]$. La suma $\sin t + \cos(2\pi t/5)$ no es periódica: los periodos son $2\pi$ y 5, y su cociente $2\pi/5$ es irracional.

b) La integral $\int \sin\tau\cos\left(\frac{2\pi}{5}(t-\tau)\right)\mathrm{d}\tau$ no converge. Interpretada mediante la transformada de Fourier, el resultado es nulo: los espectros son deltas en $\omega = \pm 1$ y $\omega = \pm 2\pi/5$, que no coinciden, así que su producto es cero. Un sistema LTI con $h = x_2$ elimina la frecuencia $\omega = 1$.

c) $x_3[n]/x_4[n] = x_3[n]\,\mathrm{e}^{-j4n} = \{\underline{0}, 3\mathrm{e}^{-j4}, \mathrm{e}^{-j8}, \mathrm{e}^{-j12}\}$ y vale 0 fuera de $0 \le n \le 3$. $x_3[n] - x_4[n]$

vale $x_3[n] - e^{j4n}$ en $0 \le n \le 3$ y $-e^{j4n}$ en el resto.

d) $(x_3 * x_4)[n] = \sum_k x_3[k]\, e^{j4(n-k)} = e^{j4n} \sum_k x_3[k]\, e^{-j4k} = \left(3e^{-j4} + e^{-j8} + e^{-j12}\right) e^{j4n}$. La exponencial sale multiplicada por una constante: es una autofunción del sistema LTI con $h = x_3$, y la constante es $X_3(e^{j4})$.

e) $x_3[n-2] = \{\underline{0}, 0, 0, 3, 1, 1\}$ (no nula en $n = 3, 4, 5$). $x_3[-n] = \{1, 1, 3, \underline{0}\}$ (de $n = -3$ a $n = 0$). $x_3[2n] = \{\underline{0}, 1\}$: se conservan $x_3[0]$ y $x_3[2]$ y se pierden las demás. Expansión: $\{\underline{0}, 0, 3, 0, 1, 0, 1\}$.

**Ejercicio 6** (convolución) Calcula las siguientes convoluciones.

a) $x[n] = \{\underline{1}, 4, 2, 0, 1\}$ e $y[n] = \{\underline{4}, 2, 3\}$

b) $x[n] = 4\,\delta[n-6]$ e $y[n] = e^{j\frac{2\pi}{5}n}$

c) $x[n] = u[n]$ e $y[n] = a^n u[n]$

d) $x(t) = u(t) - u(t-2)$ e $y(t) = u(t) - u(t-1)$

**Solución**

a) $(x * y)[n] = \{\underline{4}, 18, 19, 16, 10, 2, 3\}$, con $5 + 3 - 1 = 7$ muestras desde $n = 0$. Se comprueba con `np.convolve` o `conv`.

b) Convolucionar con $\delta[n-6]$ desplaza: $4\, e^{j\frac{2\pi}{5}(n-6)} = 4\, e^{-j\frac{12\pi}{5}} e^{j\frac{2\pi}{5}n} = 4\, e^{-j\frac{2\pi}{5}} e^{j\frac{2\pi}{5}n}$.

c) $\sum_{k=0}^{n} a^k = \dfrac{1 - a^{n+1}}{1 - a}$ para $n \ge 0$ y $a \ne 1$, de modo que $(x * y)[n] = \dfrac{1 - a^{n+1}}{1 - a}\, u[n]$. Si $a = 1$ se obtiene $(n+1)\, u[n] = r[n+1]$.

d) Un trapecio: 0 para $t < 0$, $t$ en $[0, 1]$, 1 en $[1, 2]$, $3 - t$ en $[2, 3]$ y 0 para $t > 3$. La duración es la suma de las duraciones.

**Ejercicio 7** (serie de Fourier) Calcula la serie de Fourier de las siguientes señales de periodo $T_0$ y comprueba el teorema de Parseval.

a) La onda cuadrada par que vale 1 en $|t| < T_0/4$ y $-1$ en $T_0/4 < |t| < T_0/2$.

b) La onda triangular par $x(t) = 1 - 4|t|/T_0$ para $|t| \le T_0/2$.

¿Por qué los coeficientes de la segunda decaen más deprisa?

**Solución**

a) $c_0 = 0$ y $c_k = \mathrm{sinc}(k/2)$: nulos para $k$ par y $\pm 2/(\pi k)$ para $k$ impar. Por tanto $x(t) = \frac{4}{\pi}\left(\cos\omega_0 t - \frac{1}{3}\cos 3\omega_0 t + \frac{1}{5}\cos 5\omega_0 t - \cdots\right)$. Parseval: $P = 1 = 2\sum_{k\ \text{impar}} \frac{4}{\pi^2 k^2}$, porque $\sum_{k\ \text{impar}} 1/k^2 = \pi^2/8$.

b) $c_0 = 0$ y, por ser par, $c_k = \frac{2}{T_0}\int_0^{T_0/2}\left(1 - \frac{4t}{T_0}\right)\cos(k\omega_0 t)\,\mathrm{d}t = \dfrac{2\left(1 - (-1)^k\right)}{\pi^2 k^2}$: nulos para $k$ par y $4/(\pi^2 k^2)$ para $k$ impar. Así, $x(t) = \frac{8}{\pi^2}\left(\cos\omega_0 t + \frac{1}{9}\cos 3\omega_0 t + \frac{1}{25}\cos 5\omega_0 t + \cdots\right)$. Parseval: $P = \frac{1}{T_0}\int x^2\,\mathrm{d}t = \frac{1}{3}$ y $2\sum_{k\ \text{impar}} \frac{16}{\pi^4 k^4} = \frac{32}{\pi^4}\cdot\frac{\pi^4}{96} = \frac{1}{3}$.

La onda triangular es continua (su derivada es una onda cuadrada) y sus coeficientes decaen como $1/k^2$; la cuadrada tiene saltos y decae como $1/k$. Cuanto más suave es la señal, más rápido decae su espectro. De hecho, la derivada de la triangular es $-\frac{4}{T_0}$ por la cuadrada retrasada un cuarto de periodo, y los coeficientes $jk\omega_0 c_k$ de la derivada coinciden con los de esa señal.

**Ejercicio 8** (transformada de Fourier) Calcula la transformada de Fourier de las siguientes señales, en su

versión continua y discreta.

a) Impulso: $\delta(t-t_0)$ y $\delta[n-n_0]$.

b) Pulso rectangular, centrado y causal: $\Pi(t/T)$ y $\Pi\big((t-T/2)/T\big)$; en discreto, el pulso centrado de $2N+1$ muestras y el causal $u[n]-u[n-L]$.

c) Pulso triangular, centrado y causal: $\Lambda(t/T)$ (duración $2T$) y $\Lambda\big((t-T)/T\big)$; en discreto, $t_L[n]=L-|n|$ para $|n|<L$ y su versión causal $t_L[n-L+1]$. *Pista*: usa que el pulso triangular es la convolución de dos rectangulares.

**Solución**

a) $\mathrm{e}^{-j\omega t_0}$ y $\mathrm{e}^{-j\Omega n_0}$: módulo 1 en todas las frecuencias y fase lineal.

b) $\Pi(t/T) \overset{\mathscr{F}}{\longleftrightarrow} T\,\mathrm{sinc}\left(\frac{\omega T}{2\pi}\right)$, y el causal añade el factor $\mathrm{e}^{-j\omega T/2}$. En discreto, el centrado da $\sum_{n=-N}^{N}\mathrm{e}^{-j\Omega n}=\dfrac{\sin\big(\Omega(2N+1)/2\big)}{\sin(\Omega/2)}$, real y par; el causal, $\mathrm{e}^{-j\Omega\frac{L-1}{2}}\dfrac{\sin(\Omega L/2)}{\sin(\Omega/2)}$.

c) $\Lambda(t/T)=\frac{1}{T}\big(\Pi(\cdot/T)*\Pi(\cdot/T)\big)(t)$, así que $\Lambda(t/T)\overset{\mathscr{F}}{\longleftrightarrow} T\,\mathrm{sinc}^2\left(\frac{\omega T}{2\pi}\right)$, y el causal añade $\mathrm{e}^{-j\omega T}$. En discreto, $t_L$ es la convolución del pulso de $L$ muestras con su reflejado, y su transformada es $\left(\dfrac{\sin(\Omega L/2)}{\sin(\Omega/2)}\right)^2$; la versión causal añade $\mathrm{e}^{-j\Omega(L-1)}$.

En los dos casos la transformada discreta es periódica de periodo $2\pi$ y, cerca de $\Omega=0$, se parece a la continua.

**Ejercicio 9** (exponenciales discretas) Determina si las siguientes secuencias son periódicas y, si lo son, su periodo fundamental. Para cada sinusoide, indica su frecuencia equivalente en $[-\pi,\pi)$.

a) $x[n]=2\,\mathrm{e}^{j(2\pi/8)n}$

b) $x[n]=5\,\mathrm{e}^{j(3/4)n}$

c) $x[n]=\cos(2\pi n)$

d) $x[n]=1/n$ para $n\neq 0$, con $x[0]=0$

e) $x[n]=\cos(3\pi n/8)+\sin(\pi n/6)$

f) $x[n]=\cos(11\pi n/6)$

**Solución**

a) $F_0=1/8$: periódica con $N_0=8$; $\Omega_0=\pi/4$.

b) $F_0=3/(8\pi)$ es irracional: no es periódica; $\Omega_0=0{,}75$ ya está en $[-\pi,\pi)$.

c) $\cos(2\pi n)=1$ para todo $n$: es constante, periódica con $N_0=1$, y su frecuencia equivalente es $\Omega_0=0$.

d) Aperiódica (e impar).

e) El primer término tiene $F=3/16$ y periodo 16; el segundo, $F=1/12$ y periodo 12. La suma es periódica con $N_0=\mathrm{mcm}(16,12)=48$.

f) $11\pi/6-2\pi=-\pi/6$, así que $x[n]=\cos(-\pi n/6)=\cos(\pi n/6)$: periódica con $N_0=12$. Aunque $11\pi/6$ parezca una frecuencia alta, la secuencia oscila despacio.

## Sistemas

**Ejercicio 10** (propiedades de los sistemas) Estudia si los siguientes sistemas tienen memoria y si son causales, estables, lineales, invariantes en el tiempo e invertibles. Para los que sean LTI, da su respuesta al impulso.

a) $y(t) = x(t-2)$

b) $y[n] = x[n] + 2$

c) $y(t) = x(t)\cos(\omega_0 t)$

d) $y[n] = \sum_{k=n-2}^{n+2} x[k]$

e) $y[n] = x[2n]$

f) $y(t) = \mathrm{e}^{x(t)}$

g) $y[n] = x[n]\,x[n-1]$

**Solución**

| | Memoria | Causal | Estable | Lineal | Invariante | Invertible |
|---|---|---|---|---|---|---|
| a) | sí | sí | sí | sí | sí | sí |
| b) | no | sí | sí | no | sí | sí |
| c) | no | sí | sí | sí | no | no |
| d) | sí | no | sí | sí | sí | no |
| e) | sí | no | sí | sí | no | no |
| f) | no | sí | sí | no | sí | sí |
| g) | sí | sí | sí | no | sí | no |

a) Retardo puro: $h(t) = \delta(t-2)$; el inverso es el adelanto $x(t) = y(t+2)$.

b) No es lineal porque la entrada nula da $y = 2$. Es invertible: $x = y - 2$.

c) El modulador es lineal, pero no invariante: la respuesta a $x(t-t_0)$ es $x(t-t_0)\cos(\omega_0 t)$ y no $x(t-t_0)\cos(\omega_0(t-t_0))$. No es invertible porque en los instantes en que $\cos(\omega_0 t) = 0$ se pierde el valor de la entrada.

d) Media (sin normalizar) de cinco muestras centrada: usa $x[n+1]$ y $x[n+2]$, así que no es causal. Es LTI con $h[n] = u[n+2] - u[n-3]$, y estable porque $\sum|h| = 5$. No es invertible: la entrada $\cos(2\pi n/5)$ da salida nula, porque cinco muestras consecutivas de esa sinusoide suman cero.

e) El diezmado: $y[1] = x[2]$ usa una entrada futura. No es invariante: si la entrada se retrasa una muestra, la salida no se retrasa (se toman las muestras impares en lugar de las pares). Pierde la mitad de las muestras: no es invertible.

f) No es lineal ($\mathrm{e}^{2x} \neq 2\mathrm{e}^{x}$). Es estable: si $|x| \leq B$, $y \leq \mathrm{e}^{B}$. Es invertible: $x = \ln y$, porque $y > 0$.

g) No es lineal (al duplicar la entrada la salida se multiplica por cuatro) ni invertible ($x$ y $-x$ dan la misma salida). Es estable: $|y| \leq B^2$.

**Ejercicio 11** (estabilidad de sistemas continuos) Determina la estabilidad de los siguientes sistemas continuos causales.

a) $H(s) = \dfrac{1}{s}$

b) $H(s) = \dfrac{s+2}{s+4}$

c) $H(s) = \dfrac{1}{s-3}$

d) $H(s) = \dfrac{s+1}{s^2+25}$

e) $H(s) = \dfrac{s-1}{s^2+2s+5}$

**Solución**

a) Polo en $s = 0$: no es BIBO estable. Es el integrador: el escalón produce una rampa.

b) Polo en $s = -4$: estable.

c) Polo en $s = 3$: inestable, $h(t) = \mathrm{e}^{3t}u(t)$.

d) Polos en $s = \pm 5j$, sobre el eje imaginario: no es BIBO estable. $h(t)$ es una oscilación mantenida y una entrada de frecuencia 5 rad/s produce una salida que crece linealmente.

e) Polos en $s = -1 \pm 2j$: estable. El cero en $s = 1$, en el semiplano derecho, no afecta a la estabilidad.

**Ejercicio 12** (estabilidad de sistemas discretos) Determina la estabilidad de los siguientes sistemas discretos causales.

a) $H(z) = \dfrac{z}{z - 1{,}5}$

b) $H(z) = \dfrac{z}{(z-1)^2}$

c) $H(z) = \dfrac{z}{z - 0{,}9}$

d) $H(z) = \dfrac{z}{z^2 + 1}$

e) $H(z) = \dfrac{z - 2}{z^2 - z + 0{,}5}$

**Solución**

a) Polo en $z = 1{,}5$, fuera de la circunferencia unidad: inestable, $h[n] = 1{,}5^n u[n]$.

b) Polo doble en $z = 1$: inestable, $h[n] = n\,u[n]$.

c) Polo en $z = 0{,}9$: estable, $h[n] = 0{,}9^n u[n]$.

d) Polos en $z = \pm j$, sobre la circunferencia unidad: no es BIBO estable. $h[n] = \sin(\pi n/2)\,u[n]$.

e) Polos en $z = 0{,}5 \pm 0{,}5j$, con $|z| = 0{,}707$: estable. El cero en $z = 2$ no influye.

**Ejercicio 13** (respuesta acotada) Determina si la respuesta de cada sistema a la entrada indicada (aplicada desde $t = 0$ o $n = 0$) es acotada.

a) $H(s) = \dfrac{s}{s+4}$, $x(t) = \sin(4t)\,u(t)$

b) $H(s) = \dfrac{s^2}{s^2 + 16}$, $x(t) = \big(\cos(2t) + \sin(4t)\big)\,u(t)$

c) $H(z) = \dfrac{z\sin(2\pi/7)}{z^2 - 2z\cos(2\pi/7) + 1}$, $x[n] = 3\cos\big(-\frac{4\pi}{14}n\big)\,u[n]$

d) $H(z) = \dfrac{z}{z^2 + 2}$, $x[n] = 0{,}9^n u[n]$

**Solución**

a) Sí: el sistema es estable (polo en $-4$) y la entrada está acotada. En régimen permanente, $H(j4) = \frac{4j}{4+4j} = \frac{1+j}{2}$, y $y(t) \to \frac{1}{\sqrt{2}}\sin(4t + \pi/4)$.

b) No. Los polos $\pm 4j$ están en el eje imaginario y la entrada contiene justamente la frecuencia 4 rad/s: $Y(s)$ tiene polos dobles en $\pm 4j$ y la salida contiene un término $t\cos(4t)$ que crece sin límite (resonancia). La parte de $\cos(2t)$ da una respuesta acotada.

c) No. $H(z)$ es la transformada de $\sin(2\pi n/7)u[n]$, con polos $\mathrm{e}^{\pm j2\pi/7}$ sobre la circunferencia unidad, y la entrada es $3\cos(2\pi n/7)$ (el coseno es par), de la misma frecuencia: hay resonancia

y la salida crece linealmente con $n$.

d) No. Los polos $\pm j\sqrt{2}$ tienen módulo 1,41 $>$ 1: el sistema es inestable y la salida crece como $(\sqrt{2})^n$ para esta entrada, cuya transformada no tiene ceros que cancelen esos polos.

**Ejercicio 14** (respuesta de sistemas continuos) Obtén la respuesta de los siguientes sistemas a las entradas indicadas, todas nulas para $t < 0$. ¿En qué casos se puede usar la transformada de Fourier y en cuáles es necesaria la de Laplace?

a) $h(t) = \mathrm{e}^{-2t}u(t)$, $x(t) = \big(4\cos(3t) + 10\sin(5t)\big)\,u(t)$

b) $h(t) = \sin(4t)\,u(t)$, $X(s) = \dfrac{s}{s^2+5}$

c) $H(s) = \dfrac{s+3}{s+10}$, $x(t) = t^2u(t)$

**Solución**

a) $H(s) = \frac{1}{s+2}$. Fourier da el régimen permanente: con $H(j\omega) = \frac{1}{2+j\omega}$,

$$y_{\mathrm{ss}}(t) = \frac{4}{\sqrt{13}}\cos\left(3t - \arctan\tfrac{3}{2}\right) + \frac{10}{\sqrt{29}}\sin\left(5t - \arctan\tfrac{5}{2}\right).$$

Laplace da además el transitorio, un término $A\mathrm{e}^{-2t}$ con $A = -y_{\mathrm{ss}}(0) = -\left(\frac{8}{13} - \frac{50}{29}\right) = \frac{418}{377} \approx 1{,}109$, de modo que $y(t) = \left(\frac{418}{377}\mathrm{e}^{-2t} + y_{\mathrm{ss}}(t)\right)u(t)$.

b) $H(s) = \frac{4}{s^2+16}$ y $Y(s) = \frac{4s}{(s^2+5)(s^2+16)} = \frac{4}{11}\left(\frac{s}{s^2+5} - \frac{s}{s^2+16}\right)$, de donde $y(t) = \frac{4}{11}\big(\cos(\sqrt{5}\,t) - \cos(4t)\big)u(t)$. Es acotada porque la frecuencia de la entrada, $\sqrt{5}$, no coincide con la del polo, 4; pero el sistema no es estable y Fourier no se puede aplicar.

c) $Y(s) = \frac{2(s+3)}{s^3(s+10)} = \frac{3}{5s^3} + \frac{7}{50s^2} - \frac{7}{500s} + \frac{7}{500(s+10)}$, y

$$y(t) = \left(\frac{3}{10}t^2 + \frac{7}{50}t - \frac{7}{500} + \frac{7}{500}\mathrm{e}^{-10t}\right)u(t).$$

La entrada $t^2u(t)$ no tiene transformada de Fourier: hace falta Laplace.

**Ejercicio 15** (respuesta de sistemas discretos) Obtén la respuesta de los siguientes sistemas a las entradas indicadas, todas nulas para $n < 0$.

a) $H(z) = \dfrac{z}{z+0{,}8}$, $x[n] = \left(0{,}5^n + 4\cdot 0{,}8^{n-1}\right)u[n]$

b) $H(z) = \dfrac{z}{z-0{,}8}$, $x[n] = n\,u[n]$

c) $H(z) = \dfrac{z^2}{z^2 - 0{,}3z + 0{,}4}$, $x[n] = \cos\left(\frac{\pi}{6}n\right)u[n]$. Obtén al menos el régimen permanente con la respuesta en frecuencia y la forma del transitorio.

**Solución**

a) $X(z) = \frac{z}{z-0{,}5} + 5\,\frac{z}{z-0{,}8}$. Descomponiendo $Y(z)/z$ en fracciones simples,

$$y[n] = \left(\tfrac{81}{26}(-0{,}8)^n + \tfrac{5}{2}(0{,}8)^n + \tfrac{5}{13}(0{,}5)^n\right)u[n].$$

Comprobación: $y[0] = 6 = x[0]$ y $y[1] = -0{,}3$, igual que con `lfilter([1], [1, 0.8], x)`.

b) $Y(z) = \frac{z}{z-0{,}8}\cdot\frac{z}{(z-1)^2} = \frac{20z}{z-0{,}8} - \frac{20z}{z-1} + \frac{5z}{(z-1)^2}$, de donde $y[n] = \big(20\cdot 0{,}8^n - 20 + 5n\big)u[n]$, con $y[0] = 0$, $y[1] = 1$, $y[2] = 2{,}8$, …Crece porque la entrada crece; la ganancia en continua es $H(1) = 5$.

c) Los polos $0{,}15 \pm 0{,}614j$ tienen módulo $\sqrt{0{,}4} = 0{,}632 < 1$: el sistema es estable. Con $H(\mathrm{e}^{j\Omega}) = \frac{1}{1-0{,}3\mathrm{e}^{-j\Omega}+0{,}4\mathrm{e}^{-j2\Omega}}$, en $\Omega = \pi/6$ se obtiene $|H| = 1{,}041$ y $\arg H = 0{,}206$ rad, así que

$$y[n] = 1{,}041\cos\left(\tfrac{\pi}{6}n + 0{,}206\right) + 0{,}632^n\big(A\cos(1{,}331n) + B\sin(1{,}331n)\big),$$

donde $1{,}331 = \arg(0{,}15+0{,}614j)$. Imponiendo $y[0] = 1$ e $y[1] = 1{,}166$ (de la recursión) resulta $A = -0{,}019$ y $B = 0{,}639$. El transitorio se reduce a menos del 1 % en unas diez muestras.

**Ejercicio 16** (régimen permanente) Obtén con la transformada de Fourier la respuesta en régimen permanente de los siguientes sistemas a las entradas indicadas.

a) $H(s) = \dfrac{1}{s+4}$, $x(t)$ la onda cuadrada del ejercicio 7 con $T_0 = 2$ s.

b) $H(z) = \dfrac{z}{z-0{,}8}$, $x[n] = 2\cos\left(\tfrac{\pi}{6}n\right) + \sin\left(\tfrac{\pi}{2}n\right)$.

c) $H(z) = \dfrac{z}{z-1{,}2}$, $x[n] = \cos\left(\tfrac{2\pi}{10}n\right)$.

**Solución**

a) $\omega_0 = \pi$ rad/s. Cada armónico impar, de amplitud $\frac{4}{\pi k}(-1)^{(k-1)/2}$, se multiplica por $|H(jk\pi)| = 1/\sqrt{16 + k^2\pi^2}$ y se desfasa $-\arctan(k\pi/4)$:

$$y_{\mathrm{ss}}(t) = 0{,}250\cos(\pi t - 0{,}666) - 0{,}041\cos(3\pi t - 1{,}169) + 0{,}016\cos(5\pi t - 1{,}321) - \cdots$$

El paso bajo atenúa más los armónicos altos: la salida se parece mucho más a una sinusoide que la entrada.

b) $H(\mathrm{e}^{j\Omega}) = \frac{1}{1-0{,}8\mathrm{e}^{-j\Omega}}$. En $\pi/6$: $|H| = 1{,}983$, $\arg H = -0{,}916$; en $\pi/2$: $|H| = 0{,}781$, $\arg H = -0{,}675$. Por tanto

$$y_{\mathrm{ss}}[n] = 3{,}966\cos\left(\tfrac{\pi}{6}n - 0{,}916\right) + 0{,}781\sin\left(\tfrac{\pi}{2}n - 0{,}675\right).$$

c) El polo $z = 1{,}2$ está fuera de la circunferencia unidad: el sistema es inestable, la salida crece como $1{,}2^n$ y no hay régimen permanente. La respuesta en frecuencia $H(\mathrm{e}^{j\Omega})$ se puede evaluar, pero no describe la respuesta del sistema.

# 2 Muestreo y reconstrucción de señales

**Cómo usar estos apuntes**

Estos apuntes desarrollan el contenido de las cuatro presentaciones del tema: las secciones 2.1–2.3 corresponden a la parte 1 (muestreo y aliasing), la sección 2.4 a la parte 2 (reconstrucción), la sección 2.5 a la parte 3 (cuantificación y codificación) y la sección 2.6 a la parte 4 (muestreo de señales discretas). Los ejemplos de código están en `teoría/T2/codigo/`, en Python y en Matlab, y se pueden ejecutar tal cual. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

## 2.1 Del mundo analógico al digital

Casi todas las señales que interesa procesar son analógicas: la presión acústica, la tensión de un electrodo, la temperatura o el campo electromagnético que recibe una antena. Durante buena parte del siglo XX se procesaron con circuitos analógicos: el predictor de mareas de Lord Kelvin (1873) sumaba sinusoides con poleas y engranajes, y desde los años cuarenta los amplificadores operacionales permitieron construir sumadores, integradores y filtros. Son rápidos, pero sensibles al ruido y a la deriva de los componentes, y poco flexibles: cambiar el procesado exige cambiar el circuito.

El procesado digital sustituye el circuito por un programa. Su despegue se apoya en tres hitos: el algoritmo FFT de Cooley y Tukey (1965), que hizo práctico calcular espectros (Gauss había usado una idea equivalente hacia 1805), el Intel 2920 (1979), un procesador de señal con conversores integrados, y el TMS32010 de Texas Instruments (1983), el primer DSP de éxito comercial, con arquitectura Harvard y una instrucción de multiplicar y acumular en un ciclo.

### 2.1.1 La cadena de procesado

Para procesar digitalmente una señal analógica se sigue la cadena de la figura 2.1. El filtro antialiasing limita el ancho de banda de la entrada; el conversor A/D la *muestrea* (toma valores en instantes discretos), la *cuantifica* (redondea cada valor a uno de un conjunto finito) y la *codifica* (asigna a cada valor una palabra binaria). El procesador opera sobre la secuencia de números y, si la salida tiene que volver a ser analógica, el conversor D/A y el filtro de suavizado la reconstruyen.




**Figura 2.1.** Cadena de procesado digital de una señal analógica.

La figura 2.2 muestra los tres pasos de la conversión A/D. Cada uno introduce un tipo de error distinto, y este tema estudia cuándo y cuánto:

- El **muestreo** no pierde información si la señal tiene banda limitada y se muestrea lo bastante deprisa (apartados 2.2 y 2.3). Si no, aparece el *aliasing*.
- La **reconstrucción** ideal recupera la señal exactamente, pero no es realizable; los retenedores la aproximan (apartado 2.4).
- La **cuantificación** siempre pierde información, pero el error se puede hacer tan pequeño como se quiera añadiendo bits (apartado 2.5).

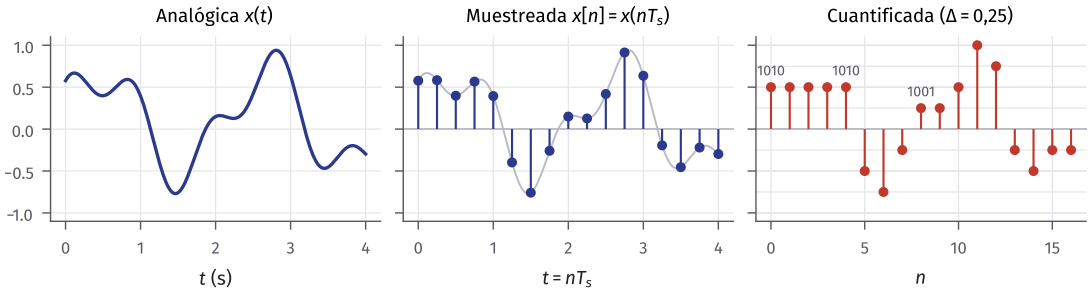



**Figura 2.2.** Señal analógica, señal muestreada (tiempo discreto, amplitud continua) y señal cuantificada (digital).

## 2.2 Muestreo periódico y aliasing

**Definición 2.2.1: Muestreo periódico**

Muestrear periódicamente una señal $x(t)$ con *periodo de muestreo* $T_s$ es formar la secuencia

$$x[n] = x(nT_s), \qquad n \in \mathbb{Z}.$$

La *frecuencia de muestreo* es $f_s = 1/T_s$ en Hz, o $\omega_s = 2\pi f_s = 2\pi/T_s$ en rad/s.

La secuencia no guarda $T_s$: es una lista de números. Los mismos números representan una señal lenta muestreada despacio o una rápida muestreada deprisa, y solo al conocer $T_s$ (o $f_s$) se puede volver a la escala de tiempo física (figura 2.3).

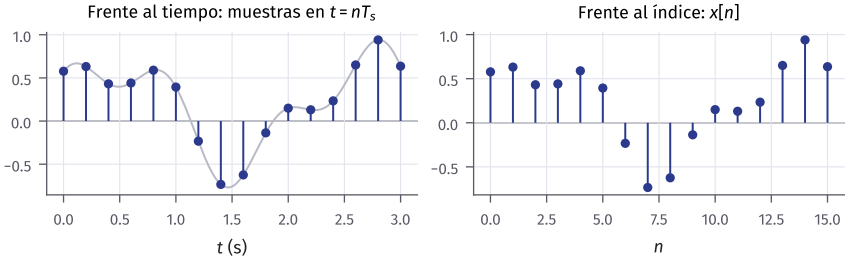



**Figura 2.3.** Muestreo periódico de una señal continua.

### 2.2.1 Frecuencia continua y frecuencia discreta

Si se muestrea una sinusoide $x(t) = A\cos(\omega_0 t + \phi)$, se obtiene

$$x[n] = A\cos(\omega_0 T_s n + \phi) = A\cos(\Omega_0 n + \phi), \qquad \Omega_0 = \omega_0 T_s = 2\pi\frac{f_0}{f_s}.$$

La frecuencia discreta $\Omega$ (rad/muestra) es la frecuencia continua $\omega$ (rad/s) *normalizada* por la frecuencia de muestreo. En ciclos por muestra, $F_0 = f_0/f_s$. Esta relación,

$$\boxed{\Omega = \omega T_s = \frac{2\pi\omega}{\omega_s}} \tag{2.1}$$

es la clave de todo el tema y la que permite traducir especificaciones en hercios a frecuencias discretas y viceversa.

**Ejemplo 2.2.2: Una frecuencia en tres unidades**

Un tono de $f_0 = 1\,\text{kHz}$ muestreado a $f_s = 8\,\text{kHz}$ tiene $\omega_0 = 2000\pi$ rad/s, $F_0 = 1/8$ ciclos por muestra y $\Omega_0 = \pi/4$ rad/muestra: cada periodo del tono ocupa ocho muestras. Si el mismo tono se muestrea a $f_s = 48\,\text{kHz}$, $\Omega_0 = \pi/24$.

### 2.2.2 Alias

En el Tema 1 vimos que las frecuencias discretas $\Omega_0$ y $\Omega_0 + 2\pi k$ dan la misma secuencia, porque $e^{j2\pi kn} = 1$. Deshaciendo el cambio (2.1), todas las frecuencias continuas

$$\omega_k = \omega_0 + k\,\omega_s, \qquad k \in \mathbb{Z},$$

producen exactamente las mismas muestras. Como el coseno es par, también $-\omega_0 + k\omega_s$. Cada una de estas sinusoides es un *alias* de las demás: al muestrear se vuelven indistinguibles (figura 2.4).




**Figura 2.4.** Dos sinusoides de frecuencias distintas que dan las mismas muestras.

**Definición 2.2.3: Alias fundamental**

El *alias fundamental* de una sinusoide muestreada es el de menor frecuencia: el único con $|\Omega| \le \pi$ o, lo que es lo mismo, con $|\omega| \le \omega_s/2$. Se obtiene llevando $\Omega_0$ al intervalo $[-\pi, \pi)$ con

$$\Omega_a = \big((\Omega_0 + \pi) \bmod 2\pi\big) - \pi.$$

Si la sinusoide original cumple $|\omega_0| < \omega_s/2$, ella misma es el alias fundamental y no hay ambigüedad. Si no, existe una sinusoide de frecuencia menor con las mismas muestras y cualquier reconstrucción que produzca señales de frecuencia menor que $\omega_s/2$ devolverá esa y no la original. Este fenómeno se llama *aliasing* (en castellano, a veces, solapamiento).

En hercios, una sinusoide de frecuencia $f_0$ aparece en la *frecuencia aparente*

$$f_a = |f_0 - k f_s| \in [0, f_s/2], \qquad k = \text{el entero más próximo a } f_0/f_s.$$

El eje de frecuencias «se pliega» en los múltiplos de $f_s/2$, que se llama *frecuencia de Nyquist* (figura 2.5). Es lo que hace que las ruedas de un coche parezcan girar despacio o al revés en un vídeo, o que un tono de 30 kHz grabado a 48 kHz sin filtrar suene a 18 kHz.

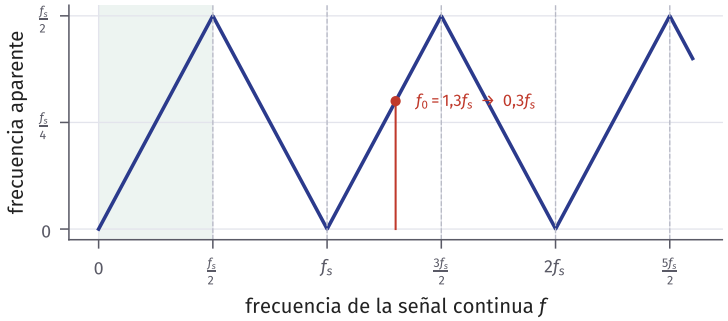



**Figura 2.5.** Frecuencia aparente en función de la frecuencia de la sinusoide muestreada.

**Ejemplo 2.2.4: Alias de una sinusoide y de una suma**

Con $T_s = 2$ s:

- $\sin\left(\frac{2\pi}{10}t\right)$ tiene $\Omega_0 = \frac{4\pi}{10} < \pi$: ya es su alias fundamental.
- $\sin\left(\frac{2\pi}{10/6}t\right)$ tiene $\Omega_0 = \frac{24\pi}{10}$. Restando $2\pi$, $\Omega_a = \frac{4\pi}{10}$, que corresponde a $\omega = \Omega_a/T_s = \frac{2\pi}{10}$ rad/s. Las dos señales dan las mismas muestras.
- $x(t) = 3\sin\left(\frac{2\pi}{5}t\right) + \sin\left(\frac{2\pi}{3}t\right)$: el primer término tiene $\Omega_1 = \frac{4\pi}{5} < \pi$ y se conserva; el segundo, $\Omega_2 = \frac{4\pi}{3}$, que se pliega a $\frac{4\pi}{3} - 2\pi = -\frac{2\pi}{3}$, es decir, $\omega = -\frac{\pi}{3}$ rad/s. Como el seno es impar, el alias fundamental es

$$3\sin\left(\frac{2\pi}{5}t\right) - \sin\left(\frac{\pi}{3}t\right).$$

**En código**

Cálculo del alias fundamental de la segunda sinusoide del ejemplo 2.2.4.

**Python**

```python
import numpy as np

def alias(w, Ts):
    """Frecuencia discreta en π[-, π)."""
    W = w * Ts
    return (W + np.pi) % (2*np.pi) - np.pi

Ts = 2.0                          # s
w1 = 2*np.pi/10                   # rad/s
w2 = 2*np.pi/(10/6)               # rad/s
print(alias(w1, Ts), alias(w2, Ts))  # π0.4
n = np.arange(20)
x1 = np.sin(w1 * Ts * n)
x2 = np.sin(w2 * Ts * n)
print(np.allclose(x1, x2))        # True
```

**Matlab**

```matlab
% Frecuencia discreta en π[-, π)
alias = @(w, Ts) mod(w*Ts + pi, 2*pi) - pi;

Ts = 2;                           % s
w1 = 2*pi/10;                     % rad/s
w2 = 2*pi/(10/6);                 % rad/s
disp([alias(w1, Ts), alias(w2, Ts)])  % π0.4
n = 0:19;
x1 = sin(w1 * Ts * n);
x2 = sin(w2 * Ts * n);
disp(max(abs(x1 - x2)) < 1e-9)  % 1
```

Ejercicios 1, 2 y 6. *Herramienta interactiva:* `T2-aliasing`.

## 2.3 El teorema del muestreo

La sección anterior estudió sinusoides. Para una señal cualquiera hace falta ver qué le ocurre a su espectro al muestrear.

### 2.3.1 Modelo con un tren de impulsos

La secuencia $x[n]$ no tiene transformada de Fourier continua, así que se construye una señal continua que contenga la misma información: el producto de $x(t)$ por un tren de impulsos (figura 2.6),

$$x_s(t) = x(t)\,p(t) = \sum_{n=-\infty}^{\infty} x(nT_s)\,\delta(t - nT_s), \qquad p(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s).$$

Por la propiedad $x(t)\delta(t-t_0) = x(t_0)\delta(t-t_0)$, cada impulso queda ponderado por una muestra.

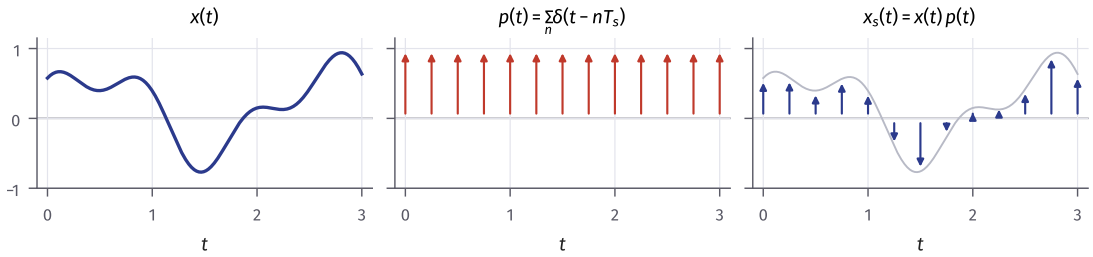



**Figura 2.6.** Muestreo ideal como producto por un tren de impulsos.

El tren $p(t)$ es periódico de periodo $T_s$, con coeficientes de Fourier

$$c_k = \frac{1}{T_s}\int_{-T_s/2}^{T_s/2} \delta(t)\,\mathrm{e}^{-jk\omega_s t}\,\mathrm{d}t = \frac{1}{T_s} \quad \text{para todo } k,$$

y una señal periódica tiene transformada $\sum_k 2\pi c_k \delta(\omega - k\omega_s)$. Por tanto:

**Propiedad 2.3.1: Transformada del tren de impulsos**

$$\sum_{n=-\infty}^{\infty} \delta(t - nT_s) \overset{\mathscr{F}}{\longleftrightarrow} \omega_s \sum_{k=-\infty}^{\infty} \delta(\omega - k\omega_s).$$

Impulsos separados $T_s$ en el tiempo se transforman en impulsos separados $\omega_s = 2\pi/T_s$ en frecuencia.

### 2.3.2 Espectro de la señal muestreada

Un producto en el tiempo es una convolución en frecuencia dividida por $2\pi$, y convolucionar con $\delta(\omega - k\omega_s)$ desplaza el espectro a $k\omega_s$:

$$\boxed{X_s(j\omega) = \frac{1}{2\pi}\,X(j\omega) * P(j\omega) = \frac{1}{T_s}\sum_{k=-\infty}^{\infty} X\big(j(\omega - k\omega_s)\big)} \tag{2.2}$$

El espectro de la señal muestreada es el de la original, escalado por $1/T_s$ y *replicado* cada $\omega_s$ (figura 2.7). Si las réplicas se solapan, se suman, y en la zona de solape ya no se puede saber qué parte venía de cada una: es el aliasing visto en frecuencia.

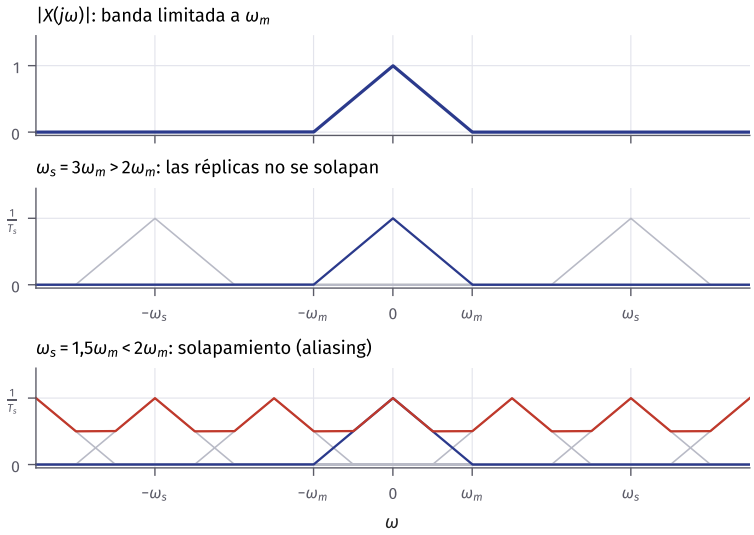



**Figura 2.7.** Espectro de la señal muestreada con muestreo suficiente (arriba) e insuficiente (abajo).

**Definición 2.3.2: Teorema del muestreo (Nyquist–Shannon)**

Sea $x(t)$ de *banda limitada*, $X(j\omega) = 0$ para $|\omega| > \omega_m$. Si

$$\omega_s > 2\,\omega_m \qquad \left(f_s > 2f_m\right),$$

$x(t)$ queda determinada de forma única por sus muestras $x(nT_s)$.

La demostración se lee en la figura 2.7: la réplica $k$ ocupa $[k\omega_s - \omega_m, k\omega_s + \omega_m]$, y la central y la primera no se solapan si $\omega_m < \omega_s - \omega_m$. Entonces la réplica central es $X(j\omega)/T_s$ y un filtro paso bajo la aísla (apartado 2.4). La frecuencia $2f_m$ se llama *tasa de Nyquist*.

La desigualdad tiene que ser estricta: $\sin(\omega_m t)$ muestreada con $\omega_s = 2\omega_m$ da $\sin(\pi n) = 0$ en todas las muestras. Con una componente justo en $\omega_s/2$ las dos réplicas se tocan y la amplitud y la fase de esa componente quedan indeterminadas.

### 2.3.3 Visión continua y visión discreta

Transformando $x_s(t)$ término a término y comparando con la DTFT de $x[n]$,

$$X_s(j\omega) = \sum_n x(nT_s)\,\mathrm{e}^{-j\omega T_s n}, \qquad X(\mathrm{e}^{j\Omega}) = \sum_n x[n]\,\mathrm{e}^{-j\Omega n},$$

se ve que son la misma función con el cambio $\Omega = \omega T_s$:

$$X(\mathrm{e}^{j\Omega}) = X_s(j\omega)\big|_{\omega=\Omega/T_s} = \frac{1}{T_s}\sum_{k=-\infty}^{\infty} X\Big(j\,\frac{\Omega - 2\pi k}{T_s}\Big). \tag{2.3}$$

Esta relación tiene tres consecuencias que se usan continuamente:

- La banda $|\omega| \le \omega_s/2$ se transforma en $|\Omega| \le \pi$. Toda la información de la señal muestreada está en un intervalo de anchura $2\pi$.
- La periodicidad $2\pi$ de la DTFT, que en el Tema 1 apareció como una propiedad matemática, *es* la replicación del espectro que produce el muestreo.

- Si $\omega_s = 2\omega_m$, la señal ocupa todo $[-\pi, \pi]$. Con *sobremuestreo*, por ejemplo $\omega_s = 4\omega_m$, ocupa $|\Omega| \le \pi/2$ y queda una banda libre que facilita los filtros analógicos (apartado 2.6.5).

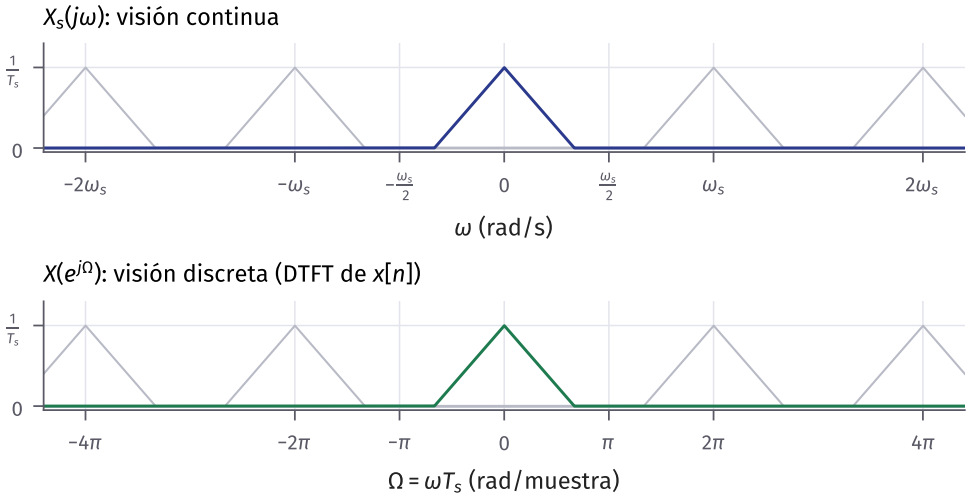



**Figura 2.8.** El mismo espectro en función de $\omega$ (rad/s) y de $\Omega$ (rad/muestra).

**En código**

Se muestrea a $f_s = 3\,\text{Hz}$ una señal $\text{sinc}^2$ de banda $f_m = 1\,\text{Hz}$ y se compara $T_s|X(e^{j\Omega})|$, dibujada frente a $\omega = \Omega/T_s$, con $|X(j\omega)|$.

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

fm = 1.0                          # banda (Hz)
fs = 3.0                          # fs > 2 fm
Ts = 1 / fs
n = np.arange(-300, 301)
x = np.sinc(fm * n * Ts)**2       # banda fm
W, X = signal.freqz(x, 1, worN=2048,
                    whole=True)     # [0, π2)
W = W - np.pi                     # π[-, π)
X = np.fft.fftshift(X)
plt.plot(W / Ts, np.abs(X) * Ts)  # ω = Ω/Ts
plt.xlabel("ω (rad/s)")
plt.show()
```

**Matlab**

```matlab
fm = 1.0;                         % banda (Hz)
fs = 3.0;                         % fs > 2 fm
Ts = 1 / fs;
n = -300:300;
x = sinc(fm * n * Ts).^2;         % banda fm
[X, W] = freqz(x, 1, 2048, 'whole');
W = W - pi;                       % π[-, π)
X = fftshift(X);
plot(W / Ts, abs(X) * Ts)         % ω = Ω/Ts
xlabel('\omega (rad/s)')
```

### 2.3.4 Filtro antialiasing

Las señales reales no son de banda limitada. Una señal de duración finita tiene espectro infinito, y el ruido térmico ocupa todas las frecuencias. Lo que esté por encima de $f_s/2$ se plegará sobre la banda útil y, una vez muestreado, es imposible separarlo. Por eso, antes del conversor A/D se coloca un filtro analógico paso bajo, el *filtro antialiasing*, que atenúa todo lo que hay por encima de $f_s/2$ (figura 2.9).

Un filtro real no corta en seco: necesita una banda de transición. Por eso se muestrea con margen sobre la tasa de Nyquist. El audio de calidad CD ocupa hasta 20 kHz y se muestrea a 44,1 kHz, lo que deja una transición de 20 a 22,05 kHz; la telefonía limita la voz a 3,4 kHz y muestrea a 8 kHz.

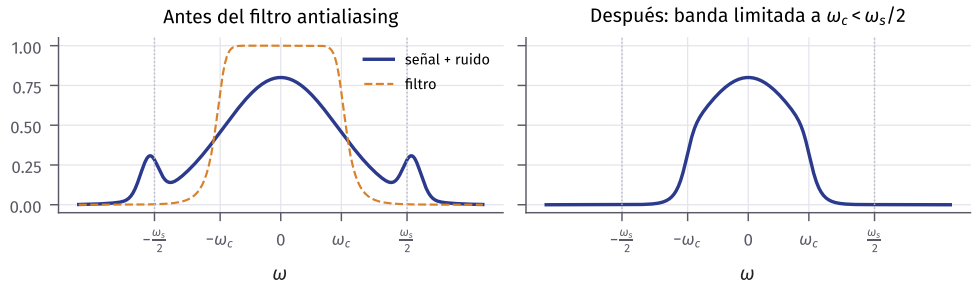



**Figura 2.9.** El filtro antialiasing elimina la parte del espectro que se plegaría sobre la banda útil.

### 2.3.5 Muestreo no ideal

Un circuito no puede generar impulsos ideales; se muestrea con pulsos de duración $\tau$ (figura 2.10). Hay dos variantes:

- **Muestreo natural**: $x_n(t) = x(t)\, p_\tau(t)$, donde $p_\tau$ es un tren de pulsos rectangulares de anchura $\tau$. Sus coeficientes de Fourier son $c_k = \frac{\tau}{T_s}\,\mathrm{sinc}\left(\frac{k\tau}{T_s}\right)$, así que

$$X_n(j\omega) = \sum_k c_k\, X\big(j(\omega - k\omega_s)\big).$$

  Las réplicas no se deforman: solo se ponderan por $c_k$, y la central sigue siendo proporcional a $X(j\omega)$.

- **Muestreo de techo plano**: cada pulso mantiene el valor de la muestra, como hace un circuito de muestreo y retención (*sample and hold*). Es $x_s(t)$ convolucionada con un pulso de anchura $\tau$, así que

$$X_{tp}(j\omega) = X_s(j\omega) \cdot \tau\ \mathrm{sinc}\left(\frac{\omega\tau}{2\pi}\right) e^{-j\omega\tau/2}.$$

  Ahora todas las réplicas, incluida la central, se multiplican por un sinc: es el *efecto de apertura*, que se corrige con un ecualizador o se hace despreciable con $\tau \ll T_s$. Con $\tau = T_s$ es exactamente el retenedor de orden cero de la apartado 2.4.4.

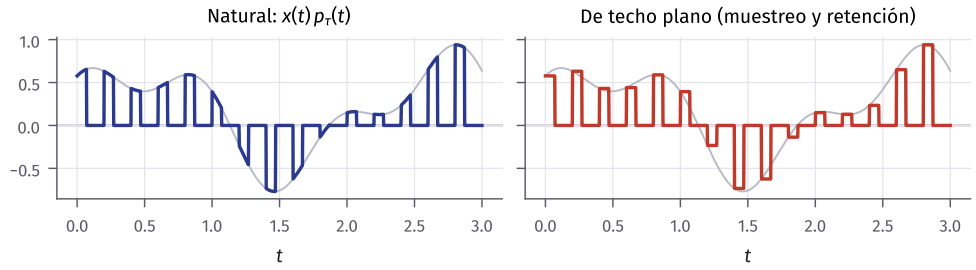



**Figura 2.10.** Muestreo natural y de techo plano.

Ejercicios 3–5. *Herramientas interactivas:* `T2-muestreo-ideal` y `T2-muestreo-natural`.

## 2.4 Reconstrucción

Reconstruir es obtener una señal continua $y(t)$ a partir de la secuencia $x[n]$ y del periodo $T_s$. En general no tiene solución única, porque infinitas señales tienen las mismas muestras. Si $x(t)$ es de banda limitada

y se cumple el teorema del muestreo, la réplica central de $X_s(j\omega)$ es $X(j\omega)/T_s$ y basta un filtro que se quede con ella.

### 2.4.1 Reconstrucción ideal

**Definición 2.4.1: Filtro de reconstrucción ideal**

$$H(j\omega) = T_s\,\Pi\!\left(\frac{\omega}{2\omega_c}\right) = \begin{cases} T_s & |\omega| \le \omega_c \\ 0 & |\omega| > \omega_c \end{cases}, \qquad \omega_m < \omega_c < \omega_s - \omega_m.$$

La ganancia $T_s$ compensa el factor $1/T_s$ de (2.2), y la frecuencia de corte puede ser cualquiera entre el final de la réplica central y el principio de la siguiente (figura 2.11). La elección habitual es $\omega_c = \omega_s/2$, la única posible cuando el margen es mínimo.

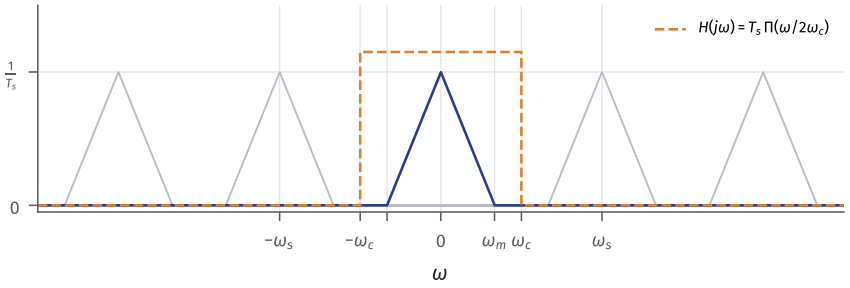



**Figura 2.11.** El filtro ideal de reconstrucción se queda con la réplica central.

Su respuesta al impulso es

$$h(t) = \frac{1}{2\pi}\int_{-\omega_c}^{\omega_c} T_s\,e^{j\omega t}\,d\omega = \frac{T_s}{\pi t}\sin(\omega_c t) = \frac{\omega_c T_s}{\pi}\,\mathrm{Sa}(\omega_c t),$$

con $\mathrm{Sa}(x) = \sin x/x$. Con $\omega_c = \omega_s/2 = \pi/T_s$ queda $h(t) = \mathrm{sinc}(t/T_s)$, con $\mathrm{sinc}(x) = \sin(\pi x)/(\pi x)$: vale 1 en $t = 0$ y se anula en todos los demás múltiplos de $T_s$.

La salida del filtro es la convolución de $x_s(t)$ con $h(t)$, es decir, una suma de copias de $h$ centradas en cada muestra:

**Propiedad 2.4.2: Fórmula de interpolación de Whittaker–Shannon**

Si $x(t)$ es de banda limitada a $\omega_m < \omega_s/2$,

$$x(t) = \sum_{n=-\infty}^{\infty} x[n]\ \mathrm{sinc}\left(\frac{t - nT_s}{T_s}\right).$$

En $t = nT_s$ solo contribuye la muestra $n$, porque las demás funciones sinc se anulan ahí; entre muestras contribuyen todas, ponderadas según su distancia (figura 2.12).

**Ejemplo 2.4.3: Elección de la frecuencia de corte**

Se quiere reconstruir $x(t) = \sin(\omega_0 t)$, con $\omega_0 = 2\pi/10$ rad/s, a partir de sus muestras con $T_s = 4$ s. Como $\omega_s = 2\pi/4 = 5\pi/10 > 2\omega_0 = 4\pi/10$, se cumple el teorema. Las réplicas de la sinusoide están en $\pm\omega_0 + k\omega_s$; la más próxima a la central por arriba es $\omega_s - \omega_0 = 3\pi/10$. Cualquier $\omega_c$ con

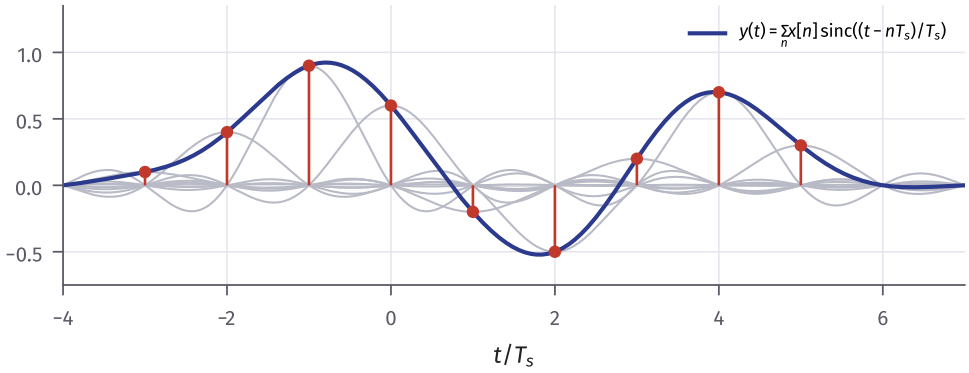
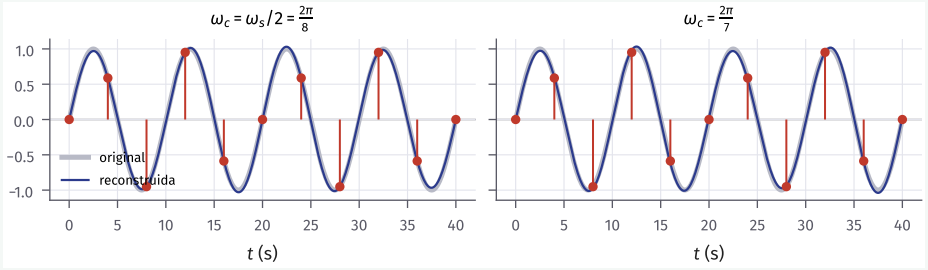



**Figura 2.12.** Interpolación con funciones sinc: la suma de las copias reproduce la señal.

$2\pi/10 < \omega_c < 3\pi/10$ reconstruye exactamente la señal:

$$y(t) = \sum_n x[n]\,\frac{\omega_c T_s}{\pi}\,\mathrm{Sa}\left(\omega_c(t - nT_s)\right).$$




### 2.4.2 Por qué la reconstrucción ideal no es realizable

El filtro ideal tiene tres problemas prácticos:

- Es *no causal*: $h(t) \neq 0$ para $t < 0$, así que $y(t)$ depende de muestras futuras.
- Tiene *duración infinita*, y sinc decae solo como $1/t$: truncar la suma deja un error apreciable.
- Supone una señal de banda limitada, y ninguna señal de duración finita lo es.

Además, ningún filtro analógico tiene un corte abrupto. Fuera de línea (al remuestrear audio o escalar imágenes) se usa la fórmula truncada o con ventana. En tiempo real se usan *retenedores*, que interpolan con pocas muestras, seguidos de un filtro analógico de suavizado.

**En código**

Reconstrucción de un coseno de 2 Hz muestreado con $T_s = 0{,}1$ s con 101 muestras. El error no es exactamente cero porque la suma está truncada.

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt

Ts = 0.1                          # s
n = np.arange(-50, 51)
xn = np.cos(2*np.pi*2*n*Ts)       # 2 Hz < fs/2
t = np.linspace(-1, 1, 1001)
# y(t) = Σ x[n] sinc((t - nTs)/Ts)
S = np.sinc((t[:, None] - n*Ts) / Ts)
y = S @ xn
print(np.max(np.abs(y - np.cos(4*np.pi*t))))
plt.plot(t, y); plt.stem(n*Ts, xn)
plt.xlim(-1, 1); plt.show()
```

**Matlab**

```matlab
Ts = 0.1;                         % s
n = -50:50;
xn = cos(2*pi*2*n*Ts);            % 2 Hz < fs/2
t = linspace(-1, 1, 1001);
% y(t) = Σ x[n] sinc((t - nTs)/Ts)
S = sinc((t.' - n*Ts) / Ts);
y = S * xn.';
disp(max(abs(y.' - cos(4*pi*t))))
plot(t, y); hold on; stem(n*Ts, xn)
xlim([-1 1])
```

### 2.4.3 La reconstrucción como interpolación

Cualquier reconstrucción lineal e invariante con el desplazamiento de las muestras tiene la forma

$$y(t) = \sum_n x[n]\, h(t - nT_s) \quad \Longleftrightarrow \quad Y(j\omega) = H(j\omega)\, X_s(j\omega).$$

La *función de interpolación* $h(t)$ dice cuánto contribuye cada muestra según su distancia al instante $t$, y su transformada $H(j\omega)$ dice cuánto se deforma la réplica central y cuánto se atenúan las demás. El filtro ideal es el caso $h = \text{sinc}$; los retenedores usan funciones más cortas (figura 2.13).

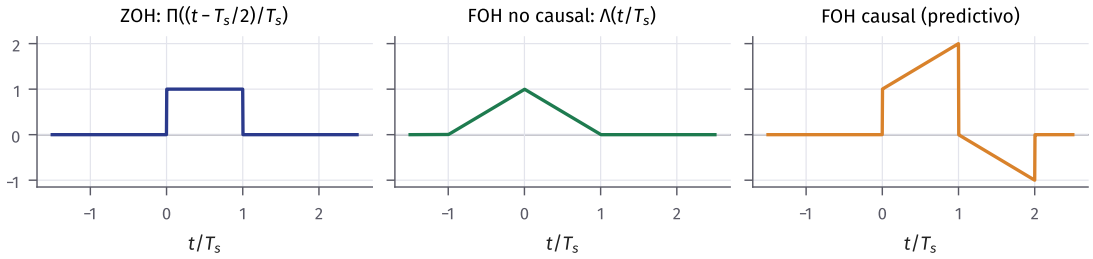



**Figura 2.13.** Funciones de interpolación de los retenedores de orden cero y uno.

### 2.4.4 Retenedor de orden cero

El retenedor de orden cero (ZOH, *zero-order hold*) mantiene cada muestra hasta la siguiente: $y(t) = x[n]$ para $nT_s \le t < (n+1)T_s$. Su función de interpolación es un pulso de anchura $T_s$ que empieza en el origen:

$$h_0(t) = \Pi\!\left(\frac{t - T_s/2}{T_s}\right).$$

Su transformada es la de un pulso centrado, $T_s\,\text{sinc}(\omega/\omega_s)$, retrasada $T_s/2$:

$$\boxed{H_0(j\omega) = T_s\, e^{-j\omega T_s/2}\ \text{sinc}\left(\frac{\omega}{\omega_s}\right) = T_s\, e^{-j\omega T_s/2}\ \text{Sa}\left(\frac{\omega T_s}{2}\right)} \tag{2.4}$$

**Errata frecuente**

El retardo del ZOH es $e^{-j\omega T_s/2}$, con signo negativo: el pulso está a la *derecha* del origen. Con signo positivo el retenedor sería anticipativo.

El ZOH es causal y muy sencillo: es lo que hace un conversor D/A al mantener su salida hasta el siguiente código. Pero no es un buen filtro de reconstrucción (figura 2.14):

- **Atenúa la banda útil**. En $\omega_s/2$, $|H_0|/T_s = \operatorname{sinc}(1/2) = 2/\pi$, unos $-3{,}9$ dB. Para audio a 44,1 kHz, en 20 kHz la atenuación es de 3,2 dB.
- **No elimina las réplicas**, solo las atenúa como $1/\omega$. La primera imagen de un tono de 20 kHz, en 24,1 kHz, sale solo 4,8 dB por debajo.
- Introduce un **retardo** de media muestra.

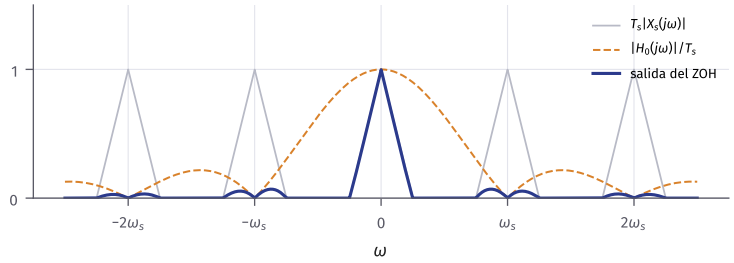



**Figura 2.14.** Módulo de la respuesta del ZOH frente al filtro ideal.

### 2.4.5 Retenedor de orden uno

El retenedor de orden uno (FOH, *first-order hold*) une las muestras con segmentos rectos. Hay dos versiones:

**FOH no causal (interpolación lineal).** Entre $nT_s$ y $(n+1)T_s$,

$$y(t) = x[n] + \big(x[n+1] - x[n]\big)\frac{t - nT_s}{T_s}.$$

La función de interpolación es un triángulo de base $2T_s$ centrado en el origen, que es la convolución de dos pulsos de anchura $T_s$ dividida por $T_s$:

$$h_1(t) = \Lambda\Big(\frac{t}{T_s}\Big) = \frac{1}{T_s}\,\Pi\Big(\frac{t}{T_s}\Big) * \Pi\Big(\frac{t}{T_s}\Big) \quad\Longrightarrow\quad \boxed{H_1(j\omega) = T_s\,\operatorname{sinc}^2\Big(\frac{\omega}{\omega_s}\Big)}$$

Es real y positiva (fase nula) y las réplicas caen como $1/\omega^2$. Usa la muestra siguiente, así que no es causal; retrasándolo una muestra se vuelve causal, con $H = T_s e^{-j\omega T_s}\operatorname{sinc}^2(\omega/\omega_s)$, a costa de un retardo $T_s$.

**FOH causal (predictivo).** Prolonga la recta que pasa por las dos últimas muestras:

$$y(t) = x[n] + \big(x[n] - x[n-1]\big)\frac{t - nT_s}{T_s}, \qquad nT_s \le t < (n+1)T_s.$$

La contribución de una muestra aislada es una rampa ascendente seguida de una descendente,

$$h(t) = \begin{cases} 1 + t/T_s & 0 \le t < T_s \\ 1 - t/T_s & T_s \le t < 2T_s \\ 0 & \text{en otro caso} \end{cases} \quad \Longrightarrow \quad H(j\omega) = T_s\,(1 + j\omega T_s)\,\mathrm{e}^{-j\omega T_s}\,\operatorname{sinc}^2\left(\frac{\omega}{\omega_s}\right).$$

El factor $1 + j\omega T_s$ amplifica las altas frecuencias: la salida tiene saltos en cada instante de muestreo y sobrepasa la señal cuando cambia la pendiente (figura 2.15).

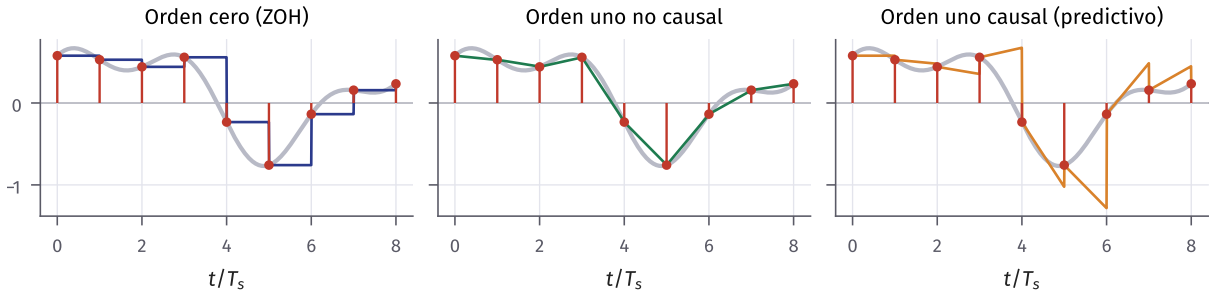



**Figura 2.15.** Los tres retenedores aplicados a la misma secuencia.

### 2.4.6 Comparación y conversión D/A

El cuadro 2.1 y la figura 2.16 resumen los retenedores. Ninguno reconstruye exactamente: deforman la banda útil y dejan pasar parte de las réplicas. Cuanto mayor es el orden, mejor atenúan las réplicas, pero necesitan más muestras y más retardo. Una cota útil del error para una señal suave: el ZOH comete un error de hasta $\max|x'|\,T_s$, y la interpolación lineal, de hasta $\max|x''|\,T_s^2/8$.

| | $h(t)$ | $\lvert H(j\omega)\rvert/T_s$ | Causal |
|---|---|---|---|
| Ideal | $\operatorname{sinc}(t/T_s)$ | $\Pi(\omega/\omega_s)$ | No |
| ZOH | $\Pi\left(\frac{t-T_s/2}{T_s}\right)$ | $\lvert\operatorname{sinc}(\omega/\omega_s)\rvert$ | Sí |
| FOH no causal | $\Lambda(t/T_s)$ | $\operatorname{sinc}^2(\omega/\omega_s)$ | No (sí con retardo $T_s$) |
| FOH predictivo | dos rampas en $[0, 2T_s)$ | $\lvert 1 + j\omega T_s\rvert\operatorname{sinc}^2(\omega/\omega_s)$ | Sí |

**Cuadro 2.1.** Comparación de los métodos de reconstrucción.

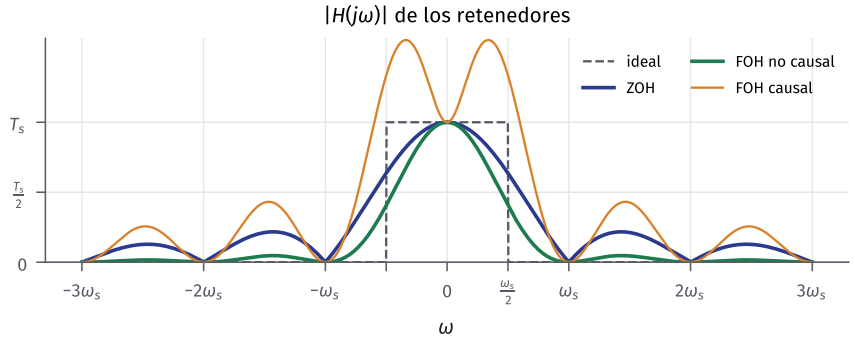



**Figura 2.16.** Respuesta en frecuencia de los retenedores.

Un conversor D/A real combina las piezas: una *compensación* digital con respuesta $1/\operatorname{sinc}(\omega/\omega_s)$ en la banda útil corrige la caída del ZOH; el conversor mantiene cada valor (ZOH); y un *filtro de suavizado* analógico elimina las réplicas que el ZOH deja pasar. Con sobremuestreo (apartado 2.6.5) las réplicas se alejan y ese filtro puede ser muy sencillo.

**En código**

Reconstrucción de una sinusoide con diez muestras por periodo con ZOH y FOH no causal. El error máximo del ZOH es 0,59 y el del FOH, 0,05.

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt

Ts = 0.1                          # s
n = np.arange(0, 11)
xn = np.sin(2*np.pi*1*n*Ts)       # 1 Hz
t = np.linspace(0, 1, 1001)
# Orden cero: se mantiene la última muestra
k = np.floor(t / Ts).astype(int)
y0 = xn[np.minimum(k, n[-1])]
# Orden uno (no causal): une las muestras
y1 = np.interp(t, n*Ts, xn)
x = np.sin(2*np.pi*t)
print(np.max(abs(y0 - x)), np.max(abs(y1 - x))
    )
plt.plot(t, y0, t, y1, t, x); plt.show()
```

**Matlab**

```matlab
Ts = 0.1;                         % s
n = 0:10;
xn = sin(2*pi*1*n*Ts);            % 1 Hz
t = linspace(0, 1, 1001);
% Orden cero: se mantiene la última muestra
y0 = interp1(n*Ts, xn, t, 'previous');
% Orden uno (no causal): une las muestras
y1 = interp1(n*Ts, xn, t, 'linear');
x = sin(2*pi*t);
disp([max(abs(y0 - x)), max(abs(y1 - x))])
plot(t, y0, t, y1, t, x)
```

Ejercicios 7–10. *Herramientas interactivas:* `T2-reconstruccion` y `T2-retenedores`.

## 2.5 Cuantificación y codificación

El muestreo discretiza el tiempo, pero la amplitud de $x[n]$ sigue siendo *continua*. Un procesador solo puede almacenar números con un número finito de bits, así que cada muestra se aproxima por uno de un conjunto finito de valores. A diferencia del muestreo, esta aproximación siempre pierde información.

### 2.5.1 Cuantificador uniforme

**Definición 2.5.1: Cuantificación**

La *cuantificación* asigna a cada muestra $x[n]$ uno de $L$ valores permitidos, los *niveles de cuantificación*:

$$x_q[n] = Q\big[x[n]\big].$$

En un cuantificador *uniforme* los niveles están separados por un *escalón* o *resolución* constante $\Delta$. La diferencia $e[n] = x_q[n] - x[n]$ es el *error de cuantificación*.

Con $b$ bits hay $L = 2^b$ niveles. Si la entrada está en el margen $[-V, V)$, de *fondo de escala* $FS = 2V$,

$$\Delta = \frac{FS}{2^b} = \frac{2V}{2^b}. \tag{2.5}$$

Es el convenio de los conversores A/D, donde $\Delta$ es el valor del bit menos significativo (1 LSB). En el margen bipolar los niveles son $k\Delta$ con $k = -2^{b-1}, \ldots, 2^{b-1} - 1$: hay un nivel en cero y el margen es ligeramente asimétrico.

**Otro convenio**

Algunos textos colocan el primer y el último nivel exactamente en los extremos del intervalo $[x_{\min}, x_{\max}]$, y entonces $\Delta = \frac{x_{\max} - x_{\min}}{L-1}$. Las dos definiciones difieren en un nivel y dan las mismas conclusiones para $L$ grande. En este curso se usa (2.5).

Si la entrada se sale del margen, el cuantificador *satura*: devuelve el nivel extremo y el error deja de estar acotado por $\Delta$. Por eso el margen del conversor debe ajustarse a la señal, sin quedarse corto (saturación) ni largo (se desperdician niveles).

### 2.5.2 Redondeo y truncamiento

Hay dos formas habituales de elegir el nivel (figura 2.17):

- **Redondeo**: el nivel más próximo, $Q[x] = \Delta\ \mathrm{round}(x/\Delta)$. El error está en $-\Delta/2 \le e < \Delta/2$ y tiene media nula.
- **Truncamiento**: el nivel inmediatamente inferior, $Q[x] = \Delta\lfloor x/\Delta \rfloor$. El error está en $-\Delta < e \le 0$: tiene un sesgo de $-\Delta/2$.

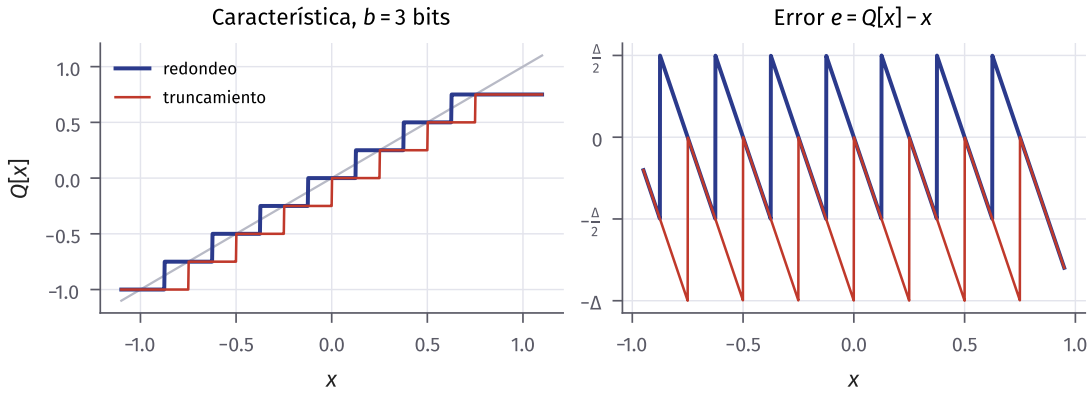



**Figura 2.17.** Característica de un cuantificador de 3 bits y error de cuantificación.

**Ejemplo 2.5.2: Cuantificar $x[n] = 0{,}9^n$**

Para representar exactamente los diez primeros valores ($n = 0, \ldots, 9$) hace falta $\Delta = 10^{-9}$, porque $0{,}9^9 = 0{,}387420489$ tiene nueve decimales. Con un decimal, $\Delta = 0{,}1$:

| | | $x_q[n]$ | | $e[n]$ | |
|---|---|---|---|---|---|
| $n$ | $x[n]$ | truncado | redondeado | truncado | redondeado |
| 0 | 1 | 1,0 | 1,0 | 0 | 0 |
| 1 | 0,9 | 0,9 | 0,9 | 0 | 0 |
| 2 | 0,81 | 0,8 | 0,8 | −0,01 | −0,01 |
| 3 | 0,729 | 0,7 | 0,7 | −0,029 | −0,029 |
| 4 | 0,6561 | 0,6 | 0,7 | −0,0561 | 0,0439 |
| 5 | 0,59049 | 0,5 | 0,6 | −0,09049 | 0,00951 |
| 6 | 0,531441 | 0,5 | 0,5 | −0,031441 | −0,031441 |
| 7 | 0,4782969 | 0,4 | 0,5 | −0,0782969 | 0,0217031 |
| 8 | 0,43046721 | 0,4 | 0,4 | −0,03046721 | −0,03046721 |
| 9 | 0,387420489 | 0,3 | 0,4 | −0,087420489 | 0,012579511 |

Truncando, todos los errores son negativos y de hasta −0,09; redondeando, cambian de signo y no superan 0,05 en valor absoluto.

### 2.5.3 Ruido de cuantificación

El error de cuantificación es una función determinista de la señal, pero si la señal es activa (cruza muchos niveles entre muestra y muestra) y no satura, se comporta como un ruido aditivo (figura 2.18):

$$x_q[n] = x[n] + e[n],$$

con $e[n]$ uniformemente distribuido en $[-\Delta/2, \Delta/2)$, incorrelado con $x[n]$ y blanco (sus muestras son independientes entre sí). Su media es nula y su potencia es

$$\boxed{\sigma_e^2 = \frac{1}{\Delta}\int_{-\Delta/2}^{\Delta/2} e^2 \,\mathrm{d}e = \frac{\Delta^2}{12}} \tag{2.6}$$

Con truncamiento la varianza es la misma, pero la media es $-\Delta/2$. El modelo falla si la señal es lenta o pequeña frente a $\Delta$: entonces el error se parece a la propia señal y aparece como distorsión armónica. Las herramientas para describir señales aleatorias se estudian en el Tema 6.

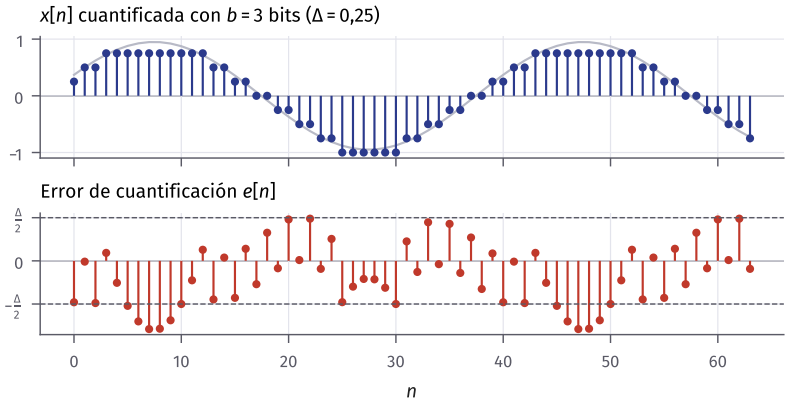



**Figura 2.18.** Señal cuantificada con 3 bits y su error de cuantificación.

La calidad de la cuantificación se mide con la *relación señal a ruido de cuantificación*,

$$\mathrm{SQNR} = 10\log_{10}\frac{P_x}{\sigma_e^2}\ \mathrm{dB}.$$

Para una sinusoide que ocupa todo el margen, de amplitud $V$, $P_x = V^2/2$, y con (2.5) y (2.6):

$$\boxed{\mathrm{SQNR} = 10\log_{10}\frac{V^2/2}{(2V)^2/(12\cdot 2^{2b})} = 10\log_{10}\left(\frac{3}{2}\,2^{2b}\right) \approx 6{,}02\,b + 1{,}76\ \mathrm{dB}} \tag{2.7}$$

Cada bit añade unos 6 dB (figura 2.19). Un CD, con 16 bits, alcanza unos 98 dB. La fórmula supone que la señal aprovecha todo el margen: si su amplitud es $V/10$, la SQNR baja 20 dB, lo que equivale a perder más de tres bits.

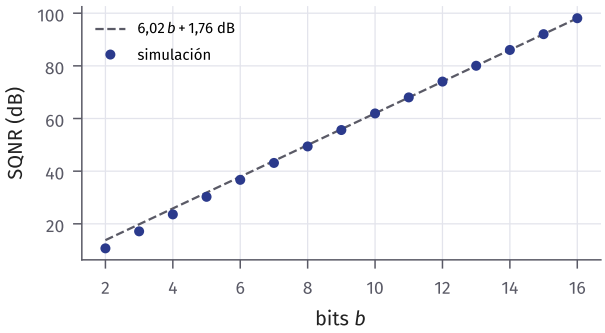



**Figura 2.19.** SQNR medida y teórica en función del número de bits.

**En código**

Cuantificación por redondeo con 8 bits de una sinusoide de amplitud $0{,}99\,V$. La varianza del error coincide con $\Delta^2/12$ y la SQNR medida, 49,7 dB, con los 49,9 dB de (2.7).

**Python**

```python
import numpy as np

b = 8                          # bits
V = 1.0                        # margen [-V, V)
D = 2*V / 2**b                 # escalón Δ
n = np.arange(100_000)
x = 0.99*V*np.sin(2*np.pi*0.01234*n)
xq = D * np.round(x / D)       # redondeo
e = xq - x
print(np.var(e), D**2/12)      # ≈ Δ²/12
snr = 10*np.log10(np.var(x) / np.var(e))
print(snr, 6.02*b + 1.76)      # dB
```

**Matlab**

```matlab
b = 8;                         % bits
V = 1.0;                       % margen [-V, V)
D = 2*V / 2^b;                 % escalón Δ
n = 0:99999;
x = 0.99*V*sin(2*pi*0.01234*n);
xq = D * round(x / D);         % redondeo
e = xq - x;
disp([var(e), D^2/12])         % ≈ Δ²/12
snr = 10*log10(var(x) / var(e));
disp([snr, 6.02*b + 1.76])     % dB
```

### 2.5.4 Codificación

La codificación asigna a cada nivel una palabra binaria. Con $b$ bits hay $2^b$ palabras, así que para $L$ niveles hace falta

$$2^b \geq L \quad\Longrightarrow\quad b \geq \log_2 L.$$

Por ejemplo, 6 niveles necesitan $b \geq \log_2 6 \approx 2{,}585$, es decir, 3 bits (y sobran dos palabras).

Para un margen bipolar, con niveles $k\Delta$, se usan sobre todo dos códigos (cuadro 2.2): el *binario desplazado*, que codifica $k + 2^{b-1}$ en binario natural y es el que entregan muchos conversores, y el *complemento a dos*, que es el de la aritmética entera de los procesadores. Pasar de uno a otro es tan sencillo como invertir el bit más significativo.

| $k$ | $x_q = k\Delta$ | binario desplazado | complemento a dos |
|---|---|---|---|
| 3 | $3\Delta$ | 111 | 011 |
| 2 | $2\Delta$ | 110 | 010 |
| 1 | $\Delta$ | 101 | 001 |
| 0 | 0 | 100 | 000 |
| $-1$ | $-\Delta$ | 011 | 111 |
| $-2$ | $-2\Delta$ | 010 | 110 |
| $-3$ | $-3\Delta$ | 001 | 101 |
| $-4$ | $-4\Delta$ | 000 | 100 |

**Cuadro 2.2.** Códigos de un cuantificador bipolar de 3 bits.

### 2.5.5 Parámetros de un conversor A/D

Un conversor A/D se describe por:

- Su **resolución** $b$ y su **tensión de referencia** $V_{\text{ref}}$. Con un margen unipolar $[0, V_{\text{ref}})$, $1\,\text{LSB} = \Delta = V_{\text{ref}}/2^b$.
- Su **rango dinámico**: el cociente entre la mayor y la menor amplitud que distingue,

$$\text{DR} = 20\log_{10}\frac{V_{\text{ref}}}{\Delta} = 20\log_{10} 2^b \approx 6{,}02\,b\ \text{dB}.$$

  Ojo: «rango dinámico» designa a veces el intervalo de valores de la entrada. Aquí lo reservamos para el cociente en decibelios y llamamos *margen* al intervalo.
- Su **frecuencia de muestreo** máxima, ligada al tiempo de conversión.
- Su **arquitectura**: *flash* (muy rápida, pocos bits), de aproximaciones sucesivas (SAR, el compromiso habitual en microcontroladores) o sigma-delta (muchos bits gracias al sobremuestreo, apartado 2.6.5).

**Ejemplo 2.5.3: Conversor de un microcontrolador**

Un conversor de 10 bits con $V_{\text{ref}} = 3{,}3\,\text{V}$ tiene $\Delta = 3{,}3/1024 \approx 3{,}22\,\text{mV}$ y $\text{DR} \approx 60{,}2\,\text{dB}$. Para una sinusoide que ocupe todo el margen, la SQNR es de unos 62 dB. Si se quiere distinguir variaciones de 0,1 mV, hacen falta $b \geq \log_2(3{,}3/10^{-4}) \approx 15{,}01$, es decir, 16 bits.

Ejercicios 11–13. *Herramienta interactiva:* `T2-cuantificacion`.

## 2.6 Muestreo de señales discretas

Muchas veces hay que cambiar la frecuencia de muestreo de una señal que ya es discreta: pasar audio de 48 kHz a los 8 kHz de la telefonía, o de los 44,1 kHz del CD a los 48 kHz del vídeo; cambiar el tamaño de una imagen; o procesar a la menor frecuencia posible para ahorrar cálculo. Volver al dominio analógico y muestrear de nuevo añadiría ruido y coste. Todo lo visto para señales continuas tiene su versión discreta: réplicas, aliasing y filtros (figura 2.20).

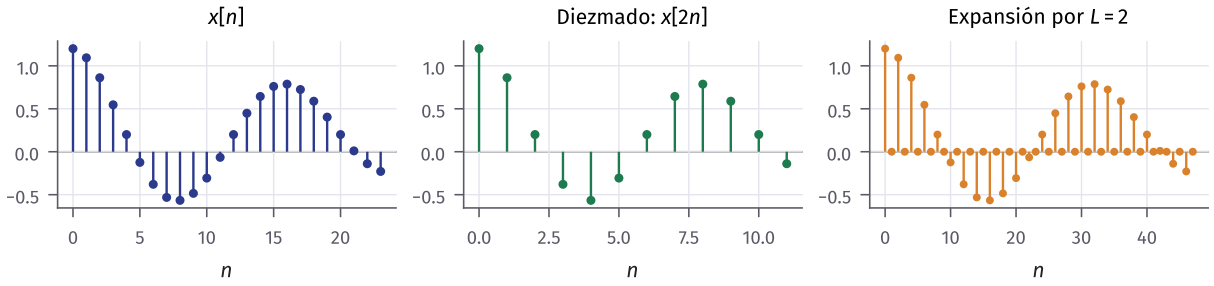



**Figura 2.20.** Una secuencia, su versión diezmada por 2 y su versión expandida por 2.

### 2.6.1 Muestreo de una secuencia

Muestrear una secuencia con periodo $N$ es conservar una de cada $N$ muestras y poner a cero las demás:

$$x_p[n] = x[n]\,p[n], \qquad p[n] = \sum_{k=-\infty}^{\infty} \delta[n - kN].$$

El tren $p[n]$ es periódico de periodo $N$, con desarrollo en serie de Fourier discreta

$$p[n] = \frac{1}{N} \sum_{k=0}^{N-1} \mathrm{e}^{j2\pi kn/N}.$$

Multiplicar por $\mathrm{e}^{j2\pi kn/N}$ desplaza el espectro $2\pi k/N$, así que

**Propiedad 2.6.1: Espectro de una secuencia muestreada**

$$X_p(\mathrm{e}^{j\Omega}) = \frac{1}{N} \sum_{k=0}^{N-1} X\big(\mathrm{e}^{j(\Omega - 2\pi k/N)}\big).$$

Es la versión discreta de (2.2): aparecen $N-1$ réplicas separadas $\Omega_s = 2\pi/N$. Si $x[n]$ tiene banda limitada a $\Omega_m$, no se solapan cuando $\Omega_m < \pi/N$, y entonces $x[n]$ se recupera de $x_p[n]$ con un filtro paso bajo ideal de ganancia $N$ y corte $\pi/N$.

### 2.6.2 Diezmado

Las muestras nulas de $x_p[n]$ no aportan nada; eliminarlas es *diezmar* (o submuestrear) por un factor $M$:

$$x_d[n] = x[nM] = x_p[nM].$$

La nueva frecuencia de muestreo es $f_s/M$. Como $X_d(\mathrm{e}^{j\Omega}) = \sum_n x_p[nM]\mathrm{e}^{-j\Omega n} = X_p(\mathrm{e}^{j\Omega/M})$, el espectro se *expande* por $M$:

**Propiedad 2.6.2: Diezmado**

$$X_d(\mathrm{e}^{j\Omega}) = \frac{1}{M} \sum_{k=0}^{M-1} X\big(\mathrm{e}^{j(\Omega - 2\pi k)/M}\big).$$

No hay aliasing si $\Omega_m < \pi/M$.

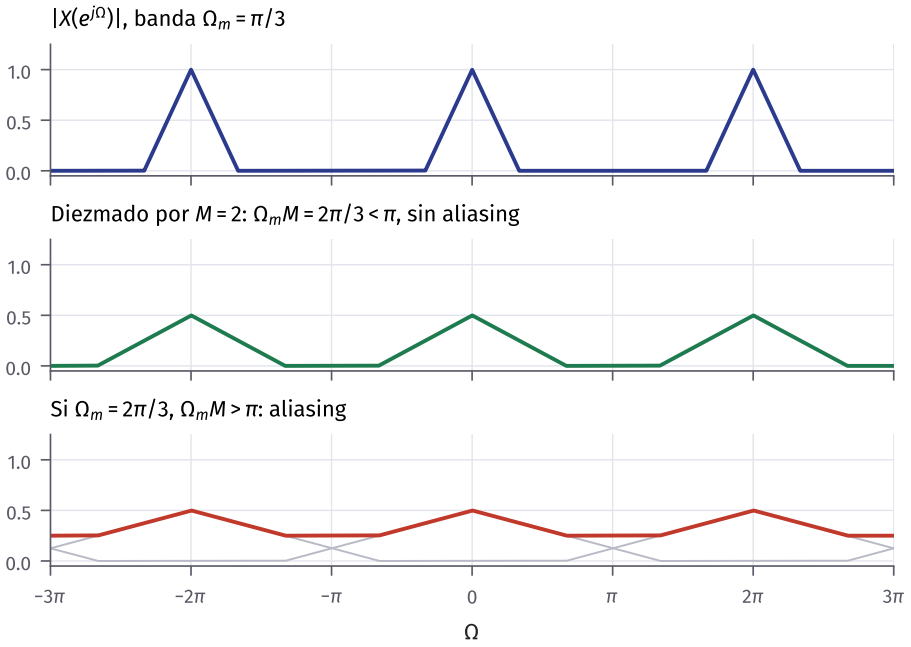



**Figura 2.21.** Diezmado por $M = 2$ sin aliasing (centro) y con aliasing (abajo).

El resultado sigue siendo periódico de periodo $2\pi$, como debe ser la DTFT de cualquier secuencia: la réplica $k$ queda centrada en $2\pi k$ (figura 2.21). Es lo esperable de (2.1): con la nueva frecuencia de muestreo, cada frecuencia en hercios corresponde a una frecuencia discreta $M$ veces mayor.

Si $x[n]$ ocupa más de $\pi/M$, antes de diezmar hay que filtrarla con un paso bajo de ganancia 1 y corte $\pi/M$ (figura 2.22). Es el mismo papel que el filtro antialiasing analógico: se pierde la banda alta, pero no se mezcla con la baja. El filtro trabaja a la frecuencia alta, aunque solo hace falta calcular una de cada $M$ salidas.

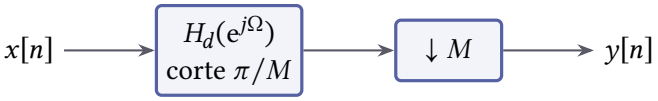



**Figura 2.22.** Diezmador con filtro antialiasing.

**Ejemplo 2.6.3: De 48 kHz a 8 kHz**

El factor es $M = 6$. El filtro corta en $\pi/6$, que a $f_s = 48\,\text{kHz}$ equivale a $\frac{\pi/6}{2\pi} \cdot 48 = 4\,\text{kHz}$: justo la frecuencia de Nyquist de la señal de salida.

### 2.6.3 Expansión e interpolación

Para aumentar la frecuencia de muestreo por un factor entero $L$, primero se insertan $L-1$ ceros entre cada par de muestras (*expansión*):

$$x_e[n] = \begin{cases} x[n/L] & n = 0, \pm L, \pm 2L, \dots \\ 0 & \text{en otro caso} \end{cases} \qquad \Longrightarrow \qquad X_e(e^{j\Omega}) = \sum_k x[k] e^{-j\Omega k L} = X(e^{j\Omega L}).$$

El espectro se *comprime* por $L$ y en $[-\pi, \pi]$ aparecen $L-1$ copias adicionales, las *imágenes* (figura 2.23). Un *filtro de interpolación* paso bajo de ganancia $L$ y corte $\pi/L$ las elimina y «rellena» los ceros. La ganancia $L$ compensa los ceros insertados: el valor medio de $x_e[n]$ es $1/L$ del de $x[n]$.

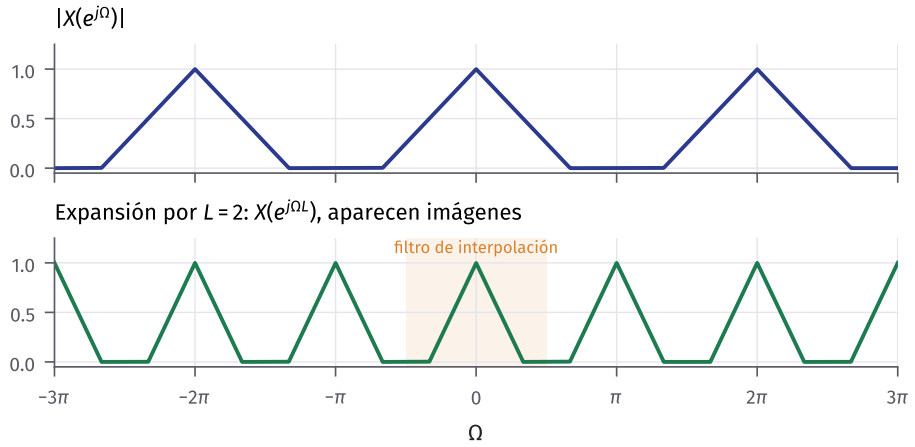



**Figura 2.23.** Expansión por $L = 2$: el espectro se comprime y aparecen imágenes.

Con el filtro ideal, cuya respuesta al impulso es $h[n] = \mathrm{sinc}(n/L)$, la salida es la interpolación de banda limitada de las muestras,

$$y[n] = \sum_k x[k]\ \mathrm{sinc}\left(\frac{n - kL}{L}\right),$$

que es la fórmula de Whittaker–Shannon evaluada en los nuevos instantes; en particular, $y[kL] = x[k]$. Con filtros más sencillos se obtienen los retenedores discretos: repetir cada muestra $L$ veces ($h[n] = 1$ para $0 \le n < L$, orden cero) o interpolar linealmente ($h[n] = \Lambda(n/L)$, orden uno).

### 2.6.4 Cambio de frecuencia por un factor racional

Para cambiar la frecuencia por un factor racional $L/M$ se interpola por $L$ y después se diezma por $M$ (figura 2.24). En ese orden no se pierde banda innecesariamente, y los dos filtros, el de interpolación y el antialiasing, se funden en uno solo con ganancia $L$ y corte $\min(\pi/L, \pi/M)$.

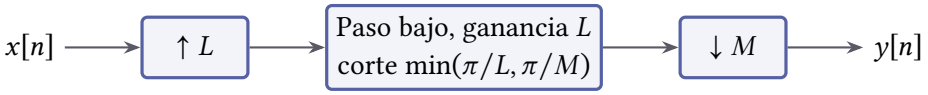



**Figura 2.24.** Cambio de frecuencia de muestreo por un factor $L/M$.

**Ejemplo 2.6.4: De 44,1 kHz a 48 kHz**

$48\,000/44\,100 = 160/147$, que ya es irreducible. Se interpola por $L = 160$, lo que da una frecuencia intermedia de 7,056 MHz, se filtra con corte $\pi/160$ y se diezma por $M = 147$. Calcular directamente así sería muy caro; en la práctica se usan estructuras *polifásicas*, que no calculan las muestras que se van a descartar ni multiplican por los ceros insertados.

**En código**

Diezmado por 4 con filtro antialiasing incluido (`decimate`) y cambio de 44,1 a 48 kHz con una estructura polifásica (`resample_poly` en Python, `resample` en Matlab).

**Python**

```python
import numpy as np
from scipy import signal

fs = 44_100                       # Hz (CD)
t = np.arange(0, 0.1, 1/fs)
x = np.sin(2*np.pi*1000*t)        # 1 kHz
# Diezmado por 4 con filtro antialiasing
x4 = signal.decimate(x, 4)        # fs/4
# Cambio racional 44,1 kHz -> 48 kHz
y = signal.resample_poly(x, 160, 147)
print(len(x), len(x4), len(y))
```

**Matlab**

```matlab
fs = 44100;                       % Hz (CD)
t = 0:1/fs:0.1-1/fs;
x = sin(2*pi*1000*t);             % 1 kHz
% Diezmado por 4 con filtro antialiasing
x4 = decimate(x, 4);              % fs/4
% Cambio racional 44,1 kHz -> 48 kHz
y = resample(x, 160, 147);
disp([length(x), length(x4), length(y)])
```

### 2.6.5 Conversores con sobremuestreo

Los conversores modernos muestrean mucho más deprisa de lo que exige el teorema y ajustan la frecuencia en digital:

- **A/D**: se muestrea a $Kf_s$ y se diezma por $K$. El filtro antialiasing analógico puede ser muy suave, porque su transición va de $f_m$ a $Kf_s - f_m$; el filtro abrupto lo hace el diezmador digital. Además, el ruido de cuantificación, de potencia $\Delta^2/12$, se reparte uniformemente en $[-\pi, \pi]$ y el filtro del diezmador solo deja pasar la fracción $1/K$:

$$\mathrm{SQNR} \approx 6{,}02\,b + 1{,}76 + 10\log_{10} K \ \mathrm{dB}.$$

  Cada factor 4 de sobremuestreo gana unos 6 dB, es decir, un bit.

- **D/A**: se interpola por $K$ antes del conversor. Las imágenes se alejan hasta $Kf_s$, el filtro de suavizado es sencillo y la caída del ZOH en la banda útil es casi nula.

Los conversores *sigma-delta* van más allá: un lazo de realimentación *conforma* el ruido de cuantificación para empujarlo fuera de la banda útil, de modo que con un cuantificador de un solo bit a varios megahercios se logran de 16 a 24 bits efectivos en la banda de audio.

Ejercicios 14–16. *Herramientas interactivas:* `T2-diezmado-interpolacion` y `T2-sobremuestreo`.

## Resumen del tema

- Muestreo periódico: $x[n] = x(nT_s)$; frecuencia discreta $\Omega = \omega T_s = 2\pi f/f_s$.
- Las frecuencias $\pm\omega_0 + k\omega_s$ dan las mismas muestras (aliasing). El alias fundamental es el único con $|\omega| \le \omega_s/2$.
- Espectro de la señal muestreada: $X_s(j\omega) = \frac{1}{T_s}\sum_k X(j(\omega - k\omega_s))$, y $X(e^{j\Omega}) = X_s(j\Omega/T_s)$.
- Teorema del muestreo: una señal de banda limitada a $\omega_m$ queda determinada por sus muestras si $\omega_s > 2\omega_m$. En la práctica, filtro antialiasing y margen.
- Reconstrucción ideal: filtro $T_s\Pi(\omega/2\omega_c)$ e interpolación con sinc. No es causal ni realizable.

- Retenedores: ZOH, $H_0 = T_s e^{-j\omega T_s/2}\,\mathrm{sinc}(\omega/\omega_s)$; FOH, $H_1 = T_s\,\mathrm{sinc}^2(\omega/\omega_s)$. Un D/A real usa ZOH, compensación y filtro de suavizado.
- Cuantificación uniforme: $\Delta = FS/2^b$; error uniforme de potencia $\Delta^2/12$; SQNR $\approx 6{,}02\,b + 1{,}76$ dB. Codificar $L$ niveles requiere $b \geq \log_2 L$ bits.
- Diezmado: $X_d = \frac{1}{M}\sum_k X(e^{j(\Omega-2\pi k)/M})$, sin aliasing si $\Omega_m < \pi/M$. Expansión: $X_e = X(e^{j\Omega L})$, con imágenes que elimina un filtro de ganancia $L$ y corte $\pi/L$. Cambio racional: interpolar, filtrar y diezmar.
- El sobremuestreo simplifica los filtros analógicos y mejora la SQNR en $10\log_{10} K$ dB.

## Bibliografía


- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulo de aplicaciones de la transformada de Fourier, apartados de muestreo. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.
- A. V. Oppenheim y A. S. Willsky, *Señales y sistemas*, 2.ª ed. Prentice Hall, 1997. Capítulo 7, la referencia para el muestreo, la reconstrucción y el muestreo de señales discretas.
- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos 1 (conversión A/D y D/A), 6 (muestreo y reconstrucción) y 11 (cambio de frecuencia de muestreo).
- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Capítulo 4, con el diezmado, la interpolación, el sobremuestreo y el ruido de cuantificación.


## 2.7 Ejercicios

Los ejercicios 1–6 corresponden a la parte 1 del tema (muestreo y aliasing), los ejercicios 7–10 a la parte 2 (reconstrucción), los ejercicios 11–13 a la parte 3 (cuantificación y codificación) y los ejercicios 14–16 a la parte 4 (muestreo de señales discretas). Se usa $\mathrm{sinc}(x) = \sin(\pi x)/(\pi x)$ y $\mathrm{Sa}(x) = \sin x/x$. Muchos resultados se pueden comprobar numéricamente con los ejemplos de `teoría/T2/codigo/`.

### Muestreo y aliasing

**Ejercicio 1** (frecuencias discretas) La señal $x(t) = 3\cos(50\pi t) + 10\sin(300\pi t) - \cos(100\pi t)$ ($t$ en segundos) se muestrea con $f_s = 200$ Hz.

a) ¿Cuál es la tasa de Nyquist de $x(t)$?

b) Obtén la secuencia $x[n]$ expresando cada término con su alias fundamental, $|\Omega| \leq \pi$.

c) ¿Qué señal se obtiene si se reconstruye $x[n]$ con el filtro ideal de corte $f_s/2$?

**Solución**

a) Las frecuencias son 25, 150 y 50 Hz. La máxima es $f_m = 150$ Hz, así que la tasa de Nyquist es $2f_m = 300$ Hz.

b) $\Omega = 2\pi f/f_s$ da $\pi/4$, $3\pi/2$ y $\pi/2$. La segunda se pliega a $3\pi/2 - 2\pi = -\pi/2$, y como el seno es impar,

$$x[n] = 3\cos\left(\tfrac{\pi}{4}n\right) - 10\sin\left(\tfrac{\pi}{2}n\right) - \cos\left(\tfrac{\pi}{2}n\right).$$

c) Cada $\Omega$ vuelve a $f = \Omega f_s/2\pi$: $y(t) = 3\cos(50\pi t) - 10\sin(100\pi t) - \cos(100\pi t)$. El tono de 150 Hz aparece en 50 Hz con el signo cambiado y se suma con el de 50 Hz original.

**Ejercicio 2** (alias fundamental) Las sinusoides $x(t) = \sin(2\pi f_0 t)$ con $f_0 = 200$, 800, 1200 y 1700 Hz se muestrean a $f_s = 1$ kHz. Para cada una, calcula la frecuencia discreta de su alias fundamental y la señal que se obtendría al reconstruir con el filtro ideal de corte $f_s/2$.

**Solución**

Con $\Omega_0 = 2\pi f_0/f_s$ llevada a $[-\pi, \pi)$:

| $f_0$ (Hz) | $\Omega_0$ | alias fundamental | señal reconstruida |
|---|---|---|---|
| 200 | $0{,}4\pi$ | $0{,}4\pi$ | $\sin(400\pi t)$ |
| 800 | $1{,}6\pi$ | $-0{,}4\pi$ | $-\sin(400\pi t)$ |
| 1200 | $2{,}4\pi$ | $0{,}4\pi$ | $\sin(400\pi t)$ |
| 1700 | $3{,}4\pi$ | $-0{,}6\pi$ | $-\sin(600\pi t)$ |

Las de 200, 800 y 1200 Hz dan las mismas muestras salvo el signo; la de 1700 Hz aparece en 300 Hz. Un alias negativo invierte el signo de un seno (no el de un coseno). Se comprueba con el ejemplo `alias_fundamental`.

**Ejercicio 3** (ancho de banda de señales derivadas) La señal $x(t)$ tiene banda limitada a $\omega_m$. Determina la frecuencia de muestreo mínima para muestrear sin aliasing:

a) $x(t) + x(t-1)$

b) $\dfrac{\mathrm{d}x(t)}{\mathrm{d}t}$

c) $x^2(t)$

d) $x(t)\cos(\omega_0 t)$, con $\omega_0 > \omega_m$

**Solución**

a) $X(j\omega)(1 + \mathrm{e}^{-j\omega})$: multiplicar por una función no añade frecuencias. La banda sigue siendo $\omega_m$ (como mucho, se anulan algunas): $\omega_s > 2\omega_m$.

b) $j\omega X(j\omega)$: misma banda, $\omega_s > 2\omega_m$.

c) El producto en el tiempo es una convolución en frecuencia, $\frac{1}{2\pi}X * X$, que ocupa $|\omega| \le 2\omega_m$: $\omega_s > 4\omega_m$.

d) $\frac{1}{2}\big[X(j(\omega-\omega_0)) + X(j(\omega+\omega_0))\big]$: el espectro se traslada a $\pm\omega_0$ y ocupa $\omega_0 - \omega_m \le |\omega| \le \omega_0 + \omega_m$. La frecuencia máxima es $\omega_0 + \omega_m$, así que el teorema del muestreo exige $\omega_s > 2(\omega_0 + \omega_m)$. *No* basta con $2\omega_m$: con esa frecuencia las réplicas de las bandas en $\pm\omega_0$ se solapan. (Para señales paso banda existe un muestreo paso banda que permite frecuencias menores con condiciones adicionales; queda fuera de este tema.)

**Ejercicio 4** (periodo de muestreo) Determina el periodo de muestreo necesario para poder reconstruir

las siguientes señales. Si no es posible evitar el aliasing, propón un criterio práctico.

a) $x_1(t) = \mathrm{e}^{-t}u(t)$ b) $x_2(t) = \mathrm{sinc}(f_c t)$ c) $x_3(t) = \mathrm{sinc}^2(f_c t)$

**Solución**

a) $X_1(j\omega) = 1/(1+j\omega)$ no se anula nunca: la señal no es de banda limitada y siempre hay aliasing. Como $|X_1| = 1/\sqrt{1+\omega^2}$ decae, se puede elegir $\omega_s/2$ donde $|X_1|$ sea despreciable. Por ejemplo, para que $|X_1| < 0{,}01\,|X_1(0)|$ hace falta $\omega > \sqrt{10^4 - 1} \approx 100$ rad/s, así que $\omega_s \approx 200$ rad/s y $T_s \approx 0{,}03$ s.

b) $\mathrm{sinc}(f_c t) \overset{\mathscr{F}}{\longleftrightarrow} \frac{1}{f_c}\Pi(f/f_c)$, que ocupa $|f| \le f_c/2$. La banda es $f_m = f_c/2$, no $f_c$: hace falta $f_s > f_c$, es decir, $T_s < 1/f_c$.

c) Su transformada es la convolución de dos pulsos, un triángulo que ocupa $|f| \le f_c$: $f_m = f_c$, $f_s > 2f_c$ y $T_s < 1/(2f_c)$. El ejemplo `espectro_muestreado` usa esta señal con $f_c = 1$ Hz y $f_s = 3$ Hz.

**Ejercicio 5** (espectro de la señal muestreada) Una señal $x(t)$ tiene espectro triangular: $X(f) = 1 - |f|/100$ para $|f| \le 100$ Hz y cero fuera.

a) Dibuja el espectro de $x_s(t)$ para $f_s = 300$ Hz y para $f_s = 150$ Hz.

b) Con $f_s = 150$ Hz, ¿en qué banda hay aliasing? ¿Qué forma tiene allí el espectro? ¿Qué banda se conserva intacta?

c) Si antes de muestrear a 150 Hz se aplica un filtro antialiasing ideal, ¿cuál debe ser su corte y qué se pierde?

**Solución**

a) $X_s(f) = f_s \sum_k X(f - k f_s)$: triángulos de altura $f_s$ centrados en $k f_s$. Con 300 Hz, cada triángulo ocupa $[k\cdot 300 - 100, k\cdot 300 + 100]$ y quedan huecos de 100 Hz entre ellos. Con 150 Hz, cada triángulo se solapa con sus vecinos.

b) La réplica en 150 Hz ocupa $[50, 250]$ y se solapa con la central en $50 \le f \le 100$. Allí la suma vale $f_s\big[(1 - \frac{f}{100}) + (1 - \frac{150-f}{100})\big] = 0{,}5\, f_s$, constante: el espectro se aplana. La banda $|f| < 50$ Hz se conserva intacta.

c) Corte en $f_s/2 = 75$ Hz. Entonces $X$ ocupa $|f| \le 75$, las réplicas empiezan en $150 - 75 = 75$ Hz y no se solapan. La banda $|f| < 75$ Hz se conserva y se pierde la de 75 a 100 Hz: es preferible perderla a que se mezcle con la banda útil.

**Ejercicio 6** (frecuencias en un electrocardiograma) Un electrocardiograma (ECG), con información útil entre 0,05 y 40 Hz, se muestrea a $f_s = 500$ Hz. La red eléctrica añade una interferencia de 50 Hz y sus armónicos.

a) ¿A qué frecuencias discretas $\Omega$ aparecen la interferencia y sus armónicos de 150 y 250 Hz?

b) Un filtro digital tiene un cero en $\Omega = 0{,}1\pi$. ¿Qué frecuencia en hercios elimina?

c) Si el mismo ECG se muestreara a 120 Hz sin filtro antialiasing, ¿dónde aparecería el armónico de 150 Hz? ¿Se podría eliminar después con un filtro digital?

**Solución**

a) $\Omega = 2\pi f/f_s$: 50 Hz en $\pi/5$, 150 Hz en $3\pi/5$ y 250 Hz en $\pi$, que es la frecuencia de Nyquist.

b) $f = \Omega f_s/2\pi = 0{,}05 \cdot 500 = 25$ Hz.

c) $150/120 = 1{,}25$, así que el entero más próximo es $k = 1$ y $f_a = |150 - 120| = 30$ Hz, es decir, $\Omega = \pi/2$. Cae dentro de la banda útil del ECG: un filtro digital que lo eliminara se llevaría también la parte de la señal en 30 Hz. Por eso el filtro antialiasing es imprescindible antes de muestrear.

## Reconstrucción

**Ejercicio 7** (reconstrucción ideal) La señal $x(t) = \cos(6\pi t) + \cos(14\pi t)$ se muestrea y se reconstruye con un filtro paso bajo ideal.

a) Con $f_s = 12$ Hz y corte $f_s/2$, ¿qué señal se obtiene?

b) ¿Cuál es la frecuencia de muestreo mínima para reconstruirla exactamente?

c) Con $f_s = 20$ Hz, ¿entre qué valores puede estar la frecuencia de corte del filtro?

**Solución**

a) Las frecuencias son 3 y 7 Hz. La de 7 Hz supera $f_s/2 = 6$ Hz y se pliega a $|7 - 12| = 5$ Hz; al ser un coseno no cambia de signo. Se obtiene $y(t) = \cos(6\pi t) + \cos(10\pi t)$, que tiene las mismas muestras que $x(t)$ a 12 Hz.

b) $f_s > 2 \cdot 7 = 14$ Hz.

c) Hay que dejar pasar 7 Hz y eliminar la réplica más baja, en $20 - 7 = 13$ Hz: $7 < f_c < 13$ Hz.

**Ejercicio 8** (retenedores de orden uno) Obtén la respuesta al impulso y la respuesta en frecuencia de los tres retenedores de orden uno:

a) El no causal, que une las muestras $x[n]$ y $x[n+1]$ con una recta.

b) El mismo retrasado una muestra, para hacerlo causal.

c) El causal predictivo, que prolonga la recta que pasa por $x[n-1]$ y $x[n]$.

Compara sus espectros con el del retenedor de orden cero. *Sugerencia*: escribe $h(t)$ como convolución o suma de pulsos rectangulares.

**Solución**

a) Una muestra aislada $x[0] = 1$ produce un triángulo que vale 1 en $t = 0$ y 0 en $t = \pm T_s$: $h_1(t) = \Lambda(t/T_s)$. Como $\Lambda(t/T_s) = \frac{1}{T_s}\Pi(t/T_s) * \Pi(t/T_s)$ y $\Pi(t/T_s) \overset{\mathscr{F}}{\longleftrightarrow} T_s \operatorname{sinc}(\omega/\omega_s)$,

$$H_1(j\omega) = \frac{1}{T_s}\big(T_s \operatorname{sinc}(\omega/\omega_s)\big)^2 = T_s \operatorname{sinc}^2\left(\frac{\omega}{\omega_s}\right).$$

Es real y no negativa (fase nula).

b) $h(t) = \Lambda\big(\frac{t-T_s}{T_s}\big)$, no nula en $[0, 2T_s]$, y $H(j\omega) = T_s\, e^{-j\omega T_s} \operatorname{sinc}^2(\omega/\omega_s)$: el mismo módulo con un retardo de una muestra.

c) $y(t) = x[n] + (x[n] - x[n-1])\frac{t-nT_s}{T_s}$ en $[nT_s, (n+1)T_s)$. La muestra $x[0] = 1$ interviene como $x[n]$ en $[0, T_s)$, con contribución $1 + t/T_s$, y como $x[n-1]$ en $[T_s, 2T_s)$, con $-(t - T_s)/T_s = 1 - t/T_s$:

$$h(t) = \begin{cases} 1 + t/T_s & 0 \le t < T_s \\ 1 - t/T_s & T_s \le t < 2T_s \\ 0 & \text{en otro caso} \end{cases}$$

Integrando, $H(j\omega) = T_s(1 + j\omega T_s)\, \mathrm{e}^{-j\omega T_s}\, \mathrm{sinc}^2(\omega/\omega_s)$ (comprobado simbólicamente).

El ZOH tiene $|H_0| = T_s|\mathrm{sinc}(\omega/\omega_s)|$, con réplicas que caen como $1/\omega$; los FOH caen como $1/\omega^2$ y las atenúan mejor. El predictivo tiene un factor $|1 + j\omega T_s|$ que realza las altas frecuencias: su salida tiene saltos.

**Ejercicio 9** (conversor D/A de audio) Un conversor D/A de audio trabaja a $f_s = 44{,}1$ kHz con un retenedor de orden cero.

a) ¿Cuánto atenúa el ZOH los tonos de 1, 10 y 20 kHz?

b) Para un tono de 20 kHz, ¿a qué frecuencia aparece la primera imagen y a qué nivel respecto al tono?

c) ¿Qué ganancia debe tener un filtro digital de compensación en 20 kHz?

d) Repite b) si antes del conversor se interpola por 4, de modo que el ZOH trabaja a 176,4 kHz.

**Solución**

$|H_0|/T_s = |\mathrm{sinc}(f/f_s)|$.

a) $-0{,}007$ dB en 1 kHz, $-0{,}75$ dB en 10 kHz y $-3{,}17$ dB en 20 kHz.

b) La réplica en $f_s$ de la componente en $-20$ kHz cae en $44{,}1 - 20 = 24{,}1$ kHz, atenuada $-4{,}79$ dB: solo 1,6 dB por debajo del tono. El ZOH no basta; hace falta un filtro de suavizado que pase 20 kHz y elimine 24,1 kHz, muy abrupto.

c) $+3{,}17$ dB, es decir, $1/\mathrm{sinc}(20/44{,}1) \approx 1{,}44$.

d) Ahora el tono se atenúa solo 0,18 dB y la primera imagen está en $176{,}4 - 20 = 156{,}4$ kHz, 17,9 dB por debajo del tono. El filtro analógico de suavizado tiene una transición de 20 a 156 kHz y puede ser muy sencillo.

**Ejercicio 10** (reconstrucción con retenedores) Muestrea las siguientes señales, eligiendo un valor adecuado de $T_s$. Indica en cada caso si es posible evitar el aliasing y estima el error de reconstruir con retenedores de orden cero y uno.

a) $x_1(t) = \sin\left(\frac{2\pi}{4}t\right)$

b) $x_2(t) = \sin\left(\frac{2\pi}{4}t\right)$ para $0 \le t \le 7$ y cero fuera

c) $x_3(t) = \sin\left(\frac{20\pi}{700}t\right)\cos\left(\frac{20\pi}{35}t\right)$

*Sugerencia*: para una señal suave, el error del ZOH es como mucho $\max|x'|\, T_s$ y el de la interpolación lineal, $\max|x''|\, T_s^2/8$. Comprueba los resultados modificando el ejemplo `retenedores`.

**Solución**

a) $f_0 = 0{,}25\,\text{Hz}$: basta $T_s < 2\,\text{s}$, aunque con $T_s$ cerca de ese límite los retenedores dan una reconstrucción muy pobre. Con $T_s = 0{,}25\,\text{s}$ (16 muestras por periodo) y $\omega_0 = \pi/2$, el error del ZOH es como mucho $\omega_0 T_s \approx 0{,}39$ y el del FOH, $\omega_0^2 T_s^2/8 \approx 0{,}019$; la simulación da 0,38 y 0,019. El ZOH necesita muchas más muestras para el mismo error.

b) Es una señal de duración finita, así que no tiene banda limitada: el aliasing no se puede evitar del todo. Su espectro está concentrado alrededor de 0,25 Hz y decae, de modo que con el mismo $T_s = 0{,}25\,\text{s}$ el aliasing es pequeño. Los errores son los del apartado anterior, salvo cerca de $t = 7$, donde la señal salta de $\sin(3{,}5\pi) = -1$ a 0 y cualquier retenedor comete un error del orden del salto.

c) Por la fórmula del producto, $x_3(t) = \frac{1}{2}\left[\sin\left(\frac{3\pi}{5}t\right) - \sin\left(\frac{19\pi}{35}t\right)\right]$, con frecuencias 0,3 y $19/70 \approx 0{,}27\,\text{Hz}$. Hace falta $f_s > 0{,}6\,\text{Hz}$, $T_s < 5/3\,\text{s}$. La envolvente es lenta (0,014 Hz) y la portadora, de $2/7\,\text{Hz}$; con $T_s = 0{,}2\,\text{s}$, unas 17 muestras por periodo de portadora, el FOH reproduce bien la señal.

## Cuantificación y codificación

**Ejercicio 11** (cuantificador de 3 bits) Un cuantificador de $b = 3$ bits tiene margen $[-1, 1)$.

a) Calcula $\Delta$ y los niveles.

b) Cuantifica $x[n] = 0{,}95\cos(\pi n/4)$ para $n = 0, \ldots, 7$ por redondeo y por truncamiento, y calcula el error.

c) Codifica en complemento a dos el resultado del redondeo.

**Solución**

a) $\Delta = 2/2^3 = 0{,}25$; los niveles son $k\Delta$ con $k = -4, \ldots, 3$, de $-1$ a 0,75.

b) Los valores son 0,95, 0,672, 0, −0,672, −0,95, −0,672, 0, 0,672. Para $n = 0, \ldots, 4$ (el resto se repite por simetría):

| $n$ | $x[n]$ | redondeo | $e$ | truncamiento | $e$ |
|---|---|---|---|---|---|
| 0 | 0,950 | 0,75 | −0,200 | 0,75 | −0,200 |
| 1 | 0,672 | 0,75 | 0,078 | 0,50 | −0,172 |
| 2 | 0 | 0 | 0 | 0 | 0 |
| 3 | −0,672 | −0,75 | −0,078 | −0,75 | −0,078 |
| 4 | −0,950 | −1,00 | −0,050 | −1,00 | −0,050 |

En $n = 0$ el redondeo daría 1,0, que no es un nivel: el cuantificador *satura* en 0,75 y el error, −0,2, supera $\Delta/2 = 0{,}125$. El margen bipolar es asimétrico: llega a $-1$ pero no a $+1$.

c) $k = 3, 3, 0, -3, -4, -3, 0, 3$: `011`, `011`, `000`, `101`, `100`, `101`, `000`, `011`.

**Ejercicio 12** (conversor A/D de un microcontrolador) Un conversor A/D de 10 bits admite una entrada entre 0 y $V_{\text{ref}}$, que se puede configurar en 1,1, 3,3 o 5 V.

a) Indica la resolución, el número de niveles y el rango dinámico de cada configuración.

b) ¿Qué configuración es la más adecuada si la entrada varía entre 0 y 3 V? ¿Y entre 0 y 5 V? ¿Y entre −2,5 y 2,5 V?

c) ¿Cuántos bits harían falta, en cada configuración, para una resolución de $10^{-4}$ V?

**Solución**

a) Siempre hay $2^{10} = 1024$ niveles y DR $= 20\log_{10} 1024 \approx 60{,}2$ dB. La resolución es $\Delta = V_{\text{ref}}/2^{10}$: 1,07 mV, 3,22 mV y 4,88 mV.

b) De 0 a 3 V, $V_{\text{ref}} = 3{,}3$ V: cubre la señal sin saturar y con la menor $\Delta$ posible (con 1,1 V saturaría y con 5 V se desperdiciaría el 40 % de los niveles). De 0 a 5 V, $V_{\text{ref}} = 5$ V. Entre −2,5 y 2,5 V ninguna sirve directamente: hay que sumar 2,5 V con un circuito de adaptación (o atenuar y desplazar) y usar $V_{\text{ref}} = 5$ V, o un conversor bipolar.

c) $b \geq \log_2(V_{\text{ref}}/10^{-4})$: 13,43, 15,01 y 15,61, es decir, 14, 16 y 16 bits.

**Ejercicio 13** (relación señal a ruido de cuantificación)

a) ¿Cuántos bits hacen falta para que una sinusoide que ocupa todo el margen tenga SQNR ≥ 80 dB?

b) ¿Y si su amplitud es solo la cuarta parte del margen?

c) ¿Qué SQNR tiene una grabación de 16 bits a fondo de escala?

d) Calcula la SQNR de una señal cuyas muestras se distribuyen uniformemente en todo el margen $[-V, V)$.

**Solución**

a) $6{,}02\,b + 1{,}76 \geq 80$ da $b \geq 12{,}997$: 13 bits.

b) La potencia baja en un factor 16, es decir, 12,04 dB, que hay que compensar: $b \geq (80 + 12{,}04 - 1{,}76)/6{,}02 = 14{,}997$, 15 bits.

c) $6{,}02 \cdot 16 + 1{,}76 \approx 98{,}1$ dB.

d) La potencia de una variable uniforme en $[-V, V)$ es $P_x = (2V)^2/12$, y $\sigma_e^2 = \Delta^2/12 = (2V)^2/(12 \cdot 2^{2b})$. Entonces SQNR $= 10\log_{10} 2^{2b} \approx 6{,}02\,b$ dB: 1,76 dB menos que la sinusoide, porque su factor de cresta es mayor.

## Muestreo de señales discretas

**Ejercicio 14** (diezmado) Sea $x[n] = \cos(0{,}2\pi n) + \cos(0{,}3\pi n)$.

a) ¿Cuál es el máximo factor de diezmado sin aliasing?

b) Obtén $x[2n]$ y $x[4n]$. ¿Hay aliasing en algún caso?

c) Antes de diezmar por 4 se aplica un paso bajo ideal de corte $\pi/4$. ¿Qué secuencia se obtiene?

**Solución**

a) $\Omega_m = 0{,}3\pi < \pi/M$ exige $M < 3{,}33$: $M = 3$, que da $\cos(0{,}6\pi n) + \cos(0{,}9\pi n)$.

b) $x[2n] = \cos(0{,}4\pi n) + \cos(0{,}6\pi n)$, sin aliasing. $x[4n] = \cos(0{,}8\pi n) + \cos(1{,}2\pi n)$, y $1{,}2\pi$ se

pliega a $-0{,}8\pi$: $x[4n] = 2\cos(0{,}8\pi n)$. Las dos componentes se han mezclado y ya no se pueden separar.

c) El filtro elimina $\cos(0{,}3\pi n)$, porque $0{,}3\pi > \pi/4$, y deja $\cos(0{,}2\pi n)$. Después del diezmado queda $\cos(0{,}8\pi n)$: se pierde una componente, pero la otra no se contamina.

**Ejercicio 15** (expansión e interpolación)

a) Se expande $x[n] = \cos(0{,}4\pi n)$ por $L = 2$. ¿Qué frecuencias contiene $x_e[n]$ en $[0, \pi]$? ¿Qué se obtiene tras un filtro ideal de ganancia 2 y corte $\pi/2$?

b) Una secuencia tiene espectro triangular de banda $\Omega_m = \pi/2$. Dibuja el espectro tras expandirla por $L = 3$ e indica dónde están las imágenes.

**Solución**

a) $x_e[n] = \cos(0{,}2\pi n) \cdot \frac{1+(-1)^n}{2}$, ya que vale $\cos(0{,}4\pi\, n/2)$ en los pares y 0 en los impares. Como $(-1)^n \cos(0{,}2\pi n) = \cos(1{,}2\pi n) = \cos(0{,}8\pi n)$,

$$x_e[n] = \tfrac{1}{2}\cos(0{,}2\pi n) + \tfrac{1}{2}\cos(0{,}8\pi n).$$

Es lo que predice $X(e^{j2\Omega})$: picos donde $2\Omega = \pm 0{,}4\pi + 2\pi k$, es decir, en $0{,}2\pi$ y en la imagen $0{,}8\pi$. El filtro elimina la imagen y duplica el resto: $y[n] = \cos(0{,}2\pi n)$, que es $x[n]$ a doble frecuencia de muestreo.

b) $X(e^{j3\Omega})$ repite el triángulo cada $2\pi/3$ con banda $\pi/6$: el original queda en $|\Omega| \le \pi/6$ y las imágenes, centradas en $\pm 2\pi/3$, ocupan $\pi/2 \le |\Omega| \le 5\pi/6$. El filtro de interpolación, con corte $\pi/3$ y ganancia 3, deja solo la banda central.

**Ejercicio 16** (cambio racional y sobremuestreo)

a) Diseña el esquema para pasar audio de 48 kHz a 32 kHz: factores $L$ y $M$, frecuencia intermedia y frecuencia de corte del filtro, en radianes por muestra y en hercios.

b) Un conversor A/D de 12 bits sobremuestrea por $K = 16$ y después diezma. ¿Qué SQNR alcanza para una sinusoide a fondo de escala? ¿A cuántos bits equivale?

c) ¿Qué factor de sobremuestreo haría falta para ganar 4 bits?

**Solución**

a) $32/48 = 2/3$: $L = 2$, $M = 3$. Se interpola a 96 kHz y se filtra con ganancia 2 y corte $\min(\pi/2, \pi/3) = \pi/3$, que a 96 kHz son $\frac{1}{6} \cdot 96 = 16$ kHz, la frecuencia de Nyquist de la salida. Después se diezma por 3.

b) $6{,}02 \cdot 12 + 1{,}76 + 10\log_{10} 16 = 72{,}0 + 12{,}0 \approx 86{,}0$ dB, unos 2 bits más: equivale a un conversor de 14 bits.

c) 4 bits son unos 24 dB, así que $10\log_{10} K = 24$ y $K \approx 256$. Ganar resolución solo con sobremuestreo es caro; los conversores sigma-delta lo consiguen con mucho menos gracias al conformado de ruido.

# 3 Transformada discreta de Fourier

**Cómo usar estos apuntes**

Estos apuntes desarrollan el contenido de las cuatro presentaciones del tema: las secciones 3.1 y 3.2 corresponden a la parte 1 (de la DTFT a la DFT), la sección 3.3 a la parte 2 (propiedades y convolución circular), la sección 3.4 a la parte 3 (análisis espectral) y la sección 3.5 a la parte 4 (FFT y convolución rápida). Los ejemplos de código están en `teoría/T3/codigo/`, en Python y en Matlab, y se pueden ejecutar tal cual. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

## 3.1 De la DTFT a la DFT

En los temas anteriores hemos descrito el contenido en frecuencia de una secuencia con su transformada de Fourier de tiempo discreto (DTFT),

$$X(\mathrm{e}^{j\Omega}) = \sum_{n=-\infty}^{\infty} x[n]\,\mathrm{e}^{-j\Omega n}, \qquad x[n] = \frac{1}{2\pi}\int_0^{2\pi} X(\mathrm{e}^{j\Omega})\,\mathrm{e}^{j\Omega n}\,\mathrm{d}\Omega.$$

$X(\mathrm{e}^{j\Omega})$ es una función continua de $\Omega$ y periódica de periodo $2\pi$; en el Tema 2 vimos que esa periodicidad es la huella del muestreo en el tiempo. La DTFT es la herramienta adecuada para razonar, pero no se puede calcular con un computador por dos motivos: la suma tiene infinitos términos, y el resultado es una función de variable continua. La transformada discreta de Fourier (DFT) resuelve los dos problemas: toma $N$ muestras en el tiempo y devuelve $N$ muestras en frecuencia. Esta sección estudia qué se pierde en cada paso.

### 3.1.1 Observar un número finito de muestras

En la práctica solo conocemos $N$ muestras de la señal. Observar $x[n]$ durante $0 \le n < N$ es multiplicarla por la *ventana rectangular*

$$w[n] = \begin{cases} 1 & 0 \le n < N \\ 0 & \text{en otro caso} \end{cases}, \qquad W(\mathrm{e}^{j\Omega}) = \sum_{n=0}^{N-1} \mathrm{e}^{-j\Omega n} = \frac{1-\mathrm{e}^{-j\Omega N}}{1-\mathrm{e}^{-j\Omega}} = \mathrm{e}^{-j\Omega\frac{N-1}{2}}\,\frac{\sin(\Omega N/2)}{\sin(\Omega/2)}.$$

La transformada de la ventana (figura 3.1) tiene un *lóbulo principal* centrado en $\Omega = 0$, de altura $N$ y anchura $4\pi/N$ entre sus primeros ceros, y *lóbulos secundarios* que decaen lentamente; el primero está solo 13 dB por debajo del principal.

Un producto en el tiempo es una convolución periódica en frecuencia:

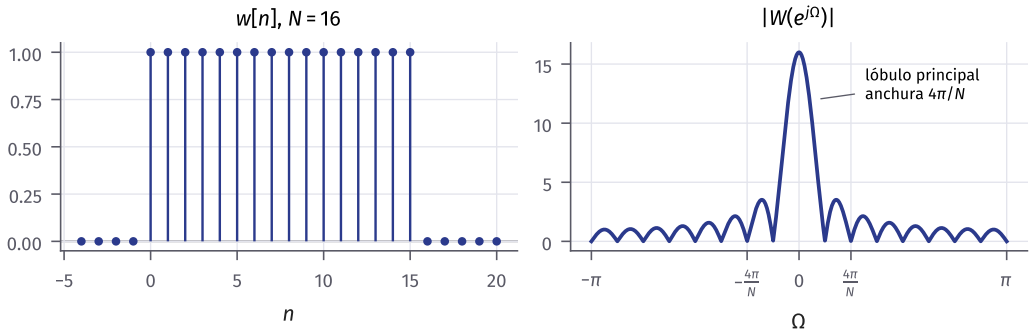



**Figura 3.1.** Ventana rectangular de $N = 16$ muestras y módulo de su DTFT.

**Propiedad 3.1.1: Efecto del truncamiento**

Si $x_N[n] = x[n]\, w[n]$,

$$X_N(\mathrm{e}^{j\Omega}) = \frac{1}{2\pi}\int_0^{2\pi} X(\mathrm{e}^{j\theta})\, W\big(\mathrm{e}^{j(\Omega-\theta)}\big)\, \mathrm{d}\theta.$$

Cada componente del espectro se sustituye por una copia de $W$. Por ejemplo, la DTFT de $\cos(\Omega_0 n)$ son dos impulsos en $\pm\Omega_0$; la del coseno truncado son dos copias de $W$ centradas en $\pm\Omega_0$ (figura 3.2). El truncamiento tiene dos efectos: *emborrona* el espectro, porque una línea se convierte en un lóbulo de anchura $4\pi/N$, y lo *contamina*, porque los lóbulos secundarios reparten energía por todas las frecuencias. Los dos disminuyen al aumentar $N$, y la apartado 3.4 muestra cómo controlarlos con otras ventanas.

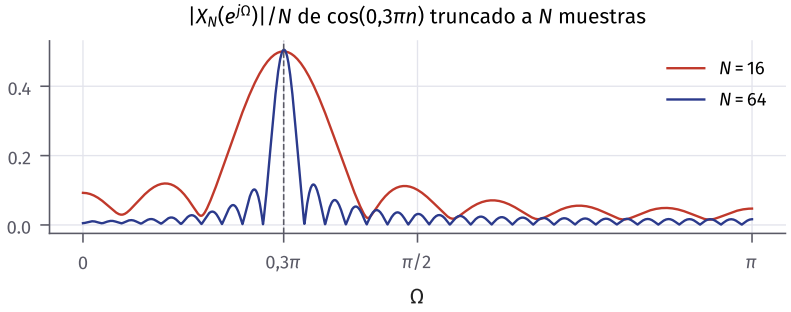



**Figura 3.2.** DTFT de un coseno truncado a $N = 16$ y $N = 64$ muestras.

### 3.1.2 Muestrear la DTFT

El segundo paso es quedarnos con $N$ valores de $X_N(\mathrm{e}^{j\Omega})$. Como es periódica, basta muestrear un periodo, $[0, 2\pi)$, en $N$ puntos equiespaciados:

$$\Omega_k = \frac{2\pi}{N}k, \qquad k = 0, 1, \ldots, N-1,$$

separados $\Delta\Omega = 2\pi/N$. Si la secuencia solo es no nula en $0 \le n < N$,

$$X[k] = X(\mathrm{e}^{j\Omega})\big|_{\Omega=2\pi k/N} = \sum_{n=0}^{N-1} x[n]\, \mathrm{e}^{-j\frac{2\pi}{N}kn}.$$

Esta es la DFT. La figura 3.3 muestra la DTFT de una secuencia de 5 muestras y sus muestras con $N = 8$ y $N = 16$: con más muestras se aprecia mejor la forma de la DTFT, pero la información es la misma, porque la secuencia es la misma.

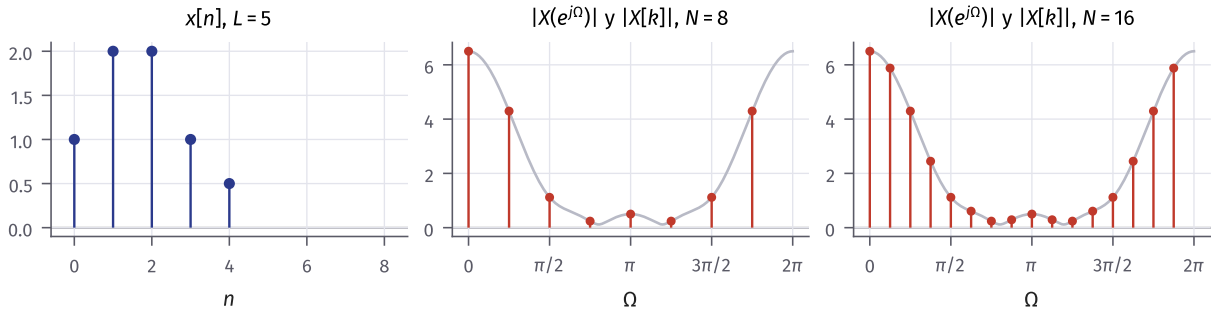



**Figura 3.3.** Una secuencia de longitud 5, su DTFT y la DFT de 8 y 16 puntos (rellenando con ceros).

## 3.2 La transformada discreta de Fourier

**Definición 3.2.1: DFT e IDFT**

La DFT de $N$ puntos de una secuencia $x[n]$, $n = 0, \dots, N-1$, es

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad k = 0, \dots, N-1,$$

con $W_N = \mathrm{e}^{-j2\pi/N}$. La transformada inversa (IDFT) es

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, W_N^{-kn}, \qquad n = 0, \dots, N-1.$$

La IDFT suma sobre $k$, lleva el factor $1/N$ y tiene el signo del exponente cambiado. Para comprobar que invierte la DFT basta la ortogonalidad de las exponenciales discretas:

$$\frac{1}{N} \sum_{k=0}^{N-1} W_N^{k(m-n)} = \begin{cases} 1 & m - n = 0, \pm N, \pm 2N, \dots \\ 0 & \text{en otro caso} \end{cases} \tag{3.1}$$

que es una suma geométrica de razón $W_N^{m-n}$: vale $N$ si la razón es 1 y $\frac{1-W_N^{N(m-n)}}{1-W_N^{m-n}} = 0$ si no. Sustituyendo la DFT en la IDFT,

$$\frac{1}{N} \sum_k X[k] W_N^{-kn} = \sum_m x[m]\, \frac{1}{N} \sum_k W_N^{k(m-n)} = x[n].$$

El factor $W_N$ es una raíz $N$-ésima de la unidad, y sus propiedades son la base de la FFT:

$$W_N^N = 1, \qquad W_N^{k+N} = W_N^k, \qquad W_N^{N/2} = -1, \qquad W_N^{k+N/2} = -W_N^k, \qquad W_N^2 = W_{N/2}.$$

### 3.2.1 Forma matricial

La DFT es una transformación lineal de $\mathbb{C}^N$ en $\mathbb{C}^N$: $\mathbf{X} = \mathbf{W}_N \mathbf{x}$, con $(\mathbf{W}_N)_{kn} = W_N^{kn}$:

$$\begin{pmatrix} X[0] \\ X[1] \\ \vdots \\ X[N-1] \end{pmatrix} = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 1 & W_N & \cdots & W_N^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & W_N^{N-1} & \cdots & W_N^{(N-1)(N-1)} \end{pmatrix} \begin{pmatrix} x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{pmatrix}.$$

La matriz es simétrica y, por (3.1), $\mathbf{W}_N^* \mathbf{W}_N = N\mathbf{I}$: sus columnas son ortogonales y la inversa es $\mathbf{W}_N^{-1} = \frac{1}{N}\mathbf{W}_N^*$. Calcular el producto directamente cuesta $N^2$ productos complejos; la FFT de la apartado 3.5 lo reduce a unos $\frac{N}{2}\log_2 N$.

**Ejemplo 3.2.2: DFT de cuatro puntos**

Con $N = 4$, $W_4 = \mathrm{e}^{-j\pi/2} = -j$ y

$$\mathbf{W}_4 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{pmatrix}.$$

Para $x = (1, 3, 4, 0)$: $X[0] = 8$, $X[1] = 1 - 3j - 4 = -3 - 3j$, $X[2] = 1 - 3 + 4 = 2$ y $X[3] = 1 + 3j - 4 = -3 + 3j$. $X[0]$ es la suma de las muestras (la componente continua multiplicada por $N$), y $X[3] = X^*[1]$ porque la secuencia es real.

**En código**

La DFT del ejemplo 3.2.2 como producto de matrices, su comparación con `fft` y la IDFT con $\frac{1}{N}\mathbf{W}_N^*$.

**Python**

```python
import numpy as np

x = np.array([1, 3, 4, 0])
N = len(x)
n = np.arange(N)
k = n.reshape(-1, 1)
W = np.exp(-2j*np.pi*k*n/N)      # matriz DFT
X = W @ x                        # O(N²)
print(np.round(X, 10))  # 8, -3-3j, 2, -3+3j
print(np.allclose(X, np.fft.fft(x)))
xr = (W.conj() @ X) / N          # IDFT
print(np.real_if_close(np.round(xr, 10)))
```

**Matlab**

```matlab
x = [1; 3; 4; 0];
N = length(x);
n = 0:N-1;
k = n.';
W = exp(-2j*pi*k*n/N);           % matriz DFT
X = W * x;                       % O(N²)
disp(X.')                % 8, -3-3j, 2, -3+3j
disp(max(abs(X - fft(x))))       % ≈ 0
xr = (W' * X) / N;               % IDFT
disp(real(xr).')
```

### 3.2.2 Extensión periódica y aliasing temporal

¿Qué ocurre si $x[n]$ es más larga que $N$, o infinita? Las muestras de la DTFT siguen estando definidas, pero agrupando la suma en bloques de $N$ y usando que $W_N^{k(n-mN)} = W_N^{kn}$,

$$X(\mathrm{e}^{j2\pi k/N}) = \sum_{n=-\infty}^{\infty} x[n] W_N^{kn} = \sum_{n=0}^{N-1} \Big[ \sum_{m=-\infty}^{\infty} x[n - mN] \Big] W_N^{kn}.$$

La IDFT de esas $N$ muestras no devuelve $x[n]$, sino un periodo de su *extensión periódica*.

**Propiedad 3.2.3: Muestreo en frecuencia**

La IDFT de $N$ muestras equiespaciadas de $X(\mathrm{e}^{j\Omega})$ es, para $0 \le n < N$,

$$\tilde{x}[n] = \sum_{m=-\infty}^{\infty} x[n - mN].$$

Si $x[n]$ tiene longitud finita $L$ y $N \ge L$, $\tilde{x}[n] = x[n]$ en $0 \le n < N$ y la secuencia se recupera exactamente.

Es el dual del teorema del muestreo: muestrear en el tiempo periodiza el espectro, y muestrear el espectro periodiza la secuencia. Si $N < L$, las copias desplazadas se solapan y se suman: es el *aliasing temporal* (figura 3.4).

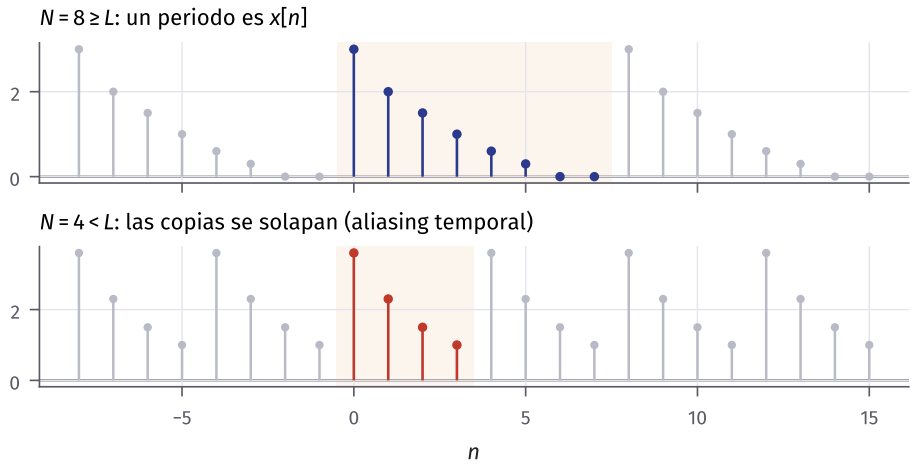



**Figura 3.4.** Extensión periódica de una secuencia de longitud 6 con $N = 8$ y con $N = 4$.

**Ejemplo 3.2.4: Aliasing temporal de una exponencial**

$x[n] = a^n u[n]$ con $|a| < 1$ tiene $X(e^{j\Omega}) = 1/(1 - ae^{-j\Omega})$. Muestreando en $N$ puntos, $X[k] = 1/(1 - aW_N^k)$, y la IDFT da

$$\tilde{x}[n] = \sum_{m\geq 0} a^{n+mN} = \frac{a^n}{1-a^N}, \qquad 0 \leq n < N.$$

Con $a = 0{,}5$ y $N = 4$, $\tilde{x} = (1{,}067;\ 0{,}533;\ 0{,}267;\ 0{,}133)$ en lugar de $(1;\ 0{,}5;\ 0{,}25;\ 0{,}125)$. El error es del orden de $a^N$ y desaparece al aumentar $N$.

### 3.2.3 Relación con la serie de Fourier discreta

Una secuencia periódica de periodo $N$ se desarrolla en serie de Fourier discreta (Tema 1):

$$\tilde{x}[n] = \sum_{k=0}^{N-1} c_k\, e^{j\frac{2\pi}{N}kn}, \qquad c_k = \frac{1}{N}\sum_{n=0}^{N-1} \tilde{x}[n]\, e^{-j\frac{2\pi}{N}kn}.$$

Comparando con la DFT de un periodo, $X[k] = N\, c_k$. La DFT trata la secuencia como *un periodo de una señal periódica*. Esta interpretación es la clave para entender sus propiedades: un desplazamiento saca muestras por un extremo y las mete por el otro, y la convolución que corresponde al producto de DFT es circular. Por eso los índices se entienden siempre módulo $N$, y escribiremos

$$((n))_N = n \bmod N \in \{0, \ldots, N-1\}.$$

### 3.2.4 Relación con la transformada Z

Como $X(e^{j\Omega}) = X(z)|_{z=e^{j\Omega}}$, la DFT son $N$ muestras de la transformada Z en puntos equiespaciados de la circunferencia unidad, $z_k = e^{j2\pi k/N} = W_N^{-k}$. Recíprocamente, si $x[n]$ tiene longitud $\leq N$, las $N$

muestras determinan $X(z)$ en todo el plano:

$$X(z) = \sum_{n=0}^{N-1} \Big[\frac{1}{N}\sum_{k} X[k]W_N^{-kn}\Big] z^{-n} = \frac{1-z^{-N}}{N}\sum_{k=0}^{N-1}\frac{X[k]}{1-W_N^{-k}z^{-1}},$$

sumando la serie geométrica en $n$. Esta fórmula es la base de los filtros diseñados por muestreo en frecuencia (Tema 4).

Ejercicios 1–4. *Herramientas interactivas:* `T3-dft` y `T3-extension-periodica`.

## 3.3 Propiedades de la DFT

Escribimos $x[n] \overset{\text{DFT}_N}{\longleftrightarrow} X[k]$. Si dos secuencias tienen longitudes distintas, se completa la más corta con ceros hasta $N$; la DFT de $N$ puntos de una secuencia de longitud $L < N$ son $N$ muestras de su DTFT (*relleno con ceros*). Las propiedades se demuestran directamente de la definición, y son las de la DTFT con los desplazamientos y las convoluciones convertidos en circulares. El cuadro 3.1 las resume.

### 3.3.1 Linealidad y desplazamientos circulares

La DFT es lineal: $a\,x_1[n] + b\,x_2[n] \leftrightarrow a\,X_1[k] + b\,X_2[k]$, siempre que las dos se calculen con la misma $N$. Tanto $X[k]$ como la secuencia que devuelve la IDFT son periódicas de periodo $N$.

**Propiedad 3.3.1: Desplazamiento circular**

$$x[((n-m))_N] \longleftrightarrow W_N^{km}X[k], \qquad W_N^{-ln}x[n] \longleftrightarrow X[((k-l))_N].$$

Para demostrar la primera, se sustituye $r = ((n-m))_N$ en la definición: como $W_N^{kn} = W_N^{k(r+m)}$ (la periodicidad absorbe el módulo), $\sum_n x[((n-m))_N]W_N^{kn} = W_N^{km}\sum_r x[r]W_N^{kr}$. El desplazamiento circular es un desplazamiento de la extensión periódica visto en un periodo: lo que sale por un extremo entra por el otro (figura 3.5). Como caso particular de la segunda, con $N$ par, $(-1)^n x[n] = W_N^{-nN/2}x[n] \leftrightarrow X[((k-N/2))_N]$: multiplicar por $(-1)^n$ intercambia las frecuencias bajas y las altas.

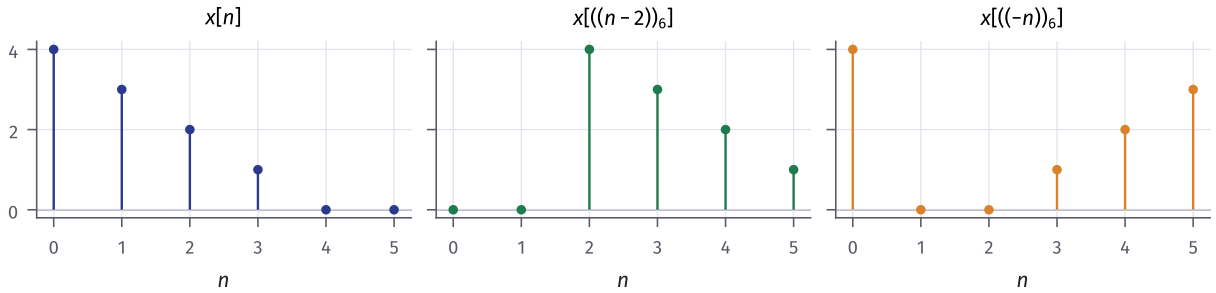



**Figura 3.5.** Una secuencia, su desplazamiento circular y su inversión circular con $N = 6$.

### 3.3.2 Inversión, conjugación, simetría y dualidad

La inversión circular $x[((-n))_N] = (x[0], x[N-1], \ldots, x[1])$ tiene DFT $X[((-k))_N]$, y la conjugada $x^*[n]$ tiene DFT $X^*[((-k))_N]$. De esta última se deduce la propiedad más usada en la práctica:

**Propiedad 3.3.2: Simetría hermítica**

Si $x[n]$ es real, $X[k] = X^*[((-k))_N] = X^*[N-k]$. Por tanto, $|X[k]| = |X[N-k]|$ y $\arg X[k] = -\arg X[N-k]$.

Consecuencias (figura 3.6):

- $X[0] = \sum_n x[n]$ es real y, si $N$ es par, también lo es $X[N/2] = \sum_n (-1)^n x[n]$.
- Las muestras $k > N/2$ son el espejo de las $k < N/2$ y corresponden a las frecuencias negativas $\Omega_k - 2\pi$. Basta calcular y representar $k = 0, \ldots, N/2$, que es lo que hacen `rfft` en Python y la mayoría de los analizadores.
- La parte par circular de $x$, $\frac{1}{2}(x[n] + x[((-n))_N])$, tiene DFT $\operatorname{Re} X[k]$, y la parte impar circular, $j \operatorname{Im} X[k]$. Una secuencia real y par circular tiene una DFT real.

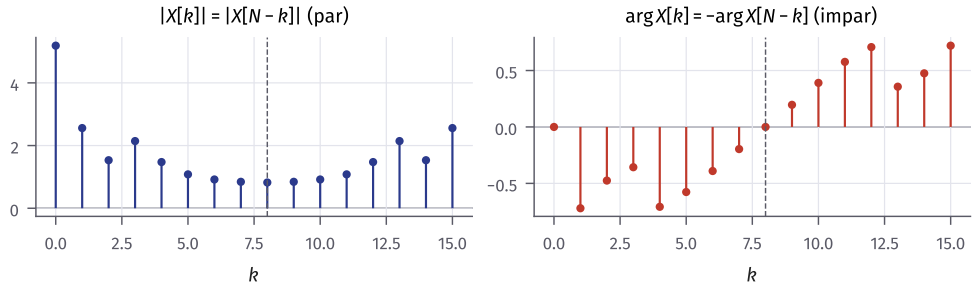



**Figura 3.6.** Módulo y fase de la DFT de una secuencia real.

Por último, la *dualidad*: si $x[n] \leftrightarrow X[k]$, la secuencia $X[n]$ tiene DFT $N\, x[((-k))_N]$. Por ejemplo, $\delta[n] \leftrightarrow 1$ y, por dualidad, $1 \leftrightarrow N\delta[k]$. Combinando con el desplazamiento, una exponencial que da un número entero $k_0$ de vueltas en $N$ muestras ocupa una sola muestra de la DFT:

$$e^{j\frac{2\pi}{N}k_0 n} \longleftrightarrow N\, \delta[((k-k_0))_N].$$

### 3.3.3 Identidad de Parseval

**Propiedad 3.3.3: Parseval**

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2.$$

Se demuestra sustituyendo $x^*[n] = \frac{1}{N}\sum_k X^*[k] W_N^{kn}$ en $\sum_n x[n]x^*[n]$ e intercambiando las sumas: $\frac{1}{N}\sum_k X^*[k] \sum_n x[n] W_N^{kn} = \frac{1}{N}\sum_k |X[k]|^2$. La energía de las $N$ muestras se reparte entre las frecuencias $\Omega_k$, con $|X[k]|^2/N$ en cada una. En la apartado 3.4.6 usaremos esta identidad para estimar potencias y densidades espectrales.

### 3.3.4 Convolución circular

**Definición 3.3.4: Convolución circular**

La convolución circular de $N$ puntos de dos secuencias de longitud $N$ es

$$(x_1 \circledast_N x_2)[n] = \sum_{m=0}^{N-1} x_1[m]\, x_2[((n-m))_N], \qquad n = 0, \dots, N-1.$$

**Propiedad 3.3.5: Convolución circular y producto de DFT**

$$(x_1 \circledast_N x_2)[n] \longleftrightarrow X_1[k]\, X_2[k], \qquad x_1[n]\, x_2[n] \longleftrightarrow \frac{1}{N}(X_1 \circledast_N X_2)[k].$$

Para demostrarlo, se calcula la IDFT del producto y se usa (3.1):

$$\frac{1}{N}\sum_k X_1[k]X_2[k]W_N^{-kn} = \sum_{m=0}^{N-1} x_1[m] \sum_{l=0}^{N-1} x_2[l]\, \frac{1}{N}\sum_{k=0}^{N-1} W_N^{-k(n-m-l)} = \sum_{m=0}^{N-1} x_1[m]\, x_2[((n-m))_N],$$

porque la suma interior vale 1 solo cuando $l = ((n-m))_N$.

### 3.3.5 Convolución circular y convolución lineal

La respuesta de un sistema LTI es una convolución *lineal*. Si $x[n]$ tiene longitud $L$ y $h[n]$ longitud $M$, $y = x * h$ tiene longitud $L+M-1$. Como el producto de DFT es una convolución circular, interesa saber cuándo coinciden. Rellenando las dos secuencias con ceros hasta $N$, la convolución circular es la lineal *plegada* con periodo $N$ (figura 3.7):

$$(x \circledast_N h)[n] = \sum_{r=-\infty}^{\infty} (x * h)[n - rN], \qquad 0 \le n < N.$$

**Propiedad 3.3.6: Convolución lineal con la DFT**

Si $N \ge L+M-1$, la convolución circular de $N$ puntos de $x$ y $h$, rellenadas con ceros, coincide con la convolución lineal:

$$x * h = \mathrm{IDFT}_N\left\{\mathrm{DFT}_N\{x\} \cdot \mathrm{DFT}_N\{h\}\right\}.$$

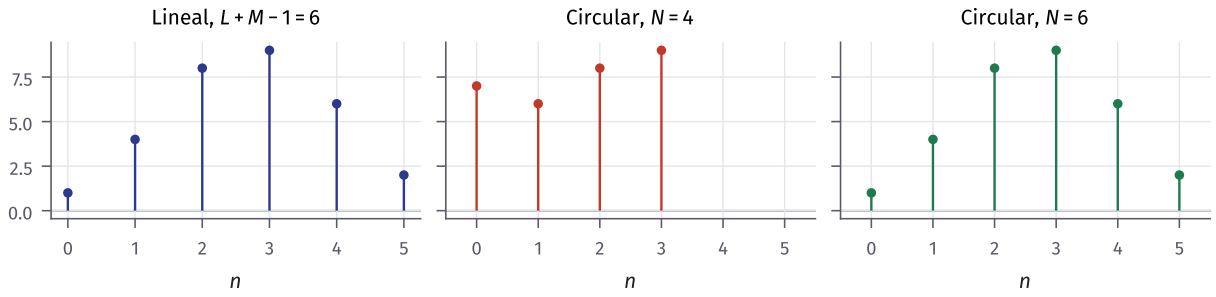



**Figura 3.7.** Convolución lineal de secuencias de longitudes 4 y 3, y convoluciones circulares de 4 y 6 puntos.

Si $N < L+M-1$, las muestras finales de la convolución lineal se suman a las primeras. Concretamente, si $L \le N < L+M-1$, solo se contaminan las $M-1$ primeras muestras; las restantes son correctas. Esta observación es la base del método de solapamiento y almacenamiento (apartado 3.5.5).

**Ejemplo 3.3.7: Respuesta de un FIR con la DFT**

Sean $h[n] = (0, 2)$ y $x[n] = (1, 3, 4)$. La convolución lineal es $y[n] = 2x[n-1] = (0, 2, 6, 8)$. Con $N = L + M - 1 = 4$:

$$X = (8,\ -3-3j,\ 2,\ -3+3j), \quad H = (2,\ -2j,\ -2,\ 2j), \quad Y = HX = (16,\ -6+6j,\ -4,\ -6-6j),$$

y la IDFT de $Y$ es $(0, 2, 6, 8)$; por ejemplo, $y[0] = \frac{1}{4}(16 - 6 + 6j - 4 - 6 - 6j) = 0$. Con $N = 3$ la muestra $y[3] = 8$ se pliega sobre $y[0]$ y se obtiene $(8, 2, 6)$.

**En código**

Convolución lineal y convolución circular de 3 y 4 puntos del ejemplo 3.3.7.

**Python**

```python
import numpy as np

x = np.array([1, 3, 4])
h = np.array([0, 2])
lin = np.convolve(x, h)      # [0, 2, 6, 8]
for N in (3, 4):             # N < 4 y N = 4
    Y = np.fft.fft(x, N) * np.fft.fft(h, N)
    y = np.real(np.fft.ifft(Y))
    print(N, np.round(y, 10))  # N=3: 8 2 6
print(lin)
```

**Matlab**

```matlab
x = [1 3 4];
h = [0 2];
lin = conv(x, h)             % [0 2 6 8]
for N = [3 4]                % N < 4 y N = 4
    Y = fft(x, N) .* fft(h, N);
    y = real(ifft(Y));
    disp([N, round(y)])      % N=3: 8 2 6
end
```

| Propiedad | Secuencia | DFT de $N$ puntos |
|---|---|---|
| Linealidad | $a\,x_1[n] + b\,x_2[n]$ | $a\,X_1[k] + b\,X_2[k]$ |
| Desplazamiento circular | $x[((n-m))_N]$ | $W_N^{km} X[k]$ |
| Modulación | $W_N^{-ln} x[n]$ | $X[((k-l))_N]$ |
| Inversión circular | $x[((-n))_N]$ | $X[((-k))_N]$ |
| Conjugación | $x^*[n]$ | $X^*[((-k))_N]$ |
| Señal real | $x[n] \in \mathbb{R}$ | $X[k] = X^*[N-k]$ |
| Parte par / impar (señal real) | $x_e[n]$, $x_o[n]$ | $\mathrm{Re}\,X[k]$, $j\,\mathrm{Im}\,X[k]$ |
| Dualidad | $X[n]$ | $N\,x[((-k))_N]$ |
| Convolución circular | $(x_1 \circledast_N x_2)[n]$ | $X_1[k]\,X_2[k]$ |
| Producto | $x_1[n]\,x_2[n]$ | $\frac{1}{N}(X_1 \circledast_N X_2)[k]$ |
| Parseval | $\sum_n \lvert x[n]\rvert^2$ | $\frac{1}{N}\sum_k \lvert X[k]\rvert^2$ |

**Cuadro 3.1.** Propiedades de la DFT.

Ejercicios 5–9. *Herramientas interactivas:* `T3-editor-dft`, `T3-propiedades-dft` y `T3-convolucion-circular`.

## 3.4 Análisis espectral con la DFT

El uso más frecuente de la DFT es estimar el contenido en frecuencia de una señal muestreada. Para hacerlo bien hay que saber leer el eje de frecuencias, entender qué hace el truncamiento y escalar correctamente amplitudes y potencias.

### 3.4.1 Eje de frecuencias

Si $x[n] = x(nT_s)$, la muestra $k$ de la DFT corresponde a la frecuencia discreta $\Omega_k = 2\pi k/N$ y, por $\Omega = \omega T_s$ (Tema 2), a

$$f_k = \frac{k}{N} f_s \text{ Hz}, \qquad \omega_k = \frac{2\pi k}{NT_s} \text{ rad/s}.$$

La separación entre muestras es

$$\Delta f = \frac{f_s}{N} = \frac{1}{NT_s} = \frac{1}{T_{\text{obs}}},$$

el inverso de la duración del registro. Las muestras $k = 0, \ldots, N/2$ cubren de 0 a $f_s/2$; las $k > N/2$ corresponden a frecuencias negativas, $f_k - f_s$. Para dibujar el espectro centrado en $[-f_s/2, f_s/2)$ se reordena con `fftshift` (figura 3.8).

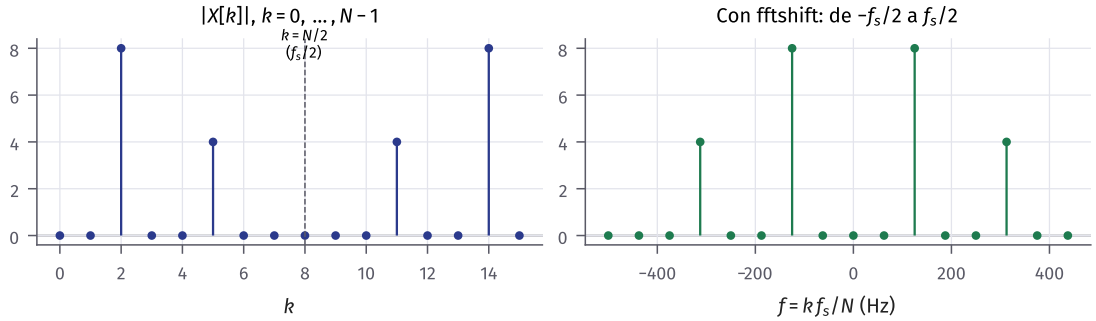



**Figura 3.8.** DFT de 16 puntos de dos tonos de 125 y 312,5 Hz muestreados a 1 kHz, frente a $k$ y frente a la frecuencia.

### 3.4.2 Amplitud de una sinusoide

Si $x[n] = A\cos(\Omega_0 n + \phi)$ y $\Omega_0 = 2\pi k_0/N$, la ventana contiene un número entero de periodos y, por la dualidad,

$$X[k_0] = \frac{AN}{2} e^{j\phi}, \qquad X[N - k_0] = \frac{AN}{2} e^{-j\phi},$$

con el resto de las muestras nulas. La amplitud se lee como $A = 2|X[k_0]|/N$ y la fase como $\arg X[k_0]$; la componente continua es $X[0]/N$ (sin el factor 2). Si $\Omega_0$ cae entre dos muestras, la amplitud se reparte entre varias: es la fuga espectral.

### 3.4.3 Fuga espectral

La DFT muestrea la DTFT de la señal *truncada*, que es la convolución del espectro real con $W(e^{j\Omega})$ (propiedad 3.1.1). Cuando $\Omega_0 = 2\pi k_0/N$, las demás muestras caen exactamente en los ceros de $W$ y la fuga no se ve. Cuando no, aparecen valores en todas las muestras, y el pico es menor que $AN/2$ (figura 3.9). En el peor caso, a media muestra, la pérdida es de 3,9 dB con la ventana rectangular. Con datos reales casi nunca hay un número entero de periodos en la ventana: la fuga es la norma.

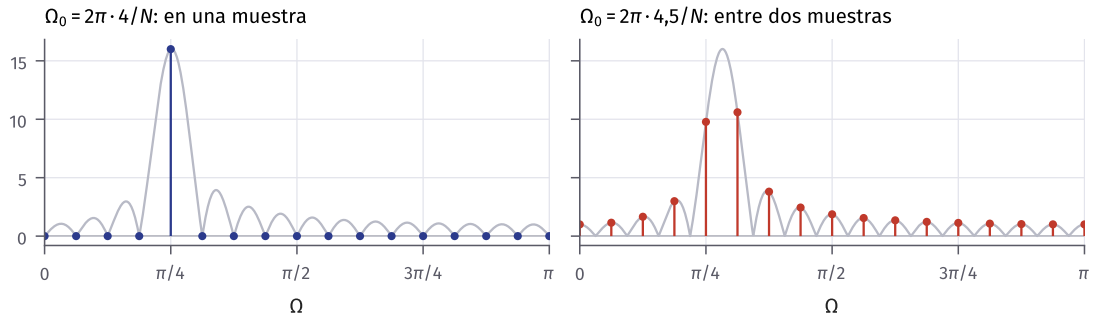



**Figura 3.9.** DFT de un coseno con un número entero de periodos en la ventana (izquierda) y con un número no entero (derecha).

### 3.4.4 Ventanas

Para reducir la fuga se multiplica la señal por una ventana $w[n]$ que va suavemente a cero en los extremos, y se calcula la DFT de $x[n]w[n]$. Las más usadas son

$$w[n] = a_0 - a_1 \cos\frac{2\pi n}{N-1} + a_2 \cos\frac{4\pi n}{N-1}, \qquad 0 \le n < N,$$

con $(a_0, a_1, a_2) = (0{,}5;\ 0{,}5;\ 0)$ para Hann, $(0{,}54;\ 0{,}46;\ 0)$ para Hamming y $(0{,}42;\ 0{,}5;\ 0{,}08)$ para Blackman. Todas tienen lóbulos secundarios mucho más bajos que la rectangular, a cambio de un lóbulo principal más ancho (figura 3.10 y cuadro 3.2).

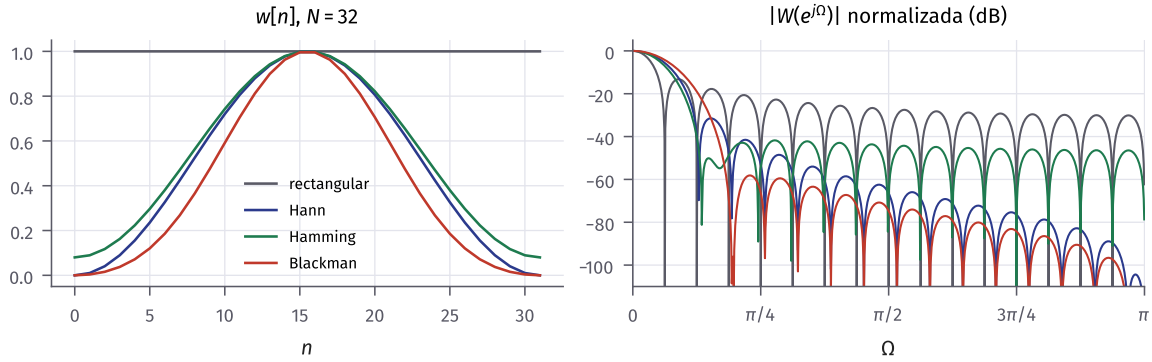



**Figura 3.10.** Ventanas de 32 muestras y módulo normalizado de su DTFT en decibelios.

| Ventana | lóbulo principal | primer lóbulo secundario | pérdida a media muestra |
|---|---|---|---|
| Rectangular | $4\pi/N$ | $-13$ dB | 3,9 dB |
| Hann | $8\pi/N$ | $-31$ dB | 1,4 dB |
| Hamming | $8\pi/N$ | $-42$ dB | 1,7 dB |
| Blackman | $12\pi/N$ | $-58$ dB | 1,1 dB |

**Cuadro 3.2.** Características de las ventanas más comunes (anchura del lóbulo principal entre ceros).

La ventana cambia la escala: para una sinusoide en una muestra, $|X_w[k_0]| = \frac{A}{2}\sum_n w[n]$, así que la amplitud se lee como $A = 2|X_w[k_0]|/\sum_n w[n]$. La figura 3.11 muestra para qué sirve una buena ventana: un tono 40 dB más débil que otro, a 80 Hz de distancia, queda oculto por la fuga de la ventana rectangular y se ve claramente con Hann o Blackman. Como regla práctica, Hann es una buena elección general; la rectangular solo conviene si la señal contiene un número entero de periodos o es un transitorio que cabe entero en la ventana.

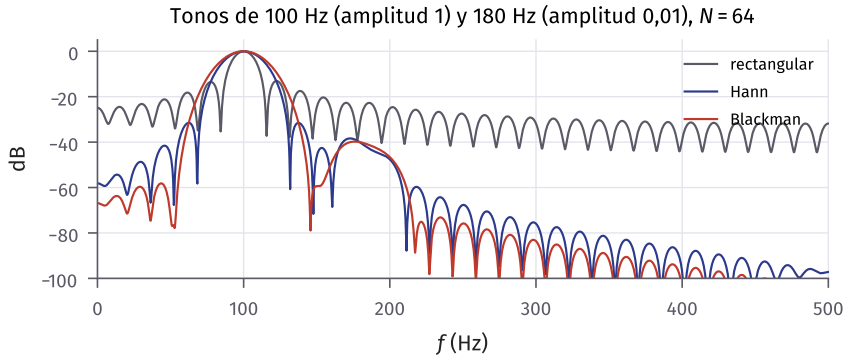



**Figura 3.11.** Espectro de dos tonos de amplitudes 1 y 0,01 con tres ventanas.

**En código**

Espectro de amplitud de dos tonos con ventana de Hann. Con $N = 200$ a 1 kHz, $\Delta f = 5$ Hz: el tono de 152 Hz cae entre dos muestras, pero la ventana limita la fuga.

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt

fs = 1000                       # Hz
N = 200                         # Δf = 5 Hz
n = np.arange(N)
x = 2*np.cos(2*np.pi*100*n/fs) \
    + 0.5*np.cos(2*np.pi*152*n/fs)
w = np.hanning(N)               # ventana
X = np.fft.rfft(x * w)          # k = 0..N/2
f = np.arange(len(X)) * fs / N  # Hz
A = 2 * np.abs(X) / np.sum(w)   # amplitud
print(f[np.argmax(A)], A.max()) # 100 Hz, 2
plt.plot(f, 20*np.log10(A + 1e-12))
plt.xlabel("f (Hz)"); plt.show()
```

**Matlab**

```matlab
fs = 1000;                      % Hz
N = 200;                        % Δf = 5 Hz
n = 0:N-1;
x = 2*cos(2*pi*100*n/fs) ...
    + 0.5*cos(2*pi*152*n/fs);
w = hanning(N).';               % ventana
X = fft(x .* w);
X = X(1:N/2+1);                 % k = 0..N/2
f = (0:N/2) * fs / N;           % Hz
A = 2 * abs(X) / sum(w);        % amplitud
[Am, i] = max(A);
disp([f(i), Am])                % 100 Hz, 2
plot(f, 20*log10(A + 1e-12))
xlabel('f (Hz)')
```

### 3.4.5 Resolución y relleno con ceros

Dos tonos se distinguen si sus lóbulos principales no se funden. Con la ventana rectangular, el lóbulo tiene semianchura $2\pi/N$, es decir, $f_s/N$ en hercios, y la condición aproximada es

$$|f_2 - f_1| \gtrsim \frac{f_s}{N} = \frac{1}{T_{\text{obs}}};$$

con Hann o Hamming hace falta aproximadamente el doble. La resolución en frecuencia está fijada por la *duración del registro*, no por la frecuencia de muestreo.

Rellenar la secuencia con ceros hasta $N' > N$ y calcular la DFT de $N'$ puntos da más muestras de la misma DTFT: interpola el espectro y facilita localizar los picos, pero *no mejora la resolución*, porque no añade información (figura 3.12). También se rellena con ceros para llegar a una longitud que sea potencia de 2 y la FFT sea más rápida.

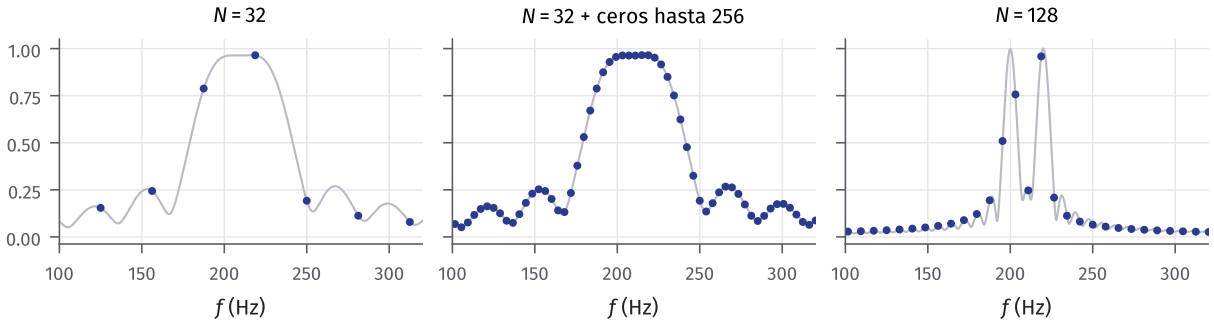



**Figura 3.12.** Tonos de 200 y 220 Hz a $f_s = 1$ kHz: con $N = 32$ no se separan, ni siquiera rellenando con ceros; con $N = 128$ sí.

### 3.4.6 Potencia y densidad espectral de potencia

Por Parseval, la potencia media de las $N$ muestras se reparte entre las frecuencias:

$$P = \frac{1}{N}\sum_{n=0}^{N-1}|x[n]|^2 = \sum_{k=0}^{N-1}\left|\frac{X[k]}{N}\right|^2.$$

- La *potencia en la muestra* $k$ es $|X[k]/N|^2$. Una sinusoide de amplitud $A$ en una muestra aporta $A^2/4$ en $k_0$ y otro tanto en $N-k_0$: en total, $A^2/2$, como debe ser. En decibelios, $P_k = 20\log_{10}(|X[k]|/N)$.
- La *densidad espectral de potencia* se estima con el *periodograma*,

$$\hat{\Phi}_{xx}(f_k) = \frac{|X[k]|^2}{N\,f_s},$$

  en unidades$^2$/Hz, de modo que $\sum_k \hat{\Phi}_{xx}(f_k)\,\Delta f = P$. En decibelios por hercio, $10\log_{10}\hat{\Phi}_{xx}$.

La diferencia importa con señales de banda ancha. Para un ruido blanco de varianza $\sigma^2$, $E|X[k]|^2 = N\sigma^2$: la potencia por muestra, $\sigma^2/N$, baja al aumentar $N$, pero la densidad, $\sigma^2/f_s$, no depende de $N$. Para un tono pasa lo contrario: su potencia está en una muestra y no depende de $N$. Por eso, al aumentar $N$, un tono sobresale más del ruido en la DFT.

Con ventana, la densidad se normaliza con $\sum w^2[n]$ en lugar de $N$: $\hat{\Phi}_{xx} = |X_w[k]|^2/(f_s\sum_n w^2[n])$. El periodograma de una señal aleatoria fluctúa mucho alrededor de su valor medio y no mejora al aumentar $N$; se reduce la varianza promediando los periodogramas de varios tramos (método de Welch, Tema 6).

### 3.4.7 Espectrograma

La DFT de una señal larga mezcla lo que ocurre en todos los instantes: no dice cuándo aparece cada frecuencia. Para señales cuyo contenido cambia con el tiempo (voz, música, vibraciones de una máquina que acelera) se usa la *transformada de Fourier de tiempo corto* (STFT), que calcula la DFT de tramos sucesivos con ventana:

$$X_m[k] = \sum_{n=0}^{N_w-1} x[n+mR]\,w[n]\,W_{N_w}^{kn}.$$

$N_w$ es la longitud de los tramos y $R$ el paso entre tramos; con $R < N_w$ se solapan, y lo habitual es un solape del 50 al 75 %. El *espectrograma* es la imagen de $|X_m[k]|^2$, con el tiempo $mRT_s$ en abscisas y la frecuencia $kf_s/N_w$ en ordenadas (figura 3.13).

La elección de $N_w$ es un compromiso: tramos largos dan buena resolución en frecuencia ($\Delta f = f_s/N_w$) pero mala en el tiempo, y tramos cortos al revés. No es una limitación del método sino del propio concepto de «frecuencia en un instante».

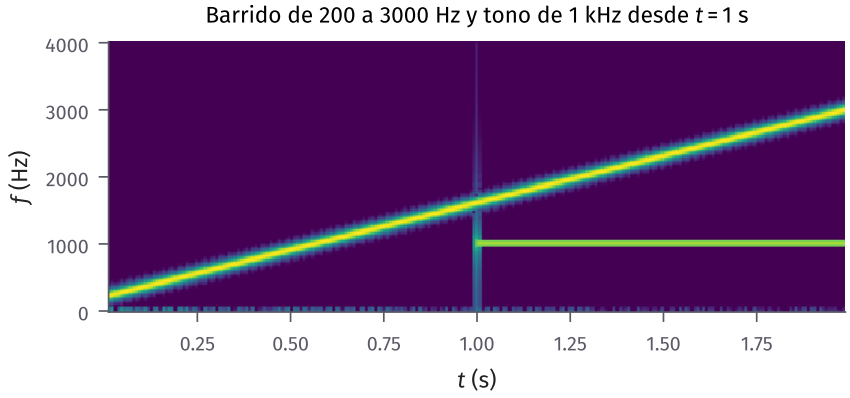



**Figura 3.13.** Espectrograma de un barrido de frecuencia y de un tono que empieza en $t = 1$ s ($N_w = 256$, solape del 75 %).

**En código**

STFT calculada a mano, tramo a tramo. `scipy.signal.spectrogram` y `spectrogram` de Matlab hacen lo mismo con más opciones.

**Python**

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

fs = 8000
t = np.arange(0, 2, 1/fs)
x = signal.chirp(t, 200, 2, 3000)
# STFT: tramos de 256, solape del 75 %
Nw, paso = 256, 64
w = np.hanning(Nw)
ini = range(0, len(x) - Nw, paso)
S = np.array([np.fft.rfft(w*x[i:i+Nw])
              for i in ini]).T
S = np.abs(S)
f = np.arange(Nw//2 + 1) * fs / Nw
tt = (np.array(ini) + Nw/2) / fs
print(S.shape)                  # (129, 121)
plt.pcolormesh(tt, f, 20*np.log10(S + 1e-9))
plt.xlabel("t (s)"); plt.ylabel("f (Hz)")
plt.show()
```

**Matlab**

```matlab
fs = 8000;
t = 0:1/fs:2-1/fs;
x = chirp(t, 200, 2, 3000);
% STFT: tramos de 256, solape del 75 %
Nw = 256; paso = 64;
w = hanning(Nw).';
ini = 1:paso:length(x)-Nw;
S = zeros(Nw/2+1, length(ini));
for m = 1:length(ini)
    X = fft(w .* x(ini(m):ini(m)+Nw-1));
    S(:, m) = abs(X(1:Nw/2+1));
end
f = (0:Nw/2) * fs / Nw;
tt = (ini - 1 + Nw/2) / fs;
disp(size(S))                   % 129 x 121
imagesc(tt, f, 20*log10(S + 1e-9)); axis xy
xlabel('t (s)'); ylabel('f (Hz)')
```

Ejercicios 10–13. *Herramientas interactivas:* `T3-dft` y `T3-ventanas`.

## 3.5 FFT y convolución rápida

### 3.5.1 Coste de la DFT directa

Calcular las $N$ muestras de la DFT con la definición cuesta $N^2$ productos complejos y $N(N-1)$ sumas. Con $N = 1024$ es un millón de productos por espectro. La *transformada rápida de Fourier* (FFT) es una familia de algoritmos que calculan exactamente la misma DFT con un número de operaciones proporcional a $N \log_2 N$. Su publicación por Cooley y Tukey en 1965 hizo práctico el procesado digital de señales; más tarde se descubrió que Gauss había usado la misma idea hacia 1805 para interpolar órbitas de asteroides.

### 3.5.2 Diezmado en el tiempo en base 2

Sea $N = 2^p$. Separamos las muestras de índice par, $n = 2r$, y las de índice impar, $n = 2r+1$:

$$X[k] = \sum_{r=0}^{N/2-1} x[2r]\, W_N^{2rk} + \sum_{r=0}^{N/2-1} x[2r+1]\, W_N^{(2r+1)k}.$$

Como $W_N^2 = W_{N/2}$,

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r]\, W_{N/2}^{rk}}_{F[k]} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1]\, W_{N/2}^{rk}}_{G[k]}, \qquad k = 0, \dots, N-1,$$

donde $F[k]$ y $G[k]$ son las DFT de $N/2$ puntos de las muestras pares e impares. Son periódicas de periodo $N/2$, y $W_N^{k+N/2} = -W_N^k$, así que basta calcularlas para $k = 0, \dots, N/2-1$:

**Propiedad 3.5.1: Mariposa del diezmado en el tiempo**

$$X[k] = F[k] + W_N^k\, G[k], \qquad X[k + N/2] = F[k] - W_N^k\, G[k], \qquad k = 0, \dots, \frac{N}{2} - 1.$$

Cada par de salidas cuesta un producto complejo ($W_N^k G[k]$) y dos sumas: es la *mariposa*. Las DFT de $N/2$ puntos se descomponen del mismo modo en DFT de $N/4$, y así hasta DFT de un punto, que son la propia muestra. Hay $\log_2 N$ etapas de $N/2$ mariposas, lo que da

$$\frac{N}{2} \log_2 N \text{ productos complejos} \qquad \text{y} \qquad N \log_2 N \text{ sumas.}$$

Para $N = 1024$ son 5120 productos en lugar de un millón (figura 3.14). Muchos de esos productos son por $W_N^0 = 1$ o por $W_N^{N/4} = -j$, que no cuestan nada, así que el número real es aún menor.

Al aplicar la separación en pares e impares repetidamente, la entrada queda en *orden de bits invertidos*: la muestra $n$ va a la posición cuyo índice en binario es el de $n$ leído al revés. Con $N = 8$, el orden es $x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]$.

**Ejemplo 3.5.2: FFT de cuatro puntos**

Para $x = (1, 3, 4, 0)$: las muestras pares son $(1, 4)$, con DFT $F = (5, -3)$; las impares, $(3, 0)$, con $G = (3, 3)$. Con $W_4^0 = 1$ y $W_4^1 = -j$:

$$X[0] = 5 + 3 = 8, \quad X[1] = -3 - 3j, \quad X[2] = 5 - 3 = 2, \quad X[3] = -3 + 3j,$$

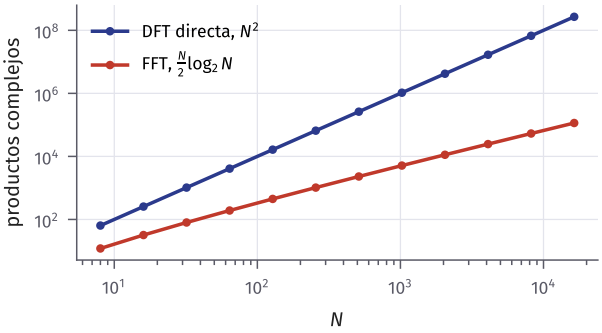



**Figura 3.14.** Productos complejos de la DFT directa y de la FFT en base 2.

como en el ejemplo 3.2.2, con 4 productos (dos de ellos triviales) en lugar de 16.

**En código**

FFT iterativa en base 2: reordenación por inversión de bits y $\log_2 N$ etapas de mariposas.

**Python**

```python
import numpy as np

rng = np.random.default_rng(0)
x = rng.standard_normal(1024)
N = len(x)
b = int(np.log2(N))             # N = 2^b
# Reordenación por inversión de bits
r = [int(f"{i:0{b}b}"[::-1], 2)
     for i in range(N)]
X = x[r].astype(complex)
m = 2                           # tamaño DFT
while m <= N:                   # b etapas
    B = X.reshape(N // m, m)    # bloques
    F, G = B[:, :m//2], B[:, m//2:]
    W = np.exp(-2j*np.pi*np.arange(m//2)/m)
    X = np.hstack([F + W*G, F - W*G]).ravel()
    m *= 2                      # mariposas
print(np.allclose(X, np.fft.fft(x)))
```

**Matlab**

```matlab
x = randn(1024, 1);
N = length(x);
% Reordenación por inversión de bits
X = x(bitrevorder(1:N));
m = 2;                          % tamaño DFT
while m <= N                % log2(N) etapas
    B = reshape(X, m, N/m);     % bloques
    F = B(1:m/2, :);
    G = B(m/2+1:end, :);
    W = exp(-2j*pi*(0:m/2-1).'/m);
    B = [F + W.*G; F - W.*G];   % mariposas
    X = B(:);
    m = 2*m;
end
disp(max(abs(X - fft(x))))      % ≈ 1e-12
```

### 3.5.3 Otras variantes

El *diezmado en frecuencia* separa las muestras pares e impares de $X[k]$ en lugar de las de $x[n]$, y deja la salida en orden de bits invertidos. Hay algoritmos en base 4 y de base mixta, y otros para cualquier $N$, incluso primo (Rader, Bluestein). Las bibliotecas actuales, como FFTW (la que usa Matlab) o la de NumPy, eligen el algoritmo según $N$; son más rápidas cuando $N$ es potencia de 2 o tiene factores pequeños. La IFFT no necesita un algoritmo propio:

$$x[n] = \frac{1}{N}\Big(\sum_k X^*[k]W_N^{kn}\Big)^* = \frac{1}{N}\big(\mathrm{FFT}\{X^*\}\big)^*,$$

y con señales reales se aprovecha la simetría hermítica para hacer la mitad de trabajo (`rfft`).

### 3.5.4 Convolución rápida

Por la propiedad 3.3.6, la convolución lineal de $x$ (longitud $L$) y $h$ (longitud $M$) se puede calcular rellenando ambas con ceros hasta $N \geq L + M - 1$, multiplicando sus FFT y calculando la IFFT. El coste es de tres FFT y $N$ productos, unos $\frac{3}{2} N \log_2 N + N$ productos complejos, frente a los $LM$ de la suma directa. Con $L = M = 1000$ y $N = 2048$, unos $3{,}6 \cdot 10^4$ frente a $10^6$. Para filtros muy cortos (unas decenas de coeficientes) la suma directa sigue siendo competitiva, sobre todo en un DSP con instrucciones de multiplicar y acumular. Si se filtran muchas señales con el mismo $h$, $H[k]$ se calcula una sola vez.

### 3.5.5 Señales largas: procesado por bloques

Si la entrada es muy larga o no termina (audio en tiempo real), no se puede esperar a tenerla entera. Se divide en bloques y se combinan los resultados.

**Solapamiento y suma (*overlap-add*).** Se divide la entrada en bloques de $L$ muestras, $x[n] = \sum_i x_i[n - iL]$. Por linealidad, $y = \sum_i (x_i * h)[n - iL]$. Cada $x_i * h$ se calcula por FFT de $N \geq L + M - 1$ puntos y tiene $L + M - 1$ muestras, así que las salidas de bloques consecutivos se solapan en $M - 1$ muestras, que se suman (figura 3.15).

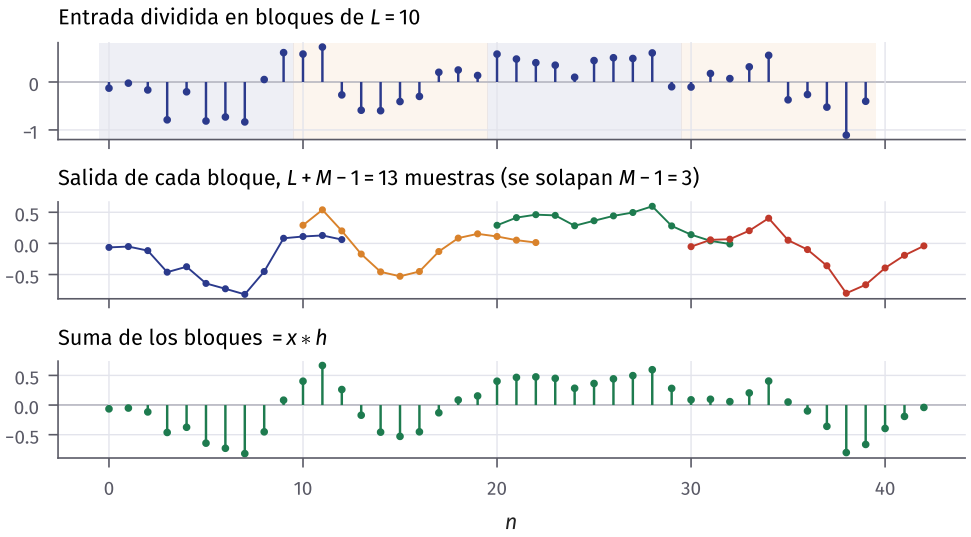



**Figura 3.15.** Solapamiento y suma con bloques de 10 muestras y un filtro de 4 coeficientes.

**Solapamiento y almacenamiento (*overlap-save*).** Se toman bloques de entrada de $N$ muestras que se solapan $M-1$ con el anterior, y se calcula su convolución *circular* de $N$ puntos con $h$. Según la apartado 3.3.5, las $M - 1$ primeras muestras están contaminadas por el plegado circular y se descartan; las $N - M + 1$ restantes coinciden con la convolución lineal y se guardan.

Los dos métodos dan el mismo resultado con un coste parecido, y el retardo es de un bloque. Los implementan `oaconvolve` (de `scipy.signal`) y `fftfilt` (de Matlab).

**En código**

Solapamiento y suma con bloques de 226 muestras, un FIR de 31 coeficientes y FFT de $N = 256$. El resultado coincide con la convolución directa.

**Python**

```python
import numpy as np

rng = np.random.default_rng(1)
x = rng.standard_normal(10_000)
h = np.ones(31) / 31           # FIR, M = 31
L = 226                         # bloque
M = len(h)
N = L + M - 1                   # 256 = 2^8
H = np.fft.fft(h, N)
y = np.zeros(len(x) + M - 1)
for i in range(0, len(x), L):
    xb = x[i:i+L]
    Yb = np.fft.fft(xb, N) * H
    yb = np.real(np.fft.ifft(Yb))
    n = min(N, len(y) - i)
    y[i:i+n] += yb[:n]      # solapa y suma
print(np.allclose(y, np.convolve(x, h)))
```

**Matlab**

```matlab
x = randn(1, 10000);
h = ones(1, 31) / 31;          % FIR, M = 31
L = 226;                        % bloque
M = length(h);
N = L + M - 1;                  % 256 = 2^8
H = fft(h, N);
y = zeros(1, length(x) + M - 1);
for i = 1:L:length(x)
    xb = x(i:min(i+L-1, end));
    yb = real(ifft(fft(xb, N) .* H));
    n = min(N, length(y) - i + 1);
    y(i:i+n-1) = y(i:i+n-1) + yb(1:n);
end
disp(max(abs(y - conv(x, h))))  % ≈ 1e-15
```

### 3.5.6 Sistemas IIR

Si $h[n]$ tiene duración infinita, las muestras $H[k] = H(\mathrm{e}^{j2\pi k/N})$ existen, pero su IDFT es la extensión periódica $h_N[n] = \sum_r h[n - rN]$ (propiedad 3.2.3). Para $h[n] = a^n u[n]$ es $h_N[n] = a^n/(1 - a^N)$ (ejemplo 3.2.4), y la DFT filtra con un error del orden de $a^N$ (figura 3.16). Con $N$ suficientemente grande el error es despreciable, y por eso el filtrado en frecuencia con la FFT se usa también con sistemas IIR o con respuestas medidas, que son largas. Pero lo exacto, y normalmente mucho más barato, es implementar el sistema con su ecuación en diferencias (Tema 4).

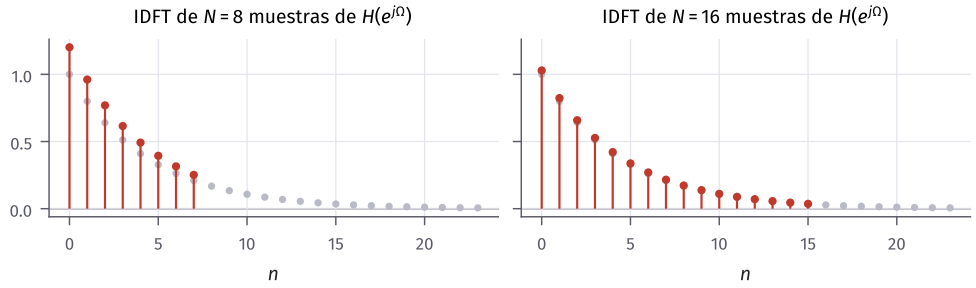



**Figura 3.16.** IDFT de 8 y 16 muestras de $H(\mathrm{e}^{j\Omega}) = 1/(1 - 0{,}8\mathrm{e}^{-j\Omega})$ frente a $h[n] = 0{,}8^n u[n]$.

Ejercicios 14–16. *Herramientas interactivas:* `T3-fft-mariposa` y `T3-solapamiento-suma`.

## Resumen del tema

- Observar $N$ muestras es enventanar: el espectro se convoluciona con $W(\mathrm{e}^{j\Omega})$, de lóbulo principal $4\pi/N$.
- DFT: $X[k] = \sum_{n=0}^{N-1} x[n]W_N^{kn}$; IDFT: $x[n] = \frac{1}{N}\sum_{k=0}^{N-1} X[k]W_N^{-kn}$; $W_N = \mathrm{e}^{-j2\pi/N}$. $X[k]$ es la DTFT en $2\pi k/N$.

- Muestrear en frecuencia periodiza en el tiempo; sin aliasing temporal si $N \geq L$. $X[k] = Nc_k$ y los índices se toman módulo $N$.
- Propiedades circulares; señal real: $X[k] = X^*[N-k]$; Parseval: $\sum|x|^2 = \frac{1}{N}\sum|X|^2$.
- El producto de DFT es una convolución circular, que coincide con la lineal si $N \geq L + M - 1$.
- $f_k = kf_s/N$, $\Delta f = 1/T_{\text{obs}}$; amplitud $2|X[k_0]|/N$; potencia por muestra $|X[k]/N|^2$; periodograma $|X[k]|^2/(Nf_s)$.
- Fuga espectral y ventanas: menos fuga a cambio de un lóbulo principal más ancho. El relleno con ceros interpola pero no resuelve.
- Espectrograma: DFT de tramos con ventana, con un compromiso entre resolución en tiempo y en frecuencia.
- FFT: $X[k] = F[k] + W_N^k G[k]$, $X[k+N/2] = F[k] - W_N^k G[k]$; $\frac{N}{2}\log_2 N$ productos.
- Convolución rápida con FFT; señales largas por solapamiento y suma o almacenamiento.

## Bibliografía


- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos 7 (DFT) y 8 (FFT).
- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Capítulos 8 (DFT), 9 (FFT) y 10 (análisis de Fourier con la DFT).
- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulo de señales en tiempo discreto, apartados de DFT y FFT. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.
- V. K. Ingle y J. G. Proakis, *Digital Signal Processing Using MATLAB*, 3.ª ed. Cengage Learning, 2012. Capítulo 5, con ejemplos en Matlab.
- F. J. Harris, «On the use of windows for harmonic analysis with the discrete Fourier transform», *Proceedings of the IEEE*, 66(1), 1978. La referencia clásica sobre ventanas.


## 3.6 Ejercicios

Los ejercicios 1–4 corresponden a la parte 1 del tema (de la DTFT a la DFT), los ejercicios 5–9 a la parte 2 (propiedades y convolución circular), los ejercicios 10–13 a la parte 3 (análisis espectral) y los ejercicios 14–16 a la parte 4 (FFT y convolución rápida). Se usa $W_N = \mathrm{e}^{-j2\pi/N}$ y $((n))_N = n \bmod N$. Las secuencias se dan como listas que empiezan en $n = 0$. Muchos resultados se pueden comprobar con `fft` y con los ejemplos de `teoría/T3/codigo/`.

## De la DTFT a la DFT

**Ejercicio 1** (DFT a mano) Calcula la DFT de 4 puntos de $x = (1, 2, 0, 1)$ con la matriz $\mathbf{W}_4$, y comprueba el resultado aplicando la IDFT.

**Solución**

Con $W_4 = -j$:

$$\begin{pmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 1-j \\ -2 \\ 1+j \end{pmatrix}.$$

$X[3] = X^*[1]$ porque $x$ es real. La IDFT, $\frac{1}{4}\mathbf{W}_4^*\mathbf{X}$, da por ejemplo $x[0] = \frac{1}{4}(4 + 1 - j - 2 + 1 + j) = 1$ y $x[1] = \frac{1}{4}\big(4 + j(1-j) + 2 - j(1+j)\big) = \frac{1}{4}(4 + 1 + j + 2 + 1 - j) = 2$.

**Ejercicio 2** (DFT de secuencias básicas) Calcula, sin hacer las sumas término a término, la DFT de $N = 8$ puntos de:

a) $\delta[n]$

b) $\delta[n-2]$

c) 1 para todo $n$

d) $\cos(\pi n/2)$

e) $e^{j\pi n/4}$

f) $(-1)^n$

**Solución**

a) $X[k] = 1$ para todo $k$.

b) $X[k] = W_8^{2k} = (-j)^k = (1, -j, -1, j, 1, -j, -1, j)$.

c) $X[k] = 8\,\delta[k]$: solo la componente continua.

d) $\cos(\pi n/2) = \frac{1}{2}\big(e^{j\frac{2\pi}{8}2n} + e^{-j\frac{2\pi}{8}2n}\big)$ da dos vueltas en 8 muestras: $X[2] = X[6] = 4$ y el resto, cero.

e) Da una vuelta: $X[k] = 8\,\delta[k-1]$.

f) $(-1)^n = e^{j\frac{2\pi}{8}4n}$: $X[k] = 8\,\delta[k-4]$, la frecuencia $\Omega = \pi$.

**Ejercicio 3** (aliasing temporal) La secuencia $x = (1, 2, 3, 4, 5, 6)$ tiene DTFT $X(e^{j\Omega})$. Se toman $N = 4$ muestras equiespaciadas de $X(e^{j\Omega})$ en $[0, 2\pi)$ y se calcula su IDFT.

a) ¿Qué secuencia se obtiene?

b) ¿Cuál es el menor $N$ que permite recuperar $x[n]$?

**Solución**

a) Un periodo de la extensión periódica de periodo 4: $\tilde{x}[n] = x[n] + x[n+4]$, es decir, $(1+5,\ 2+6,\ 3,\ 4) = (6, 8, 3, 4)$.

b) $N \geq L = 6$.

**Ejercicio 4** (DFT de una secuencia infinita) Sea $x[n] = 0{,}5^n u[n]$.

a) Obtén $X[k]$, las $N = 4$ muestras de su DTFT en $\Omega_k = 2\pi k/4$.

b) Calcula la IDFT de esas muestras y compárala con $x[n]$ para $n = 0, \dots, 3$.

c) ¿Qué $N$ hace falta para que el error sea menor que $10^{-3}$?

**Solución**

a) $X[k] = \dfrac{1}{1 - 0{,}5W_4^k}$: $X = (2;\ 0{,}8 - 0{,}4j;\ 0{,}667;\ 0{,}8 + 0{,}4j)$.

b) $\tilde{x}[n] = \sum_m x[n+4m] = \frac{0{,}5^n}{1-0{,}5^4} = 1{,}0667 \cdot 0{,}5^n$: $(1{,}067;\ 0{,}533;\ 0{,}267;\ 0{,}133)$, cada valor un 6,7 % mayor que $x[n]$ por el aliasing temporal.

c) El error relativo es $\frac{0{,}5^N}{1-0{,}5^N} \approx 0{,}5^N$, y el absoluto, como mucho en $n = 0$, lo mismo. $0{,}5^N < 10^{-3}$ exige $N \geq 10$.

## Propiedades y convolución circular

**Ejercicio 5** (simetría y Parseval) Una secuencia real de longitud 8 tiene $X[0] = 10$, $X[1] = 2 + j$, $X[2] = -1 - 2j$, $X[3] = 3$ y $X[4] = -2$.

a) Completa $X[5]$, $X[6]$ y $X[7]$.

b) Calcula $x[0]$ y $\sum_n x[n]$ sin calcular la IDFT completa.

c) Calcula la energía $\sum_n |x[n]|^2$.

**Solución**

a) $X[k] = X^*[8-k]$: $X[5] = X^*[3] = 3$, $X[6] = X^*[2] = -1 + 2j$, $X[7] = X^*[1] = 2 - j$.

b) $\sum_n x[n] = X[0] = 10$. $x[0] = \frac{1}{8}\sum_k X[k] = \frac{1}{8}(10+2+j-1-2j+3-2+3-1+2j+2-j) = \frac{16}{8} = 2$.

c) Por Parseval, $\frac{1}{8}\sum_k |X[k]|^2 = \frac{1}{8}(100 + 5 + 5 + 9 + 4 + 9 + 5 + 5) = \frac{142}{8} = 17{,}75$.

**Ejercicio 6** (desplazamiento y modulación) Para $x = (1, 2, 0, 1)$, con la DFT del ejercicio 1, obtén sin calcular ninguna DFT nueva:

a) la DFT de $y[n] = x[((n-2))_4]$;

b) la DFT de $z[n] = (-1)^n x[n]$;

c) la secuencia cuya DFT es $X[((-k))_4]$.

**Solución**

a) $y = (0, 1, 1, 2)$ y $Y[k] = W_4^{2k} X[k] = (-1)^k X[k] = (4,\ -1 + j,\ -2,\ -1 - j)$.

b) $(-1)^n = W_4^{-2n}$, así que $Z[k] = X[((k-2))_4] = (X[2], X[3], X[0], X[1]) = (-2,\ 1+j,\ 4,\ 1-j)$; $z = (1, -2, 0, -1)$.

c) La inversión circular $x[((-n))_4] = (x[0], x[3], x[2], x[1]) = (1, 1, 0, 2)$. Como $x$ es real, $X[((-k))_4] = X^*[k]$.

**Ejercicio 7** (convolución circular a mano) Sean $x = (1, 2, 3, 4)$ y $h = (1, 1)$.

a) Calcula la convolución lineal $x * h$.

b) Calcula la convolución circular de 4 puntos y compárala con la lineal.

c) ¿Qué $N$ hace falta para que coincidan?

**Solución**

a) $y[n] = x[n] + x[n-1] = (1, 3, 5, 7, 4)$.

b) $(x \circledast_4 h)[n] = x[n] + x[((n-1))_4]$: $(1+4,\ 2+1,\ 3+2,\ 4+3) = (5, 3, 5, 7)$. Es la lineal plegada con periodo 4: la muestra $y[4] = 4$ se suma a $y[0] = 1$.

c) $N \geq L + M - 1 = 5$.

**Ejercicio 8** (longitud de la DFT para filtrar) Una señal de $L = 100$ muestras se filtra con un FIR de $M = 30$ coeficientes usando la DFT.

a) ¿Cuál es el menor $N$ que da la convolución lineal? ¿Y la menor potencia de 2?

b) Si por error se usa $N = 128$, ¿qué muestras de la salida son incorrectas?

c) Si se usa $N = 100$, ¿qué muestras de la salida son correctas?

**Solución**

a) La salida tiene $L + M - 1 = 129$ muestras: $N \geq 129$, y la menor potencia de 2 es 256.

b) La circular de 128 puntos es $y[n] + y[n + 128]$, y solo $y[128]$ cae fuera: se suma a $y[0]$. Solo $y[0]$ es incorrecta.

c) Ahora $y_c[n] = y[n] + y[n + 100]$ con $y[n + 100] \neq 0$ para $n + 100 \leq 128$, es decir, $n \leq 28$. Son correctas las muestras $n = 29, \ldots, 99$: todas menos las $M - 1 = 29$ primeras. Es el fundamento del solapamiento y almacenamiento.

**Ejercicio 9** (secuencias pares)

a) Calcula la DFT de 4 puntos de $x = (4, 1, 0, 1)$ y explica por qué es real.

b) Demuestra que si $x[n]$ es real y par circular ($x[n] = x[((-n))_N]$), su DFT es real y par.

**Solución**

a) $X[k] = 4 + W_4^k + W_4^{3k} = 4 + 2\cos(\pi k/2)$: $X = (6, 4, 2, 4)$. Es real porque $x[1] = x[3]$: la secuencia es par circular.

b) Por la inversión, $x[((-n))_N] \leftrightarrow X[((-k))_N]$, así que $X[k] = X[((-k))_N]$ (par). Por ser real, $X[k] = X^*[((-k))_N]$. Juntando las dos, $X[k] = X^*[k]$: es real.

## Análisis espectral

**Ejercicio 10** (eje de frecuencias) Se calcula la DFT de $N = 512$ muestras de una señal muestreada a $f_s = 8\,\text{kHz}$.

a) ¿Cuál es la separación entre muestras de la DFT y la duración del registro?

b) ¿En qué muestras aparece un tono de 1 kHz?

c) ¿A qué frecuencia corresponde la muestra $k = 300$?

**Solución**

a) $\Delta f = 8000/512 = 15{,}625\,\text{Hz}$; $T_{\text{obs}} = 512/8000 = 64\,\text{ms} = 1/\Delta f$.

b) $k_0 = 1000/15{,}625 = 64$ y su imagen $N - k_0 = 448$.

c) $k = 300 > N/2 = 256$: es una frecuencia negativa, $(300 - 512) \cdot 15{,}625 = -3312{,}5\,\text{Hz}$. Si la señal es real, $X[300] = X^*[212]$, que corresponde a $+3312{,}5\,\text{Hz}$.

**Ejercicio 11** (amplitudes con y sin ventana) La señal $x(t) = 3\cos(2\pi\,1000\,t) + \cos(2\pi\,2500\,t + \pi/4)$ se muestrea a $f_s = 8\,\text{kHz}$ y se toman $N = 64$ muestras.

a) ¿En qué muestras $k \le N/2$ aparecen los tonos? ¿Cuánto valen $|X[k]|$ y $\arg X[k]$ en ellas?

b) Repite el cálculo con la ventana de Hann periódica, $w[n] = \frac{1}{2} - \frac{1}{2}\cos(2\pi n/N)$. ¿Qué valen las muestras vecinas de cada pico?

**Solución**

a) $\Delta f = 125\,\text{Hz}$: los tonos caen exactamente en $k = 8$ y $k = 20$. $|X[8]| = \frac{3 \cdot 64}{2} = 96$ con fase 0, y $|X[20]| = \frac{64}{2} = 32$ con fase $\pi/4$. El resto de las muestras son nulas.

b) $w[n] = \frac{1}{2} - \frac{1}{4}e^{j2\pi n/N} - \frac{1}{4}e^{-j2\pi n/N}$, así que cada línea se convierte en tres: $\frac{1}{2}$ en $k_0$ y $-\frac{1}{4}$ en $k_0 \pm 1$, multiplicando la DFT sin ventana. Con $\sum w = N/2 = 32$: $X_w[8] = 48$, $X_w[7] = X_w[9] = -24$; $|X_w[20]| = 16$ y $|X_w[19]| = |X_w[21]| = 8$. La amplitud se recupera con $2|X_w[k_0]|/\sum w$: $2 \cdot 48/32 = 3$.

**Ejercicio 12** (resolución) Se quieren distinguir dos tonos de 1000 y 1030 Hz muestreados a $f_s = 8\,\text{kHz}$.

a) ¿Cuántas muestras hacen falta, aproximadamente, con la ventana rectangular? ¿Y con la de Hann?

b) ¿Se pueden distinguir con $N = 128$ muestras rellenando con ceros hasta 4096? Justifícalo.

**Solución**

a) Con la rectangular, $f_s/N < 30\,\text{Hz}$: $N > 267$, unos 33 ms de señal. Con Hann, el lóbulo principal es el doble de ancho: $N > 533$, unos 67 ms.

b) No. El relleno con ceros da más muestras de la DTFT de las mismas 128 muestras, cuyo lóbulo principal (semianchura $f_s/N = 62{,}5\,\text{Hz}$) es mucho más ancho que la separación de los tonos. Se ve un único pico.

**Ejercicio 13** (potencia y densidad espectral) La señal $x[n] = 2\cos(2\pi\,100\,n/f_s) + v[n]$, con $f_s = 1\,\text{kHz}$ y $v[n]$ ruido blanco de varianza $\sigma^2 = 0{,}01$, se analiza con $N = 1000$ muestras.

a) ¿Cuánto vale la potencia por muestra, $|X[k]/N|^2$, en el tono y, en promedio, en el ruido? Exprésalo en dB.

b) ¿Y la densidad espectral estimada por el periodograma, en dB/Hz?

c) ¿Qué cambia si se duplica $N$?

**Solución**

a) El tono cae en $k_0 = 100$: $|X[k_0]| = AN/2$ y la potencia por muestra es $A^2/4 = 1$, 0 dB. Para el ruido, $E|V[k]|^2 = N\sigma^2$, así que la potencia media por muestra es $\sigma^2/N = 10^{-5}$, $-50\,\text{dB}$.

b) Ruido: $\sigma^2/f_s = 10^{-5}$ /Hz, $-50$ dB/Hz. Tono: $\frac{(AN/2)^2}{Nf_s} = \frac{A^2N}{4f_s} = 1,0$ dB/Hz.

c) La potencia por muestra del tono sigue siendo 0 dB y la del ruido baja a $-53$ dB. En densidad, el ruido sigue en $-50$ dB/Hz y el pico del tono sube a 3 dB/Hz. En los dos casos el tono sobresale 3 dB más: alargar el registro ayuda a detectar tonos en ruido. (Simulado promediando 200 realizaciones.)

### FFT y convolución rápida

**Ejercicio 14** (coste de la FFT) Un procesador hace un producto complejo cada nanosegundo. ¿Cuánto tarda en calcular una DFT de $N = 4096$ puntos directamente y con una FFT en base 2? Ignora las sumas.

**Solución**

Directa: $N^2 = 1,68 \cdot 10^7$ productos, 16,8 ms. FFT: $\frac{N}{2}\log_2 N = 2048 \cdot 12 = 24\,576$ productos, 24,6 µs. Es unas 680 veces más rápida.

**Ejercicio 15** (FFT de cuatro puntos) Calcula la DFT de $x = (1, 3, 4, 0)$ con el algoritmo de diezmado en el tiempo: separa las muestras pares e impares, calcula sus DFT de 2 puntos y combínalas con las mariposas. Dibuja el diagrama de flujo y cuenta los productos.

**Solución**

Pares $(1, 4)$: $F = (1 + 4,\ 1 - 4) = (5, -3)$. Impares $(3, 0)$: $G = (3, 3)$. Con $W_4^0 = 1$ y $W_4^1 = -j$:

$$X[0] = F[0] + G[0] = 8, \qquad X[1] = F[1] - jG[1] = -3 - 3j,$$
$$X[2] = F[0] - G[0] = 2, \qquad X[3] = F[1] + jG[1] = -3 + 3j.$$

Coincide con la DFT directa. Hay $\frac{N}{2}\log_2 N = 4$ productos, todos por $\pm 1$ o $\pm j$, es decir, triviales; la DFT directa necesita 16.

**Ejercicio 16** (solapamiento y suma) Un sistema de audio a 48 kHz filtra en tiempo real con un FIR de $M = 101$ coeficientes por solapamiento y suma con FFT de $N = 1024$ puntos.

a) ¿Qué longitud $L$ deben tener los bloques? ¿Cuántos bloques se procesan por segundo?

b) ¿Cuántos productos complejos por muestra de salida necesita, si $H[k]$ se calcula una vez? Compáralo con la convolución directa.

c) ¿Qué retardo introduce el procesado por bloques?

**Solución**

a) $L = N - M + 1 = 924$ muestras; $48\,000/924 \approx 52$ bloques por segundo.

b) Por bloque: una FFT, $N$ productos y una IFFT, $2 \cdot 512 \cdot 10 + 1024 = 11\,264$ productos complejos, unos 12,2 por muestra de salida (unos 49 reales). La convolución directa necesita 101 productos reales por muestra: la FFT es unas dos veces más barata, y la ventaja crece con $M$.

c) Al menos un bloque, $924/48\,000 \approx 19$ ms, más el tiempo de cálculo.

# 4 Filtros

**Cómo usar estos apuntes**

Estos apuntes desarrollan el contenido de las cinco presentaciones del tema: la sección 4.1 corresponde a la parte 1 (filtros ideales, especificaciones y fase), la sección 4.2 a la parte 2 (filtros continuos), la sección 4.3 a la parte 3 (filtros IIR), la sección 4.4 a la parte 4 (filtros FIR) y la sección 4.5 a la parte 5 (fase cero, fase mínima, paso todo y peine). Los ejemplos de código están en `teoría/T4/codigo/`, en Python (`scipy.signal`) y en Matlab, y se pueden ejecutar tal cual. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

## 4.1 Filtros ideales, especificaciones y fase

Un *filtro* es un sistema LTI estable que modifica selectivamente las componentes en frecuencia de una señal. Lo describen su respuesta al impulso o su respuesta en frecuencia,

$$h(t) \overset{\mathscr{F}}{\longleftrightarrow} H(j\omega), \qquad h[n] \overset{\mathscr{F}}{\longleftrightarrow} H(\mathrm{e}^{j\Omega}),$$

y, como vimos en el Tema 1, una sinusoide de frecuencia $\omega_0$ sale de él multiplicada por $|H(j\omega_0)|$ y desfasada $\arg H(j\omega_0)$. Seguimos la notación del curso: $\omega$ (rad/s) es la frecuencia de las señales continuas y $\Omega$ (rad/muestra) la de las discretas; con muestreo a $f_s$, $\Omega = \omega/f_s = 2\pi f/f_s$. La respuesta de un filtro discreto es periódica de periodo $2\pi$, así que basta describirla en $[-\pi, \pi]$, y si $h[n]$ es real, en $[0, \pi]$, porque $|H|$ es par y $\arg H$ impar.

### 4.1.1 Filtros ideales

Los cuatro filtros selectivos ideales (figura 4.1) dejan pasar sin cambios una banda de frecuencias y anulan el resto:

$$H_{\mathrm{PB}}(\mathrm{e}^{j\Omega}) = \begin{cases} 1 & |\Omega| < \Omega_c \\ 0 & \Omega_c < |\Omega| \le \pi \end{cases}, \qquad H_{\mathrm{BP}}(\mathrm{e}^{j\Omega}) = \begin{cases} 1 & \Omega_1 < |\Omega| < \Omega_2 \\ 0 & \text{en el resto de } [-\pi, \pi] \end{cases},$$

con el paso alto $H_{\mathrm{PA}} = 1 - H_{\mathrm{PB}}$ y el supresor de banda $H_{\mathrm{SB}} = 1 - H_{\mathrm{BP}}$. En continuo se definen igual con $\omega$ en lugar de $\Omega$. El valor absoluto es imprescindible: un filtro con respuesta al impulso real actúa igual sobre $\omega$ y $-\omega$.

Ningún filtro ideal se puede construir. La respuesta al impulso del paso bajo ideal es

$$h[n] = \frac{1}{2\pi} \int_{-\Omega_c}^{\Omega_c} \mathrm{e}^{j\Omega n}\, \mathrm{d}\Omega = \frac{\sin(\Omega_c n)}{\pi n}$$

(con $h[0] = \Omega_c/\pi$), que es distinta de cero para $n < 0$, de modo que el sistema no es causal, y tiene duración infinita. En continuo ocurre lo mismo con $h(t) = \sin(\omega_c t)/(\pi t)$. Diseñar un filtro consiste en *aproximar* el ideal con un sistema causal, estable y de orden finito, admitiendo una tolerancia.

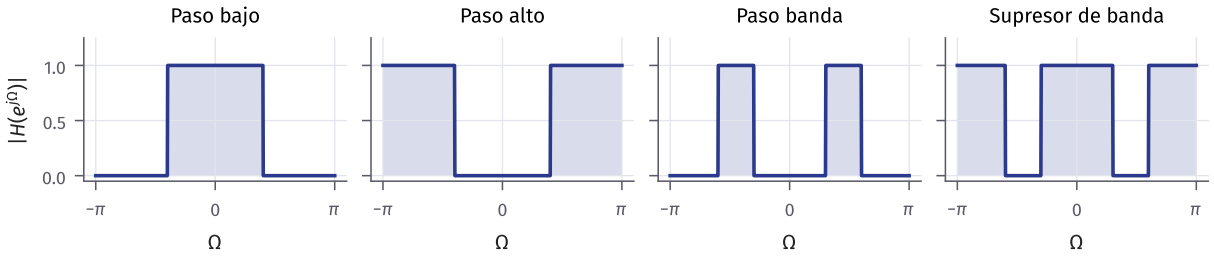



**Figura 4.1.** Los cuatro filtros ideales en discreto.

### 4.1.2 Especificaciones

La tolerancia se expresa con una *plantilla* (figura 4.2). Para un paso bajo:

- en la banda de paso, $0 \le \omega \le \omega_p$, la atenuación no supera $\alpha_p$ dB: $1/E \le |H| \le 1$ con $\alpha_p = 20\log_{10} E$;
- en la banda de supresión, $\omega \ge \omega_s$, la atenuación es al menos $\alpha_s$ dB: $|H| \le 1/A$ con $\alpha_s = 20\log_{10} A$;
- en la banda de transición, $\omega_p < \omega < \omega_s$, no se impone nada.

La *frecuencia de corte* $\omega_c$ es aquella en la que $|H(j\omega_c)| = 1/\sqrt{2}$, es decir, la atenuación es de 3 dB. Dos cocientes resumen la dificultad del diseño: la *selectividad* $k = \omega_p/\omega_s < 1$, más exigente cuanto más se acerca a 1, y la *discriminación*

$$k_1 = \frac{\varepsilon}{\sqrt{A^2-1}}, \qquad \varepsilon = \sqrt{E^2-1},$$

más exigente cuanto más pequeña. En los filtros FIR es más habitual dar los rizados: $1-\delta_p \le |H| \le 1+\delta_p$ en la banda de paso y $|H| \le \delta_s$ en la de supresión, con $\alpha_s = -20\log_{10}\delta_s$. Para los demás tipos de filtro hay un par de bordes por cada transición ($\omega_{p1}, \omega_{p2}, \omega_{s1}, \omega_{s2}$ en un paso banda).

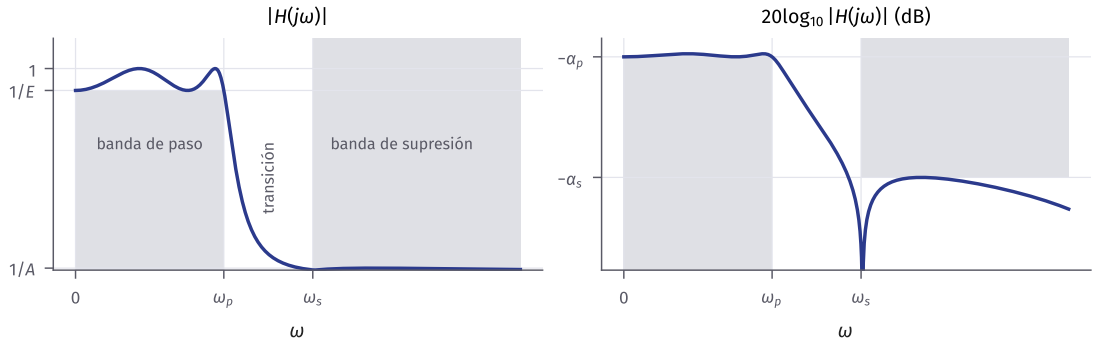



**Figura 4.2.** Plantilla de un paso bajo en escala lineal y en decibelios, con un filtro elíptico que la cumple. Las zonas sombreadas están prohibidas.

### 4.1.3 Fase y retardo de grupo

Un sistema que no distorsiona solo escala y retrasa: $y[n] = K\,x[n-n_0]$, con $H(\mathrm{e}^{j\Omega}) = K\mathrm{e}^{-j\Omega n_0}$. Su fase es *lineal* en $\Omega$. En general se define el *retardo de grupo*

$$\tau(\Omega) = -\frac{\mathrm{d}\arg H(\mathrm{e}^{j\Omega})}{\mathrm{d}\Omega} \quad \text{(muestras)},$$

que mide cuánto se retrasa la envolvente de un paquete de frecuencias en torno a $\Omega$. Si $\tau$ es constante, todas las componentes se retrasan lo mismo y la forma de onda se conserva; si no, se produce *distorsión de fase*. La figura 4.3 lo ilustra con una onda cuadrada: un IIR y un FIR con módulos parecidos dan salidas muy distintas, porque el IIR retrasa más los armónicos cercanos al corte. La distorsión de fase importa en transmisión de datos, en electrocardiografía o en audio de alta calidad; el oído es poco sensible a ella en régimen permanente, pero no en los transitorios.

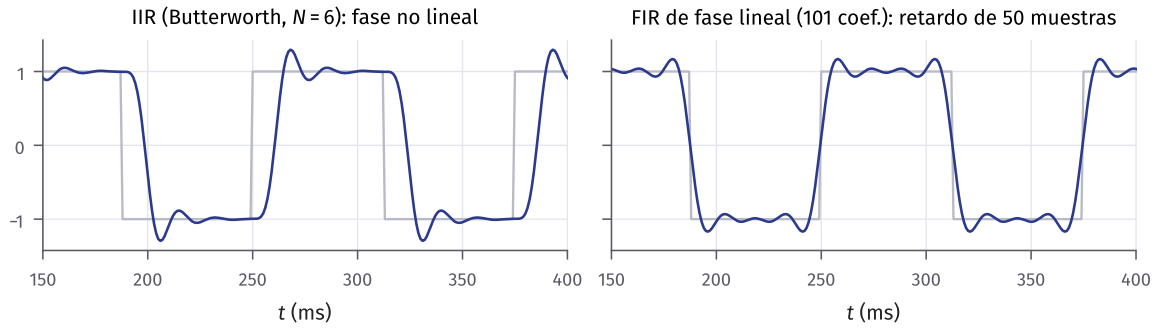



**Figura 4.3.** Onda cuadrada de 8 Hz ($f_s = 1$ kHz) filtrada con un Butterworth de orden 6 y con un FIR de fase lineal de 101 coeficientes, ambos con corte en 60 Hz. La salida del FIR se ha adelantado 50 muestras para compararla.

### 4.1.4 Polos, ceros y diseño a mano

La respuesta en frecuencia de un sistema racional queda determinada por la posición de sus polos y ceros, que conocemos del Tema 1 (figura 4.4):

- un *polo real* en $-a$, $H(s) = a/(s+a)$, atenúa a partir de $\omega \approx a$ con una pendiente de $-20$ dB/década, y cada polo adicional añade otros $-20$ dB/década;
- un *par de polos* $-\sigma \pm j\omega_a$ produce una resonancia cerca de $\omega_a$, tanto más aguda cuanto menor es $\sigma$. Con $\sigma = 0$, $H(s) = \omega_n^2/(s^2 + \omega_n^2)$ tiene los polos sobre el eje imaginario: es *marginalmente estable*, su respuesta al impulso es una sinusoide que no se amortigua y no sirve como filtro;
- un *par de ceros* en $\pm j\omega_n$ anula exactamente la frecuencia $\omega_n$. Sin polos, $|H|$ crecería como $\omega^2$; con dos polos con parte real negativa el sistema es propio (tantos polos como ceros, condición para que sea realizable) y la ganancia vuelve a 1 lejos de $\omega_n$.

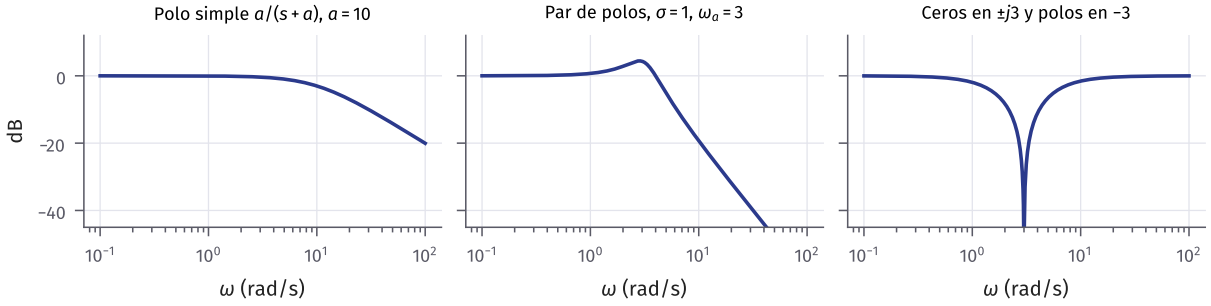



**Figura 4.4.** Efecto de un polo real, de un par de polos complejos y de un par de ceros imaginarios compensado con dos polos.

En discreto se razona igual con la circunferencia unidad en lugar del eje $j\omega$. Un *filtro de muesca* que

elimine $\Omega_0$ tiene ceros en $e^{\pm j\Omega_0}$ y polos en el mismo ángulo con radio $r < 1$:

$$H(z) = \frac{(1 - e^{j\Omega_0} z^{-1})(1 - e^{-j\Omega_0} z^{-1})}{(1 - r e^{j\Omega_0} z^{-1})(1 - r e^{-j\Omega_0} z^{-1})} = \frac{1 - 2\cos\Omega_0\, z^{-1} + z^{-2}}{1 - 2r\cos\Omega_0\, z^{-1} + r^2 z^{-2}}.$$

Lejos de $\Omega_0$ cada polo está casi a la misma distancia del punto $e^{j\Omega}$ que su cero, y $|H| \approx 1$; la anchura de la muesca es aproximadamente $2(1 - r)$ rad (figura 4.5). Este diseño a mano funciona para filtros muy sencillos, pero no para filtros selectivos con especificaciones exigentes: para ellos hacen falta los métodos sistemáticos de las secciones siguientes.

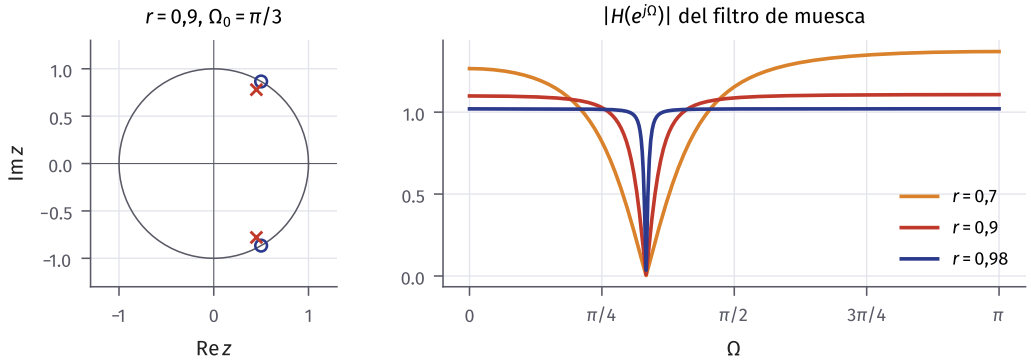



**Figura 4.5.** Filtro de muesca en $\Omega_0 = \pi/3$: diagrama de polos y ceros y respuesta para tres radios.

### 4.1.5 Filtros FIR e IIR

Un filtro discreto causal de orden finito tiene función de transferencia y ecuación en diferencias

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}}, \qquad y[n] = \sum_{k=0}^{M} b_k\, x[n-k] - \sum_{k=1}^{N} a_k\, y[n-k].$$

Si algún $a_k \neq 0$ hay realimentación y la respuesta al impulso es infinita: filtro *IIR*. Si todos los $a_k$ son nulos, $h[n] = b_n$ tiene $M + 1$ muestras: filtro *FIR*. El cuadro 4.1 resume sus diferencias, que guían la elección.

**Cuadro 4.1.** Comparación entre filtros IIR y FIR.

| | IIR | FIR |
|---|---|---|
| Diseño | A partir de un filtro continuo | Directo en discreto (ventanas, optimización) |
| Orden | Bajo | Alto: decenas o cientos de coeficientes |
| Fase | No lineal | Puede ser exactamente lineal |
| Estabilidad | Hay que comprobarla | Siempre estables |
| Implementación | Sensible a la cuantificación; secciones de orden 2 | Robusta; admite FFT para órdenes altos |

Ejercicios 1–3. *Herramientas interactivas:* `T4-media-movil`, `T4-polos-ceros-z` y `T4-fase-lineal`.

## 4.2 Filtros continuos

### 4.2.1 Aproximación del módulo

Diseñamos un paso bajo continuo, el *prototipo*, y obtendremos los demás tipos con transformaciones de frecuencia (apartado 4.2.6). Como $|H(j\omega)|^2$ es una función par de $\omega$, se busca de la forma

$$|H(j\omega)|^2 = \frac{1}{1 + P(\omega^2)},$$

con $P$ pequeño en la banda de paso y grande en la de supresión. Cada familia de filtros corresponde a una elección de $P$. Conocido $|H(j\omega)|^2 = H(s)H(-s)\big|_{s=j\omega}$, se calculan las raíces de $H(s)H(-s)$, que aparecen en parejas simétricas respecto al eje imaginario, y se asignan a $H(s)$ las del semiplano izquierdo: el filtro resulta estable.

Los datos del diseño son $\omega_p$, $\omega_s$, $\alpha_p$ y $\alpha_s$, y de ellos se derivan

$$E = 10^{\alpha_p/20}, \quad A = 10^{\alpha_s/20}, \quad \varepsilon = \sqrt{E^2 - 1}, \quad k = \frac{\omega_p}{\omega_s}, \quad k_1 = \frac{\varepsilon}{\sqrt{A^2 - 1}}.$$

Usaremos como ejemplo común un paso bajo con $\alpha_p = 1\,\text{dB}$ hasta $1\,\text{kHz}$ y $\alpha_s = 40\,\text{dB}$ desde $5\,\text{kHz}$: $\omega_p = 2\pi \cdot 1000\,\text{rad/s}$, $\omega_s = 2\pi \cdot 5000\,\text{rad/s}$, $E = 1{,}122$, $A = 100$, $\varepsilon = 0{,}5088$, $k = 0{,}2$ y $k_1 = 0{,}00509$.

### 4.2.2 Butterworth

**Definición 4.2.1: Filtro de Butterworth**

$$|H(j\omega)|^2 = \frac{1}{1 + (\omega/\omega_c)^{2N}}$$

Es *máximamente plano*: las $2N - 1$ primeras derivadas de $|H|^2$ se anulan en $\omega = 0$. Es monótono, vale $1/\sqrt{2}$ en $\omega_c$ sea cual sea el orden y cae $20N$ dB/década. Las raíces de $1 + (s/j\omega_c)^{2N} = 0$ están sobre una circunferencia de radio $\omega_c$; las del semiplano izquierdo son los polos

$$p_k = \omega_c\, e^{j\pi \frac{2k+N-1}{2N}}, \qquad k = 1, \ldots, N,$$

separados un ángulo $\pi/N$ (figura 4.6).

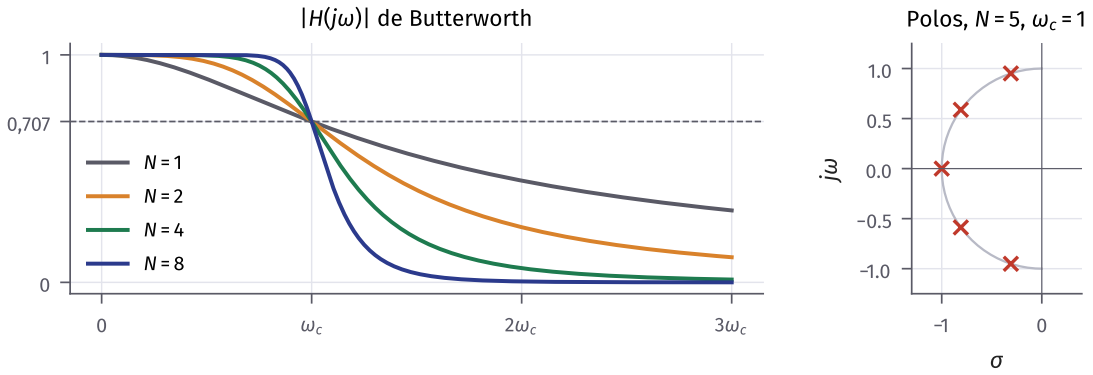



**Figura 4.6.** Módulo del filtro de Butterworth para varios órdenes y polos para $N = 5$.

Para obtener el orden se imponen los dos bordes de la plantilla:

$$\frac{1}{1+(\omega_p/\omega_c)^{2N}} \geq \frac{1}{E^2} \;\Leftrightarrow\; \left(\frac{\omega_p}{\omega_c}\right)^{2N} \leq E^2-1, \qquad \frac{1}{1+(\omega_s/\omega_c)^{2N}} \leq \frac{1}{A^2} \;\Leftrightarrow\; \left(\frac{\omega_s}{\omega_c}\right)^{2N} \geq A^2-1.$$

Dividiendo una entre otra se elimina $\omega_c$.

**Propiedad 4.2.2: Orden y corte de un Butterworth**

$$N \geq \frac{\log_{10}\dfrac{A^2-1}{E^2-1}}{2\log_{10}\dfrac{\omega_s}{\omega_p}} = \frac{\log(1/k_1)}{\log(1/k)}, \qquad \frac{\omega_p}{(E^2-1)^{1/2N}} \leq \omega_c \leq \frac{\omega_s}{(A^2-1)^{1/2N}}.$$

Al redondear $N$ hacia arriba queda margen y $\omega_c$ se puede elegir en un intervalo: en el extremo izquierdo se cumple exactamente la banda de paso; en el derecho, la de supresión. `buttord` de Matlab y de `scipy.signal` devuelve el izquierdo.

**Ejemplo 4.2.3: Butterworth**

Con las especificaciones comunes, $E^2-1 = 0{,}2589$ y $A^2-1 = 9999$:

$$N \geq \frac{\log_{10}(9999/0{,}2589)}{2\log_{10}5} = \frac{4{,}587}{1{,}398} = 3{,}28 \;\rightarrow\; N = 4,$$

$$\omega_c \in \left[\frac{6283}{0{,}2589^{1/8}},\; \frac{31416}{9999^{1/8}}\right] = [7439,\; 9935]\ \text{rad/s},$$

es decir, $f_c$ entre 1184 y 1581 Hz.

**En código**

Orden, corte y respuesta del Butterworth del ejemplo 4.2.3.

**Python**

```python
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt

wp = 2*np.pi*1e3; ws = 2*np.pi*5e3  # rad/s
ap, as_ = 1, 40                     # dB
N, wc = signal.buttord(wp, ws, ap, as_,
                       analog=True)
print(N, wc)                        # 4, 7439
b, a = signal.butter(N, wc, analog=True)
w = np.linspace(0, 2*np.pi*8e3, 1000)
w, H = signal.freqs(b, a, w)
plt.plot(w/(2*np.pi), 20*np.log10(abs(H)))
plt.xlabel("f (Hz)"); plt.ylabel("dB")
plt.show()
```

**Matlab**

```matlab
wp = 2*pi*1e3; ws = 2*pi*5e3;       % rad/s
ap = 1; as = 40;                    % dB
[N, wc] = buttord(wp, ws, ap, as, 's');
disp([N, wc])                       % 4, 7439
[b, a] = butter(N, wc, 's');
w = linspace(0, 2*pi*8e3, 1000);
H = freqs(b, a, w);
plot(w/(2*pi), 20*log10(abs(H)))
xlabel('f (Hz)'); ylabel('dB')
```

### 4.2.3 Chebyshev

Los *polinomios de Chebyshev* se definen por

$$T_N(x) = \begin{cases} \cos(N\arccos x) & |x| \leq 1 \\ \cosh(N\,\operatorname{arccosh} x) & |x| > 1 \end{cases}, \qquad T_0 = 1,\; T_1 = x,\; T_{N+1} = 2xT_N - T_{N-1}.$$

En $[-1,1]$ oscilan entre $-1$ y 1, con $N$ ceros; fuera crecen como $2^{N-1}x^N$, mucho más deprisa que $x^N$.

**Definición 4.2.4: Filtros de Chebyshev**

$$\text{Tipo I: } |H(j\omega)|^2 = \frac{1}{1+\varepsilon^2 T_N^2(\omega/\omega_p)}, \qquad \text{Tipo II: } |H(j\omega)|^2 = \frac{\varepsilon_2^2\, T_N^2(\omega_s/\omega)}{1+\varepsilon_2^2\, T_N^2(\omega_s/\omega)},\ \varepsilon_2^2 = A^2-1.$$

El tipo I tiene rizado entre 1 y $1/E$ en la banda de paso (equirrizado) y cae monótonamente en la de supresión; sus polos están sobre una elipse. El tipo II, o inverso, es plano en la banda de paso y tiene rizado en la de supresión, donde alcanza exactamente $1/A$ en sus máximos; tiene ceros sobre el eje $j\omega$. Para el tipo I, la condición en $\omega_s$ es $1+\varepsilon^2\cosh^2\left(N\,\text{arccosh}(\omega_s/\omega_p)\right) \geq A^2$, que da

**Propiedad 4.2.5: Orden de un Chebyshev**

$$N \geq \frac{\text{arccosh}(1/k_1)}{\text{arccosh}(1/k)} = \frac{\text{arccosh}\left(\sqrt{A^2-1}/\varepsilon\right)}{\text{arccosh}(\omega_s/\omega_p)},$$

la misma para los dos tipos.

Con las especificaciones comunes, $1/k_1 = 196{,}5$ y $N \geq \text{arccosh}(196{,}5)/\text{arccosh}(5) = 5{,}97/2{,}29 = 2{,}61$, de modo que $N=3$: uno menos que Butterworth. En Matlab y en `scipy.signal`, `cheby1(N,Rp,Wp)` recibe el rizado de la banda de paso y su borde, y `cheby2(N,Rs,Ws)` la atenuación de la banda de supresión y el suyo.

### 4.2.4 Elíptico

El filtro *elíptico* o de Cauer usa una función racional de Chebyshev $R_N$, construida con funciones elípticas de Jacobi:

$$|H(j\omega)|^2 = \frac{1}{1+\varepsilon^2 R_N^2(\omega/\omega_p)}.$$

Tiene rizado en las dos bandas y ceros en el eje $j\omega$, y es el filtro de *menor orden* que cumple unas especificaciones dadas. El orden exacto es

$$N \geq \frac{K(k)\,K\left(\sqrt{1-k_1^2}\right)}{K(k_1)\,K\left(\sqrt{1-k^2}\right)},$$

con $K$ la integral elíptica completa de primera especie. Una aproximación muy precisa es

$$k' = \sqrt{1-k^2}, \quad \rho_0 = \frac{1-\sqrt{k'}}{2(1+\sqrt{k'})}, \quad \rho = \rho_0 + 2\rho_0^5 + 15\rho_0^9 + 150\rho_0^{13}, \quad N \geq \frac{2\log_{10}(4/k_1)}{\log_{10}(1/\rho)}.$$

Para las especificaciones comunes, $k = \omega_p/\omega_s = 0{,}2$, $k' = 0{,}9798$, $\rho = 0{,}002551$ y $N \geq 2{,}23$, así que $N=3$, el mismo orden que Chebyshev pero con más margen.

### 4.2.5 Comparación

La figura 4.7 muestra las cuatro familias con el mismo orden, y la figura 4.8 sus polos y ceros. Cuanto más selectivo es un filtro, menos lineal es su fase: Butterworth es el de fase más lineal y el elíptico el de menos. La figura 4.9 compara los diseños de orden mínimo para la plantilla común.

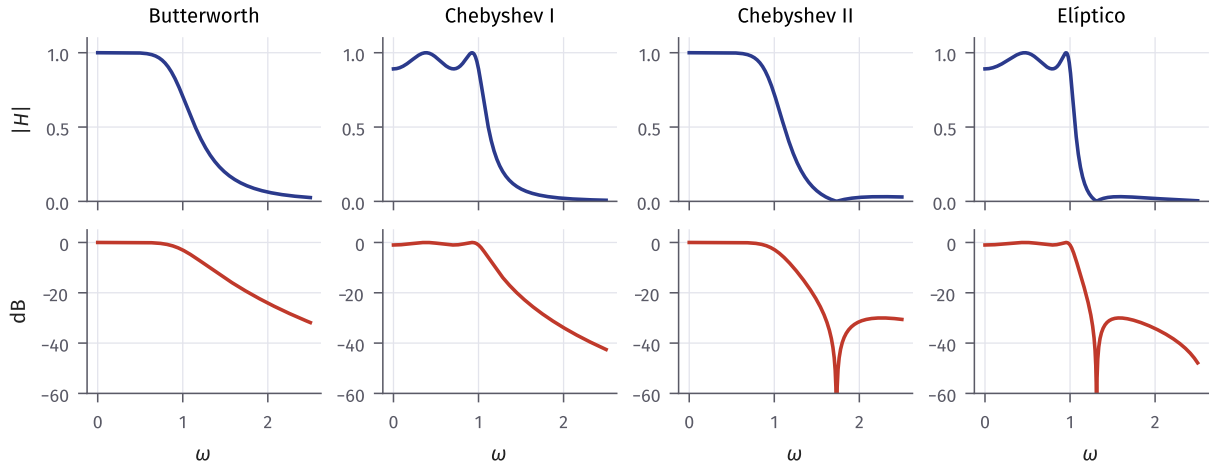



**Figura 4.7.** Butterworth, Chebyshev I y II y elíptico de orden 4, en escala lineal y en dB.

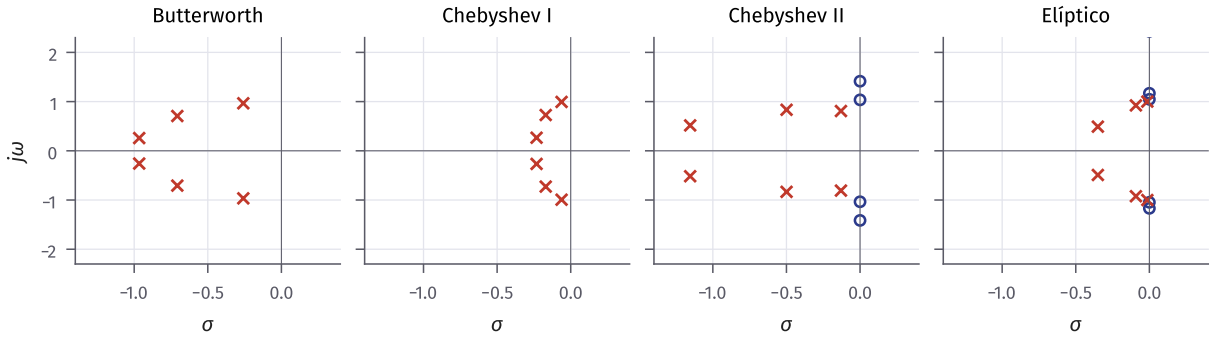



**Figura 4.8.** Polos (×) y ceros (∘) de las cuatro familias para $N = 6$.

**En código**

Orden mínimo de cada familia y llamadas de diseño, con el orden correcto de los argumentos.

**Python**

```python
import numpy as np
from scipy import signal

wp = 2*np.pi*1e3; ws = 2*np.pi*5e3
ap, as_ = 1, 40
for f in (signal.buttord, signal.cheb1ord,
          signal.cheb2ord, signal.ellipord):
    N, wn = f(wp, ws, ap, as_, analog=True)
    print(f.__name__, N)   # 4, 3, 3, 3
b, a = signal.cheby1(3, ap, wp, analog=True)
b, a = signal.cheby2(3, as_, ws,
                     analog=True)
b, a = signal.ellip(3, ap, as_, wp,
                    analog=True)
```

**Matlab**

```matlab
wp = 2*pi*1e3; ws = 2*pi*5e3;
ap = 1; as = 40;
N1 = buttord(wp, ws, ap, as, 's')   % 4
N2 = cheb1ord(wp, ws, ap, as, 's')  % 3
N3 = cheb2ord(wp, ws, ap, as, 's')  % 3
N4 = ellipord(wp, ws, ap, as, 's')  % 3
% Rizado de la banda de paso (ap) antes
% que la atenuación (as):
[b, a] = cheby1(3, ap, wp, 's');
[b, a] = cheby2(3, as, ws, 's');
[b, a] = ellip(3, ap, as, wp, 's');
```

### 4.2.6 Transformaciones de frecuencia

Paso alto, paso banda y supresor se obtienen a partir de un prototipo paso bajo $H_{\mathrm{LP}}(s)$ con $\omega_p = 1$ mediante una sustitución $s = F(\hat{s})$, donde $\hat{s}$ y $\hat{\omega}$ son la variable y la frecuencia del filtro de destino (cua-

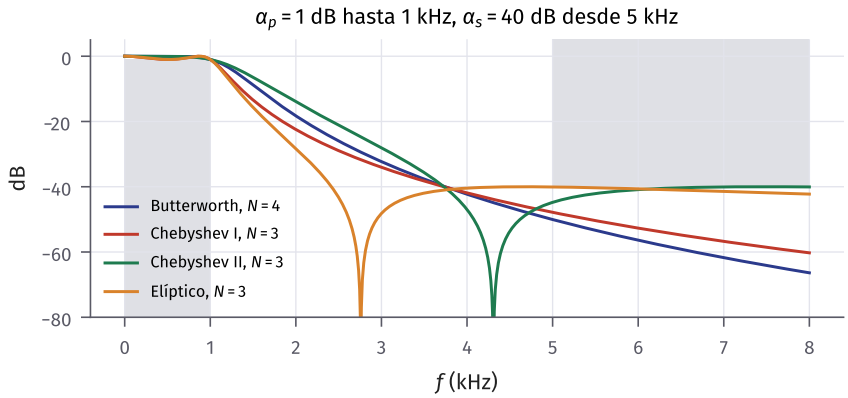



**Figura 4.9.** Diseños de orden mínimo para la plantilla común.

dro 4.2). Las transformaciones solo mueven las frecuencias: conservan los rizados y las atenuaciones.

**Cuadro 4.2.** Transformaciones de frecuencia desde un prototipo con $\omega_p = 1$. En paso banda y supresor, $\hat{\omega}_0^2 = \hat{\omega}_{p1}\hat{\omega}_{p2}$ y $B = \hat{\omega}_{p2} - \hat{\omega}_{p1}$.

| Destino | Sustitución | Relación de frecuencias | Matlab / `scipy.signal` |
|---|---|---|---|
| Paso bajo | $s = \hat{s}/\hat{\omega}_p$ | $\omega = \hat{\omega}/\hat{\omega}_p$ | `lp2lp` |
| Paso alto | $s = \hat{\omega}_p/\hat{s}$ | $\omega = -\hat{\omega}_p/\hat{\omega}$ | `lp2hp` |
| Paso banda | $s = \dfrac{\hat{s}^2 + \hat{\omega}_0^2}{B\,\hat{s}}$ | $\omega = \dfrac{\hat{\omega}^2 - \hat{\omega}_0^2}{B\,\hat{\omega}}$ | `lp2bp` |
| Supresor | $s = \dfrac{B\,\hat{s}}{\hat{s}^2 + \hat{\omega}_0^2}$ | $\omega = \dfrac{B\,\hat{\omega}}{\hat{\omega}_0^2 - \hat{\omega}^2}$ | `lp2bs` |

El procedimiento es:

1. llevar los bordes de supresión al prototipo con la relación de frecuencias y quedarse con el más exigente, $\omega_s = \min|\omega(\hat{\omega}_s)|$;
2. diseñar el prototipo con $\omega_p = 1$, ese $\omega_s$, $\alpha_p$ y $\alpha_s$;
3. sustituir: $H(\hat{s}) = H_{\text{LP}}(F(\hat{s}))$.

El paso banda que resulta es geométricamente simétrico respecto a $\hat{\omega}_0$: dos frecuencias con la misma atenuación cumplen $\hat{\omega}_1\hat{\omega}_2 = \hat{\omega}_0^2$. Si los bordes de supresión dados no lo son, uno de los dos se cumple con margen; por eso se toma el mínimo. El orden del paso banda y del supresor es el doble del del prototipo (figura 4.10).

**Ejemplo 4.2.6: Paso alto de Butterworth**

Banda de paso desde 4 kHz con rizado de 0,1 dB y atenuación mínima de 40 dB por debajo de 1 kHz. El prototipo tiene $\omega_p = 1$ y $\omega_s = \hat{\omega}_p/\hat{\omega}_s = 4$ (el signo de $\omega = -\hat{\omega}_p/\hat{\omega}$ no importa, porque $|H|$ es par). Con $E^2 - 1 = 10^{0{,}01} - 1 = 0{,}0233$,

$$N \geq \frac{\log_{10}(9999/0{,}0233)}{2\log_{10} 4} = 4{,}68 \;\rightarrow\; N = 5,$$

y el paso alto es $H(\hat{s}) = H_{\text{LP}}(2\pi \cdot 4000/\hat{s})$.

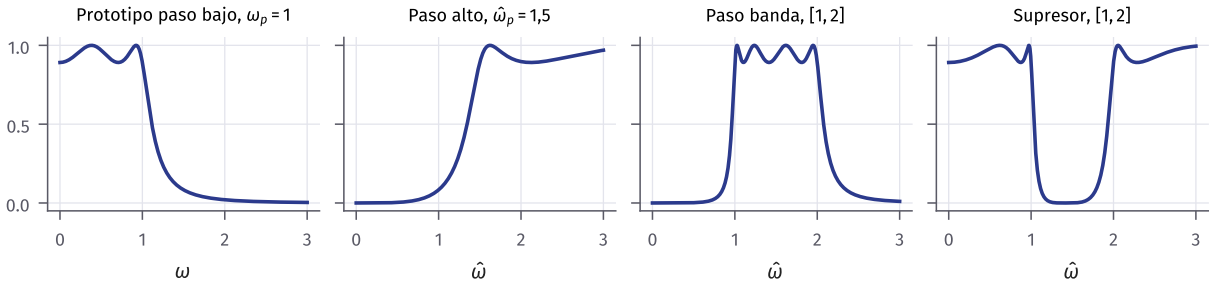



**Figura 4.10.** Un prototipo de Chebyshev I de orden 4 y los filtros que se obtienen con cada transformación.

**En código**

El ejemplo 4.2.6 con el prototipo y `lp2hp`, y en un solo paso con `butter(N,wn,'high','s')`.

**Python**

```python
import numpy as np
from scipy import signal

# Paso 4 kHz, 0,1 dB; supresión 1 kHz, 40 dB
wp_ = 2*np.pi*4e3; ws_ = 2*np.pi*1e3
# 1) Prototipo paso bajo con wp = 1
ws = wp_ / ws_                          # = 4
N, wc = signal.buttord(1, ws, 0.1, 40,
                       analog=True)
b, a = signal.butter(N, wc, analog=True)
# 2) Paso bajo -> paso alto: s = wp_/s
bh, ah = signal.lp2hp(b, a, wp_)
# Equivale a hacerlo en un solo paso:
N2, wn = signal.buttord(wp_, ws_, 0.1, 40,
                        analog=True)
b2, a2 = signal.butter(N2, wn, "highpass",
                       analog=True)
print(N, N2)                            # 5, 5
```

**Matlab**

```matlab
% Paso 4 kHz, 0,1 dB; supresión 1 kHz, 40 dB
wp_ = 2*pi*4e3; ws_ = 2*pi*1e3;
% 1) Prototipo paso bajo con wp = 1
ws = wp_ / ws_;                         % = 4
[N, wc] = buttord(1, ws, 0.1, 40, 's');
[b, a] = butter(N, wc, 's');
% 2) Paso bajo -> paso alto (Matlab):
%    [bh, ah] = lp2hp(b, a, wp_);
% Equivale a hacerlo en un solo paso:
[N2, wn] = buttord(wp_, ws_, 0.1, 40, 's');
[b2, a2] = butter(N2, wn, 'high', 's');
disp([N, N2])                           % 5, 5
```

**Ejemplo 4.2.7: Paso banda elíptico y supresor de Chebyshev II**

*Paso banda*: paso de 4 a 7 kHz con 1 dB; supresión por debajo de 3 kHz y por encima de 8 kHz con 22 dB. Trabajando en kHz (el factor $2\pi$ se simplifica), $\hat{\omega}_0^2 = 28$ y $B = 3$:

$$\omega(3) = \frac{|9-28|}{3\cdot 3} = 2{,}11, \qquad \omega(8) = \frac{64-28}{8\cdot 3} = 1{,}5,$$

así que $\omega_s = 1{,}5$ y un elíptico con $\alpha_p = 1$, $\alpha_s = 22$ necesita $N = 3$ (orden 6).
*Supresor*: paso por debajo de 4 y por encima de 7 kHz con 2 dB; supresión entre 5 y 6 kHz con 30 dB. De nuevo $\hat{\omega}_0^2 = 28$, $B = 3$:

$$\omega(5) = \frac{5\cdot 3}{28-25} = 5, \qquad \omega(6) = \frac{6\cdot 3}{36-28} = 2{,}25,$$

y $\omega_s = 2{,}25$. El orden de Chebyshev es $N \geq \operatorname{arccosh}(41{,}3)/\operatorname{arccosh}(2{,}25) = 3{,}02$, es decir, $N = 4$. `cheb2ord` aplicada a las especificaciones originales da $N = 3$, porque busca los bordes de paso que minimizan el orden: también es un diseño válido.

Ejercicios 4–7. *Herramientas interactivas:* `T4-polos-ceros-s`, `T4-filtros-continuos` y `T4-transformaciones-frecuencia`.

## 4.3 Filtros IIR

Un filtro IIR se diseña casi siempre a partir de uno continuo: se obtiene $H_a(s)$ con los métodos de la apartado 4.2 y se convierte en $H(z)$. La conversión debe llevar el eje $j\omega$ a la circunferencia unidad, para conservar la respuesta en frecuencia; el semiplano izquierdo al interior del círculo, para conservar la estabilidad; y dar un $H(z)$ racional.

### 4.3.1 Invarianza al impulso

La idea más directa es muestrear la respuesta al impulso: $h[n] = T\,h_a(nT)$ (el factor $T$ mantiene la ganancia en la banda de paso). Si $H_a$ tiene polos simples, por fracciones simples,

$$H_a(s) = \sum_k \frac{r_k}{s - p_k} \;\Rightarrow\; h_a(t) = \sum_k r_k \mathrm{e}^{p_k t} u(t) \;\Rightarrow\; H(z) = \sum_k \frac{T\,r_k}{1 - \mathrm{e}^{p_k T} z^{-1}}.$$

Cada polo $p_k$ se convierte en $\mathrm{e}^{p_k T}$, que está dentro del círculo si $\operatorname{Re} p_k < 0$: la estabilidad se conserva. Pero, por el teorema del muestreo (Tema 2),

$$H(\mathrm{e}^{j\Omega}) = \sum_{m=-\infty}^{\infty} H_a\Big(j\frac{\Omega - 2\pi m}{T}\Big),$$

y como ningún filtro continuo de orden finito está limitado en banda, siempre hay *aliasing* (figura 4.11). Es tolerable si $H_a$ es despreciable por encima de $\pi/T$, lo que ocurre con paso bajo y paso banda; con paso alto y supresor de banda el método no sirve. Se usa sobre todo cuando interesa reproducir la forma de $h_a(t)$, por ejemplo al simular un sistema continuo.

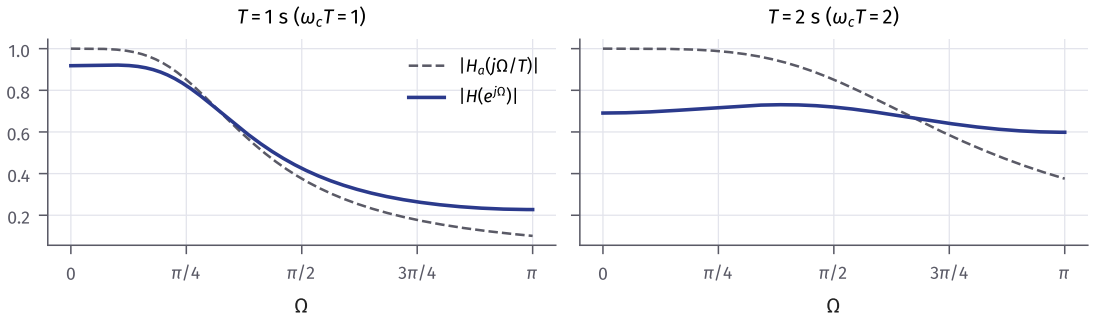



**Figura 4.11.** Invarianza al impulso de un Butterworth de orden 2 con $\omega_c = 1$ rad/s y dos periodos de muestreo. Con $T = 2$ s el aliasing deforma por completo la respuesta.

### 4.3.2 Transformada bilineal

**Definición 4.3.1: Transformada bilineal**

$$s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}} \qquad \Longleftrightarrow \qquad z = \frac{1 + sT/2}{1 - sT/2}, \qquad H(z) = H_a(s)\Big|_{s=\frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}}.$$

Aparece al integrar numéricamente con la regla del trapecio: $y[n] = y[n-1] + \frac{T}{2}(x[n] + x[n-1])$ tiene función de transferencia $\frac{T}{2}\frac{1+z^{-1}}{1-z^{-1}}$, que aproxima al integrador $1/s$. Con $s = \sigma + j\omega$, $|z|^2 = \frac{(1+\sigma T/2)^2 + (\omega T/2)^2}{(1-\sigma T/2)^2 + (\omega T/2)^2}$,

que es menor que 1 si y solo si $\sigma < 0$: el semiplano izquierdo va al *interior* del círculo y el eje $j\omega$ *sobre* la circunferencia. En $z = e^{j\Omega}$,

$$j\omega = \frac{2}{T}\,\frac{1 - e^{-j\Omega}}{1 + e^{-j\Omega}} = \frac{2}{T}\,\frac{e^{j\Omega/2} - e^{-j\Omega/2}}{e^{j\Omega/2} + e^{-j\Omega/2}} = j\,\frac{2}{T}\tan\frac{\Omega}{2}.$$

**Propiedad 4.3.2: Relación de frecuencias de la bilineal**

$$\omega = \frac{2}{T}\tan\frac{\Omega}{2}, \qquad \Omega = 2\arctan\frac{\omega T}{2}.$$

Todo el eje $\omega \in [0, \infty)$ se comprime en $\Omega \in [0, \pi)$, sin aliasing.

La compresión deforma las frecuencias (figura 4.12), pero conserva los niveles: un rizado de 1 dB del filtro continuo es un rizado de 1 dB del discreto. Para que los bordes del discreto queden en $\Omega_p$ y $\Omega_s$, hay que *predistorsionar*: el filtro continuo se diseña con bordes $\omega_p = \frac{2}{T}\tan\frac{\Omega_p}{2}$ y $\omega_s = \frac{2}{T}\tan\frac{\Omega_s}{2}$. Como $T$ aparece en la predistorsión y en la bilineal, se cancela y no tiene por qué ser el periodo de muestreo; lo habitual es tomar $T = 2$, con lo que $\omega = \tan(\Omega/2)$ y $s = \frac{1-z^{-1}}{1+z^{-1}}$. Los ceros en el infinito de un paso bajo continuo van a $\Omega = \pi$, es decir, a $z = -1$.

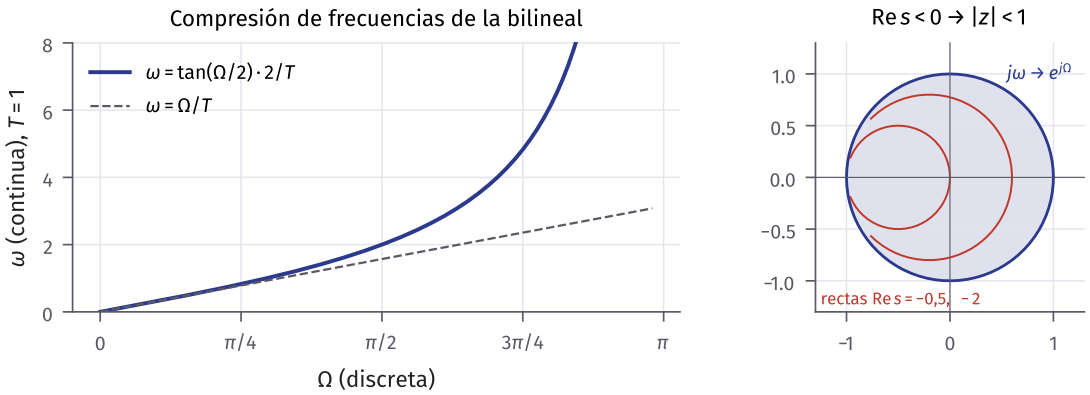



**Figura 4.12.** Relación de frecuencias de la bilineal ($T = 1$) y transformación del semiplano izquierdo en el interior del círculo unidad.

### 4.3.3 Procedimiento de diseño

1. Pasar los bordes a frecuencia discreta, $\Omega = 2\pi f / f_s$. Las atenuaciones no cambian.
2. Predistorsionar con $T = 2$: $\omega = \tan(\Omega/2)$ en cada borde.
3. Diseñar el filtro continuo: prototipo paso bajo y, si hace falta, transformación de frecuencias.
4. Aplicar la bilineal, $H(z) = H_a\big(\frac{1-z^{-1}}{1+z^{-1}}\big)$; con `bilinear` se indica $f_s = 1/T = 0{,}5$.
5. Comprobar la respuesta con `freqz`.

**Ejemplo 4.3.3: Paso bajo de Butterworth**

$f_s = 200$ kHz, banda de paso hasta 25 kHz con rizado de 0,5 dB y atenuación mínima de 15 dB desde 55 kHz. Los bordes discretos son $\Omega_p = 2\pi \cdot 25/200 = 0{,}25\pi$ y $\Omega_s = 0{,}55\pi$, y predistorsionados

$\omega_p = \tan(0{,}125\pi) = 0{,}4142$ y $\omega_s = \tan(0{,}275\pi) = 1{,}1708$. Con $E^2 - 1 = 0{,}1220$ y $A^2 - 1 = 30{,}62$,

$$N \geq \frac{\log_{10}(30{,}62/0{,}1220)}{2\log_{10}(1{,}1708/0{,}4142)} = 2{,}66 \;\rightarrow\; N = 3, \qquad \omega_c \in [0{,}588,\ 0{,}663].$$

Con $\omega_c = 0{,}588$ y la bilineal,

$$H(z) = \frac{0{,}0662\,(1+z^{-1})^3}{1 - 0{,}9357z^{-1} + 0{,}5672z^{-2} - 0{,}1016z^{-3}},$$

que atenúa 18 dB a 55 kHz (figura 4.13).

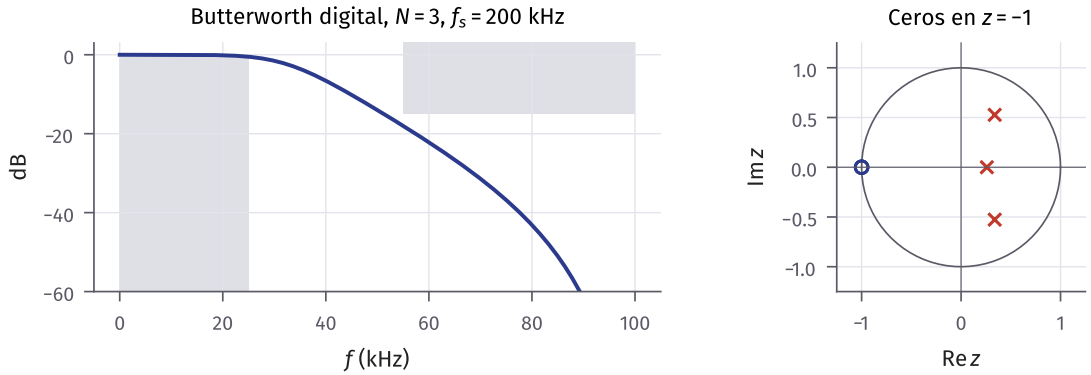



**Figura 4.13.** Respuesta y polos y ceros del ejemplo 4.3.3.

**En código**

El ejemplo 4.3.3 paso a paso (predistorsión, diseño continuo, bilineal) y con una sola llamada a `butter` con frecuencias discretas.

**Python**

```python
import numpy as np
from scipy import signal

fs = 200e3                      # Hz
Wp = 2*np.pi*25e3/fs            # π0,25
Ws = 2*np.pi*55e3/fs            # π0,55
# 1) Predistorsión con T = 2
wp, ws = np.tan(Wp/2), np.tan(Ws/2)
# 2) Filtro continuo
N, wc = signal.buttord(wp, ws, 0.5, 15,
                       analog=True)
b, a = signal.butter(N, wc, analog=True)
# 3) Bilineal: fs = 1/T = 0,5
bz, az = signal.bilinear(b, a, fs=0.5)
# Lo mismo en un paso (frecuencias en Hz):
N2, Wn = signal.buttord(25e3, 55e3, 0.5,
                        15, fs=fs)
bz2, az2 = signal.butter(N2, Wn, fs=fs)
print(N, np.allclose(az, az2))  # 3 True
```

**Matlab**

```matlab
fs = 200e3;                     % Hz
Wp = 2*pi*25e3/fs;              % π0,25
Ws = 2*pi*55e3/fs;              % π0,55
% 1) Predistorsión con T = 2
wp = tan(Wp/2); ws = tan(Ws/2);
% 2) Filtro continuo
[N, wc] = buttord(wp, ws, 0.5, 15, 's');
[b, a] = butter(N, wc, 's');
% 3) Bilineal: fs = 1/T = 0,5
[bz, az] = bilinear(b, a, 0.5);
% Lo mismo en un paso (frecuencias
% normalizadas a fs/2):
[N2, Wn] = buttord(Wp/pi, Ws/pi, 0.5, 15);
[bz2, az2] = butter(N2, Wn);
disp([N, N2])                   % 3, 3
```

Si se omite `'s'` (o `analog=True`), las funciones de orden y diseño de Matlab y `scipy.signal` hacen la predistorsión y la bilineal internamente; las frecuencias se dan normalizadas a la de Nyquist, $\Omega/\pi \in [0, 1]$, o en hercios con el argumento `fs` de `scipy`. El tipo se indica con `'high'` y `'stop'` en Matlab, y

con `'highpass'`, `'bandpass'` y `'bandstop'` en Python.

**Ejemplo 4.3.4: Paso alto de Chebyshev I y paso banda de Butterworth**

*Paso alto*, $f_s = 2$ kHz: paso desde 700 Hz con 1 dB, supresión por debajo de 500 Hz con 32 dB. $\Omega_p = 0{,}7\pi$ y $\Omega_s = 0{,}5\pi$ dan $\hat{\omega}_p = \tan(0{,}35\pi) = 1{,}9626$ y $\hat{\omega}_s = 1$. En el prototipo, $\omega_s = \hat{\omega}_p/\hat{\omega}_s = 1{,}9626$ y, con $\sqrt{A^2-1}/\varepsilon = 78{,}2$, $N \geq \operatorname{arccosh}(78{,}2)/\operatorname{arccosh}(1{,}9626) = 3{,}90$, es decir, $N = 4$.
*Paso banda*: paso en $[0{,}45\pi, 0{,}65\pi]$ con 1 dB y supresión fuera de $[0{,}3\pi, 0{,}75\pi]$ con 40 dB. Predistorsionando, $\hat{\omega}_{p1} = 0{,}8541$, $\hat{\omega}_{p2} = 1{,}6319$, $\hat{\omega}_{s1} = 0{,}5095$ y $\hat{\omega}_{s2} = 2{,}4142$; $\hat{\omega}_0^2 = 1{,}3937$ y $B = 0{,}7778$. Los bordes de supresión van a 2,86 y 2,36 en el prototipo, así que $\omega_s = 2{,}36$ y $N \geq 6{,}14$: $N = 7$, un filtro de orden 14 (figura 4.14).

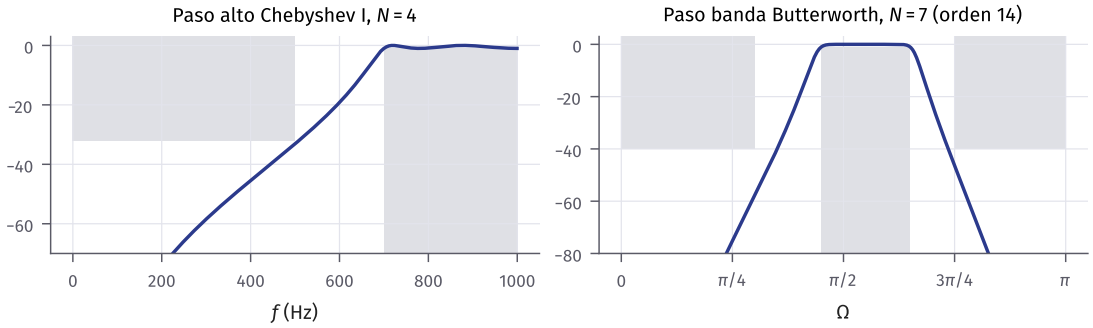



**Figura 4.14.** Filtros del ejemplo 4.3.4.

**En código**

Diseño directo del paso banda del ejemplo 4.3.4 en secciones de orden 2.

**Python**

```python
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt

# Bordes normalizados a π (1 = fs/2)
Wp = [0.45, 0.65]; Ws = [0.3, 0.75]
N, Wn = signal.buttord(Wp, Ws, 1, 40)
print(N)                    # 7: orden 14
sos = signal.butter(N, Wn, "bandpass",
                    output="sos")
W, H = signal.sosfreqz(sos, 2000)
plt.plot(W/np.pi, 20*np.log10(abs(H) + 1e-12))

plt.xlabel("Ωπ/"); plt.ylim(-80, 3)
plt.show()
# Filtrar una señal: signal.sosfilt(sos, x)
```

**Matlab**

```matlab
% Bordes normalizados a π (1 = fs/2)
Wp = [0.45 0.65]; Ws = [0.3 0.75];
[N, Wn] = buttord(Wp, Ws, 1, 40);
disp(N)                     % 7: orden 14
[b, a] = butter(N, Wn);     % paso banda
[H, W] = freqz(b, a, 2000);
plot(W/pi, 20*log10(abs(H)))
xlabel('\Omega/\pi'); ylim([-80 3])
% Para filtrar, mejor por secciones:
[z, p, k] = butter(N, Wn);
sos = zp2sos(z, p, k);      % sosfilt(sos, x)
```

### 4.3.4 Transformaciones de frecuencia en discreto

Alternativamente, se puede diseñar un paso bajo discreto con corte $\Omega_c$ y transformarlo sustituyendo $z^{-1}$ por un filtro paso todo de $\hat{z}^{-1}$, que lleva la circunferencia unidad sobre sí misma y el interior al interior (Constantinides, 1970). Las dos más sencillas son

$$\text{a paso bajo con corte } \hat{\Omega}_c \;:\; z^{-1} \to \frac{\hat{z}^{-1} - \alpha}{1 - \alpha\hat{z}^{-1}}, \qquad \alpha = \frac{\sin\left((\Omega_c - \hat{\Omega}_c)/2\right)}{\sin\left((\Omega_c + \hat{\Omega}_c)/2\right)};$$

$$\text{a paso alto con corte } \hat{\Omega}_c \;:\; z^{-1} \to -\frac{\hat{z}^{-1}+\alpha}{1+\alpha\hat{z}^{-1}}, \quad \alpha = -\frac{\cos\big((\Omega_c+\hat{\Omega}_c)/2\big)}{\cos\big((\Omega_c-\hat{\Omega}_c)/2\big)}.$$

Hay transformaciones de orden 2 a paso banda y supresor (Oppenheim y Schafer, capítulo 7). En Matlab están en `iirlp2hp`, `iirlp2bp` e `iirlp2bs`, con frecuencias normalizadas en $[0, 1]$.

### 4.3.5 Implementación

Un IIR de orden alto en forma directa (los coeficientes $a_k$ y $b_k$ del polinomio completo) es muy sensible al redondeo: las raíces de un polinomio de grado alto cambian mucho con pequeñas perturbaciones de sus coeficientes, y en los filtros selectivos los polos están muy juntos y cerca de la circunferencia. Se implementa como una cascada de *secciones de orden 2* (`sos`), cada una con un par de polos y un par de ceros: `butter(..., output="sos")` y `sosfilt` en Python; `zp2sos` y `sosfilt` en Matlab. El cuadro 4.3 compara los dos métodos de conversión.

**Cuadro 4.3.** Invarianza al impulso y transformada bilineal.

| | Invarianza al impulso | Bilineal |
|---|---|---|
| Relación | $z = \mathrm{e}^{sT}$ para los polos | $s = \frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}$ |
| Frecuencias | Lineal, $\Omega = \omega T$ | Comprimidas, $\omega = \frac{2}{T}\tan\frac{\Omega}{2}$ |
| Aliasing | Sí | No |
| Tipos | Paso bajo y paso banda | Todos |
| Conserva | La forma de $h(t)$ | Los niveles de $\lvert H\rvert$ |

Ejercicios 8–11. *Herramientas interactivas:* `T4-discretizacion-iir` y `T4-cuantificacion-coeficientes`.

## 4.4 Filtros FIR

Un FIR de $N$ coeficientes (orden $N-1$) es

$$y[n] = \sum_{k=0}^{N-1} h[k]\,x[n-k], \qquad H(\mathrm{e}^{j\Omega}) = \sum_{n=0}^{N-1} h[n]\,\mathrm{e}^{-j\Omega n}.$$

Todos sus polos están en $z = 0$, así que siempre es estable. No hay un filtro continuo del que partir, y se diseña directamente en discreto. Su mayor ventaja es que puede tener fase exactamente lineal.

### 4.4.1 Fase lineal

Si $h[n]$ es real y simétrica respecto a su centro $M = \frac{N-1}{2}$, $h[n] = h[N-1-n]$, los términos se agrupan por parejas:

$$h[n]\mathrm{e}^{-j\Omega n} + h[N-1-n]\mathrm{e}^{-j\Omega(N-1-n)} = \mathrm{e}^{-j\Omega M}\,h[n]\big(\mathrm{e}^{j\Omega(M-n)} + \mathrm{e}^{-j\Omega(M-n)}\big) = \mathrm{e}^{-j\Omega M}\,2h[n]\cos\big(\Omega(M-n)\big).$$

Sumando todas las parejas (y el término central si $N$ es impar), $H(\mathrm{e}^{j\Omega}) = \mathrm{e}^{-j\Omega M}A(\Omega)$ con $A(\Omega)$ real: la fase es lineal salvo saltos de $\pi$ donde $A$ cambia de signo, y el retardo de grupo vale $M = (N-1)/2$ muestras. Con antisimetría, $h[n] = -h[N-1-n]$, cada pareja da $2j\,h[n]\sin(\Omega(M-n))$ y $H = j\mathrm{e}^{-j\Omega M}A(\Omega)$: la fase es lineal con un desfase de $\pi/2$. Hay cuatro tipos (cuadro 4.4).

**Cuadro 4.4.** Los cuatro tipos de FIR de fase lineal, con $M = (N-1)/2$.

| Tipo | Simetría | $N$ | $A(\Omega)$ | Ceros forzados |
|---|---|---|---|---|
| I | par | impar | $h[M] + \sum_{n=0}^{M-1} 2h[n]\cos(\Omega(M-n))$ | ninguno |
| II | par | par | $\sum_{n=0}^{N/2-1} 2h[n]\cos(\Omega(M-n))$ | $z=-1$ |
| III | impar | impar | $\sum_{n=0}^{M-1} 2h[n]\sin(\Omega(M-n))$, con $h[M]=0$ | $z=1$ y $z=-1$ |
| IV | impar | par | $\sum_{n=0}^{N/2-1} 2h[n]\sin(\Omega(M-n))$ | $z=1$ |

Los ceros forzados se ven en $A(\Omega)$: en el tipo II, en $\Omega=\pi$ todos los cosenos $\cos(\pi(M-n))$ tienen argumento múltiplo impar de $\pi/2$ y se anulan; en los tipos III y IV los senos se anulan en $\Omega=0$, y en el III también en $\Omega=\pi$. Por eso el tipo II no puede ser paso alto ni supresor, el III solo puede ser paso banda, y el IV sirve para paso alto, derivadores y transformadores de Hilbert. El tipo I sirve para todo.

En el dominio $z$, la simetría equivale a $H(z) = \pm z^{-(N-1)}H(z^{-1})$: si $z_0$ es un cero, $1/z_0$ también lo es, y como $h$ es real, también $z_0^*$ y $1/z_0^*$. Los ceros aparecen en cuartetos, o en parejas recíprocas sobre el eje real, o en parejas conjugadas sobre la circunferencia (figura 4.15).

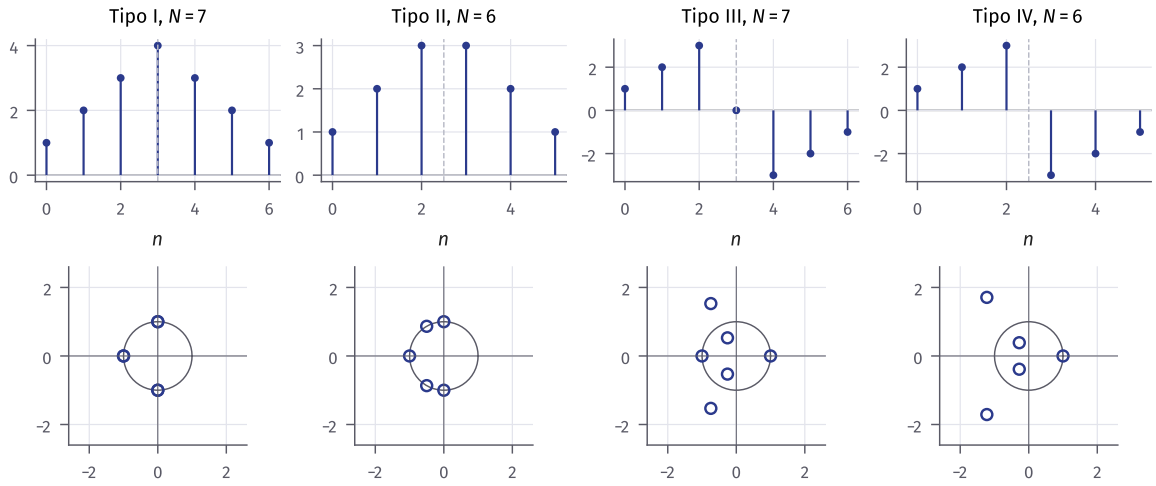



**Figura 4.15.** Un ejemplo de cada tipo y sus ceros. Obsérvense los ceros en $z=-1$ (tipos II y III) y en $z=1$ (tipos III y IV).

### 4.4.2 Método de las ventanas

Se parte de la respuesta al impulso del filtro ideal con fase cero, $h_d[n] = \frac{1}{2\pi}\int_{-\pi}^{\pi} H_d(e^{j\Omega})e^{j\Omega n}\,d\Omega$ (cuadro 4.5). Como $h_d$ es infinita y no causal:

1. se trunca a $|n| \le L$ ponderándola con una ventana $w[n]$: $h_w[n] = h_d[n]\,w[n]$, $N = 2L+1$ coeficientes;

2. se retrasa $L$ muestras: $h[n] = h_w[n-L]$, $0 \le n \le 2L$ (figura 4.16).

$h_d$ y $w$ son pares, de modo que $h$ es simétrica, de tipo I, con $H(e^{j\Omega}) = e^{-j\Omega L}H_w(e^{j\Omega})$. Por el teorema de la modulación,

$$H_w(e^{j\Omega}) = \frac{1}{2\pi}\int_{-\pi}^{\pi} H_d(e^{j\theta})\,W\big(e^{j(\Omega-\theta)}\big)\,d\theta \;:$$

el filtro ideal suavizado por la transformada de la ventana, igual que en el análisis espectral del Tema 3.

**Cuadro 4.5.** Respuestas al impulso de los filtros ideales con fase cero.

| Filtro | $h_d[0]$ | $h_d[n]$, $n \neq 0$ |
|---|---|---|
| Paso bajo ($\Omega_c$) | $\Omega_c/\pi$ | $\dfrac{\sin(\Omega_c n)}{\pi n}$ |
| Paso alto ($\Omega_c$) | $1-\Omega_c/\pi$ | $-\dfrac{\sin(\Omega_c n)}{\pi n}$ |
| Paso banda ($\Omega_1,\Omega_2$) | $(\Omega_2-\Omega_1)/\pi$ | $\dfrac{\sin(\Omega_2 n)-\sin(\Omega_1 n)}{\pi n}$ |
| Supresor ($\Omega_1,\Omega_2$) | $1-(\Omega_2-\Omega_1)/\pi$ | $-\dfrac{\sin(\Omega_2 n)-\sin(\Omega_1 n)}{\pi n}$ |
| Multibanda ($A_k$ en $[\Omega_{k-1},\Omega_k]$) | $\sum_k A_k \dfrac{\Omega_k-\Omega_{k-1}}{\pi}$ | $\sum_k (A_k - A_{k+1})\dfrac{\sin(\Omega_k n)}{\pi n}$, con $A_{M+1}=0$ |

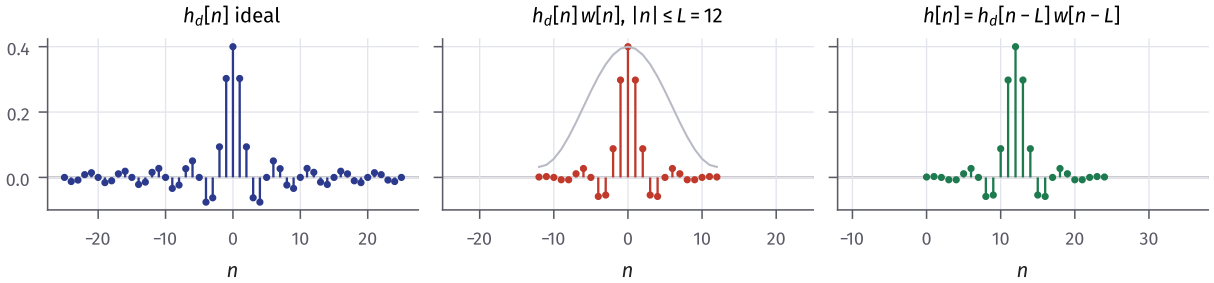



**Figura 4.16.** Método de las ventanas: respuesta ideal, truncada y ponderada (Hamming, $L = 12$), y retrasada.

### 4.4.3 Fenómeno de Gibbs y ventanas fijas

Con la ventana rectangular (truncamiento simple), el lóbulo principal de $W$, de anchura $4\pi/(2L+1)$, fija la anchura de la transición, que se estrecha al crecer $L$. Los lóbulos secundarios fijan el rizado, y como su área relativa no depende de $L$, el rizado tampoco: alrededor de un 9 % de sobreoscilación junto al borde y un primer lóbulo en la banda de supresión hacia $-21$ dB (figura 4.17). Es el *fenómeno de Gibbs*. Para reducir el rizado hay que usar ventanas que vayan suavemente a cero en los extremos (figura 4.18). Para $-L \le n \le L$:

$$w_{\text{Hann}} = \frac{1}{2} + \frac{1}{2}\cos\frac{2\pi n}{2L+1}, \qquad w_{\text{Hamming}} = 0{,}54 + 0{,}46\cos\frac{2\pi n}{2L+1},$$

$$w_{\text{Blackman}} = 0{,}42 + 0{,}5\cos\frac{2\pi n}{2L+1} + 0{,}08\cos\frac{4\pi n}{2L+1}.$$

**Cuadro 4.6.** Ventanas fijas para diseño de filtros (valores de Mitra). La atenuación mínima es la del filtro, no la del lóbulo secundario de la ventana.

| Ventana | Lóbulo principal | Lóbulo secundario | Atenuación mínima $\alpha_s$ | Transición $\Delta\Omega$ |
|---|---|---|---|---|
| Rectangular | $4\pi/(2L+1)$ | $-13{,}3$ dB | 20,9 dB | $0{,}92\pi/L$ |
| Hann | $8\pi/(2L+1)$ | $-31{,}5$ dB | 43,9 dB | $3{,}11\pi/L$ |
| Hamming | $8\pi/(2L+1)$ | $-42{,}7$ dB | 54,5 dB | $3{,}32\pi/L$ |
| Blackman | $12\pi/(2L+1)$ | $-58{,}1$ dB | 75,3 dB | $5{,}56\pi/L$ |

Con una ventana fija el rizado es el mismo en las dos bandas, $\delta_p \approx \delta_s$, y lo determina la ventana. El diseño se reduce a:

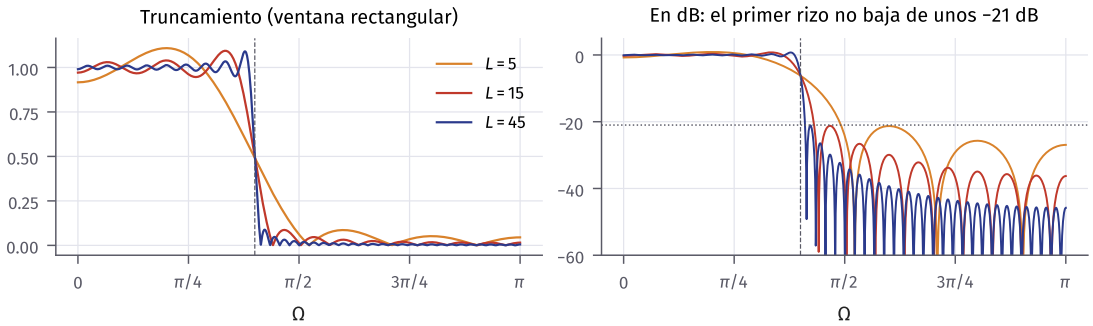



**Figura 4.17.** Truncamiento de un paso bajo ideal con $\Omega_c = 0{,}4\pi$: la transición se estrecha al crecer $L$, pero el rizado no baja.

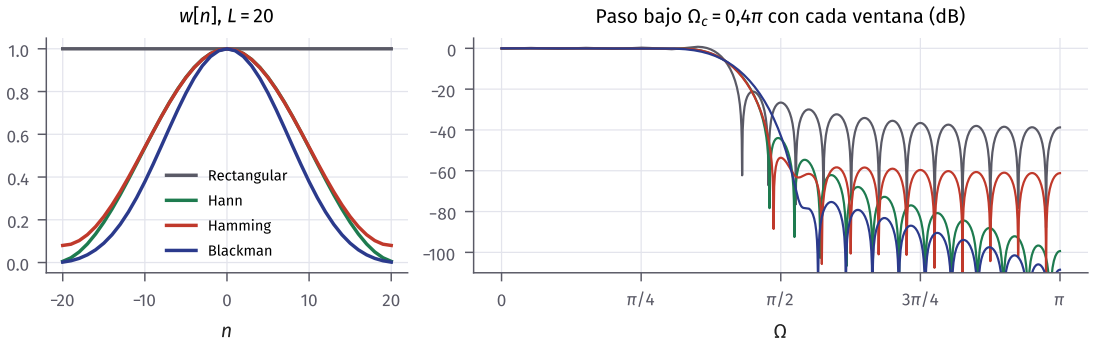



**Figura 4.18.** Ventanas fijas con $L = 20$ y el paso bajo que da cada una.

1. corte en el centro de la transición, $\Omega_c = (\Omega_p + \Omega_s)/2$, donde $|H| \approx 0{,}5$;
2. la ventana más sencilla con atenuación mínima mayor que $\alpha_s$;
3. el menor $L$ cuya transición sea menor que $\Delta\Omega = \Omega_s - \Omega_p$.

**Ejemplo 4.4.1: Paso bajo con ventana fija**

$\Omega_p = 0{,}3\pi$, $\Omega_s = 0{,}5\pi$, $\alpha_s = 40\,\text{dB}$. $\Omega_c = 0{,}4\pi$ y $\Delta\Omega = 0{,}2\pi$. Sirven Hann, Hamming y Blackman; con Hann, $L \geq 3{,}11\pi/0{,}2\pi = 15{,}6$, así que $L = 16$ y $N = 33$ coeficientes; Hamming pediría $L = 17$ y Blackman $L = 28$. La atenuación conseguida es de 44 dB, con un retardo de 16 muestras.

**En código**

El ejemplo 4.4.1 construyendo $h[n]$ a mano y con `firwin`/`fir1`.

**Python**

```python
import numpy as np
from scipy import signal

# Ωp = π0,3, Ωs = π0,5, αs = 40 dB -> Hann
Wc = 0.4*np.pi                  # Ω(p + Ωs)/2
L = 16                          # ππ3,11/(0,2)
n = np.arange(-L, L+1)
hd = Wc/np.pi * np.sinc(Wc/np.pi * n)
w = np.hanning(2*L + 3)[1:-1]
h = hd * w                      # 33 coef.
# h ya es causal si se indexa desde 0
W, H = signal.freqz(h, 1, 4096)
As = -20*np.log10(abs(H[W >= 0.5*np.pi]))
print(As.min())                 # 44 dB
# Casi igual (extremos nulos):
h2 = signal.firwin(2*L + 1, 0.4,
                   window="hann")
```

**Matlab**

```matlab
% Ωp = π0,3, Ωs = π0,5, αs = 40 dB -> Hann
Wc = 0.4*pi;                    % Ω(p + Ωs)/2
L = 16;                         % ππ3,11/(0,2)
n = -L:L;
hd = Wc/pi * sinc(Wc/pi * n);
w = hanning(2*L + 1).';         % sin ceros
h = hd .* w;                    % 33 coef.
[H, W] = freqz(h, 1, 4096);
As = -20*log10(abs(H(W >= 0.5*pi)));
disp(min(As))                   % 44 dB
% Casi igual (ganancia 1 en Ω = 0):
h2 = fir1(2*L, 0.4, hanning(2*L + 1));
```

### 4.4.4 Ventana de Kaiser

Las ventanas fijas no permiten elegir el rizado. La ventana de Kaiser tiene un parámetro $\beta$ que ajusta el compromiso entre rizado y anchura de transición (figura 4.19):

$$w[n] = \frac{I_0\!\left(\beta\sqrt{1-(n/L)^2}\right)}{I_0(\beta)}, \quad |n| \le L, \qquad I_0(u) = 1 + \sum_{r=1}^{\infty}\left(\frac{(u/2)^r}{r!}\right)^2,$$

donde $I_0$ es la función de Bessel modificada de orden cero. Kaiser obtuvo fórmulas empíricas para $\beta$ y $L$:

**Propiedad 4.4.2: Diseño con ventana de Kaiser**

$$\beta = \begin{cases} 0{,}1102\,(\alpha_s - 8{,}7) & \alpha_s > 50 \\ 0{,}5842\,(\alpha_s - 21)^{0{,}4} + 0{,}07886\,(\alpha_s - 21) & 21 \le \alpha_s \le 50 \\ 0 & \alpha_s < 21 \end{cases}, \qquad 2L \ge \frac{\alpha_s - 8}{2{,}285\,\Delta\Omega}.$$

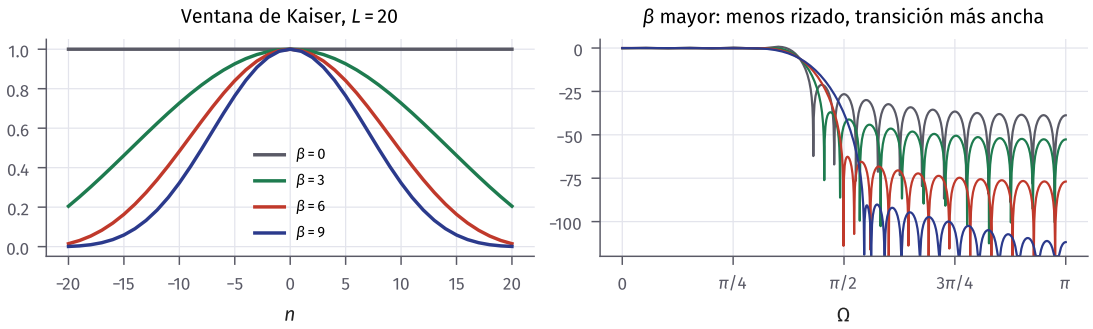



**Figura 4.19.** Ventana de Kaiser con $L = 20$ para varios valores de $\beta$, y el paso bajo correspondiente.

Para el ejemplo 4.4.1, $\beta = 0{,}5842 \cdot 19^{0{,}4} + 0{,}07886 \cdot 19 = 3{,}395$ y $2L \ge 32/(2{,}285 \cdot 0{,}2\pi) = 22{,}3$, de modo que $L = 12$: 25 coeficientes en lugar de 33, con 41,7 dB de atenuación (figura 4.20). Si se dan los rizados, $\alpha_s = -20\log_{10}\min(\delta_p, \delta_s)$, porque la ventana produce el mismo en las dos bandas. Otra ventana

ajustable es la de *Dolph–Chebyshev* (`chebwin`), que pone todos los lóbulos secundarios a la misma altura; para ella $2L \approx (2{,}056\alpha_s - 16{,}4)/(2{,}285\Delta\Omega)$.

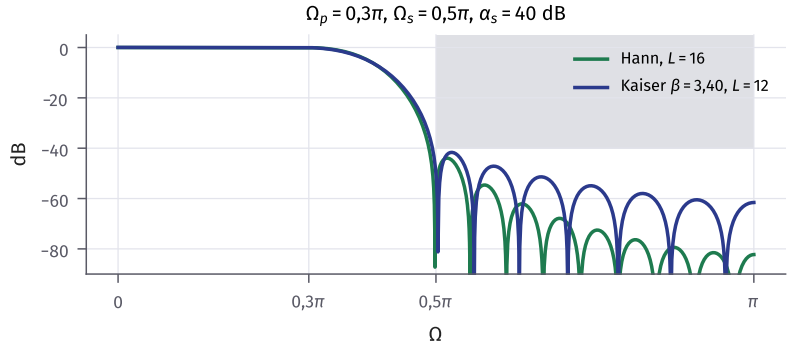



**Figura 4.20.** El paso bajo del ejemplo 4.4.1 con Hann ($L = 16$) y con Kaiser ($L = 12$).

**En código**

Paso alto con $\Omega_s = 0{,}4\pi$, $\Omega_p = 0{,}55\pi$ y $\delta = 0{,}01$ con `kaiserord`, y el mismo filtro con Parks–McClellan. El paso alto debe ser de tipo I: si `kaiserord` propone un número par de coeficientes, se añade uno.

**Python**

```python
import numpy as np
from scipy import signal

# Paso alto: Ωs = π0,4, Ωp = π0,55,
# rizado δ = 0,01 en las dos bandas
A = -20*np.log10(0.01)          # 40 dB
N, beta = signal.kaiserord(A, 0.15)
N += (N % 2 == 0)               # impar
h = signal.firwin(N, 0.475, pass_zero=False,
                  window=("kaiser", beta))
print(N, round(beta, 3))        # 31, 3.395
# Parks-McClellan: rizado uniforme
hpm = signal.remez(N, [0, 0.4, 0.55, 1],
                   [0, 1], fs=2)
```

**Matlab**

```matlab
% Paso alto: Ωs = π0,4, Ωp = π0,55,
% rizado δ = 0,01 en las dos bandas
d = 0.01;
[N, Wn, beta, tipo] = kaiserord( ...
    [0.4 0.55], [0 1], [d d]);
N = N + mod(N, 2);              % orden par
h = fir1(N, Wn, tipo, kaiser(N+1, beta));
disp([N+1, beta])               % 31, 3.395
% Parks-McClellan: rizado uniforme
hpm = firpm(N, [0 0.4 0.55 1], [0 0 1 1]);
```

### 4.4.5 Parks–McClellan y muestreo en frecuencia

El método de Parks–McClellan busca el FIR de fase lineal que minimiza el *error máximo* ponderado entre $A(\Omega)$ y la respuesta ideal en las bandas especificadas. El teorema de alternancia asegura que la solución es *equirrizada*: el error alcanza su máximo con signos alternos en al menos $M+2$ frecuencias, y el algoritmo de Remez las encuentra. Con el mismo número de coeficientes, el rizado es menor que con cualquier ventana (figura 4.21), y se pueden pedir rizados distintos en cada banda. El orden se estima con la fórmula de Kaiser

$$N - 1 \approx \frac{-20\log_{10}\sqrt{\delta_p\delta_s} - 13}{14{,}6\,\Delta\Omega/2\pi}.$$

Se usa con `firpm` en Matlab y `remez` en `scipy.signal`.

En el *muestreo en frecuencia* se fija la respuesta deseada en $N$ frecuencias $\Omega_k = 2\pi k/N$ y se obtiene $h[n]$ con la IDFT, imponiendo la simetría de fase lineal. Entre las muestras, la respuesta interpola con rizado; se mejora dejando muestras de valor intermedio en la transición o aplicando una ventana a $h[n]$.

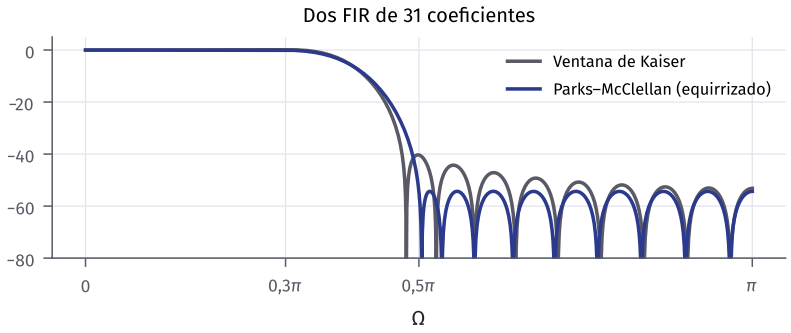



**Figura 4.21.** FIR de 31 coeficientes con Kaiser y con Parks–McClellan para las mismas bandas.

`fir2` (Matlab) y `firwin2` (`scipy.signal`) hacen esto a partir de una respuesta definida a tramos, lo que permite filtros multibanda arbitrarios (figura 4.22).

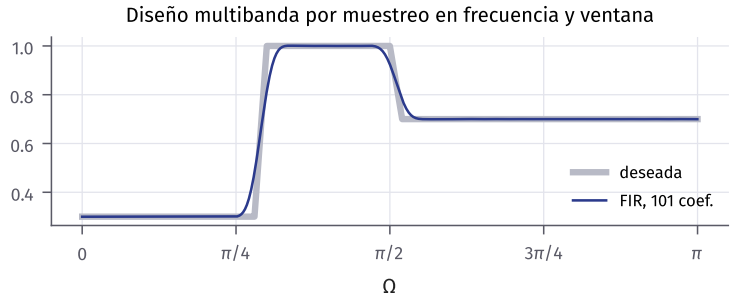



**Figura 4.22.** FIR multibanda de 101 coeficientes con `firwin2` y ventana de Hamming.

### 4.4.6 Coste y retardo

Un FIR de $N$ coeficientes cuesta $N$ productos por muestra (la mitad aprovechando la simetría) y retrasa $(N-1)/2$ muestras. En audio a 44,1–48 kHz son habituales 50–200 coeficientes en filtros básicos, 100–500 en filtros de cruce de altavoces, 500–2000 en cancelación de eco y hasta varios miles en procesado de alta calidad, con retardos de unos pocos a decenas de milisegundos. Para órdenes altos se filtra por bloques con la FFT (solapamiento y suma, Tema 3). En aplicaciones en tiempo real el retardo limita el orden; un IIR, o un FIR de fase mínima (apartado 4.5), retrasan mucho menos.

Ejercicios 12–15. *Herramientas interactivas:* `T4-tipos-fir`, `T4-fir-ventana` y `T4-parks-mcclellan`.

## 4.5 Filtros especiales

### 4.5.1 Fase cero

Un filtro de fase cero tiene $H(\mathrm{e}^{j\Omega})$ real y no negativa, lo que exige $h[n] = h[-n]$: no es causal. Dado cualquier $F(z)$, $H(z) = F(z)F(z^{-1})$ es de fase cero, porque $H(\mathrm{e}^{j\Omega}) = F(\mathrm{e}^{j\Omega})F^*(\mathrm{e}^{j\Omega}) = |F(\mathrm{e}^{j\Omega})|^2$ si $f$ es real. Sus polos y ceros aparecen en parejas recíprocas, así que $H$ no es causal y estable a la vez; pero si la señal está *grabada* se puede aplicar en dos pasos: se filtra $x$ con $F$, se invierte la salida en el tiempo, se vuelve a filtrar y se invierte otra vez. Como invertir en el tiempo una señal real conjuga su DTFT,

$$Y = \left(F\,(F\,X)^*\right)^* = F^*F\,X = |F|^2 X.$$

La atenuación en dB se duplica y la fase es nula (figura 4.23). Es lo que hace `filtfilt` en Matlab y en `scipy.signal`, que además cuida los transitorios de los extremos.

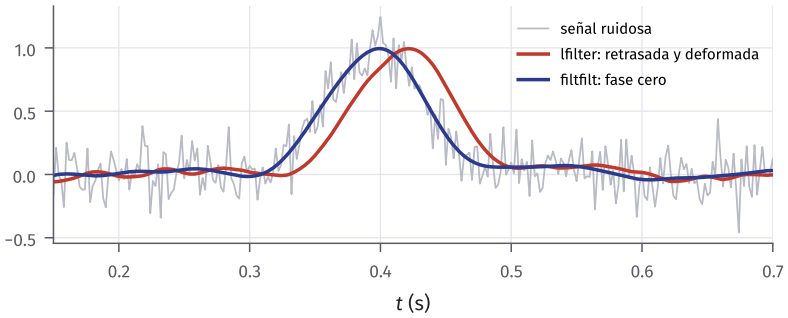



**Figura 4.23.** Un pulso con ruido filtrado con un Butterworth de orden 3 hacia delante (`lfilter`) y con fase cero (`filtfilt`).

**En código**

Filtrado normal y de fase cero; se compara la posición del máximo.

**Python**

```python
import numpy as np
from scipy import signal

fs = 500
t = np.arange(0, 1, 1/fs)
rng = np.random.default_rng(3)
x = np.exp(-((t - 0.4)/0.05)**2) \
    + 0.15*rng.standard_normal(t.size)
b, a = signal.butter(3, 15, fs=fs)
y1 = signal.lfilter(b, a, x)    # retrasa
y2 = signal.filtfilt(b, a, x)   # |H|^2
print(t[np.argmax(y1)], t[np.argmax(y2)])
```

**Matlab**

```matlab
fs = 500;
t = 0:1/fs:1-1/fs;
x = exp(-((t - 0.4)/0.05).^2) ...
    + 0.15*randn(size(t));
[b, a] = butter(3, 15/(fs/2));
y1 = filter(b, a, x);           % retrasa
y2 = filtfilt(b, a, x);         % |H|^2
[~, i1] = max(y1); [~, i2] = max(y2);
disp([t(i1), t(i2)])
```

### 4.5.2 Fase mínima, máxima y mixta

Reflejar un cero respecto a la circunferencia unidad, de $z_0$ a $1/z_0^*$, no cambia el módulo de la respuesta salvo una constante:

$$|1 - z_0 e^{-j\Omega}| = |z_0| \left|1 - \frac{1}{z_0^*} e^{-j\Omega}\right|.$$

Hay por tanto muchos sistemas con el mismo $|H|$ y distinta fase. Un sistema causal y estable es de *fase mínima* si todos sus ceros están dentro del círculo, de *fase máxima* si están todos fuera y de *fase mixta* en otro caso (figura 4.24). El de fase mínima tiene el menor retardo de grupo y la menor variación de fase de todos los que comparten módulo, concentra la energía de $h[n]$ al principio, y es el único cuya inversa $1/H(z)$ también es causal y estable, lo que importa para deshacer el efecto de un canal.

### 4.5.3 Filtros paso todo

Un filtro *paso todo* tiene $|H(e^{j\Omega})| = 1$ para toda $\Omega$. Con coeficientes reales y causal, tiene la forma

$$H_{\mathrm{PT}}(z) = \frac{a_M + a_{M-1} z^{-1} + \cdots + z^{-M}}{1 + a_1 z^{-1} + \cdots + a_M z^{-M}} = \frac{z^{-M} D(z^{-1})}{D(z)},$$

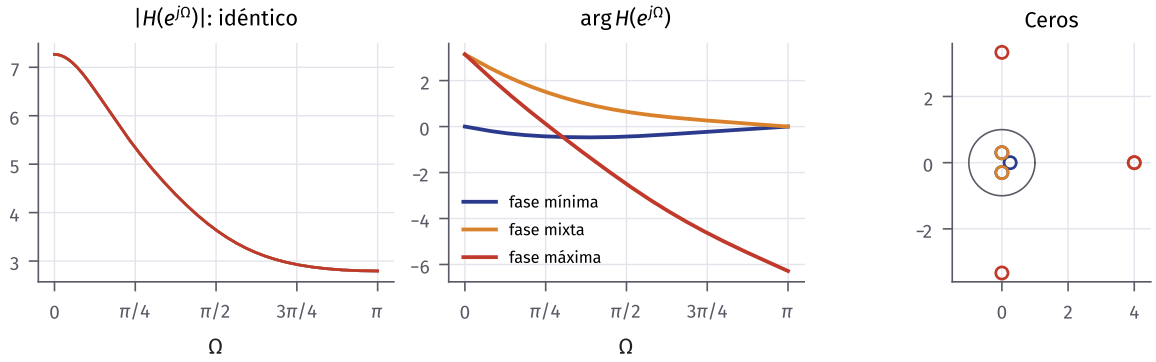



**Figura 4.24.** Tres sistemas con los mismos polos y el mismo módulo y ceros reflejados: fase mínima, mixta y máxima.

con el numerador igual al denominador con los coeficientes en orden inverso. En la circunferencia, $|e^{-j\Omega M} D(e^{-j\Omega})| = |D^*(e^{j\Omega})| = |D(e^{j\Omega})|$, y el cociente tiene módulo 1. Cada polo $p$ (dentro del círculo, si es estable) tiene enfrentado un cero en $1/p^*$. La fase decrece monótonamente $M\pi$ entre $\Omega = 0$ y $\Omega = \pi$, y el retardo de grupo es siempre positivo (figura 4.25). Se usan para corregir la fase de otro filtro sin tocar su módulo (igualadores de fase), como desfasadores y como bloques de filtros más complejos.

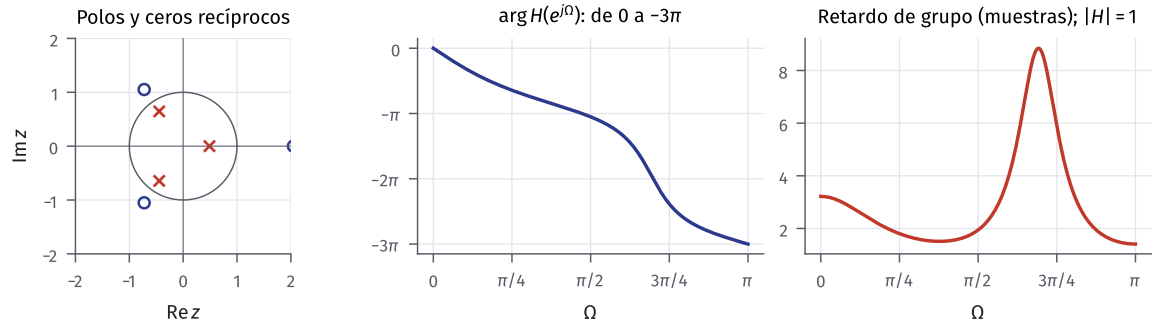



**Figura 4.25.** Paso todo de orden 3 con $D(z) = 1 + 0{,}4z^{-1} + 0{,}18z^{-2} - 0{,}3z^{-3}$: polos y ceros, fase y retardo de grupo.

**Propiedad 4.5.1: Descomposición fase mínima por paso todo**

Todo sistema racional, causal y estable se puede escribir $H(z) = H_{\min}(z)\, H_{\mathrm{PT}}(z)$, con $H_{\min}$ de fase mínima y $|H_{\min}| = |H|$.

Basta reflejar hacia dentro cada cero $z_0$ con $|z_0| > 1$ y compensarlo con un paso todo de primer orden con el polo en $1/z_0^*$ y el cero en $z_0$. Por ejemplo,

$$\frac{1 + 3z^{-1}}{1 - 0{,}5z^{-1}} = \frac{3\,(1 + \frac{1}{3}z^{-1})}{1 - 0{,}5z^{-1}} \cdot \frac{\frac{1}{3} + z^{-1}}{1 + \frac{1}{3}z^{-1}}.$$

Como el paso todo siempre añade retardo, de aquí se deduce que el de fase mínima es el de menor retardo de grupo.

### 4.5.4 Filtros en peine

Sustituyendo $z$ por $z^L$ en un filtro $G(z)$ se obtiene un *filtro en peine*:

$$H(z) = G(z^L), \qquad H(e^{j\Omega}) = G(e^{j\Omega L}).$$

La respuesta se comprime en un factor $L$ y se repite con periodo $2\pi/L$; en el tiempo, se intercalan $L-1$ ceros entre los coeficientes de $g[n]$. Con $G(z) = \frac{1}{2}(1+z^{-1})$, que tiene un cero en $\Omega = \pi$, sale $H(z) = \frac{1}{2}(1+z^{-L})$, con ceros en $\Omega = (2k+1)\pi/L$ (figura 4.26).

Para eliminar una frecuencia $f_0$ y todos sus armónicos, como el zumbido de la red eléctrica, se parte de una muesca en continua, $G(z) = \frac{1+a}{2}\frac{1-z^{-1}}{1-az^{-1}}$, y se toma $L = f_s/f_0$:

$$H(z) = \frac{1+a}{2}\,\frac{1-z^{-L}}{1-a\,z^{-L}}.$$

Sus ceros son las $L$ raíces de la unidad, en $\Omega = 2\pi k/L$, es decir, en $f = kf_0$; los polos están en los mismos ángulos con radio $a^{1/L}$. Cuanto más cerca de 1 está $a$, más estrechas son las muescas y más largo el transitorio. Es el filtro que proporciona `iircomb` en Matlab y en `scipy.signal`.

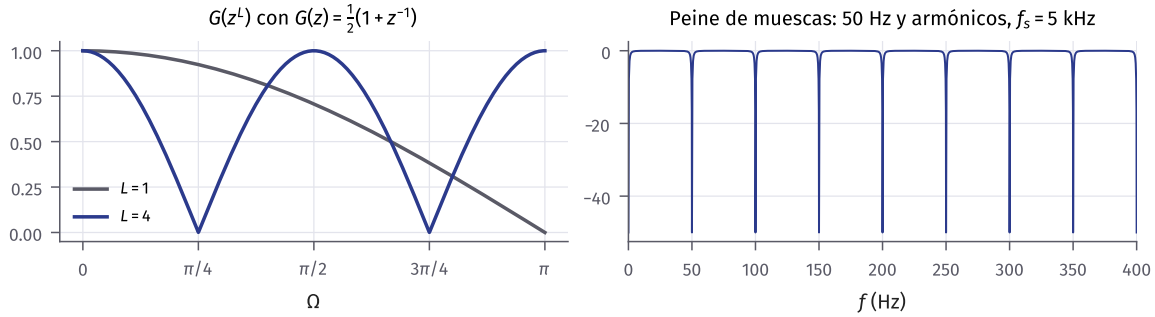



**Figura 4.26.** Peine FIR $\frac{1}{2}(1+z^{-L})$ y peine de muescas para 50 Hz y sus armónicos con $f_s = 5$ kHz.

**En código**

Peine de muescas para 50 Hz con $f_s = 5$ kHz y su respuesta en la componente útil (7 Hz), en 50 y 150 Hz y en 175 Hz.

**Python**

```python
import numpy as np
from scipy import signal

fs, f0 = 5000, 50
L = fs // f0                    # 100
a = 0.9                         # polos
# H(z) = (1+a)/2 (1 - z^-L)/(1 - a z^-L)
b = np.zeros(L + 1); b[[0, L]] = [1, -1]
d = np.zeros(L + 1); d[[0, L]] = [1, -a]
b *= (1 + a) / 2
f = np.array([7, 50, 150, 175])     # Hz
_, H = signal.freqz(b, d, f, fs=fs)
print(np.round(abs(H), 3))  # 1 0 0 1
```

**Matlab**

```matlab
fs = 5000; f0 = 50;
L = fs / f0;                    % 100
a = 0.9;                        % polos
% H(z) = (1+a)/2 (1 - z^-L)/(1 - a z^-L)
b = zeros(1, L+1); b([1 L+1]) = [1 -1];
d = zeros(1, L+1); d([1 L+1]) = [1 -a];
b = b * (1 + a) / 2;
f = [7 50 150 175];                 % Hz
H = freqz(b, d, f, fs);
disp(round(abs(H)*1000)/1000)  % 1 0 0 1
```

Ejercicios 16–18. *Herramientas interactivas:* `T4-fase-cero`, `T4-fase-minima-paso-todo` y `T4-peine`.

## Bibliografía


- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos 5 (fase mínima, paso todo) y 10 (diseño de filtros).

- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Capítulos 5 (fase lineal, fase mínima y paso todo) y 7 (diseño de filtros).
- S. K. Mitra, *Procesamiento de señales digitales: un enfoque basado en computadora*, 3.ª ed. McGraw-Hill, 2007. Capítulos de diseño de filtros IIR y FIR; de ahí proceden los valores de la tabla de ventanas.
- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulos de filtros. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.
- V. K. Ingle y J. G. Proakis, *Digital Signal Processing Using MATLAB*, 3.ª ed. Cengage Learning, 2012. Capítulos 7 y 8, con ejemplos en Matlab.

## 4.6 Ejercicios

Los ejercicios 1–3 corresponden a la parte 1 del tema (filtros ideales, especificaciones y fase), los ejercicios 4–7 a la parte 2 (filtros continuos), los ejercicios 8–11 a la parte 3 (filtros IIR), los ejercicios 12–15 a la parte 4 (filtros FIR) y los ejercicios 16–18 a la parte 5 (filtros especiales). Se usa $\omega$ para la frecuencia continua y $\Omega = 2\pi f/f_s$ para la discreta, y $E = 10^{\alpha_p/20}$, $A = 10^{\alpha_s/20}$, $\varepsilon = \sqrt{E^2 - 1}$. Los resultados se pueden comprobar con `scipy.signal` o Matlab y con los ejemplos de `teoría/T4/codigo/`.

### Filtros ideales, especificaciones y fase

**Ejercicio 1** (especificaciones de un filtro discreto) Un paso bajo discreto trabaja a $f_s = 8\,\text{kHz}$. Debe tener un rizado $\delta_p = 0{,}05$ hasta 1 kHz y $|H| \le \delta_s = 0{,}001$ desde 1,5 kHz.

a) Expresa los bordes en $\Omega$ y normalizados a la frecuencia de Nyquist, como los piden Matlab y `scipy.signal`.

b) Expresa los rizados como $\alpha_p$ y $\alpha_s$ en dB.

c) ¿Qué frecuencia de corte en hercios tendría un FIR diseñado con ventana?

**Solución**

a) $\Omega_p = 2\pi \cdot 1/8 = 0{,}25\pi$ y $\Omega_s = 0{,}375\pi$; normalizados, 0,25 y 0,375.

b) Respecto a la ganancia nominal 1, $\alpha_p = -20\log_{10}(1-\delta_p) = 0{,}45\,\text{dB}$ (si se mide entre el máximo $1+\delta_p$ y el mínimo $1-\delta_p$, $20\log_{10}\frac{1{,}05}{0{,}95} = 0{,}87\,\text{dB}$). $\alpha_s = -20\log_{10} 0{,}001 = 60\,\text{dB}$.

c) El centro de la transición: $f_c = 1{,}25\,\text{kHz}$ ($\Omega_c = 0{,}3125\pi$).

**Ejercicio 2** (retardo de grupo) Compara la fase de dos paso bajo sencillos:

a) $h[n] = \frac{1}{4}(1, 2, 1)$;

b) $H(z) = \dfrac{0{,}5}{1 - 0{,}5z^{-1}}$.

Calcula en cada caso el retardo de grupo en $\Omega = 0$, $\pi/2$ y $\pi$, y di cuál distorsiona la forma de un pulso.

**Solución**

a) $H(e^{j\Omega}) = \frac{1}{4}(1 + 2e^{-j\Omega} + e^{-j2\Omega}) = e^{-j\Omega}\frac{1+\cos\Omega}{2}$. El factor real es no negativo, así que la fase es $-\Omega$ y el retardo vale 1 muestra en todas las frecuencias: fase lineal.

b) Con $a = 0{,}5$, $\arg H = -\arctan\frac{a\sin\Omega}{1-a\cos\Omega}$ y $\tau(\Omega) = \dfrac{a\cos\Omega - a^2}{1 - 2a\cos\Omega + a^2}$: $\tau(0) = 1$, $\tau(\pi/2) = -0{,}2$ y $\tau(\pi) = -1/3$. Las frecuencias se retrasan de forma distinta, y el pulso se deforma.

**Ejercicio 3** (filtro de muesca) Diseña un filtro de muesca de orden 2 que elimine los 50 Hz de la red en una señal muestreada a $f_s = 1$ kHz, con polos de radio $r = 0{,}95$.

a) Escribe $H(z)$ con coeficientes numéricos y la ecuación en diferencias.

b) Calcula la ganancia en continua y en $\Omega = \pi$. ¿Cómo la normalizarías?

c) Estima la anchura de la muesca en hercios.

**Solución**

a) $\Omega_0 = 2\pi \cdot 50/1000 = 0{,}1\pi$ y $2\cos\Omega_0 = 1{,}9021$:

$$H(z) = \frac{1 - 1{,}9021z^{-1} + z^{-2}}{1 - 1{,}8070z^{-1} + 0{,}9025z^{-2}},$$

$$y[n] = x[n] - 1{,}9021x[n-1] + x[n-2] + 1{,}8070y[n-1] - 0{,}9025y[n-2].$$

b) $H(1) = 0{,}0979/0{,}0955 = 1{,}025$ y $H(-1) = 3{,}902/3{,}709 = 1{,}052$. Se puede multiplicar el numerador por $1/1{,}025$ para tener ganancia 1 en continua.

c) Unos $2(1-r) = 0{,}1$ rad, es decir, $0{,}1 \cdot 1000/2\pi \approx 16$ Hz entre los puntos de $-3$ dB (numéricamente, 16,4 Hz).

## Filtros continuos

**Ejercicio 4** (Butterworth) Diseña el paso bajo de Butterworth de orden mínimo con atenuación de 3 dB a 2 kHz y de al menos 30 dB a 6 kHz.

a) Calcula el orden y la frecuencia de corte.

b) Da la posición de los polos y escribe $H(s)$ como producto de factores de orden 2.

**Solución**

a) $E^2 - 1 = 10^{0{,}3} - 1 = 0{,}995 \approx 1$ y $A^2 - 1 = 999$: $N \geq \frac{\log_{10}(999/0{,}995)}{2\log_{10}3} = 3{,}15$, $N = 4$. Como $\alpha_p = 3$ dB, $\omega_c \approx \omega_p = 2\pi \cdot 2000 = 12\,566$ rad/s.

b) $p_k = \omega_c e^{j\pi(2k+3)/8}$: ángulos de $\pm 112{,}5^\circ$ y $\pm 157{,}5^\circ$, es decir, $\omega_c(-0{,}3827 \pm 0{,}9239j)$ y $\omega_c(-0{,}9239 \pm 0{,}3827j)$.

$$H(s) = \frac{\omega_c^4}{(s^2 + 0{,}7654\,\omega_c s + \omega_c^2)(s^2 + 1{,}8478\,\omega_c s + \omega_c^2)}.$$

**Ejercicio 5** (comparación de familias) Un paso bajo debe atenuar como mucho 0,5 dB hasta 10 kHz y al menos 50 dB desde 20 kHz. Calcula el orden mínimo con Butterworth, Chebyshev y elíptico.

**Solución**

$\varepsilon^2 = 10^{0{,}05} - 1 = 0{,}1220$, $A^2 - 1 \approx 10^5$ y $1/k_1 = \sqrt{A^2-1}/\varepsilon = 905$; $1/k = 2$. Butterworth: $N \geq \log(905)/\log 2 = 9{,}82$, $N = 10$. Chebyshev (I o II): $N \geq \operatorname{arccosh}(905)/\operatorname{arccosh}(2) = 7{,}50/1{,}317 = 5{,}70$, $N = 6$. Elíptico: $k' = \sqrt{1-0{,}25} = 0{,}866$, $\rho_0 = 0{,}0179$, $N \geq \frac{2\log_{10}(4\cdot 905)}{\log_{10}(1/\rho)} = \frac{7{,}12}{1{,}75} = 4{,}08$, $N = 5$. Con `buttord`, `cheb1ord` y `ellipord`: 10, 6 y 5.

**Ejercicio 6** (polinomios de Chebyshev) Un Chebyshev tipo I de orden 3 tiene $\alpha_p = 1$ dB y $\omega_p = 1$ rad/s.

a) Obtén $T_3(x)$ con la recurrencia.

b) ¿Cuánto vale $|H|$ en $\omega = 0$ y en $\omega = \omega_p$? ¿Y si el orden fuera par?

c) ¿Qué atenuación tiene en $\omega = 2$?

**Solución**

a) $T_2 = 2x \cdot x - 1 = 2x^2 - 1$ y $T_3 = 2x(2x^2-1) - x = 4x^3 - 3x$.

b) $T_3(0) = 0$, así que $|H(0)| = 1$; $T_3(1) = 1$ y $|H(j\omega_p)| = 1/\sqrt{1+\varepsilon^2} = 1/E$, la atenuación de 1 dB. Con $N$ par, $|T_N(0)| = 1$ y $|H(0)| = 1/E$: el rizado empieza en su mínimo.

c) $T_3(2) = 26$, $\varepsilon^2 = 0{,}2589$: $10\log_{10}(1 + 0{,}2589 \cdot 676) = 22{,}5$ dB.

**Ejercicio 7** (paso banda continuo) Se quiere un paso banda de Butterworth con banda de paso de 900 a 1100 Hz (1 dB) y atenuación de al menos 30 dB por debajo de 600 Hz y por encima de 1500 Hz.

a) Calcula $\hat{\omega}_0$, $B$ y el $\omega_s$ del prototipo.

b) Calcula el orden del prototipo y el del paso banda.

c) ¿Es la plantilla geométricamente simétrica? ¿Qué borde se cumple con margen?

**Solución**

a) En kHz: $\hat{\omega}_0^2 = 0{,}9 \cdot 1{,}1 = 0{,}99$ ($f_0 = 995$ Hz) y $B = 0{,}2$. Los bordes de supresión van a $|0{,}36 - 0{,}99|/(0{,}6 \cdot 0{,}2) = 5{,}25$ y $(2{,}25 - 0{,}99)/(1{,}5 \cdot 0{,}2) = 4{,}2$; $\omega_s = 4{,}2$.

b) $N \geq \frac{\log_{10}(999/0{,}2589)}{2\log_{10} 4{,}2} = 2{,}88$, $N = 3$; el paso banda es de orden 6.

c) No: $0{,}6 \cdot 1{,}5 = 0{,}9 \neq 0{,}99$. El borde inferior (600 Hz) se cumple con margen, porque su imagen en el prototipo (5,25) está más lejos que la del superior.

## Filtros IIR

**Ejercicio 8** (invarianza al impulso) Aplica la invarianza al impulso con $T = 0{,}1$ s a $H_a(s) = \dfrac{1}{(s+1)(s+2)}$.

a) Obtén $H(z)$ y sus polos.

b) Compara la ganancia en continua de $H(z)$ con la de $H_a(s)$. ¿Por qué no coinciden exactamente?

**Solución**

a) $H_a(s) = \frac{1}{s+1} - \frac{1}{s+2}$, de modo que $H(z) = T\Big[\frac{1}{1-\mathrm{e}^{-T}z^{-1}} - \frac{1}{1-\mathrm{e}^{-2T}z^{-1}}\Big]$, con polos en $\mathrm{e}^{-0{,}1} = 0{,}9048$ y $\mathrm{e}^{-0{,}2} = 0{,}8187$.

b) $H(1) = 0{,}1\big(\frac{1}{0{,}0952} - \frac{1}{0{,}1813}\big) = 0{,}4992$ frente a $H_a(0) = 0{,}5$. La diferencia es el aliasing: $H(\mathrm{e}^{j0}) = \sum_m H_a(j2\pi m/T)$ incluye las copias desplazadas, que no son nulas aunque sí pequeñas.

**Ejercicio 9** (bilineal de primer orden) Convierte $H_a(s) = \dfrac{\omega_c}{s+\omega_c}$ en un paso bajo discreto con corte $\Omega_c = 0{,}2\pi$ usando la bilineal con $T = 2$.

a) Obtén $H(z)$ y comprueba que $|H(\mathrm{e}^{j\Omega_c})| = 1/\sqrt{2}$.

b) ¿Dónde quedaría el corte si no se predistorsionara ($\omega_c = \Omega_c/T$)?

**Solución**

a) $\omega_c = \tan(0{,}1\pi) = 0{,}3249$. Con $s = \frac{1-z^{-1}}{1+z^{-1}}$,

$$H(z) = \frac{\omega_c(1+z^{-1})}{(1+\omega_c)+(\omega_c-1)z^{-1}} = \frac{0{,}2452\,(1+z^{-1})}{1-0{,}5095z^{-1}}.$$

En $\Omega_c$ la bilineal lleva a $\omega = \omega_c$, donde $|H_a| = 1/\sqrt{2}$; numéricamente, 0,7071.

b) Con $\omega_c = 0{,}1\pi = 0{,}3142$, el corte discreto es $2\arctan(0{,}3142) = 0{,}194\pi$, un 3 % por debajo. El error crece con la frecuencia.

**Ejercicio 10** (importancia de la predistorsión) Un paso bajo de Butterworth discreto a $f_s = 10\,\mathrm{kHz}$ debe atenuar como mucho 1 dB hasta 1 kHz y al menos 25 dB desde 2 kHz. Calcula el orden con predistorsión y sin ella (usando $\omega = \Omega/T$). ¿Cuál es el correcto?

**Solución**

$\Omega_p = 0{,}2\pi$, $\Omega_s = 0{,}4\pi$; $(A^2-1)/(E^2-1) = 315{,}2/0{,}2589 = 1218$. Con predistorsión, $\omega_p = \tan(0{,}1\pi) = 0{,}3249$ y $\omega_s = \tan(0{,}2\pi) = 0{,}7265$: $N \geq \dfrac{\log_{10} 1218}{2\log_{10} 2{,}236} = 4{,}41$, $N = 5$, que es lo que da `buttord(0.2,0.4,1,25)`. Sin predistorsión, la razón sería 2 y $N \geq 5{,}12$, $N = 6$: el filtro sería más largo de lo necesario y, sobre todo, sus bordes no quedarían donde se pidió. Lo correcto es predistorsionar.

**Ejercicio 11** (paso alto IIR) Diseña un paso alto de Chebyshev I discreto a $f_s = 8\,\mathrm{kHz}$ con banda de paso desde 2 kHz (1 dB) y atenuación de al menos 40 dB por debajo de 1 kHz. Calcula los bordes predistorsionados, el $\omega_s$ del prototipo y el orden.

**Solución**

$\Omega_p = 0{,}5\pi$ y $\Omega_s = 0{,}25\pi$; $\hat{\omega}_p = \tan(\pi/4) = 1$ y $\hat{\omega}_s = \tan(\pi/8) = 0{,}4142$. Prototipo: $\omega_s = \hat{\omega}_p/\hat{\omega}_s = 2{,}414$. Con $\sqrt{A^2-1}/\varepsilon = 196{,}5$, $N \geq \dfrac{\operatorname{arccosh}(196{,}5)}{\operatorname{arccosh}(2{,}414)} = \dfrac{5{,}97}{1{,}53} = 3{,}91$, $N = 4$. Se diseña el prototipo, se transforma con $s = 1/\hat{s}$ y se aplica la bilineal (o `cheby1(4,1,0.5,'high')`).

## Filtros FIR

**Ejercicio 12** (tipos de fase lineal) Para cada FIR, indica el tipo, el retardo de grupo y los ceros, y di si podría ser paso bajo o paso alto:

a) $h = (1, 3, 3, 1)$; b) $h = (1, -2, 2, -1)$.

**Solución**

a) Simétrica con $N = 4$: tipo II, retardo 1,5 muestras. $H(z) = (1 + z^{-1})^3$: triple cero en $z = -1$. $A(\Omega) = 2\cos(3\Omega/2) + 6\cos(\Omega/2)$, que vale 8 en $\Omega = 0$ y 0 en $\pi$: paso bajo, nunca paso alto.

b) Antisimétrica con $N = 4$: tipo IV, retardo 1,5 muestras y desfase adicional $\pi/2$. $H(z) = (1 - z^{-1})(1 - z^{-1} + z^{-2})$: ceros en $z = 1$ y $e^{\pm j\pi/3}$. El cero en $z = 1$ impide que sea paso bajo; $|H(-1)| = 6$: paso alto.

**Ejercicio 13** (paso alto con ventana) Diseña un paso alto FIR con $\Omega_s = 0{,}4\pi$, $\Omega_p = 0{,}6\pi$ y $\alpha_s = 50$ dB.

a) Elige la ventana fija y calcula $L$, el número de coeficientes y $h[n]$.

b) Repite con la ventana de Kaiser.

**Solución**

a) $\Omega_c = 0{,}5\pi$ y $\Delta\Omega = 0{,}2\pi$. Hann no llega (43,9 dB); Hamming sí (54,5 dB): $L \geq 3{,}32\pi/0{,}2\pi = 16{,}6$, $L = 17$ y $N = 35$. Con $m = n - 17$,

$$h[n] = \left(\delta[m] - \frac{\sin(0{,}5\pi m)}{\pi m}\right)\left(0{,}54 + 0{,}46\cos\frac{2\pi m}{35}\right), \quad 0 \leq n \leq 34,$$

con $h[17] = 0{,}5$. Numéricamente atenúa 52 dB.

b) $\beta = 0{,}5842 \cdot 29^{0{,}4} + 0{,}07886 \cdot 29 = 4{,}53$ y $2L \geq 42/(2{,}285 \cdot 0{,}2\pi) = 29{,}3$, $L = 15$: 31 coeficientes.

**Ejercicio 14** (filtro de audio) Un filtro antialiasing digital a 48 kHz debe dejar pasar hasta 20 kHz con $\delta_p = 0{,}01$ y atenuar desde 22 kHz con $\delta_s = 10^{-4}$.

a) Calcula $\beta$ y el número de coeficientes con la ventana de Kaiser, y el retardo en milisegundos.

b) Estima el número de coeficientes con Parks–McClellan.

**Solución**

a) La ventana impone el mismo rizado en las dos bandas: $\alpha_s = -20\log_{10} 10^{-4} = 80$ dB. $\Delta\Omega = 2\pi \cdot 2/48 = 0{,}0833\pi$. $\beta = 0{,}1102(80 - 8{,}7) = 7{,}86$ y $2L \geq 72/(2{,}285 \cdot 0{,}2618) = 120{,}4$, $L = 61$: 123 coeficientes y un retardo de 61 muestras, 1,27 ms.

b) $N - 1 \approx \frac{-20\log_{10}\sqrt{0{,}01 \cdot 10^{-4}} - 13}{14{,}6 \cdot 2/48} = \frac{47}{0{,}608} = 77$: unos 79 coeficientes, porque Parks–McClellan aprovecha que $\delta_p$ puede ser mayor que $\delta_s$.

**Ejercicio 15** (truncamiento y Gibbs) Se trunca el paso bajo ideal con $\Omega_c = \pi/2$ a $|n| \leq 10$ (ventana rectangular).

a) Calcula $h_d[n]$ para $n = 0, \ldots, 3$ y explica por qué los términos pares distintos de cero son nulos.

b) ¿Cuánto vale $H_w(e^{j0})$? ¿Y el máximo de $H_w$?

c) ¿Qué cambia si se usa $|n| \le 40$?

**Solución**

a) $h_d[0] = 0{,}5$, $h_d[1] = 1/\pi = 0{,}318$, $h_d[2] = 0$, $h_d[3] = -1/(3\pi) = -0{,}106$. Con $\Omega_c = \pi/2$, $\sin(\pi n/2) = 0$ para $n$ par.

b) $H_w(e^{j0}) = \sum_n h_d[n] = 1{,}032$ (no exactamente 1, por el truncamiento); el máximo, cerca del borde, es 1,091: la sobreoscilación de Gibbs, un 9 %.

c) La transición se estrecha a la cuarta parte y los rizados se acercan al borde, pero la sobreoscilación sigue siendo del 9 %.

## Filtros especiales

**Ejercicio 16** (fase cero) Se filtra una señal grabada con $F(z) = \dfrac{0{,}2}{1 - 0{,}8z^{-1}}$ con `lfilter` y con `filtfilt`. Compara la ganancia en $\Omega = 0$ y en $\Omega = \pi$ y el retardo en ambos casos. ¿Se podría hacer lo mismo en tiempo real?

**Solución**

$F(1) = 0{,}2/0{,}2 = 1$ y $|F(-1)| = 0{,}2/1{,}8 = 0{,}111$: $-19{,}1$ dB en $\Omega = \pi$ con una pasada. Con `filtfilt` la respuesta es $|F|^2$: ganancia 1 en continua y $-38{,}2$ dB en $\pi$, sin desfase; con `lfilter` el retardo de grupo en continua es $0{,}8/0{,}2 = 4$ muestras. No se puede en tiempo real: la pasada hacia atrás necesita las muestras futuras.

**Ejercicio 17** (fase mínima y paso todo) Sea $H(z) = \dfrac{1 - 2z^{-1}}{1 + 0{,}8z^{-1}}$.

a) ¿Es de fase mínima? ¿Es estable y causal su inverso?

b) Escríbelo como $H_{\min}(z)H_{\text{PT}}(z)$ y comprueba que $|H_{\text{PT}}| = 1$.

**Solución**

a) El cero está en $z = 2$, fuera del círculo: no es de fase mínima. $1/H$ tendría un polo en $z = 2$: causal pero inestable.

b) Se refleja el cero a $1/2$:

$$H(z) = \underbrace{\frac{2\,(1 - 0{,}5z^{-1})}{1 + 0{,}8z^{-1}}}_{H_{\min}} \cdot \underbrace{\frac{0{,}5 - z^{-1}}{1 - 0{,}5z^{-1}}}_{H_{\text{PT}}}.$$

$|0{,}5 - e^{-j\Omega}| = |e^{-j\Omega}|\,|0{,}5e^{j\Omega} - 1| = |1 - 0{,}5e^{-j\Omega}|$, así que $|H_{\text{PT}}| = 1$. $H_{\min}$ tiene el cero en 0,5 y su inversa es estable.

**Ejercicio 18** (peine para la red) Una señal de ECG se muestrea a $f_s = 8$ kHz y lleva interferencia de 50 Hz y sus armónicos.

a) ¿Qué $L$ hace que $1 - z^{-L}$ anule 50 Hz y sus armónicos? ¿Qué ganancia tiene a 25 Hz? ¿Qué problema tiene para el ECG?

b) Con $H(z) = \frac{1+a}{2}\frac{1-z^{-L}}{1-az^{-L}}$ y $a = 0{,}95$, ¿cuánto vale $|H|$ a 25 Hz y a 49 Hz?

**Solución**

a) $L = f_s/f_0 = 160$: ceros en $\Omega = 2\pi k/160$, es decir, en $f = 50k$ Hz. $|1 - \mathrm{e}^{-j\Omega L}| = 2|\sin(\Omega L/2)|$ vale 2 a 25 Hz: entre muescas la ganancia oscila entre 0 y 2, lo que deforma la señal útil (y también elimina la continua).

b) $|H| = 1$ a 25 Hz y 0,93 a 49 Hz: las muescas son estrechas y el resto del espectro pasa casi sin cambios.

# 5 Aplicaciones

**Cómo usar estos apuntes**

Estos apuntes desarrollan el contenido de las tres presentaciones del tema: la sección 5.1 corresponde a la parte 1 (transformada de Fourier de tiempo corto y espectrograma), la sección 5.2 a la parte 2 (wavelets) y la sección 5.3 a la parte 3 (procesamiento de imágenes). Los ejemplos de código están en `teoría/T5/codigo/`, en Python (`numpy` y `scipy`) y en Matlab, y se pueden ejecutar tal cual; ninguno necesita toolboxes ni ficheros externos. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

## 5.1 Transformada de Fourier de tiempo corto

Los temas anteriores dan las herramientas básicas: la DFT para analizar el contenido en frecuencia (Tema 3) y los filtros para modificarlo (Tema 4). Este tema las aplica a señales reales, que tienen dos rasgos que hasta ahora hemos dejado de lado: su contenido en frecuencia *cambia con el tiempo*, y a veces no dependen del tiempo sino del *espacio*, como las imágenes.

### 5.1.1 Señales no estacionarias

La DFT de una señal completa dice qué frecuencias contiene, pero no cuándo aparece cada una. En la figura 5.1, $x_1$ tiene un tono de 50 Hz durante medio segundo seguido de uno de 120 Hz, y $x_2$ los mismos tonos en orden inverso. Los módulos de sus DFT son prácticamente iguales: la información temporal está escondida en la fase, de donde es imposible leerla.

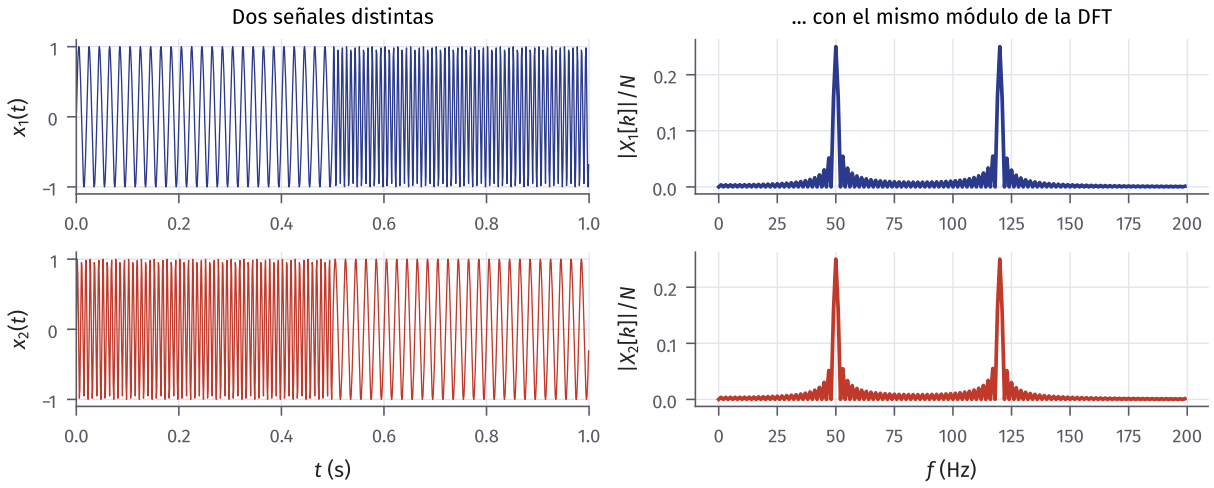



**Figura 5.1.** Dos señales distintas con el mismo módulo de la DFT.

La voz, la música, las vibraciones de una máquina que arranca o los ecos de un radar son *no estacionarias*: su espectro cambia con el tiempo (en el Tema 6 se da una definición precisa de estacionariedad). Para

estudiarlas se analiza la señal *por tramos*: se multiplica por una ventana $w[n]$ de $L$ muestras colocada en cada instante y se calcula la DFT del tramo (figura 5.2). Como vimos en el Tema 3, multiplicar por una ventana es convolucionar el espectro de la señal con el de la ventana, así que conviene usar ventanas suaves (Hann, Hamming) y no la rectangular, cuyos lóbulos laterales son altos.

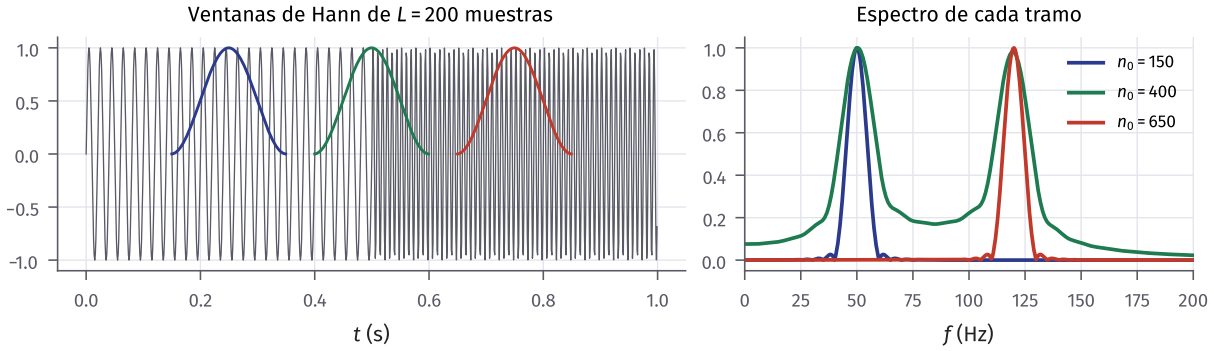



**Figura 5.2.** Tres ventanas de Hann sobre la señal $x_1$ y el espectro de cada tramo. El tramo central contiene el cambio de frecuencia y muestra los dos tonos.

### 5.1.2 Definición

**Definición 5.1.1: Transformada de Fourier de tiempo corto (STFT)**

En tiempo continuo, con una ventana $w(t)$ desplazada a $\tau$,

$$X(\tau, \omega) = \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, \mathrm{e}^{-j\omega t}\, \mathrm{d}t.$$

En tiempo discreto, con una ventana de $L$ muestras que avanza $R$ muestras de un tramo al siguiente,

$$X(m, \Omega) = \sum_{n=-\infty}^{\infty} x[n]\, w[n - mR]\, \mathrm{e}^{-j\Omega n},$$

y, muestreando la frecuencia con $N \geq L$ puntos, $X[m,k] = X(m, 2\pi k/N)$, que se calcula con una DFT de $N$ puntos de cada tramo.

Para cada tramo $m$ hay un espectro, y para cada frecuencia $k$ una secuencia en el tiempo: la STFT es una función de dos variables. Con frecuencia de muestreo $f_s$, el tramo $m$ empieza en la muestra $mR$ y está centrado en $t_m = (mR + L/2)/f_s$, y el índice $k$ corresponde a $f_k = k f_s/N$. Algunos textos escriben la exponencial como $\mathrm{e}^{-j\Omega(n-mR)}$, es decir, miden el tiempo desde el principio de cada tramo; eso multiplica $X$ por $\mathrm{e}^{j\Omega mR}$, que cambia la fase pero no el módulo.

Si la ventana se anula fuera de $0 \leq n < L$, la suma tiene solo $L$ términos:

$$X[m,k] = \sum_{r=0}^{L-1} x[mR + r]\, w[r]\, \mathrm{e}^{-j2\pi k(mR+r)/N}.$$

Para una señal de $N_x$ muestras, el número de tramos completos es

$$M = \left\lfloor \frac{N_x - L}{R} \right\rfloor + 1.$$

Una señal real de 2 s a $f_s = 1\,\mathrm{kHz}$ con $L = 128$, $R = 32$ y $N = 256$ da $M = \lfloor 1872/32 \rfloor + 1 = 59$ tramos, y de cada uno basta guardar las $N/2 + 1 = 129$ frecuencias entre 0 y $f_s/2$.

**En código**

STFT calculada tramo a tramo para un chirp que barre de 50 a 400 Hz en 2 s. La frecuencia del máximo de cada tramo sigue la frecuencia instantánea $50 + 175t$ Hz.

**Python**

```python
import numpy as np
from scipy import signal

fs = 1000
t = np.arange(0, 2, 1/fs)
x = signal.chirp(t, 50, 2, 400)  # 50->400
L, R, N = 128, 32, 256       # L, salto, FFT
w = np.hanning(L)
ini = np.arange(0, len(x) - L + 1, R)
X = np.array([np.fft.rfft(x[i:i+L]*w, N)
              for i in ini]).T   # (f, t)
f = np.fft.rfftfreq(N, 1/fs)
tm = (ini + L/2)/fs              # centros
pico = f[np.argmax(np.abs(X), axis=0)]
print(X.shape)                   # (129, 59)
print(np.c_[tm, pico][::10])     # f(t)
# Igual: signal.spectrogram(x, fs, w, ...)
```

**Matlab**

```matlab
fs = 1000;
t = 0:1/fs:2-1/fs;
x = chirp(t, 50, 2, 400);        % 50->400
L = 128; R = 32; N = 256;    % L, salto, FFT
w = hanning(L)';
ini = 1:R:numel(x)-L+1;
X = zeros(N/2+1, numel(ini));
for m = 1:numel(ini)
  S = fft(x(ini(m):ini(m)+L-1).*w, N);
  X(:, m) = S(1:N/2+1);          % (f, t)
end
f = (0:N/2)*fs/N;
tm = (ini - 1 + L/2)/fs;         % centros
[~, k] = max(abs(X));
disp(size(X))                    % 129 59
disp([tm(1:10:end); f(k(1:10:end))]')
% Igual: spectrogram(x, w, L-R, N, fs)
```

### 5.1.3 Espectrograma

El *espectrograma* es el cuadrado del módulo de la STFT, $|X[m,k]|^2$, representado sobre el plano tiempo-frecuencia con un mapa de colores, casi siempre en decibelios. En la figura 5.3 se distinguen tres componentes: el chirp aparece como una recta, el tono de 300 Hz como un segmento horizontal entre 0,6 y 1,2 s, y el clic de $t = 1{,}6$ s como una línea vertical, porque un impulso contiene todas las frecuencias. Las líneas verticales tenues en 0,6 y 1,2 s son los transitorios del encendido y apagado brusco del tono.

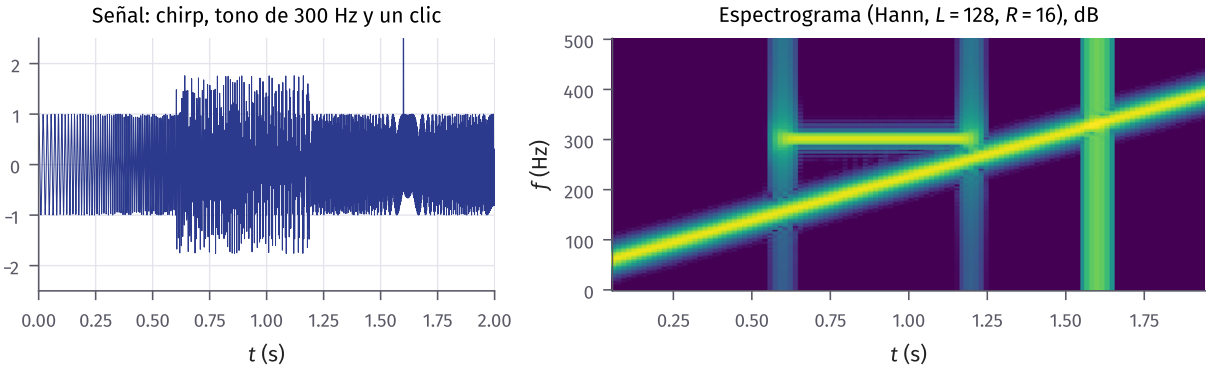



**Figura 5.3.** Señal de prueba y su espectrograma con una ventana de Hann de 128 muestras y salto de 16.

En Matlab el espectrograma lo calcula y dibuja `spectrogram(x, w, L-R, N, fs)`, cuyo tercer argumento es el solapamiento $L - R$; en Python, `scipy.signal.spectrogram` o la clase `ShortTimeFFT`. Octave no tiene `spectrogram`, sino `specgram`. Los ejemplos de estos apuntes calculan la STFT a mano para que se vea qué hacen.

### 5.1.4 Resolución en tiempo y en frecuencia

Los parámetros de la STFT son:

- La *longitud de la ventana* $L$, que fija la duración analizada, $\Delta t = L/f_s$, y la resolución en frecuencia. El lóbulo principal de una ventana de Hann mide $4f_s/L$ entre ceros, y dos tonos se distinguen si están separados al menos la mitad, $\approx 2f_s/L$ (Tema 3).
- El *tipo de ventana*, que reparte el compromiso entre la anchura del lóbulo principal y la altura de los secundarios.
- El *salto* $R$ o, equivalentemente, el *solapamiento* $L-R$. No cambia la resolución, sino lo densamente que se muestrea el eje de tiempos. Son habituales solapamientos del 50 al 75 %.
- El número de puntos de la DFT, $N \geq L$. Rellenar con ceros interpola el espectro, pero no separa mejor dos tonos.

La figura 5.4 muestra el compromiso. Con $L = 32$ (32 ms) el clic y los bordes del tono quedan bien localizados, pero el lóbulo mide 125 Hz y las frecuencias se emborronan. Con $L = 512$ el tono de 300 Hz es una raya fina, pero el chirp, que cambia 90 Hz durante la ventana, se ensancha, y el clic se extiende medio segundo.

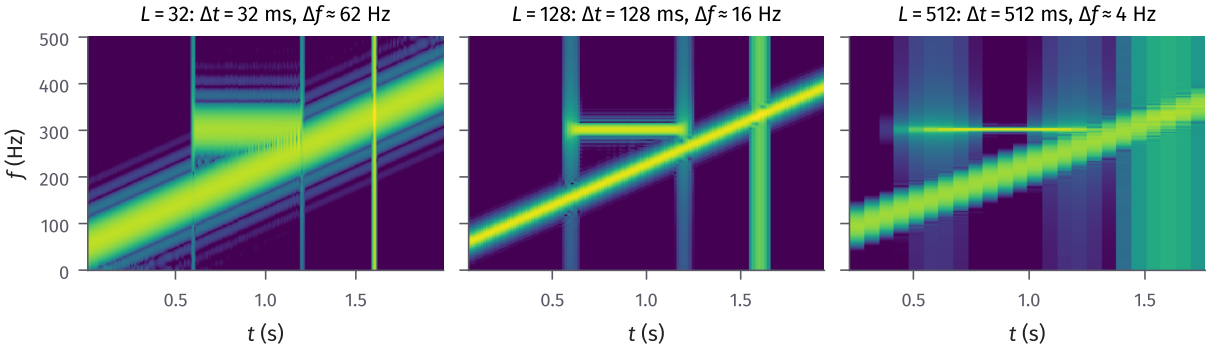



**Figura 5.4.** La misma señal con tres longitudes de ventana. $\Delta f$ es la separación mínima entre dos tonos para la ventana de Hann.

Este compromiso es inevitable. Una señal de duración $\Delta t$ tiene un espectro de anchura del orden de $1/\Delta t$: es el *principio de incertidumbre*. Si se miden las anchuras con las desviaciones típicas de $|w(t)|^2$ y de $|W(j\omega)|^2$ (normalizadas como densidades),

$$\sigma_t \, \sigma_\omega \geq \frac{1}{2},$$

con igualdad solo para la ventana gaussiana. La STFT usa la misma ventana en todas las frecuencias, de modo que su resolución es la misma en todo el plano tiempo-frecuencia. La longitud se elige según la señal: en voz se usan 20–30 ms, el tiempo durante el que el tracto vocal apenas cambia; en música, ventanas más largas para separar notas próximas.

**Ejemplo 5.1.2: Una ventana imposible**

Con $f_s = 1$ kHz y ventana de Hann, separar dos tonos a 20 Hz exige $2f_s/L \leq 20$, es decir, $L \geq 100$ muestras (100 ms). Separar dos clics a 10 ms exige una ventana más corta que su separación, $L \lesssim 10$. Ninguna longitud cumple ambas condiciones: hay que calcular dos espectrogramas o usar un análisis con resolución variable, como el de las wavelets (sección 5.2).

### 5.1.5 Síntesis por solapamiento y suma

La STFT se puede invertir. La IDFT del tramo $m$ devuelve $x[n]\,w[n-mR]$; si se suman todos los tramos,

$$\sum_m x[n]\,w[n-mR] = x[n]\sum_m w[n-mR],$$

que vale $C\,x[n]$ si las ventanas desplazadas suman una constante:

$$\sum_m w[n-mR] = C \qquad \text{para todo } n.$$

Es la condición *COLA* (*constant overlap-add*), y el método se llama *solapamiento y suma*. La cumplen la ventana de Hann periódica $w[n] = \frac{1}{2} - \frac{1}{2}\cos(2\pi n/L)$, $0 \le n < L$, con $R = L/2$ (suma 1) o $R = L/4$ (suma 2), la de Hamming periódica con $R = L/2$ (suma 1,08) y la rectangular con $R = L$ (figura 5.5). Gracias a ella se puede modificar la señal en el plano tiempo-frecuencia (atenuar el ruido en los puntos donde domina, eliminar un tono durante un intervalo) y volver al tiempo. Es la base de muchos reductores de ruido y del *vocoder* de fase, que cambia el tempo de una grabación sin cambiar su tono.

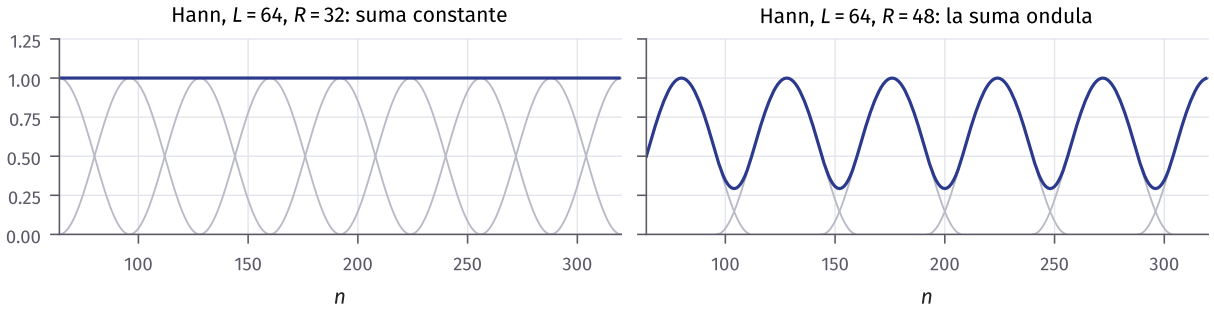



**Figura 5.5.** Ventanas de Hann desplazadas (gris) y su suma (azul). Con $R = L/2$ la suma es constante; con $R = 3L/4$, no.

**Ventana periódica y simétrica**

La ventana de Hann *simétrica* de longitud $L$, la de `hann(L)` en Matlab, tiene ceros en $n = 0$ y $n = L-1$ y no cumple COLA exactamente. La *periódica*, `hann(L,'periodic')`, es la simétrica de longitud $L+1$ sin la última muestra. Para analizar da igual; para resintetizar conviene la periódica.

### 5.1.6 Aplicaciones

**Marcación por tonos.** En la marcación DTMF cada tecla del teléfono suma una sinusoide de fila (697, 770, 852 o 941 Hz) y una de columna (1209, 1336 o 1477 Hz). El espectrograma de la figura 5.6 muestra en qué instante se pulsó cada tecla y cuál: la «2» es 697 + 1336 Hz, el «0» 941 + 1336 y el «5» 770 + 1336. Para separar 697 y 770 Hz, que distan 73 Hz, una ventana de Hann necesita $L \ge 2f_s/73 \approx 220$ muestras a 8 kHz (27 ms), menos que la duración mínima de un tono.

**En código**

Reconocimiento de una tecla a partir de 100 ms de señal con ruido: se busca el máximo del espectro en la banda de las filas y en la de las columnas. Los receptores reales usan el algoritmo de Goertzel, que calcula solo las 7 componentes de la DFT que interesan.

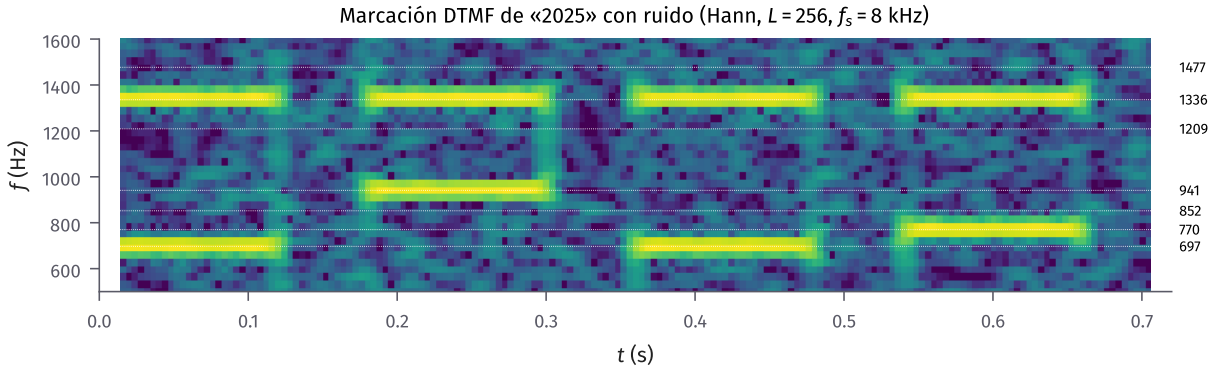



**Figura 5.6.** Espectrograma de la marcación de «2025» con ruido.

**Python**

```python
import numpy as np

fs = 8000
t = np.arange(0, 0.1, 1/fs)   # 800 muestras
x = np.sin(2*np.pi*852*t) \
    + np.sin(2*np.pi*1336*t)     # tecla 8
rng = np.random.default_rng(0)
x += 0.5*rng.standard_normal(t.size)
N = 4096
w = np.hamming(x.size)
X = np.abs(np.fft.rfft(x*w, N))
f = np.fft.rfftfreq(N, 1/fs)
baja = (f > 650) & (f < 1000)
alta = (f > 1150) & (f < 1550)
fb = f[baja][np.argmax(X[baja])]
fa = f[alta][np.argmax(X[alta])]
filas = np.array([697, 770, 852, 941])
cols = np.array([1209, 1336, 1477])
teclas = ["123", "456", "789", "*0#"]
i = np.argmin(abs(filas - fb))
j = np.argmin(abs(cols - fa))
print(fb, fa, teclas[i][j])      # ... 8
```

**Matlab**

```matlab
fs = 8000;
t = 0:1/fs:0.1-1/fs;          % 800 muestras
x = sin(2*pi*852*t) ...
    + sin(2*pi*1336*t);          % tecla 8
x = x + 0.5*randn(size(t));
N = 4096;
w = hamming(numel(x))';
X = abs(fft(x.*w, N));
X = X(1:N/2+1);
f = (0:N/2)*fs/N;
baja = find(f > 650 & f < 1000);
alta = find(f > 1150 & f < 1550);
[~, ib] = max(X(baja)); fb = f(baja(ib));
[~, ia] = max(X(alta)); fa = f(alta(ia));
filas = [697 770 852 941];
cols = [1209 1336 1477];
teclas = ['123'; '456'; '789'; '*0#'];
[~, i] = min(abs(filas - fb));
[~, j] = min(abs(cols - fa));
disp([fb fa]); disp(teclas(i, j))  % 8
```

**Otras aplicaciones.** En *voz*, el espectrograma muestra los formantes (resonancias del tracto vocal) y el tono de la voz, y los reconocedores automáticos parten de una STFT. En *música*, permite identificar notas e instrumentos. En *mantenimiento predictivo*, el arranque de un motor es un chirp y un rodamiento dañado produce armónicos que crecen con el tiempo. En *radar y sonar*, el desplazamiento Doppler a lo largo del tiempo revela la velocidad de un blanco.

Ejercicios 1–5.

## 5.2 Wavelets

### 5.2.1 De la STFT a las wavelets

La STFT fija la ventana antes de analizar y la usa en todas las frecuencias. Pero muchas señales mezclan componentes lentas y largas (una tendencia, un tono grave) con detalles rápidos y breves (un clic, un salto, un borde). Lo natural sería analizar las bajas frecuencias con ventanas largas y las altas con ventanas cortas, de forma que cada análisis abarque aproximadamente el mismo número de periodos. Además, la sinusoide no siempre es la mejor forma con la que comparar: un latido del corazón o un escalón se parecen más a otras ondas.

La figura 5.7 resume la diferencia. Cada rectángulo representa la resolución de un coeficiente: la STFT reparte el plano en rectángulos iguales; las wavelets usan rectángulos estrechos y altos en alta frecuencia y anchos y bajos en baja frecuencia. El área es la misma en todos, como exige el principio de incertidumbre.

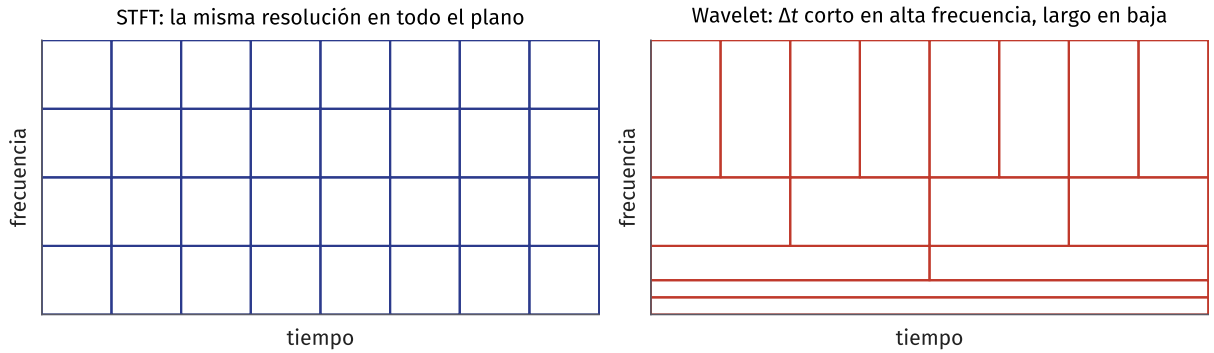



**Figura 5.7.** Reparto del plano tiempo-frecuencia de la STFT y de la transformada wavelet.

### 5.2.2 Wavelets

**Definición 5.2.1: Wavelet**

Una *wavelet madre* $\psi(t)$ es una función de energía finita, de duración limitada o que decae rápidamente, y de media nula. A partir de ella se forma la familia

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}}\,\psi\!\left(\frac{t-b}{a}\right), \qquad a>0,\ b\in\mathbb{R},$$

donde $a$ es la *escala* y $b$ el *desplazamiento*. El factor $1/\sqrt{a}$ hace que todas tengan la misma energía.

La palabra significa «ondita»: oscila, como una sinusoide, pero solo durante un intervalo. Con $a<1$ la wavelet se comprime y oscila más deprisa; con $a>1$ se dilata (figura 5.9). Si $\psi$ oscila con frecuencia $\omega_0$, $\psi_{a,b}$ oscila con $\omega_0/a$: la escala es inversamente proporcional a la frecuencia.

Las wavelets más usadas (figura 5.8) son:

- *Haar*: $\psi(t)=1$ en $[0,\frac{1}{2})$, $-1$ en $[\frac{1}{2},1)$ y 0 en el resto. La más sencilla; discontinua.
- *Daubechies* db$P$: soporte de longitud $2P-1$ y $P$ momentos nulos (véase más abajo); db1 es Haar. No tienen expresión cerrada: se definen por sus filtros.

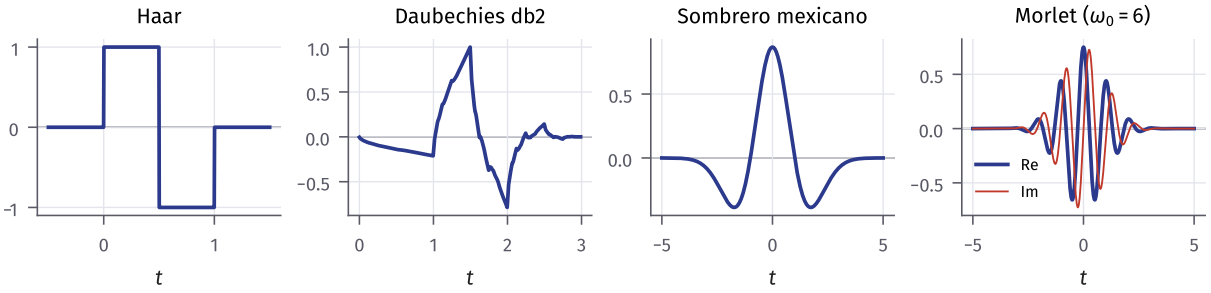



**Figura 5.8.** Algunas wavelets madre. La de Morlet es compleja.

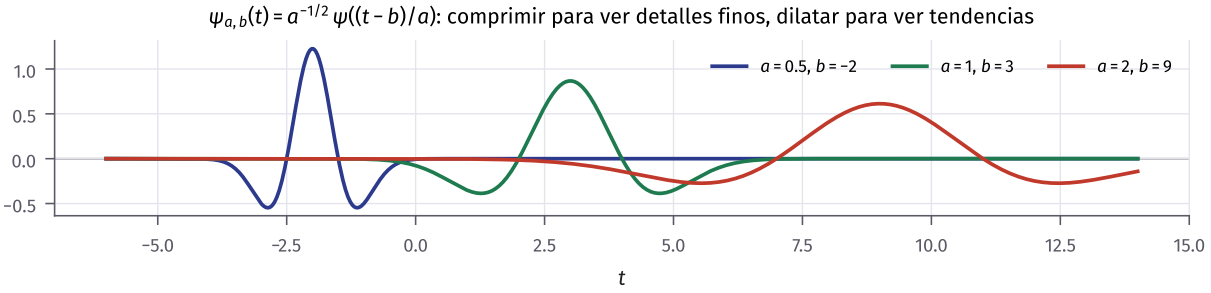



**Figura 5.9.** Sombrero mexicano con tres escalas y desplazamientos.

- *Sombrero mexicano*: la segunda derivada de una gaussiana cambiada de signo, $\psi(t) \propto (1-t^2)\mathrm{e}^{-t^2/2}$.
- *Morlet*: una exponencial compleja con envolvente gaussiana, $\psi(t) = \pi^{-1/4}\mathrm{e}^{j\omega_0 t}\mathrm{e}^{-t^2/2}$, normalmente con $\omega_0 = 6$. Es la más parecida a la STFT con ventana gaussiana.

### 5.2.3 Transformada wavelet continua

**Definición 5.2.2: Transformada wavelet continua**

$$W_x(a,b) = \int_{-\infty}^{\infty} x(t)\,\psi_{a,b}^*(t)\,\mathrm{d}t = \frac{1}{\sqrt{a}}\int_{-\infty}^{\infty} x(t)\,\psi^*\!\left(\frac{t-b}{a}\right)\mathrm{d}t.$$

$W_x(a,b)$ es la correlación de $x$ con la wavelet escalada, evaluada en $b$: mide cuánto se parece la señal, alrededor de $t = b$, a una onda de la forma de $\psi$ y anchura proporcional a $a$. La STFT correla con $w(t-\tau)\mathrm{e}^{j\omega t}$, que depende de tres parámetros (la posición, la frecuencia y la longitud de la ventana, fijada de antemano); la transformada wavelet, de dos.

El *escalograma* es $|W_x(a,b)|^2$ sobre el plano $(b,a)$. Se suele dibujar con la frecuencia equivalente en un eje logarítmico: para la Morlet, con $a$ en segundos, $f \approx \omega_0/(2\pi a)$. En la figura 5.10 se compara con el espectrograma: el clic se estrecha al subir la frecuencia y el tono de 20 Hz se resuelve en frecuencia mejor que el de 120 Hz. Para la Morlet, la envolvente $\mathrm{e}^{-t^2/(2a^2)}$ tiene desviación típica $\sigma_t = a$ y su espectro, $\sigma_\omega = 1/a$: el producto es constante y el área de cada «rectángulo» de la figura 5.7 también.

Para que la transformada se pueda invertir, $\psi$ debe cumplir la *condición de admisibilidad*

$$C_\psi = \int_{-\infty}^{\infty} \frac{|\Psi(j\omega)|^2}{|\omega|}\,\mathrm{d}\omega < \infty,$$

donde $\Psi(j\omega)$ es la CTFT de $\psi$. Si $\Psi$ es continua, la integral solo converge si $\Psi(j0) = 0$, es decir, si

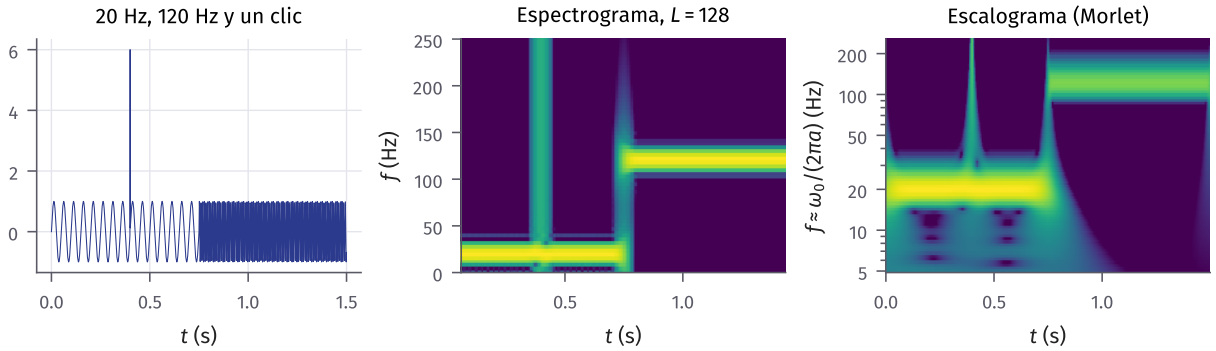



**Figura 5.10.** Señal con un tono de 20 Hz, uno de 120 Hz y un clic: espectrograma y escalograma con la wavelet de Morlet.

$\int \psi(t)\,\mathrm{d}t = 0$: la wavelet tiene media nula y por tanto oscila. Entonces

$$x(t) = \frac{1}{C_\psi}\int_0^\infty\!\!\int_{-\infty}^\infty W_x(a,b)\,\psi_{a,b}(t)\,\frac{\mathrm{d}b\;\mathrm{d}a}{a^2}.$$

**Definición 5.2.3: Momentos nulos**

Una wavelet tiene $P$ momentos nulos si

$$\int_{-\infty}^{\infty} t^p\,\psi(t)\,\mathrm{d}t = 0, \qquad p = 0, 1, \ldots, P-1.$$

Si $\psi$ tiene $P$ momentos nulos, la transformada de cualquier polinomio de grado menor que $P$ es cero. Una señal suave se parece localmente a un polinomio, así que sus coeficientes son casi nulos en las escalas finas, y solo destacan las discontinuidades y los cambios bruscos. Haar tiene un momento nulo ($\int t\,\psi(t)\,\mathrm{d}t = \frac{1}{8} - \frac{3}{8} = -\frac{1}{4} \neq 0$), el sombrero mexicano dos y db$P$, $P$.

### 5.2.4 Transformada wavelet discreta

La CWT es muy redundante: una función de una variable se convierte en una de dos. Basta muestrear escalas y desplazamientos en una rejilla *diádica*, $a = 2^j$ y $b = k\,2^j$ con $j, k \in \mathbb{Z}$:

$$\psi_{j,k}(t) = 2^{-j/2}\,\psi\!\left(2^{-j}t - k\right).$$

Para wavelets bien elegidas (Haar, Daubechies y otras) estas funciones forman una *base ortonormal*:

$$\int_{-\infty}^{\infty} \psi_{j,k}(t)\,\psi_{j',k'}(t)\,\mathrm{d}t = \delta[j-j']\,\delta[k-k'].$$

Cualquier señal de energía finita se escribe entonces como

$$x(t) = \sum_j \sum_k d_j[k]\,\psi_{j,k}(t), \qquad d_j[k] = \int_{-\infty}^{\infty} x(t)\,\psi_{j,k}(t)\,\mathrm{d}t,$$

y, por ser la base ortonormal, se cumple la igualdad de Parseval: $\int |x(t)|^2\,\mathrm{d}t = \sum_{j,k} |d_j[k]|^2$.

**Análisis multirresolución.** Además de la wavelet se define una *función de escala* $\varphi(t)$ (a veces llamada «wavelet padre»), de media 1, cuyas versiones $\varphi_{j,k}(t) = 2^{-j/2}\varphi(2^{-j}t - k)$ recogen lo que queda de la señal por encima de una escala. Para Haar, $\varphi$ es el pulso rectangular de $[0, 1)$. Se definen:

- el *detalle* del nivel $j$, $D_j(t) = \sum_k d_j[k]\,\psi_{j,k}(t)$, lo que la señal tiene a la escala $2^j$;
- la *aproximación* del nivel $J$, $A_J(t) = \sum_k a_J[k]\,\varphi_{J,k}(t)$, con $a_J[k] = \int x(t)\varphi_{J,k}(t)\,\mathrm{d}t$: la señal sin los detalles más finos que $2^J$.

Partiendo de una señal muestreada, que se identifica con la aproximación de nivel 0,

$$x = A_J + D_J + D_{J-1} + \cdots + D_1, \qquad A_{j-1} = A_j + D_j.$$

La figura 5.11 muestra una descomposición de Haar en cuatro niveles: la aproximación $A_4$ conserva la forma general; los saltos aparecen en todos los detalles; la oscilación rápida del final, en $D_2$ y $D_3$.

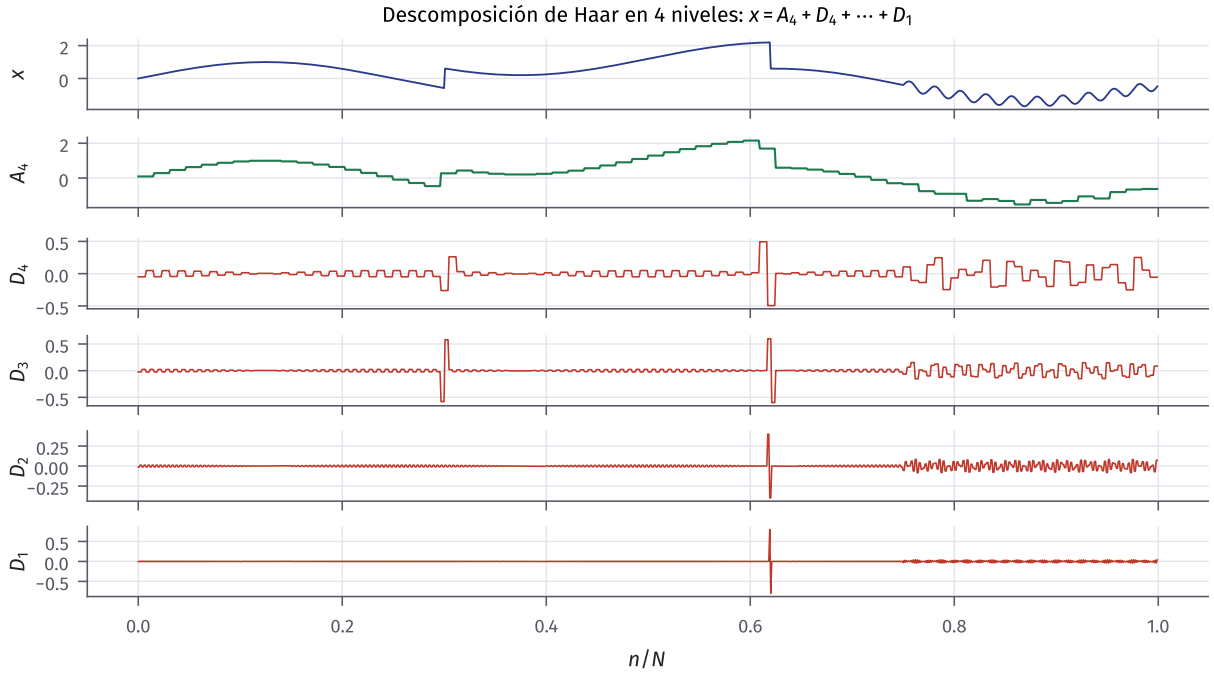



**Figura 5.11.** Señal, aproximación y detalles de una descomposición de Haar en cuatro niveles. Cada componente tiene la longitud de la señal.

### 5.2.5 La DWT como banco de filtros

$\varphi$ y $\psi$ cumplen las *ecuaciones de dos escalas*

$$\varphi(t) = \sqrt{2}\sum_n h_0[n]\,\varphi(2t - n), \qquad \psi(t) = \sqrt{2}\sum_n h_1[n]\,\varphi(2t - n),$$

donde $h_0$ es un filtro paso bajo y $h_1[n] = (-1)^n h_0[L-1-n]$ un paso alto de la misma longitud $L$. De ellas se deduce el *algoritmo de Mallat*, que calcula los coeficientes de un nivel a partir de la aproximación del anterior sin calcular ninguna integral:

$$a_j[k] = \sum_n h_0[n - 2k]\,a_{j-1}[n], \qquad d_j[k] = \sum_n h_1[n - 2k]\,a_{j-1}[n].$$

Es una convolución con $h_0[-n]$ y $h_1[-n]$ seguida de un *diezmado por 2* (quedarse con una de cada dos muestras). La inversa intercala ceros (interpola por 2), filtra con $h_0$ y $h_1$ y suma.

**Propiedad 5.2.4: DWT de Haar**

Con $h_0 = \frac{1}{\sqrt{2}}(1,1)$ y $h_1 = \frac{1}{\sqrt{2}}(1,-1)$, un nivel de la DWT de una secuencia de longitud par es

$$a_1[k] = \frac{x[2k] + x[2k+1]}{\sqrt{2}}, \qquad d_1[k] = \frac{x[2k] - x[2k+1]}{\sqrt{2}},$$

y la inversa,

$$x[2k] = \frac{a_1[k] + d_1[k]}{\sqrt{2}}, \qquad x[2k+1] = \frac{a_1[k] - d_1[k]}{\sqrt{2}}.$$

La aproximación es la media de cada pareja (por $\sqrt{2}$) y el detalle, su semidiferencia.

**Ejemplo 5.2.5: Tres niveles de Haar**

Con $x = (4, 6, 10, 12, 8, 6, 5, 5)$:

- Nivel 1: $a_1 = \frac{1}{\sqrt{2}}(10, 22, 14, 10)$, $d_1 = \frac{1}{\sqrt{2}}(-2, -2, 2, 0)$.
- Nivel 2, sobre $a_1$: $a_2 = \frac{1}{2}(10 + 22,\ 14 + 10) = (16, 12)$, $d_2 = \frac{1}{2}(10 - 22,\ 14 - 10) = (-6, 2)$.
- Nivel 3: $a_3 = \frac{28}{\sqrt{2}} = 19{,}80$ y $d_3 = \frac{4}{\sqrt{2}} = 2{,}83$.

La energía se conserva: $\sum x^2 = 446$ y $392 + 8 + (36 + 4) + \frac{4+4+4+0}{2} = 446$. Casi toda está en $a_3$, la media.

Cada nivel parte la banda en dos (figura 5.12): $d_1$ recoge aproximadamente las frecuencias de $[\pi/2, \pi]$, $d_2$ las de $[\pi/4, \pi/2]$, y así hasta $a_J$, que recoge $[0, \pi/2^J]$. Los filtros de Haar separan mal las bandas; los de Daubechies más largos, mejor. En todos los casos $|H_0|^2 + |H_1|^2 = 2$: la energía que uno quita la recoge el otro.

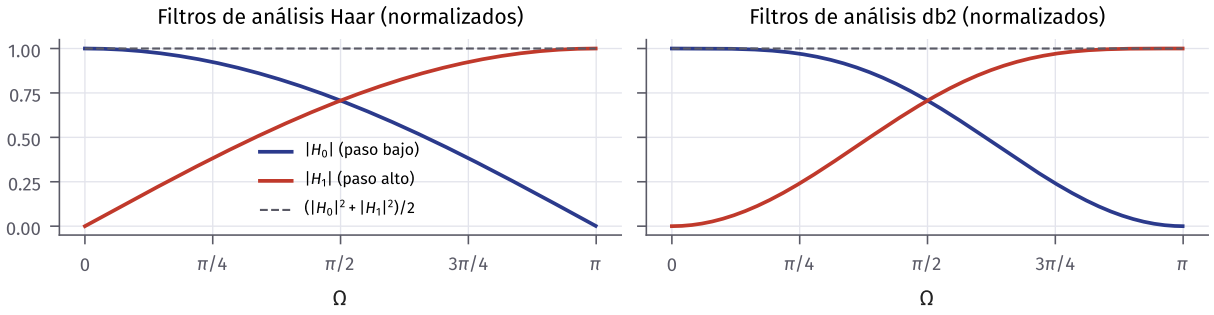



**Figura 5.12.** Respuesta de los filtros de análisis de Haar y de db2.

El número total de coeficientes es $\frac{N}{2} + \frac{N}{4} + \cdots + \frac{N}{2^J} + \frac{N}{2^J} = N$: la DWT no añade redundancia, y el coste es $O(N)$, menor aún que el de la FFT. Los coeficientes del filtro db2 son

$$h_0 = \frac{1}{4\sqrt{2}}\left(1 + \sqrt{3},\ 3 + \sqrt{3},\ 3 - \sqrt{3},\ 1 - \sqrt{3}\right),$$

y su paso alto $h_1$ cumple $\sum_n h_1[n] = 0$ y $\sum_n n\, h_1[n] = 0$: dos momentos nulos, de modo que los detalles de una rampa son nulos, salvo en los extremos. Con Haar, $x[n] = n$ da $d_1[k] = -1/\sqrt{2}$ en todos los $k$.

**En código**

DWT de Haar en cuatro niveles, comprobación de Parseval y de la reconstrucción. En Matlab, la Wavelet Toolbox ofrece `dwt`, `wavedec`, `waverec`, `wrcoef` y `cwt`; en Python, el paquete PyWavelets (`pywt.wavedec`, `pywt.cwt`).

**Python**

```python
import numpy as np
def dwt_haar(x, J):       # [d1, ..., dJ, aJ]
    a, c = np.asarray(x, float), []
    for _ in range(J):            # análisis
        c.append((a[0::2] - a[1::2])/2**.5)
        a = (a[0::2] + a[1::2])/2**.5
    return c + [a]

def idwt_haar(c):
    a = c[-1]
    for d in reversed(c[:-1]):    # síntesis
        x = np.empty(2*a.size)
        x[0::2] = (a + d)/2**.5
        x[1::2] = (a - d)/2**.5
        a = x
    return a
n = np.arange(1024)
x = np.sin(2*np.pi*n/512) + (n > 300)
c = dwt_haar(x, 4)
print([v.size for v in c])      # 512 ... 64
E = [np.sum(v**2) for v in c]
print(np.round(E, 2), np.sum(x**2))
print(abs(idwt_haar(c) - x).max())  # ~1e-15
```

**Matlab**

```matlab
n = 0:1023;
x = sin(2*pi*n/512) + (n > 300);
J = 4; c = cell(1, J+1); a = x;
for j = 1:J                       % análisis
  c{j} = (a(1:2:end)-a(2:2:end))/sqrt(2);
  a = (a(1:2:end)+a(2:2:end))/sqrt(2);
end
c{J+1} = a;
disp(cellfun(@numel, c))        % 512 ... 64
E = cellfun(@(v) sum(v.^2), c);
disp([E, sum(x.^2)])             % Parseval
for j = J:-1:1                    % síntesis
  y = zeros(1, 2*numel(a));
  y(1:2:end) = (a + c{j})/sqrt(2);
  y(2:2:end) = (a - c{j})/sqrt(2);
  a = y;
end
disp(max(abs(a - x)))             % ~1e-15
```

### 5.2.6 Aplicaciones

**Eliminación de ruido.** La DWT concentra la energía de una señal con pocas discontinuidades en unos pocos coeficientes grandes, mientras que el ruido blanco, al ser la base ortonormal, reparte la suya por igual entre todos. Anulando los coeficientes de detalle pequeños y reconstruyendo se elimina gran parte del ruido. Hay dos reglas:

$$\text{umbral duro: } \hat{d} = \begin{cases} d, & |d| > u \\ 0, & |d| \le u \end{cases} \qquad \text{umbral suave: } \hat{d} = \operatorname{sgn}(d)\max(|d| - u, 0).$$

Un umbral habitual es el universal de Donoho y Johnstone, $u = \hat{\sigma}\sqrt{2\ln N}$, con la desviación típica del ruido estimada a partir de los detalles más finos, que son casi todo ruido: $\hat{\sigma} = \text{mediana}|d_1[k]|/0{,}6745$. A diferencia de un paso bajo, que quita el ruido y también suaviza los saltos, el umbral conserva los saltos (figura 5.13).




**Figura 5.13.** Señal escalonada con ruido, filtrada con una media móvil y con umbral sobre la DWT de Haar. En rojo, la señal sin ruido.

**En código**

Umbral duro sobre cinco niveles de Haar. El error cuadrático medio baja de 0,40 a 0,12.

**Python**

```python
import numpy as np

N = 1024
n = np.arange(N)
x = 1.5*(n > 100) - 2.2*(n > 240) \
    + 1.6*(n > 410) + 1.2*(n > 512)
rng = np.random.default_rng(4)
y = x + 0.4*rng.standard_normal(N)
c, a = [], y                          # análisis
for j in range(5):
    c.append((a[0::2] - a[1::2])/2**.5)
    a = (a[0::2] + a[1::2])/2**.5
s = np.median(abs(c[0]))/0.6745   # sigma
u = s*np.sqrt(2*np.log(N))        # umbral
for d in reversed(c):             # síntesis
    d = np.where(abs(d) > u, d, 0)  # duro
    z = np.empty(2*a.size)
    z[0::2] = (a + d)/2**.5
    z[1::2] = (a - d)/2**.5
    a = z
rms = lambda e: np.sqrt(np.mean(e**2))
print(s, u)                       # 0.40 1.49
print(rms(y - x), rms(a - x))     # 0.40 0.12
```

**Matlab**

```matlab
N = 1024; n = 0:N-1;
x = 1.5*(n > 100) - 2.2*(n > 240) ...
    + 1.6*(n > 410) + 1.2*(n > 512);
y = x + 0.4*randn(1, N);
J = 5; c = cell(1, J+1); a = y;
for j = 1:J
  c{j} = (a(1:2:end)-a(2:2:end))/sqrt(2);
  a = (a(1:2:end)+a(2:2:end))/sqrt(2);
end
s = median(abs(c{1}))/0.6745;     % sigma
u = s*sqrt(2*log(N));             % umbral
for j = J:-1:1                    % duro
  d = c{j}.*(abs(c{j}) > u);
  z = zeros(1, 2*numel(a));
  z(1:2:end) = (a + d)/sqrt(2);
  z(2:2:end) = (a - d)/sqrt(2);
  a = z;
end
rms = @(e) sqrt(mean(e.^2));
disp([s u])                       % 0.40 1.49
disp([rms(y-x) rms(a-x)])         % 0.40 0.12
```

**Compresión.** Si se guardan solo los coeficientes mayores, la señal se reconstruye con poco error (figura 5.14). JPEG 2000 aplica una DWT bidimensional con wavelets biortogonales (la CDF 9/7) y codifica los coeficientes; a igual tamaño da menos artefactos que la DCT por bloques de JPEG.

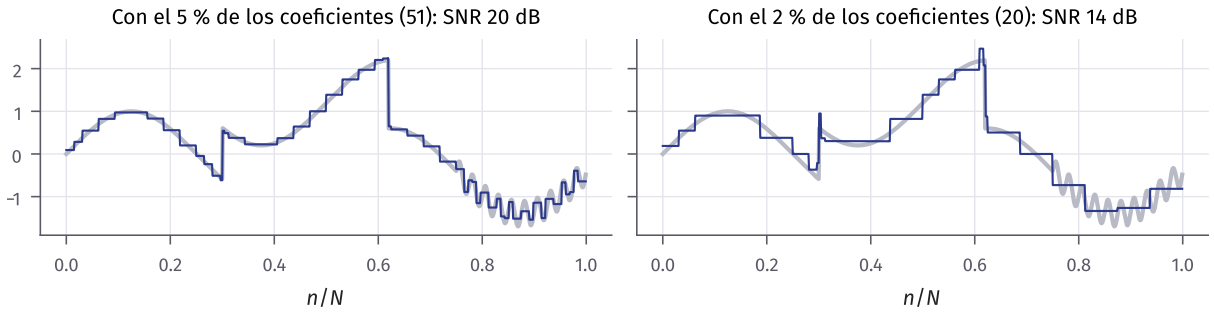



**Figura 5.14.** Reconstrucción a partir del 5 y del 2 % de los coeficientes de Haar.

**Detección.** Los saltos y picos producen coeficientes grandes en todas las escalas, y las wavelets se usan para detectar los complejos QRS de un electrocardiograma o los fallos incipientes en una máquina. La elección de la wavelet depende de la forma de lo que se busca y del número de momentos nulos necesarios.

Ejercicios 6–10.

## 5.3 Procesamiento de imágenes

### 5.3.1 La imagen como señal bidimensional

Una imagen es una señal que depende de dos variables espaciales. Una cámara la muestrea con una rejilla de sensores y cuantifica cada muestra: el resultado es una matriz $f[m, n]$ de $M \times N$ *píxeles*, con $m = 0, \dots, M-1$ la fila, contada de arriba abajo, y $n = 0, \dots, N-1$ la columna (figura 5.15). En Matlab los índices empiezan en 1: el píxel $f[m, n]$ es `f(m+1, n+1)`.

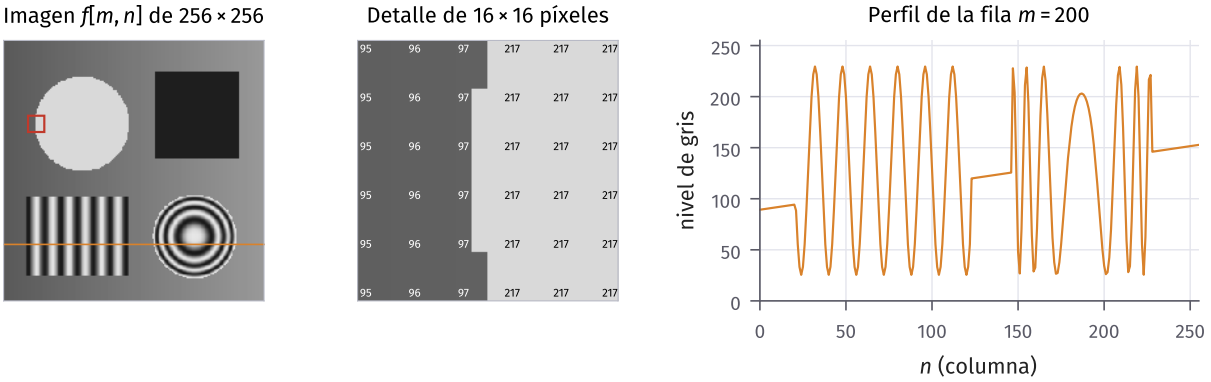



**Figura 5.15.** La imagen de prueba de los ejemplos, un detalle de 16 × 16 píxeles con algunos de sus valores (de 0 a 255) y el perfil de una fila.

- *Escala de grises*: un valor por píxel, entero de 8 bits (`uint8`, de 0, negro, a 255, blanco), de 16 bits, o real en $[0, 1]$.
- *Binaria*: 0 o 1.
- *Color*: tres canales, normalmente rojo, verde y azul (RGB), es decir, una matriz de $M \times N \times 3$. Hay otros espacios de color, como Lab (luminosidad y dos componentes de color) o YCbCr (el de JPEG).

El *muestreo espacial* tiene las mismas reglas que el temporal (Tema 2): la frecuencia espacial se mide en ciclos por píxel, o por milímetro, y un detalle con periodo menor que dos píxeles produce *aliasing*, el efecto moiré que aparece al fotografiar una tela de rayas finas. La *cuantificación* con $b$ bits da $2^b$ niveles; con pocos niveles aparecen contornos falsos en los degradados.

Sin comprimir, una imagen ocupa $M \cdot N \cdot c \cdot b/8$ bytes, con $c$ canales: una imagen en color de 1920×1080 a 8 bits ocupa 6,2 MB. Por eso se guardan comprimidas: sin pérdidas (PNG, TIFF), eliminando redundancia, o con pérdidas (JPEG, JPEG 2000), descartando lo que menos se percibe. Matlab lee y escribe imágenes con `imread` e `imwrite` y las muestra con `imshow` o `image`; Python, con `matplotlib.pyplot.imread` e `imshow` o con el paquete Pillow.

**En código**

Los ejemplos usan una imagen sintética de 256 × 256 con un degradado, un círculo, un rectángulo, unas franjas verticales y un patrón de anillos, para no depender de ficheros externos.

**Python**

```python
import numpy as np


def imagen_prueba(N=256):
    """Imagen sintética N x N en [0, 1]."""
    n = np.arange(N)/N
    x, y = np.meshgrid(n, n)
    f = 0.35 + 0.25*x            # degradado
    f[(x-.3)**2 + (y-.32)**2 < .18**2] = .85
    f[(x > 0.58) & (x < 0.9)
      & (y > 0.12) & (y < 0.45)] = 0.12
    fr = (y > 0.6) & (y < 0.9) \
        & (x > 0.08) & (x < 0.48)
    f[fr] = .5 + .4*np.cos(2*np.pi*16*x[fr])
    r = np.hypot(x - 0.73, y - 0.75)
    an = r < 0.16
    f[an] = 0.5 + 0.4*np.cos(2*np.pi*120
                             * r[an]**2)
    return f


if __name__ == "__main__":
    f = imagen_prueba()
    print(f.shape, f.min(), f.max())
```

**Matlab**

```matlab
function f = imagen_prueba(N)
% Imagen sintética de N x N en [0, 1].
if nargin < 1, N = 256; end
[x, y] = meshgrid((0:N-1)/N);
f = 0.35 + 0.25*x;               % degradado
f((x-0.3).^2 + (y-0.32).^2 < 0.18^2) = .85;
f(x > 0.58 & x < 0.9 ...
  & y > 0.12 & y < 0.45) = 0.12;
fr = y > 0.6 & y < 0.9 ...
     & x > 0.08 & x < 0.48;
f(fr) = 0.5 + 0.4*cos(2*pi*16*x(fr));
r = hypot(x - 0.73, y - 0.75);
an = r < 0.16;
f(an) = 0.5 + 0.4*cos(2*pi*120*r(an).^2);
end
```

### 5.3.2 Operaciones puntuales

Una *operación puntual* transforma cada píxel con una función de su propio valor, $s = T(r)$, sin mirar a los vecinos (figura 5.16). Con valores en $[0, 1]$:

- *negativo*, $s = 1 - r$;
- *corrección gamma*, $s = r^{\gamma}$: con $\gamma < 1$ aclara los tonos oscuros, con $\gamma > 1$ oscurece los claros;
- *estiramiento lineal* del intervalo $[r_1, r_2]$ a $[0, 1]$, recortando fuera de él (es lo que hace `imadjust` de Matlab);
- *umbralización*, $s = 1$ si $r > u$ y 0 si no, que da una imagen binaria.

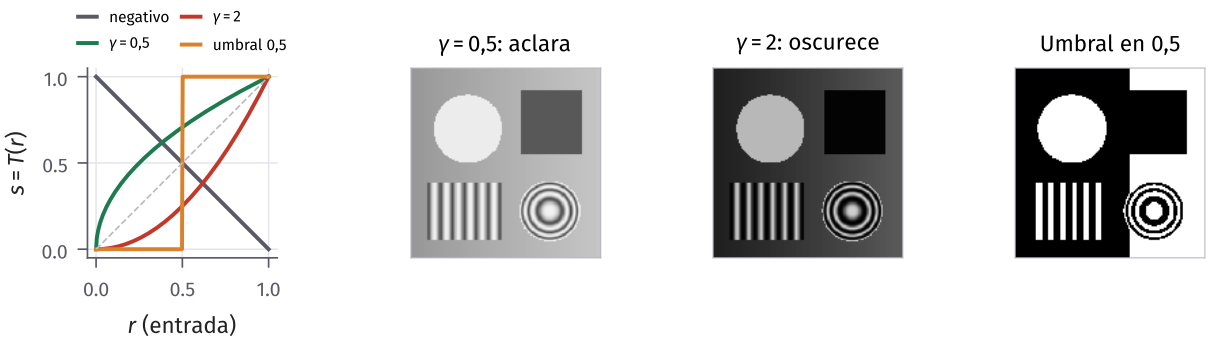



**Figura 5.16.** Cuatro transformaciones puntuales y su efecto.

### 5.3.3 Histograma y ecualización

**Definición 5.3.1: Histograma**

Si una imagen de $M \times N$ píxeles tiene $L$ niveles $r = 0, \ldots, L-1$, su *histograma* $h[r]$ es el número de píxeles de nivel $r$. Normalizado, $p[r] = h[r]/(MN)$ es la proporción de píxeles de cada nivel, y

$$F(r) = \sum_{i=0}^{r} p[i]$$

es la proporción de píxeles con nivel menor o igual que $r$.

$p[r]$ y $F(r)$ son la función de probabilidad y la función de distribución del nivel de un píxel elegido al azar (Tema 6). Un histograma concentrado a la izquierda corresponde a una imagen oscura; a la derecha, a una clara; estrecho, a una con poco contraste. El histograma no dice dónde están los píxeles: dos imágenes muy distintas pueden tener el mismo. Si el objeto y el fondo tienen niveles distintos, el histograma tiene dos picos, y un umbral en el valle que los separa segmenta la imagen; el método de Otsu elige ese umbral automáticamente.

La *ecualización* busca repartir los niveles por todo el rango con la transformación

$$s = T(r) = \text{round}\big((L-1)\,F(r)\big),$$

que es creciente, de modo que conserva el orden de los niveles. Si $r$ fuera una variable continua con función de distribución $F$, la variable $F(r)$ sería uniforme en $[0, 1]$: es el mismo resultado que se usa en el Tema 6 para generar números aleatorios con una distribución dada. Con niveles discretos el histograma resultante no es plano (los píxeles de un mismo nivel van todos al mismo nivel nuevo), pero ocupa todo el rango (figura 5.17).

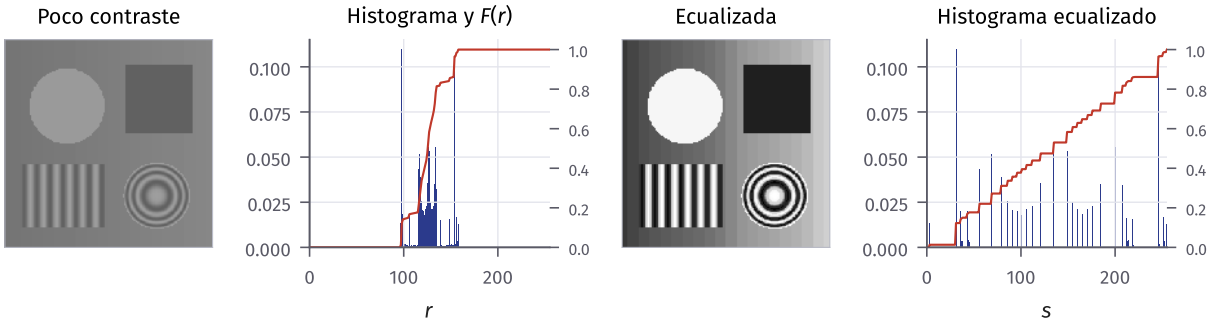



**Figura 5.17.** Imagen con poco contraste, su histograma con $F(r)$ en rojo, la imagen ecualizada y su histograma.

**En código**

Ecualización hecha a mano. Con la Image Processing Toolbox de Matlab, `imhist` dibuja el histograma e `histeq` ecualiza; `histeq` también puede llevar el histograma a una forma prefijada.

**Python**

```python
import numpy as np
from imagen_prueba import imagen_prueba

f = imagen_prueba()
g = np.round(255*(0.35 + 0.3*f))
g = g.astype(int)           # contraste bajo
h = np.bincount(g.ravel(), minlength=256)
F = np.cumsum(h)/g.size       # distribución
T = np.round(255*F).astype(int)  # s = T(r)
ge = T[g]
print(g.min(), g.max())          # 97 158
print(ge.min(), ge.max())        # 3 255
print(np.std(g), np.std(ge))     # 16.7 73.3
```

**Matlab**

```matlab
f = imagen_prueba();
g = round(255*(0.35 + 0.3*f)); % 97..158
h = accumarray(g(:) + 1, 1, [256 1]);
F = cumsum(h)/numel(g);       % distribución
T = round(255*F);                % s = T(r)
ge = T(g + 1);
disp([min(g(:)) max(g(:))])      % 97 158
disp([min(ge(:)) max(ge(:))])    % 3 255
disp([std(g(:)) std(ge(:))])     % 16.7 73.3
% Con Image Processing Toolbox: histeq
```

### 5.3.4 Convolución bidimensional

**Definición 5.3.2: Convolución 2D**

$$g[m,n] = (h * f)[m,n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} h[i,j]\, f[m-i, n-j].$$

Es la extensión directa de la convolución 1D, y como ella da la salida de un sistema lineal e invariante al desplazamiento cuya respuesta al impulso $\delta[m,n]$ es $h[m,n]$. En imágenes $h$ es casi siempre un *núcleo* pequeño ($3\times3$, $5\times5$), es decir, un filtro FIR 2D, y cada píxel de salida es una combinación lineal de sus vecinos.

Si la imagen mide $M\times N$ y el núcleo $K\times K$, la convolución completa mide $(M+K-1)\times(N+K-1)$. Lo habitual es quedarse con la parte central del tamaño de la imagen (opción `'same'`) y, para los píxeles del borde, suponer algo sobre lo que hay fuera: ceros, el valor del borde repetido o la imagen reflejada. Rellenar con ceros oscurece el marco de la imagen.

El núcleo es *separable* si $h[i,j] = u[i]\,v[j]$. Entonces basta filtrar cada fila con $v$ y después cada columna con $u$, con $2K$ productos por píxel en lugar de $K^2$: para $K = 15, 30$ en vez de 225. El gaussiano y el de Sobel son separables.

**Convolución y correlación**

`conv2` de Matlab y `scipy.signal.convolve2d` calculan la convolución; `filter2` e `imfilter` de Matlab calculan la *correlación*, $\sum_{i,j} h[i,j]\,f[m+i,n+j]$, que equivale a convolucionar con el núcleo girado 180°. Para núcleos simétricos dan lo mismo; para el de Sobel, el signo cambia.

**Núcleos habituales.** (Figura 5.18)

- *Media* de $K\times K$: $h = \frac{1}{K^2}$ en todo el núcleo. Paso bajo: reduce el ruido y emborrona los bordes. Su respuesta en frecuencia es un sinc con lóbulos secundarios.
- *Gaussiano*: $h[i,j] \propto \mathrm{e}^{-(i^2+j^2)/(2\sigma^2)}$, normalizado para sumar 1. Paso bajo sin lóbulos ni artefactos; el más usado para suavizar.

- *Sobel*:

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix},$$

una derivada horizontal ($f[m, n+1] - f[m, n-1]$, al convolucionar) combinada con un suavizado vertical; su traspuesta deriva en vertical. El módulo del gradiente $\sqrt{g_x^2 + g_y^2}$ marca los bordes en cualquier dirección.

- *Laplaciano*: $\left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{smallmatrix}\right]$, aproximación de $\partial^2 f/\partial m^2 + \partial^2 f/\partial n^2$. Paso alto: realza los detalles y también el ruido. Sumar a la imagen un múltiplo de $-\nabla^2 f$ la hace más nítida.

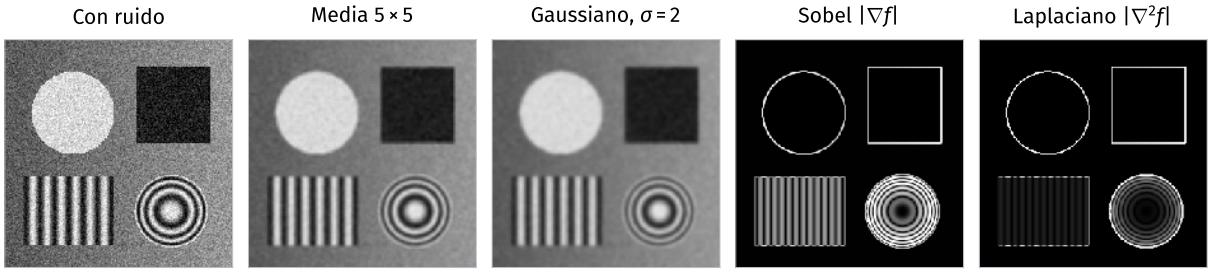



**Figura 5.18.** Imagen con ruido suavizada con una media y con un gaussiano, y bordes de la imagen sin ruido detectados con Sobel y con el laplaciano.

**En código**

Gradiente de Sobel. En el degradado la imagen crece 0,25/256 por columna y $g_x$ vale ocho veces eso: el núcleo de Sobel da la derivada multiplicada por $2 \times 4$.

**Python**

```python
import numpy as np
from scipy.signal import convolve2d
from imagen_prueba import imagen_prueba

f = imagen_prueba()
sx = np.array([[1, 0, -1],
               [2, 0, -2],
               [1, 0, -1]])      # d/dn
media = np.ones((5, 5))/25
gx = convolve2d(f, sx, mode="same")
gy = convolve2d(f, sx.T, mode="same")
g = np.hypot(gx, gy)           # |gradiente|
fs = convolve2d(f, media, mode="same")
bordes = g > 0.5
print(bordes.mean())            # 0.18
print(f[80, 10:20])             # degradado
print(gx[80, 10:20])           # 8*0.25/256
```

**Matlab**

```matlab
f = imagen_prueba();
sx = [1 0 -1;
      2 0 -2;
      1 0 -1];                    % d/dn
media = ones(5)/25;
gx = conv2(f, sx, 'same');
gy = conv2(f, sx', 'same');
g = hypot(gx, gy);              % |gradiente|
fs = conv2(f, media, 'same');
bordes = g > 0.5;
disp(mean(bordes(:)))            % 0.18
disp(f(81, 11:20))               % degradado
disp(gx(81, 11:20))             % 8*0.25/256
```

### 5.3.5 DFT bidimensional

**Definición 5.3.3: DFT 2D**

La DFT de una imagen $f[m,n]$ de $M \times N$ y su inversa son

$$F[k,l] = \sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f[m,n]\, e^{-j2\pi\left(\frac{km}{M}+\frac{ln}{N}\right)}, \qquad f[m,n] = \frac{1}{MN}\sum_{k=0}^{M-1}\sum_{l=0}^{N-1} F[k,l]\, e^{j2\pi\left(\frac{km}{M}+\frac{ln}{N}\right)}.$$

La DFT 2D descompone la imagen en *ondas planas* $e^{j2\pi(km/M+ln/N)}$: franjas que tienen $k$ ciclos a lo largo de las filas y $l$ a lo largo de las columnas, orientadas perpendicularmente al vector $(k,l)$. $F[0,0]$ es la suma de todos los píxeles, y $F[0,0]/(MN)$ el nivel medio.

Como $e^{-j2\pi(km/M+ln/N)} = e^{-j2\pi km/M}\, e^{-j2\pi ln/N}$, la DFT 2D es *separable*:

$$F[k,l] = \sum_{m=0}^{M-1}\left[\sum_{n=0}^{N-1} f[m,n]\, e^{-j2\pi ln/N}\right] e^{-j2\pi km/M} \ :$$

se calcula la DFT 1D de cada fila y después la de cada columna del resultado. Con la FFT cuesta $O(MN\log_2(MN))$. En Matlab y en Python, `fft2` e `ifft2`.

Para dibujarla se lleva el origen al centro con `fftshift` (las frecuencias $k \geq M/2$ son en realidad negativas) y se representa $\log(1+|F|)$, porque el valor en el origen es varios órdenes de magnitud mayor que el resto. La figura 5.19 muestra algunos casos: un coseno da dos puntos simétricos en la dirección perpendicular a sus franjas, tanto más alejados del centro cuanto más finas son, y un cuadrado da el producto de dos sinc, con los lóbulos en cruz. En una imagen real (figura 5.20) la energía se concentra en las bajas frecuencias, y los bordes rectos producen rayas perpendiculares a ellos.

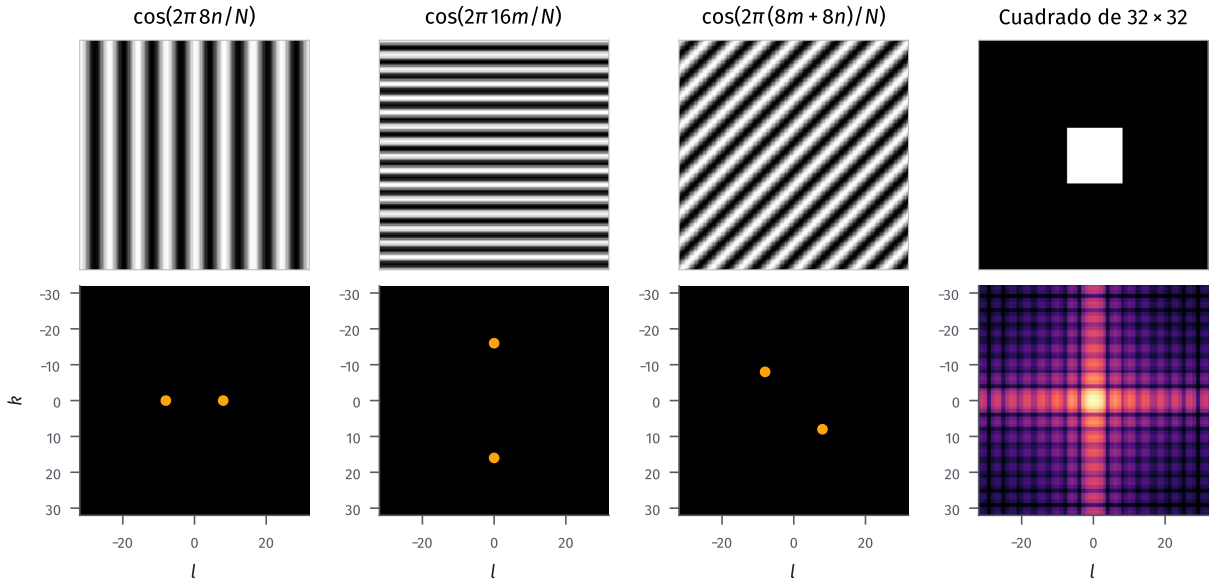



**Figura 5.19.** Cuatro imágenes de $128 \times 128$ y el módulo de su DFT centrada (zona central de $\pm 32$).

**Propiedad 5.3.4: Propiedades de la DFT 2D**

- *Periodicidad*: $F[k+M,l] = F[k,l+N] = F[k,l]$, y lo mismo $f$ al invertir.
- *Simetría*: si $f$ es real, $F[-k,-l] = F^*[k,l]$ (índices módulo $M$ y $N$), y $|F|$ es simétrico respecto al origen.

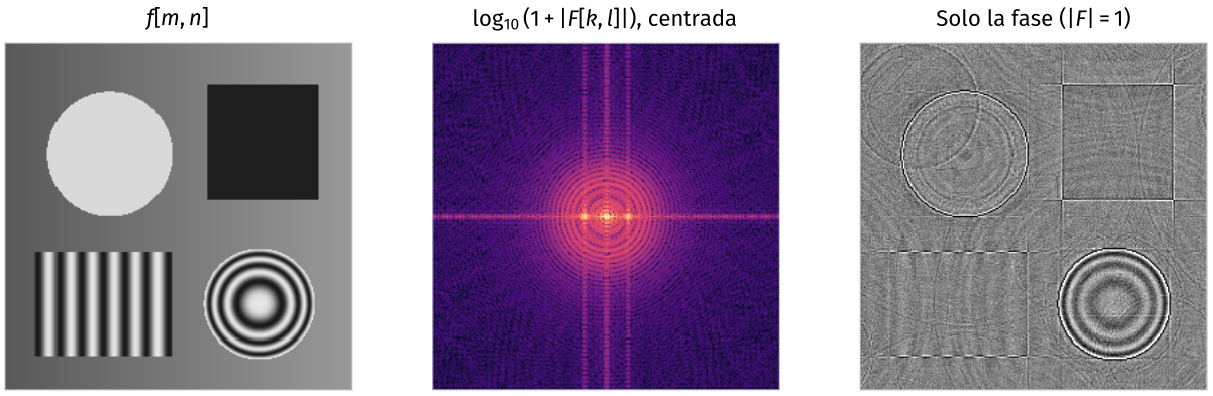



**Figura 5.20.** La imagen de prueba, el módulo de su DFT y la imagen que se obtiene conservando solo la fase.

- *Desplazamiento circular*: $f[(m-m_0)_M,(n-n_0)_N] \overset{\mathcal{F}}{\longleftrightarrow} \mathrm{e}^{-j2\pi\left(\frac{km_0}{M}+\frac{ln_0}{N}\right)}F[k,l]$. Mover la imagen no cambia $|F|$.
- *Rotación*: si se gira la imagen un ángulo $\theta$, su transformada (continua) gira el mismo ángulo.
- *Convolución circular*: la convolución circular 2D de dos imágenes de $M\times N$ tiene por DFT el producto de sus DFT.
- *Parseval*: $\sum_{m,n}|f[m,n]|^2 = \frac{1}{MN}\sum_{k,l}|F[k,l]|^2$.

La periodicidad y la simetría son las que permiten centrar la DFT. La fase es tan importante como en 1D: en la figura 5.20, la imagen reconstruida con $|F| = 1$ y la fase original conserva los bordes de los objetos, mientras que con el módulo y una fase cualquiera no se reconoce nada.

### 5.3.6 Filtrado en el dominio de la frecuencia

Por el teorema de convolución, un filtro se puede aplicar multiplicando: $G[k,l] = H[k,l]\,F[k,l]$ y $g = \mathrm{IDFT}\{G\}$. Es la opción eficiente cuando el núcleo es grande. Dos precauciones: el producto de DFT corresponde a una convolución *circular*, así que para obtener la lineal hay que rellenar con ceros hasta al menos $(M+K-1)\times(N+K-1)$; y para que $g$ sea real, $H$ debe tener la simetría $H[-k,-l] = H^*[k,l]$.

Con $D[k,l] = \sqrt{k^2+l^2}$ la distancia al origen (con las frecuencias negativas bien colocadas):

- *Paso bajo ideal*: $H = 1$ si $D \le D_0$ y 0 si no. Su respuesta al impulso es la versión 2D de un sinc, y produce *anillos* alrededor de los bordes (el fenómeno de Gibbs del Tema 3).
- *Paso bajo gaussiano*: $H = \mathrm{e}^{-D^2/(2D_0^2)}$. Su respuesta al impulso también es gaussiana, sin lóbulos: suaviza sin anillos. Corresponde a un gaussiano espacial de $\sigma = N/(2\pi D_0)$ píxeles (ejercicio 16).
- *Paso alto*: $H_{\mathrm{PA}} = 1 - H_{\mathrm{PB}}$; deja los bordes.

**En código**

Paso bajo gaussiano con $D_0 = 20$ aplicado con la DFT. $H[0,0] = 1$, así que el nivel medio no cambia; el filtro conserva el 96 % de la energía.

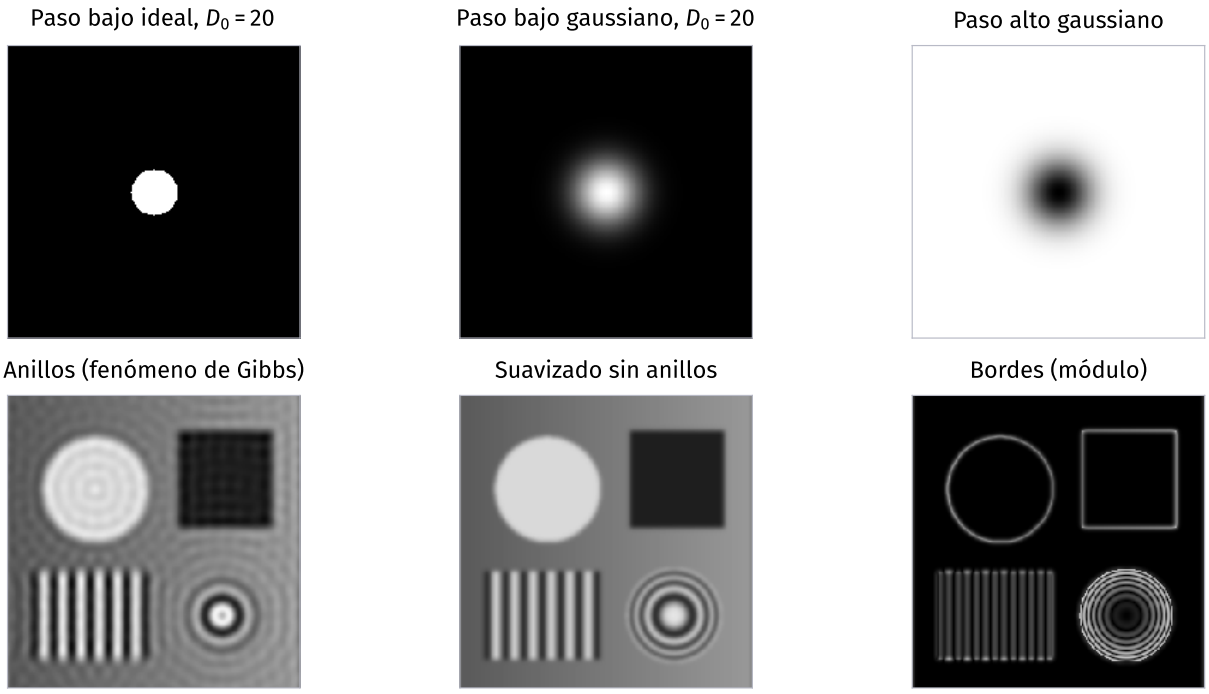



**Figura 5.21.** Respuestas en frecuencia (centradas) y resultado de tres filtros aplicados con la DFT.

**Python**

```python
import numpy as np
from imagen_prueba import imagen_prueba

f = imagen_prueba()
M, N = f.shape
F = np.fft.fft2(f)
k = np.fft.fftfreq(M)*M          # 0..-1
l = np.fft.fftfreq(N)*N
K, L = np.meshgrid(k, l, indexing="ij")
D2 = K**2 + L**2
D0 = 20
H = np.exp(-D2/(2*D0**2))        # gaussiano
g = np.real(np.fft.ifft2(H*F))
E = np.abs(F)**2
print(np.sum(E*H**2)/np.sum(E))  # 0.958
print(f.mean(), g.mean())        # H[0,0]=1
```

**Matlab**

```matlab
f = imagen_prueba();
[M, N] = size(f);
F = fft2(f);
k = [0:M/2-1, -M/2:-1];          % 0..-1
l = [0:N/2-1, -N/2:-1];
[L, K] = meshgrid(l, k);
D2 = K.^2 + L.^2;
D0 = 20;
H = exp(-D2/(2*D0^2));           % gaussiano
g = real(ifft2(H.*F));
E = abs(F).^2;
disp(sum(E(:).*H(:).^2)/sum(E(:)))
disp([mean(f(:)) mean(g(:))])    % H(1,1)=1
```

### 5.3.7 Otras aplicaciones

- *Restauración*: si se conoce el desenfoque $H$ (por movimiento de la cámara, por ejemplo), se puede intentar deshacerlo dividiendo por $H$; donde $H \approx 0$ esa división amplifica el ruido, y se usa el filtro de Wiener, que tiene en cuenta el espectro del ruido (Tema 6).
- *Compresión*: JPEG divide la imagen en bloques de $8 \times 8$, aplica a cada uno la transformada discreta del coseno (DCT) y cuantifica con más fuerza las frecuencias altas; JPEG 2000 usa una DWT 2D.
- *Segmentación y morfología*: umbralizar, detectar bordes, rellenar huecos y medir objetos.
- *Visión por computador*: las redes neuronales convolucionales encadenan muchas convoluciones

2D cuyos núcleos se aprenden a partir de ejemplos.

Ejercicios 11–16.

## Bibliografía


- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Capítulo 10 (análisis de Fourier de señales con la DFT, incluida la transformada de Fourier dependiente del tiempo).
- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos de la DFT y de bancos de filtros.
- S. Mallat, *A Wavelet Tour of Signal Processing: The Sparse Way*, 3.ª ed. Academic Press, 2009. Referencia completa sobre wavelets, bancos de filtros y eliminación de ruido.
- R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4.ª ed. Pearson, 2018. Histograma, filtrado espacial, DFT 2D y filtrado en frecuencia, wavelets en imágenes.
- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulos de procesamiento de imágenes y wavelets. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.


## 5.4 Ejercicios

Los ejercicios 1–5 corresponden a la parte 1 del tema (transformada de Fourier de tiempo corto y espectrograma), los ejercicios 6–10 a la parte 2 (wavelets) y los ejercicios 11–16 a la parte 3 (procesamiento de imágenes). Se usa $\Omega$ para la frecuencia discreta y $f$ en hercios; $L$ es la longitud de la ventana, $R$ el salto entre tramos y $N$ el número de puntos de la DFT. Los resultados se pueden comprobar con los ejemplos de `teoría/T5/codigo/`.

### Transformada de Fourier de tiempo corto

**Ejercicio 1** (parámetros de un espectrograma) Una grabación de voz de 1 s se muestrea a $f_s = 8\,\text{kHz}$ y se analiza con una ventana de Hann de $L = 256$ muestras, salto $R = 64$ y DFT de $N = 512$ puntos.

a) ¿Qué duración tiene cada tramo y qué solapamiento hay entre tramos consecutivos?

b) ¿Cuántos tramos completos hay?

c) ¿Qué separación en hercios hay entre dos frecuencias consecutivas de la DFT? ¿Y cuál es la mínima separación entre dos tonos que se pueden distinguir?

d) ¿Mejoraría esa separación mínima con $N = 2048$?

**Solución**

a) $L/f_s = 32$ ms. Dos tramos consecutivos comparten $L - R = 192$ muestras: un 75 %. Hay un espectro cada $R/f_s = 8$ ms.

b) $M = \lfloor (8000 - 256)/64 \rfloor + 1 = \lfloor 121 \rfloor + 1 = 122$.

c) Las frecuencias de la DFT están separadas $f_s/N = 15{,}6$ Hz. La separación mínima la fija la ventana: el lóbulo principal de Hann mide $4f_s/L = 125$ Hz entre ceros, y dos tonos se separan si distan al menos la mitad, unos 62,5 Hz.

d) No. Más puntos de DFT muestrean más densamente el mismo espectro del tramo (separación de 3,9 Hz), pero el lóbulo principal sigue midiendo 125 Hz. Para separar tonos más próximos hay que alargar la ventana, a costa de la resolución temporal.

**Ejercicio 2** (elegir la longitud de la ventana) Una señal muestreada a $f_s = 1$ kHz contiene dos tonos de 200 y 220 Hz y, más tarde, dos clics separados 10 ms. Se usa una ventana de Hann.

a) ¿Qué longitud mínima necesita la ventana para separar los tonos?

b) ¿Qué longitud máxima permite ver los clics como dos sucesos distintos?

c) ¿Hay una STFT que resuelva las dos cosas? ¿Qué alternativas hay?

**Solución**

a) Hace falta $2f_s/L \le 20$ Hz, es decir, $L \ge 100$ muestras (100 ms).

b) Cada clic se extiende en el espectrograma lo que dura la ventana. Para que no se solapen, $L/f_s \lesssim 10$ ms, es decir, $L \lesssim 10$ muestras.

c) No: las dos condiciones son incompatibles por el principio de incertidumbre. Se pueden calcular dos espectrogramas, uno con ventana larga para los tonos y otro con ventana corta para los clics, o usar una transformada wavelet, que analiza las altas frecuencias con ventanas cortas y las bajas con ventanas largas. (Aun así, separar 200 y 220 Hz exige unos 100 ms de señal con cualquier método.)

**Ejercicio 3** (STFT de un tono y de un chirp)

a) Demuestra que la STFT de $x[n] = \mathrm{e}^{j\Omega_0 n}$ con una ventana $w[n]$ es

$$X(m, \Omega) = \mathrm{e}^{-j(\Omega - \Omega_0)mR}\, W\!\left(\mathrm{e}^{j(\Omega - \Omega_0)}\right),$$

donde $W$ es la DTFT de la ventana. ¿Cómo es el espectrograma?

b) Un chirp lineal $x(t) = \cos\!\left(2\pi(f_0 t + \frac{k}{2}t^2)\right)$ tiene frecuencia instantánea $f_0 + kt$. Para el chirp de 50 a 400 Hz en 2 s, ¿cuánto vale $k$ y qué frecuencia debe tener el máximo del tramo centrado en $t = 1$ s?

c) ¿Cuánto cambia la frecuencia del chirp durante una ventana de 128 muestras a $f_s = 1$ kHz? Compáralo con la anchura del lóbulo principal de Hann.

**Solución**

a) $X(m, \Omega) = \sum_n \mathrm{e}^{j\Omega_0 n} w[n - mR] \mathrm{e}^{-j\Omega n}$. Con $r = n - mR$: $\mathrm{e}^{-j(\Omega - \Omega_0)mR} \sum_r w[r] \mathrm{e}^{-j(\Omega - \Omega_0)r} = \mathrm{e}^{-j(\Omega - \Omega_0)mR}\, W(\mathrm{e}^{j(\Omega - \Omega_0)})$. El módulo no depende de $m$: el espectrograma es una banda horizontal en $\Omega_0$ con la forma de $|W|^2$, igual en todos los tramos. Un coseno da dos bandas, en

$\pm\Omega_0$.

b) $k = (400 - 50)/2 = 175$ Hz/s. En $t = 1$ s, $f = 50 + 175 = 225$ Hz. El ejemplo `stft_manual` da 230 Hz para el tramo centrado en 1,024 s ($50 + 175 \cdot 1{,}024 = 229$ Hz, redondeado a la rejilla de 3,9 Hz).

c) En 128 ms la frecuencia cambia $175 \cdot 0{,}128 = 22$ Hz, menos que el lóbulo de Hann, $4f_s/L = 31$ Hz. El chirp se ve como una recta fina; con $L = 512$ cambiaría 90 Hz y se ensancharía.

**Ejercicio 4** (solapamiento y suma)

a) Demuestra que la ventana de Hann periódica $w[n] = \frac{1}{2} - \frac{1}{2}\cos\frac{2\pi n}{L}$, $0 \le n < L$ ($L$ par), cumple $\sum_m w[n - mL/2] = 1$.

b) ¿Cuánto suma la ventana de Hamming periódica $0{,}54 - 0{,}46\cos\frac{2\pi n}{L}$ con el mismo salto?

c) ¿Qué salto hace falta con la ventana rectangular?

d) ¿Por qué la reconstrucción solo necesita que la suma sea constante, y no que valga 1?

**Solución**

a) En cada instante $n$ se solapan exactamente dos ventanas, una en la posición $r$ y otra en $r+L/2$ (módulo $L$). Como $\cos\frac{2\pi(r+L/2)}{L} = -\cos\frac{2\pi r}{L}$, la suma es $\frac{1}{2} - \frac{1}{2}\cos\frac{2\pi r}{L} + \frac{1}{2} + \frac{1}{2}\cos\frac{2\pi r}{L} = 1$.

b) Por el mismo argumento, $0{,}54 + 0{,}54 = 1{,}08$, constante.

c) $R = L$: los tramos no se solapan y se tocan.

d) Si la suma es $C$, la reconstrucción da $C\,x[n]$ y basta dividir por $C$.

**Ejercicio 5** (detección DTMF) Un receptor DTMF trabaja a $f_s = 8$ kHz. Los tonos de fila son 697, 770, 852 y 941 Hz, y los de columna 1209, 1336 y 1477 Hz.

a) ¿Cuál es la pareja de tonos más próximos? ¿Qué longitud mínima de ventana necesita una ventana rectangular para separarlos? ¿Y una de Hamming?

b) Si se usa una DFT de $N = 256$ puntos, ¿qué índice $k$ y qué frecuencia de la DFT quedan más cerca de 852 Hz y de 1336 Hz?

c) La norma exige reconocer tonos de 40 ms. ¿Es compatible con el apartado a?

**Solución**

a) 697 y 770 Hz, a 73 Hz. La rectangular tiene un lóbulo de $2f_s/L$ entre ceros y separa tonos a $\approx f_s/L$: $L \ge 8000/73 \approx 110$ muestras (14 ms). La de Hamming, como la de Hann, separa a $\approx 2f_s/L$: $L \ge 220$ muestras (27 ms).

b) $f_s/N = 31{,}25$ Hz. $852/31{,}25 = 27{,}3$, así que $k = 27$, que es 843,75 Hz. $1336/31{,}25 = 42{,}75$, así que $k = 43$, que es 1343,75 Hz.

c) Sí: 27 ms caben en un tono de 40 ms. La rejilla de la DFT no coincide con los tonos, por eso se usa el algoritmo de Goertzel, que evalúa la DTFT exactamente en las 7 frecuencias.

## Wavelets

**Ejercicio 6** (DWT de Haar a mano) Calcula la DWT de Haar de tres niveles de $x = (4, 6, 10, 12, 8, 6, 5, 5)$ y comprueba que se conserva la energía.

**Solución**

Nivel 1: $a_1 = \frac{1}{\sqrt{2}}(10, 22, 14, 10) = (7{,}07;\ 15{,}56;\ 9{,}90;\ 7{,}07)$ y $d_1 = \frac{1}{\sqrt{2}}(-2, -2, 2, 0)$. Nivel 2, sobre $a_1$: $a_2 = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}}(10 + 22,\ 14 + 10) = (16, 12)$ y $d_2 = \frac{1}{2}(10 - 22,\ 14 - 10) = (-6, 2)$. Nivel 3: $a_3 = \frac{16+12}{\sqrt{2}} = 19{,}80$ y $d_3 = \frac{16-12}{\sqrt{2}} = 2{,}83$.
Energía: $\sum x^2 = 16 + 36 + 100 + 144 + 64 + 36 + 25 + 25 = 446$. En los coeficientes: $a_3^2 = 392$, $d_3^2 = 8$, $\sum d_2^2 = 40$ y $\sum d_1^2 = \frac{4+4+4+0}{2} = 6$; total 446. El 88 % está en $a_3$, que es la media de la señal multiplicada por $\sqrt{8}$.

**Ejercicio 7** (reconstrucción aproximada) Con los coeficientes del ejercicio anterior:

a) Reconstruye la señal anulando $d_1$. ¿Qué operación sencilla se obtiene?

b) ¿Cuánto vale la energía del error? Compárala con $\sum d_1^2$.

c) Si además se anula $d_2$, ¿qué señal queda?

**Solución**

a) Con $d_1 = 0$, $x[2k] = x[2k+1] = a_1[k]/\sqrt{2}$, la media de cada pareja: $(5, 5, 11, 11, 7, 7, 5, 5)$.

b) El error es $(-1, 1, -1, 1, 1, -1, 0, 0)$, de energía 6, igual a $\sum d_1^2$. Por ser la base ortonormal, la energía del error es la de los coeficientes anulados.

c) $A_2$: la media de cada grupo de cuatro, $(8, 8, 8, 8, 6, 6, 6, 6)$. El error tiene energía $6 + 40 = 46$.

**Ejercicio 8** (escalas y frecuencias de la Morlet) Se quiere analizar una señal muestreada a $f_s = 1\,\text{kHz}$ entre 10 y 200 Hz con la wavelet de Morlet de $\omega_0 = 6$.

a) ¿Qué escalas $a$ (en segundos y en muestras) corresponden a 10 y a 200 Hz?

b) La envolvente de $\psi_{a,b}$ es una gaussiana de desviación típica $a$, y la de su espectro, de desviación típica $1/a$ en rad/s. Calcula $\sigma_t$ y $\sigma_f$ a 10 y a 200 Hz. ¿Qué producto se mantiene constante?

**Solución**

a) $a = \omega_0/(2\pi f)$: a 10 Hz, $a = 95{,}5$ ms (95,5 muestras); a 200 Hz, $a = 4{,}77$ ms (4,8 muestras).

b) $\sigma_t = a$ y $\sigma_f = 1/(2\pi a) = f/\omega_0$. A 10 Hz: $\sigma_t = 95{,}5$ ms y $\sigma_f = 1{,}7$ Hz. A 200 Hz: $\sigma_t = 4{,}8$ ms y $\sigma_f = 33$ Hz. El producto $\sigma_t\sigma_f = 1/(2\pi)$ es el mismo; también la resolución relativa $\sigma_f/f = 1/\omega_0$.

**Ejercicio 9** (momentos nulos)

a) Comprueba que la wavelet de Haar tiene un momento nulo y no dos.

b) Los coeficientes del paso alto de db2 son $h_1 = \frac{1}{4\sqrt{2}}\big(1-\sqrt{3},\ -(3-\sqrt{3}),\ 3+\sqrt{3},\ -(1+\sqrt{3})\big)$. Comprueba que $\sum_n h_1[n] = 0$ y $\sum_n n\,h_1[n] = 0$.

c) ¿Qué detalles $d_1$ dan Haar y db2 para la rampa $x[n] = n$?

**Solución**

a) $\int \psi \, \mathrm{d}t = \frac{1}{2} - \frac{1}{2} = 0$, pero $\int t\,\psi \, \mathrm{d}t = \int_0^{1/2} t \, \mathrm{d}t - \int_{1/2}^{1} t \, \mathrm{d}t = \frac{1}{8} - \frac{3}{8} = -\frac{1}{4} \neq 0$.

b) Sin el factor común: $(1-\sqrt{3}) - (3-\sqrt{3}) + (3+\sqrt{3}) - (1+\sqrt{3}) = 0$, y $0 - (3-\sqrt{3}) + 2(3+\sqrt{3}) - 3(1+\sqrt{3}) = -3 + \sqrt{3} + 6 + 2\sqrt{3} - 3 - 3\sqrt{3} = 0$.

c) Haar: $d_1[k] = (2k - (2k+1))/\sqrt{2} = -1/\sqrt{2}$ en todos los $k$. db2: cada detalle es una combinación $\sum_n h_1[n]\, x[2k+n]$; con $x$ una rampa vale $2k \sum h_1 + \sum n\, h_1 = 0$, salvo en los extremos, donde el filtro se sale de la señal. Por eso db2 representa con menos coeficientes las señales suaves.

**Ejercicio 10** (banco de filtros y umbral)

a) Con $H_0(z) = \frac{1+z^{-1}}{\sqrt{2}}$ y $H_1(z) = \frac{1-z^{-1}}{\sqrt{2}}$, calcula $|H_0(\mathrm{e}^{j\Omega})|^2 + |H_1(\mathrm{e}^{j\Omega})|^2$. ¿Dónde están sus ceros?

b) Una señal de $N = 1024$ muestras se descompone en $J = 5$ niveles. ¿Cuántos coeficientes hay en cada nivel y en total? ¿Qué banda de frecuencias recoge aproximadamente $a_5$ si $f_s = 1\,\mathrm{kHz}$?

c) La mediana de $|d_1|$ vale 0,27. Estima la desviación típica del ruido y el umbral universal.

**Solución**

a) $|H_0|^2 = \frac{1}{2}|1 + \mathrm{e}^{-j\Omega}|^2 = 1 + \cos\Omega$ y $|H_1|^2 = 1 - \cos\Omega$: suman 2 en todas las frecuencias. $H_0$ tiene un cero en $\Omega = \pi$ (paso bajo) y $H_1$ en $\Omega = 0$ (paso alto).

b) $d_1$: 512, $d_2$: 256, $d_3$: 128, $d_4$: 64, $d_5$: 32 y $a_5$: 32; en total 1024, las mismas que muestras. $a_5$ recoge $[0, \pi/32]$, es decir, de 0 a $500/32 \approx 16\,\mathrm{Hz}$.

c) $\hat{\sigma} = 0{,}27/0{,}6745 = 0{,}40$ y $u = 0{,}40\sqrt{2\ln 1024} = 0{,}40 \cdot 3{,}72 = 1{,}49$, como en el ejemplo `eliminar_ruido`.

## Procesamiento de imágenes

**Ejercicio 11** (tamaño y muestreo de una imagen)

a) ¿Cuánto ocupa sin comprimir una imagen en color de $4000 \times 3000$ píxeles con 8 bits por canal? ¿Y en escala de grises con 16 bits?

b) Un sensor tiene píxeles de $4\,\mu\mathrm{m}$. ¿Cuál es la máxima frecuencia espacial que puede representar sin aliasing, en ciclos por milímetro?

c) Se fotografía una tela con rayas de periodo $6\,\mu\mathrm{m}$ en el plano del sensor. ¿Qué se ve?

**Solución**

a) $4000 \cdot 3000 \cdot 3 = 36 \cdot 10^6$ bytes, 36 MB. En grises con 16 bits, $4000 \cdot 3000 \cdot 2 = 24\,\mathrm{MB}$.

b) La frecuencia de muestreo es $1/4\,\mu\mathrm{m} = 250$ muestras/mm; la de Nyquist, 125 ciclos/mm.

c) Las rayas tienen $1/0{,}006 = 167$ ciclos/mm, por encima de 125: aparece aliasing a $250 - 167 = 83$ ciclos/mm, unas rayas falsas de periodo $12\,\mu\mathrm{m}$ (3 píxeles): el efecto moiré. Las cámaras ponen un filtro óptico paso bajo delante del sensor para evitarlo.

**Ejercicio 12** (ecualización a mano) Una imagen de $4 \times 4$ píxeles con 8 niveles ($L = 8$, de 0 a 7) tiene el histograma

$$h = (0,\ 0,\ 6,\ 5,\ 3,\ 2,\ 0,\ 0).$$

a) Calcula $F(r)$ y la transformación de ecualización $s = \text{round}(7F(r))$.

b) Da el histograma de la imagen ecualizada. ¿Es plano?

**Solución**

a) $F(2) = 6/16 = 0{,}375$, $F(3) = 11/16 = 0{,}6875$, $F(4) = 14/16 = 0{,}875$ y $F(5) = 1$. Entonces $T(2) = \text{round}(2{,}625) = 3$, $T(3) = \text{round}(4{,}81) = 5$, $T(4) = \text{round}(6{,}125) = 6$ y $T(5) = 7$.

b) Seis píxeles en el nivel 3, cinco en el 5, tres en el 6 y dos en el 7. No es plano, porque todos los píxeles de un nivel van al mismo nivel nuevo, pero ahora ocupa de 3 a 7 en lugar de 2 a 5: el contraste ha aumentado.

**Ejercicio 13** (convolución 2D a mano) La imagen de $5 \times 5$

$$f = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 9 & 9 & 9 & 0 \\ 0 & 9 & 9 & 9 & 0 \\ 0 & 9 & 9 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

se convoluciona (opción `'same'`, con ceros fuera) con:

a) la media de $3 \times 3$; b) el laplaciano; c) el Sobel $S_x$.

Calcula la fila central ($m = 2$) del resultado en los tres casos e interprétala.

**Solución**

a) Cada salida es la media de su entorno de $3 \times 3$: $(3, 6, 9, 6, 3)$. El borde del cuadrado se suaviza.

b) $f[m-1,n] + f[m+1,n] + f[m,n-1] + f[m,n+1] - 4f[m,n]$: $(9, -9, 0, -9, 9)$. Es nula en el interior uniforme y cambia de signo al cruzar el borde: los bordes son los pasos por cero.

c) Con la convolución, $S_x$ calcula $\sum_i (1,2,1)_i \, (f[m-i+1, n+1] - f[m-i+1, n-1])$: $(36, 36, 0, -36, -36)$. Positiva donde la imagen crece hacia la derecha (borde izquierdo), negativa donde decrece, y nula en el centro. Con `filter2` o `imfilter` (correlación) los signos se invierten.

**Ejercicio 14** (separabilidad)

a) Escribe el núcleo de media de $3 \times 3$ y el de Sobel $S_y$ (derivada vertical) como producto de un vector columna por un vector fila.

b) ¿Cuántos productos por píxel cuesta filtrar con un gaussiano de $15 \times 15$ directamente y de forma separable?

c) ¿Es separable el laplaciano $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$? ¿Se puede calcular con filtros 1D?

**Solución**

a) Media: $\frac{1}{9}\mathbf{1}_{3\times 3} = \frac{1}{3}(1,1,1)^T \cdot \frac{1}{3}(1,1,1)$. $S_y = S_x^T = (1,0,-1)^T(1,2,1)$.

b) $15^2 = 225$ directamente y $2 \cdot 15 = 30$ de forma separable.

c) No: una matriz separable tiene rango 1 y esta tiene rango 2 (sus filas $(0,1,0)$ y $(1,-4,1)$ no son proporcionales). Pero es la suma de dos núcleos separables, la segunda derivada en filas $(1,-2,1)$ y en columnas $(1,-2,1)^T$, así que se calcula con dos filtros 1D y una suma.

**Ejercicio 15** (DFT 2D de una onda plana) Sea la imagen de $8 \times 8$ $f[m,n] = \cos\left(2\pi\frac{3m+n}{8}\right)$.

a) Calcula su DFT 2D. ¿En qué posiciones es distinta de cero y cuánto vale?

b) ¿Cómo están orientadas las franjas de la imagen?

c) Si la imagen se desplaza circularmente dos columnas, ¿qué le pasa a la DFT?

**Solución**

a) $f = \frac{1}{2}e^{j2\pi(3m/8+n/8)} + \frac{1}{2}e^{-j2\pi(3m/8+n/8)}$. Cada exponencial da un único coeficiente igual a $MN = 64$ por su amplitud: $F[3,1] = F[5,7] = 32$ (la segunda está en $(-3,-1)$, es decir, en $(5,7)$ módulo 8), y cero en el resto.

b) La fase es constante sobre las rectas $3m + n = \text{cte}$: franjas perpendiculares al vector $(3,1)$, inclinadas, con 3 ciclos a lo largo de cada columna y 1 a lo largo de cada fila.

c) Se multiplica por $e^{-j2\pi l\cdot 2/8}$: $F[3,1]$ queda multiplicado por $e^{-j\pi/2} = -j$ y $F[5,7]$ por $e^{-j7\pi/2} = j$. El módulo no cambia.

**Ejercicio 16** (filtro gaussiano en frecuencia) Se filtra una imagen de $N \times N$ con $H[k,l] = e^{-D^2/(2D_0^2)}$, $D^2 = k^2 + l^2$ (con $k,l$ entre $-N/2$ y $N/2-1$).

a) La CTFT de $e^{-x^2/(2\sigma^2)}$ en función de la frecuencia $u$ (ciclos por píxel) es proporcional a $e^{-2\pi^2\sigma^2u^2}$, y el índice $k$ corresponde a $u = k/N$. ¿A qué $\sigma$ espacial equivale el filtro? Calcúlalo para $N = 256$ y $D_0 = 20$.

b) ¿Por qué un paso bajo ideal con el mismo $D_0$ produce anillos y este no?

c) Para filtrar con la DFT y obtener la convolución lineal con un núcleo de $K \times K$, ¿de qué tamaño deben ser las DFT?

**Solución**

a) Igualando exponentes, $2\pi^2\sigma^2k^2/N^2 = k^2/(2D_0^2)$, de donde $\sigma = \frac{N}{2\pi D_0}$. Con $N = 256$ y $D_0 = 20$, $\sigma = 2{,}04$ píxeles: el mismo suavizado que el gaussiano de $\sigma = 2$ de la figura de filtros espaciales de los apuntes.

b) La respuesta al impulso del ideal es la transformada de un disco, un «sinc circular» con lóbulos que cambian de signo: cada borde se convoluciona con ellos y aparecen anillos. La del gaussiano es otra gaussiana, positiva y sin lóbulos.

c) El producto de DFT da la convolución circular. Para que coincida con la lineal, hay que rellenar imagen y núcleo con ceros hasta al menos $(N+K-1)\times(N+K-1)$.

# 6 Señales aleatorias

**Cómo usar estos apuntes**

Estos apuntes desarrollan el contenido de las cuatro presentaciones del tema: la sección 6.1 corresponde a la parte 1 (variables aleatorias, muestreo y análisis), la sección 6.2 a la parte 2 (varias variables), la sección 6.3 a la parte 3 (señales aleatorias en el dominio del tiempo) y la sección 6.4 a la parte 4 (análisis espectral y sistemas LTI). Los ejemplos de código están en `teoría/T6/codigo/`, en Python (`numpy` y `scipy`) y en Matlab, y se pueden ejecutar tal cual; como usan números aleatorios, los resultados cambian ligeramente de una ejecución a otra y entre los dos lenguajes. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

## 6.1 Variables aleatorias

### 6.1.1 Señales deterministas y aleatorias

Hasta ahora todas las señales han sido *deterministas*: su valor en cada instante se conoce exactamente, porque viene dado por una fórmula, una tabla o una grabación. Muchas señales de interés no son así. El ruido térmico de un amplificador, la voz de un locutor, los bits de un mensaje o el error de un sensor no se pueden predecir: si se repite la observación, la señal sale distinta (figura 6.1). Se llaman señales *aleatorias* o *estocásticas*.

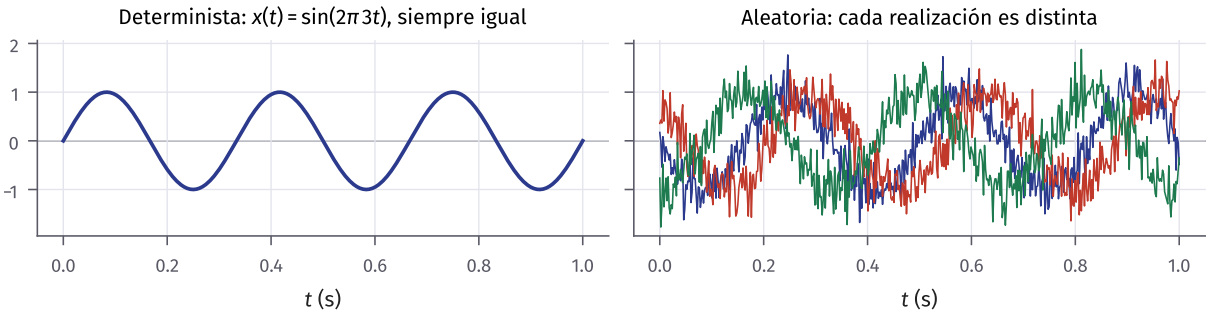



**Figura 6.1.** Una señal determinista y tres realizaciones de una señal aleatoria.

Una señal aleatoria no se describe valor a valor, sino estadísticamente: qué valores son más probables, cuánto se alejan de su media, cómo se parecen los valores próximos en el tiempo, cómo se reparte su potencia en frecuencia. Para ello se necesita el lenguaje de la probabilidad, que se repasa en esta sección y en la siguiente. Las dos últimas lo aplican a las señales.

### 6.1.2 Variable aleatoria

**Definición 6.1.1: Variable aleatoria**

Una *variable aleatoria* $X$ es una función que asigna un número real a cada resultado de un experimento aleatorio. Antes de realizar el experimento solo se sabe con qué probabilidad tomará cada valor; el valor obtenido, $x$, es una *realización* o *muestra* de $X$.

Se usan mayúsculas para la variable y minúsculas para sus valores. Una variable es *discreta* si toma valores aislados $x_1, x_2, \ldots$ (la cara de un dado, el número de errores en un paquete) y *continua* si toma valores en un intervalo (la tensión de ruido en un instante).

Una variable discreta se describe con su *función de probabilidad*

$$p_X(x_i) = P(X = x_i), \qquad p_X(x_i) \geq 0, \qquad \sum_i p_X(x_i) = 1.$$

Para una continua, la probabilidad de un valor concreto es nula, y se usa la *función de densidad* $f_X(x) \geq 0$, que da la probabilidad de un intervalo como un área:

$$P(a < X \leq b) = \int_a^b f_X(x)\,\mathrm{d}x, \qquad \int_{-\infty}^{\infty} f_X(x)\,\mathrm{d}x = 1.$$

$f_X(x)\,\mathrm{d}x$ es la probabilidad de que $X$ caiga en un intervalo de anchura $\mathrm{d}x$ alrededor de $x$. La densidad no es una probabilidad y puede valer más que 1: la de $\mathcal{U}[0; 0{,}5]$ vale 2.

Las dos descripciones se unifican con la *función de distribución*

$$F_X(x) = P(X \leq x),$$

que es no decreciente, va de $F_X(-\infty) = 0$ a $F_X(\infty) = 1$ y cumple $P(a < X \leq b) = F_X(b) - F_X(a)$. Para una variable continua, $F_X(x) = \int_{-\infty}^{x} f_X(u)\,\mathrm{d}u$ y $f_X = \mathrm{d}F_X/\,\mathrm{d}x$; para una discreta, $F_X$ es una escalera con un salto de altura $p_X(x_i)$ en cada $x_i$ (figura 6.2).

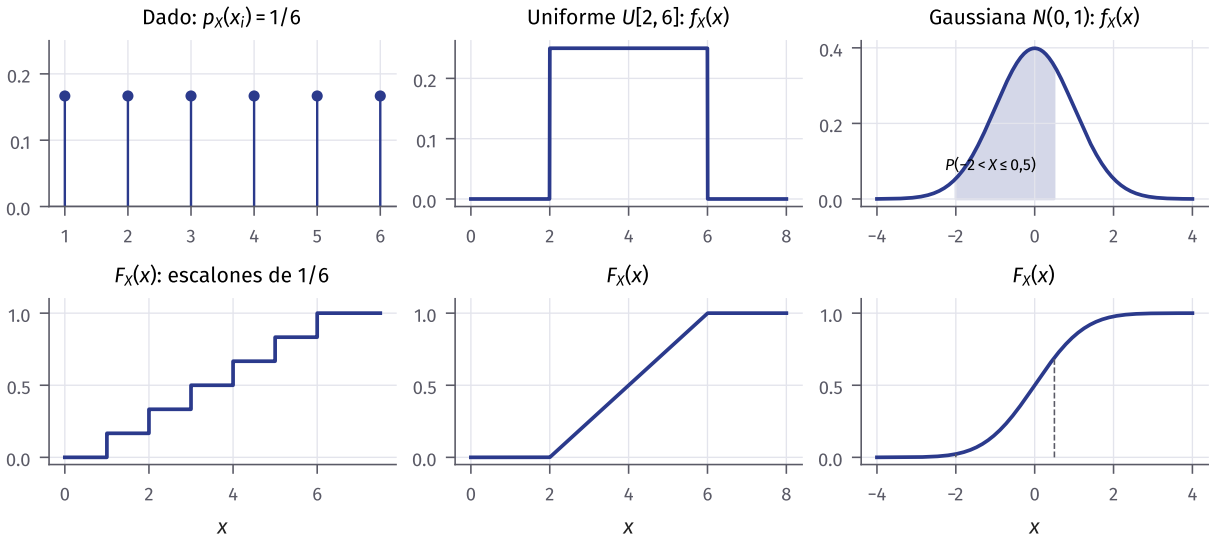



**Figura 6.2.** Función de probabilidad, de densidad y de distribución de un dado, de una uniforme y de una gaussiana.

### 6.1.3 Esperanza, media y varianza

**Definición 6.1.2: Esperanza**

La *esperanza* o valor esperado de $g(X)$ es

$$E\{g(X)\} = \sum_i g(x_i)\, p_X(x_i) \qquad \text{o} \qquad E\{g(X)\} = \int_{-\infty}^{\infty} g(x)\, f_X(x)\, \mathrm{d}x.$$

Es el promedio de $g(X)$ ponderado por la probabilidad. Los casos más usados son:

- la *media* $\mu_X = E\{X\}$, el centro de gravedad de la distribución;
- el *valor cuadrático medio* $E\{X^2\}$, que, si $X$ es una tensión aplicada a una resistencia de $1\,\Omega$, es la potencia media;
- la *varianza* $\sigma_X^2 = E\{(X-\mu_X)^2\}$, que mide la dispersión alrededor de la media. Desarrollando el cuadrado, $\sigma_X^2 = E\{X^2\} - \mu_X^2$. Su raíz $\sigma_X$ es la *desviación típica*, con las mismas unidades que $X$.

La esperanza es lineal: $E\{aX+b\} = a\mu_X + b$, y de ahí $\mathrm{Var}\{aX+b\} = a^2\sigma_X^2$. Por ejemplo, para un dado equilibrado $\mu_X = 3{,}5$, $E\{X^2\} = \frac{1+4+\cdots+36}{6} = \frac{91}{6}$ y $\sigma_X^2 = \frac{91}{6} - \frac{49}{4} = \frac{35}{12}$.

### 6.1.4 Distribuciones habituales

**Uniforme.** $X \sim \mathcal{U}[a,b]$ si su densidad es constante en $[a,b]$:

$$f_X(x) = \begin{cases} \dfrac{1}{b-a}, & a \le x \le b, \\ 0, & \text{en otro caso,} \end{cases} \qquad \mu_X = \frac{a+b}{2}, \quad E\{X^2\} = \frac{a^2+ab+b^2}{3}, \quad \sigma_X^2 = \frac{(b-a)^2}{12}.$$

Modela el error de cuantificación (Tema 2), la fase de una portadora desconocida y los números aleatorios que genera un ordenador.

**Gaussiana o normal.** $X \sim \mathcal{N}(\mu, \sigma^2)$ si

$$f_X(x) = \frac{1}{\sqrt{2\pi}\,\sigma}\, \mathrm{e}^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

Su media es $\mu$ y su varianza $\sigma^2$. La variable $Z = (X-\mu)/\sigma$ es $\mathcal{N}(0,1)$, cuya función de distribución $\Phi(z)$ está tabulada; así $P(X \le x) = \Phi\big((x-\mu)/\sigma\big)$. A una, dos y tres desviaciones típicas de la media queda el 68,3, el 95,4 y el 99,7 % de la probabilidad (figura 6.3). Una transformación lineal de una gaussiana es gaussiana: $aX+b \sim \mathcal{N}(a\mu+b,\ a^2\sigma^2)$. Su importancia se debe al teorema del límite central (sección 6.2.5): el ruido que resulta de sumar muchas contribuciones pequeñas e independientes, como el ruido térmico, es gaussiano.

**Exponencial.** $f_X(x) = \lambda \mathrm{e}^{-\lambda x}$ para $x \ge 0$, con $\mu_X = 1/\lambda$ y $\sigma_X^2 = 1/\lambda^2$. Modela tiempos de espera entre sucesos independientes.

Ejercicios 1–3.

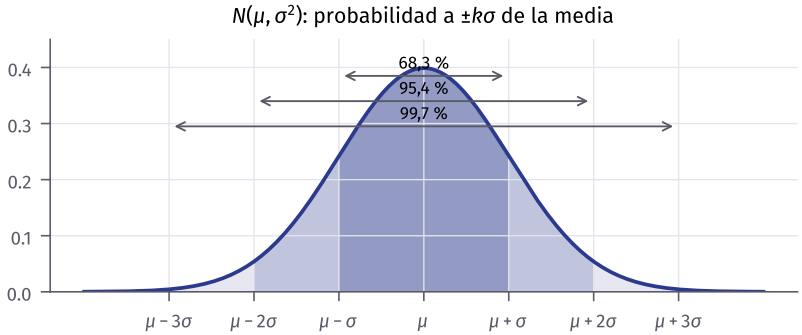



**Figura 6.3.** Densidad gaussiana y probabilidad a $\pm\sigma$, $\pm 2\sigma$ y $\pm 3\sigma$ de la media.

### 6.1.5 Muestreo de variables aleatorias

Los generadores de números pseudoaleatorios de Matlab (`rand`) y de Python (`rng.random`, con `rng = np.random.default_rng()`) producen muestras de $\mathcal{U}[0,1)$ que se comportan como independientes. A partir de ellas se pueden obtener muestras de cualquier distribución. Esto permite *simular*: repetir muchas veces un experimento con entradas aleatorias y estimar a partir de los resultados lo que sería difícil de calcular. Es el *método de Monte Carlo*. Fijando la semilla del generador (`rng(0)` en Matlab, `default_rng(0)` en Python) los resultados son reproducibles.

**Método de la ruleta.** Para una variable discreta se reparte $[0,1)$ en tramos consecutivos de longitudes $p_X(x_1), p_X(x_2), \ldots$, como los sectores de una ruleta, se genera $u \sim \mathcal{U}[0,1)$ y se devuelve el valor cuyo tramo contiene a $u$:

$$x = x_i \qquad \text{si} \qquad F_X(x_{i-1}) \le u < F_X(x_i).$$

Como el tramo $i$ mide $p_X(x_i)$, el valor $x_i$ sale con esa probabilidad.

**En código**

Dado cargado con $p = (0{,}1;\ 0{,}1;\ 0{,}1;\ 0{,}1;\ 0{,}1;\ 0{,}5)$. Las frecuencias relativas se acercan a $p$ y la media a 4,5.

**Python**

```python
import numpy as np

rng = np.random.default_rng(1)
p = np.array([.1, .1, .1, .1, .1, .5])
F = np.cumsum(p)                # 0.1 ... 1
u = rng.random(60000)
x = np.searchsorted(F, u, 'right') + 1
frec = np.bincount(x, minlength=7)[1:]
print(frec/u.size)              # ~ p
print(x.mean(), np.sum(np.arange(1, 7)*p))
```

**Matlab**

```matlab
p = [.1 .1 .1 .1 .1 .5];        % dado
F = cumsum(p);                  % 0.1 ... 1
u = rand(1, 60000);
x = zeros(size(u));
for i = 1:numel(u)
  x(i) = find(u(i) < F, 1);  % 1er F > u
end
frec = accumarray(x', 1)'/numel(u);
disp(frec)                      % ~ p
disp([mean(x), sum((1:6).*p)])
```

**Inversa de la función de distribución.** Si $U \sim \mathcal{U}[0,1]$ y $F_X$ es continua y estrictamente creciente, la variable $X = F_X^{-1}(U)$ tiene función de distribución $F_X$, porque

$$P(X \le x) = P\big(F_X^{-1}(U) \le x\big) = P\big(U \le F_X(x)\big) = F_X(x).$$

Para la exponencial, despejando $u = 1 - \mathrm{e}^{-\lambda x}$ se obtiene $x = -\frac{1}{\lambda}\ln(1-u)$ (figura 6.4). El método de la ruleta es la versión discreta de este. El resultado también funciona al revés: si $X$ tiene distribución

continua $F_X$, la variable $F_X(X)$ es uniforme en $[0,1]$. Es la base de la ecualización del histograma del Tema 5.

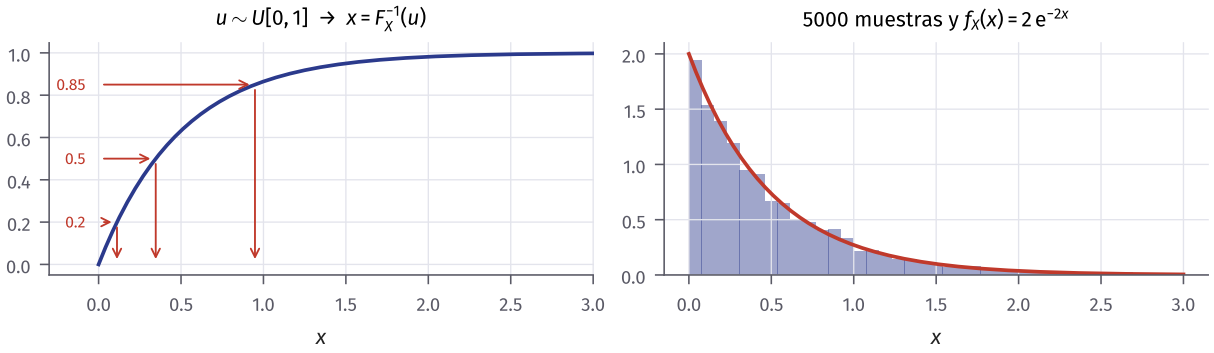



**Figura 6.4.** Método de la inversa para la exponencial de $\lambda = 2$: tres valores de $u$ y su $x$, y el histograma de 5000 muestras.

**En código**

Muestras exponenciales con $\lambda = 2$: la media se acerca a $1/\lambda = 0{,}5$, la varianza a $1/\lambda^2 = 0{,}25$ y la fracción de muestras mayores que 1 a $e^{-2} = 0{,}135$.

**Python**

```python
import numpy as np

rng = np.random.default_rng(0)
lam, N = 2.0, 100000
u = rng.random(N)                  # U[0,1)
x = -np.log(1 - u)/lam             # F^-1(u)
print(x.mean(), 1/lam)             # media
print(x.var(ddof=1), 1/lam**2)     # varianza
print(np.mean(x > 1), np.exp(-lam))
```

**Matlab**

```matlab
lam = 2; N = 100000;
u = rand(1, N);                    % U[0,1)
x = -log(1 - u)/lam;               % F^-1(u)
disp([mean(x), 1/lam])             % media
disp([var(x), 1/lam^2])            % varianza
disp([mean(x > 1), exp(-lam)])
```

**Box-Muller.** La función de distribución de la gaussiana no tiene inversa explícita. Pero un par $(Z_1, Z_2)$ de gaussianas $\mathcal{N}(0,1)$ independientes, escrito en coordenadas polares, tiene un ángulo $\Theta$ uniforme en $[0, 2\pi)$ y un radio $R$ independiente con $F_R(r) = 1 - e^{-r^2/2}$ (una distribución de Rayleigh), que sí se puede invertir. Por tanto, con $u_1, u_2 \sim \mathcal{U}[0,1)$ independientes,

$$r = \sqrt{-2\ln(1-u_1)}, \quad \theta = 2\pi u_2, \qquad z_1 = r\cos\theta, \quad z_2 = r\sin\theta$$

son dos muestras $\mathcal{N}(0,1)$ independientes, y $x = \mu + \sigma z$ es $\mathcal{N}(\mu, \sigma^2)$. En la práctica se usan `randn` (Matlab) y `rng.standard_normal` (Python), que emplean métodos más rápidos.

**En código**

Box-Muller: las dos salidas tienen media 0, desviación típica 1 y están incorreladas; el 68,3 % de las muestras cae a menos de una desviación típica.

**Python**

```python
import numpy as np

rng = np.random.default_rng(2)
N = 50000
u1, u2 = rng.random(N), rng.random(N)
r = np.sqrt(-2*np.log(1 - u1))   # radio
th = 2*np.pi*u2                  # ángulo
z1, z2 = r*np.cos(th), r*np.sin(th)
x = 5 + 2*z1                     # N(5, 4)
print(z1.mean(), z1.std(), z2.std())
print(np.corrcoef(z1, z2)[0, 1]) # ~ 0
print(x.mean(), x.var())         # ~ 5, 4
print(np.mean(abs(z1) < 1))      # ~ 0.683
```

**Matlab**

```matlab
N = 50000;
u1 = rand(1, N); u2 = rand(1, N);
r = sqrt(-2*log(1 - u1));        % radio
th = 2*pi*u2;                    % ángulo
z1 = r.*cos(th); z2 = r.*sin(th);
x = 5 + 2*z1;                    % N(5, 4)
disp([mean(z1), std(z1), std(z2)])
c = corrcoef(z1, z2); disp(c(1, 2))  % ~ 0
disp([mean(x), var(x)])          % ~ 5, 4
disp(mean(abs(z1) < 1))          % ~ 0.683
```

**Aceptación y rechazo.** Cuando $F_X$ no se puede invertir, se busca una densidad $q(x)$ fácil de muestrear y una constante $M$ tales que $Mq(x) \geq f(x)$ para todo $x$. Se genera un candidato $x \sim q$ y un $u \sim \mathcal{U}[0,1)$, y se *acepta* $x$ si $u\,Mq(x) \leq f(x)$; si no, se descarta. Los puntos $(x, uMq(x))$ se reparten uniformemente bajo la curva $Mq$, y los aceptados, uniformemente bajo $f$, así que sus abscisas siguen la densidad $f$ (figura 6.5). La fracción aceptada es el cociente de áreas, $1/M$: conviene que $Mq$ se ajuste a $f$.

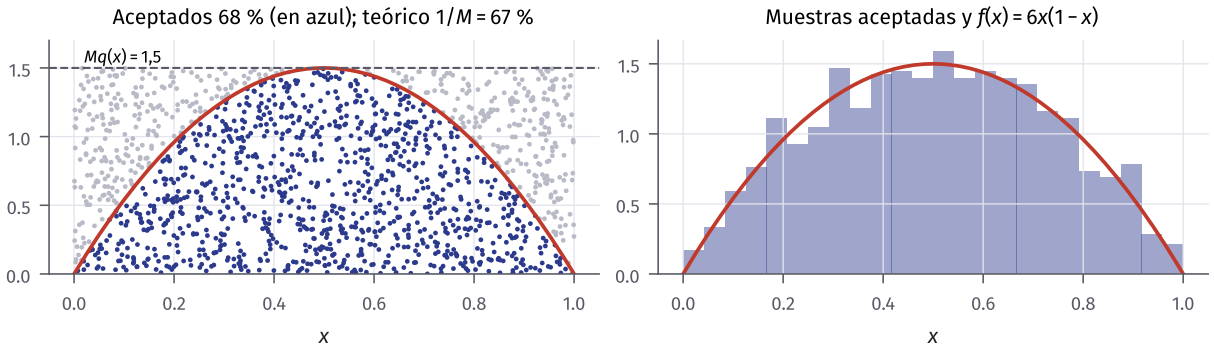



**Figura 6.5.** Aceptación y rechazo para $f(x) = 6x(1-x)$ en $[0,1]$ con $q$ uniforme y $M = 1{,}5$.

**En código**

Aceptación y rechazo para $f(x) = 6x(1-x)$: se acepta un 67 % de los candidatos y las muestras tienen media 0,5 y varianza 0,05, las de $f$.

**Python**

```python
import numpy as np

rng = np.random.default_rng(3)
f = lambda x: 6*x*(1 - x)        # objetivo
M = 1.5                  # f <= M*q, q=U[0,1]
N = 100000
x = rng.random(N)          # candidatos de q
u = rng.random(N)
acept = u*M <= f(x)
xs = x[acept]
print(acept.mean(), 1/M)         # ~ 0.667
print(xs.mean(), xs.var())       # 0.5, 0.05
```

**Matlab**

```matlab
f = @(x) 6*x.*(1 - x);           % objetivo
M = 1.5;                % f <= M*q, q=U[0,1]
N = 100000;
x = rand(1, N);            % candidatos de q
u = rand(1, N);
acept = u*M <= f(x);
xs = x(acept);
disp([mean(acept), 1/M])         % ~ 0.667
disp([mean(xs), var(xs)])        % 0.5, 0.05
```

**Mezclas.** Una *mezcla* es una combinación convexa de densidades,

$$f_X(x) = \sum_{k=1}^{K} \pi_k \, f_k(x), \qquad \pi_k \geq 0, \quad \sum_k \pi_k = 1,$$

que modela poblaciones formadas por varios grupos. Se muestrea en dos pasos: se elige la componente $k$ con probabilidad $\pi_k$ (por el método de la ruleta) y se genera una muestra de $f_k$ (figura 6.6).

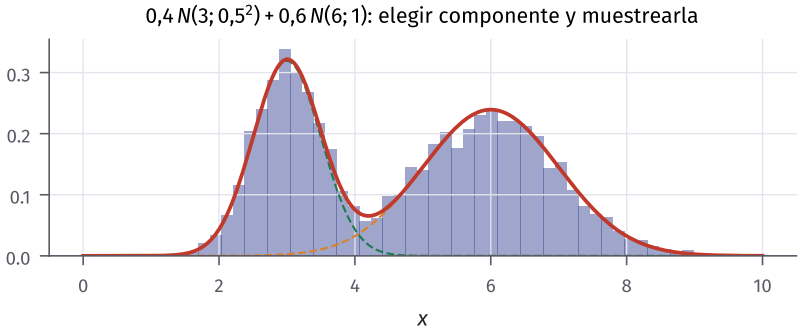



**Figura 6.6.** Histograma de 5000 muestras de una mezcla de dos gaussianas.

### 6.1.6 Análisis de muestras

**Histograma.** Dadas $N$ muestras, se divide el eje en intervalos de anchura $\Delta$ y se cuenta cuántas, $n_j$, caen en cada uno. La altura

$$\hat{f}_X(x) = \frac{n_j}{N\Delta} \qquad \text{para } x \text{ en el intervalo } j$$

estima la densidad, porque $n_j/N$ estima la probabilidad del intervalo, que es aproximadamente $f_X(x)\Delta$. Con esta normalización el área total es 1 y el histograma se puede comparar con la densidad (figura 6.7). Intervalos estrechos dan más detalle pero más ruido, porque cada uno recibe menos muestras. En Matlab, `histogram(x, 'Normalization', 'pdf')`; en Python, `plt.hist(x, density=True)`.

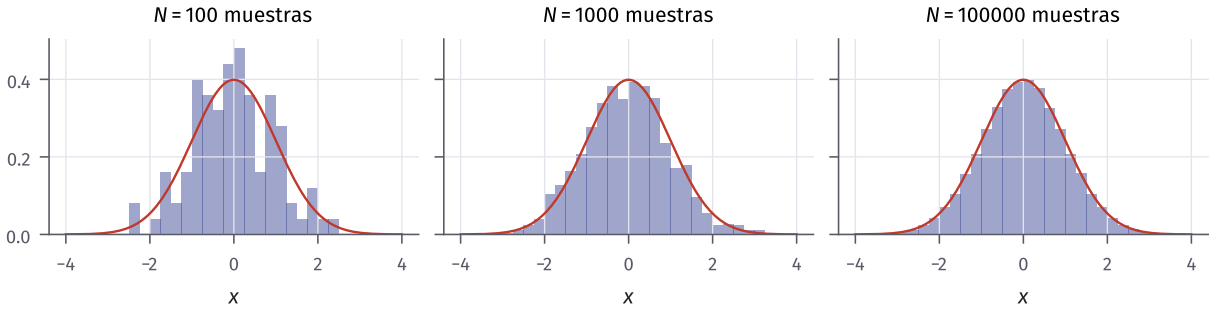



**Figura 6.7.** Histogramas normalizados de muestras $\mathcal{N}(0,1)$ y la densidad teórica.

**Estimadores.** Con $N$ muestras independientes $x_1, \ldots, x_N$ de $X$, la media y la varianza se estiman con

$$\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \hat{\sigma}^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \hat{\mu})^2.$$

Un estimador es una variable aleatoria: con otras muestras sale otro valor. $\hat{\mu}$ es *insesgado*, $E\{\hat{\mu}\} = \mu$, y su varianza es $\sigma^2/N$; su desviación típica, el error típico de la estimación, decrece como $1/\sqrt{N}$. Para la

varianza se divide por $N-1$ porque las desviaciones se miden respecto a $\hat{\mu}$, que está más cerca de las muestras que la media verdadera; dividiendo por $N$ el estimador se queda corto en media en un factor $(N-1)/N$. `var` de Matlab y `np.var(x, ddof=1)` dividen por $N-1$; `np.var(x)` y `var(x, 1)`, por $N$.

**Monte Carlo.** De la misma forma, cualquier esperanza se estima con la media de las muestras:

$$E\{g(X)\} = \int g(x)\, f_X(x)\, \mathrm{d}x \;\approx\; \frac{1}{N}\sum_{i=1}^{N} g(x_i),$$

con un error típico de $\sigma_g/\sqrt{N}$. Así se calculan integrales sin solución cerrada, probabilidades (con $g$ una función indicadora) y el comportamiento de sistemas complejos con entradas aleatorias. La figura 6.8 estima $\pi$ como cuatro veces la fracción de puntos uniformes del cuadrado unidad que caen dentro del cuarto de círculo: para ganar una cifra decimal hace falta cien veces más muestras.

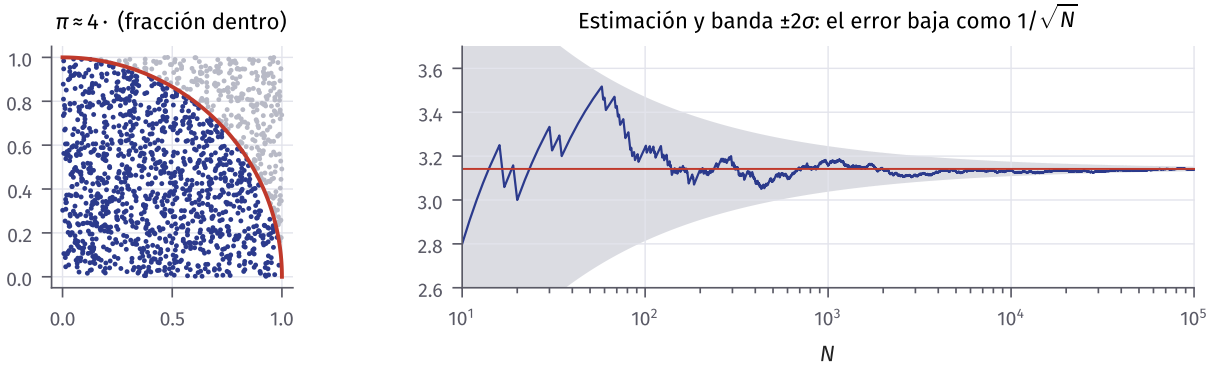



**Figura 6.8.** Estimación de $\pi$ por Monte Carlo y banda de $\pm 2$ errores típicos.

**En código**

$E\{\cos X\}$ con $X \sim \mathcal{N}(0,1)$ vale $\mathrm{e}^{-1/2} = 0{,}6065$. El error típico estimado baja diez veces al multiplicar $N$ por cien.

**Python**

```python
import numpy as np

rng = np.random.default_rng(4)
for N in (100, 10000, 1000000):
    x = rng.standard_normal(N)  # X ~ N(0,1)
    g = np.cos(x)               # E{cos X}?
    est = g.mean()
    err = g.std(ddof=1)/np.sqrt(N)
    print(N, est, err)
print(np.exp(-0.5))             # exacto
```

**Matlab**

```matlab
for N = [100 10000 1000000]
  x = randn(1, N);              % X ~ N(0,1)
  g = cos(x);                   % E{cos X}?
  est = mean(g);
  err = std(g)/sqrt(N);
  disp([N, est, err])
end
disp(exp(-0.5))                 % exacto
```

Ejercicios 4–5.

## 6.2 Varias variables aleatorias

### 6.2.1 Distribución conjunta

Muchas magnitudes aleatorias aparecen juntas: el valor de una señal en dos instantes, la entrada y la salida de un sistema, las dos coordenadas de un punto. Para describirlas no basta con cada una por separado; hace falta su *distribución conjunta*:

$$F_{XY}(x,y) = P(X \le x,\ Y \le y), \qquad f_{XY}(x,y) = \frac{\partial^2 F_{XY}(x,y)}{\partial x\,\partial y},$$

con $P\big((X,Y)\in A\big) = \iint_A f_{XY}\,\mathrm{d}x\,\mathrm{d}y$ para cualquier región $A$ del plano. Para variables discretas se usa la probabilidad conjunta $p_{XY}(x_i,y_j) = P(X=x_i,\ Y=y_j)$. Con $M$ variables se escribe $f_{X_1\cdots X_M}(x_1,\dots,x_M)$, o $f_{\mathbf{X}}(\mathbf{x})$ con notación vectorial. Cuando no hay confusión se omiten los subíndices: $f(x,y)$.

**Marginales.** La distribución de una sola variable se recupera integrando (o sumando) la conjunta en las demás:

$$f_X(x) = \int_{-\infty}^{\infty} f_{XY}(x,y)\,\mathrm{d}y, \qquad p_X(x_i) = \sum_j p_{XY}(x_i,y_j).$$

Con más variables, para obtener la marginal de un subconjunto se integran todas las que no están en él.

**Condicionadas.** La densidad de $Y$ sabiendo que $X=x$ es

$$f_{Y|X}(y|x) = \frac{f_{XY}(x,y)}{f_X(x)} \qquad (f_X(x)>0).$$

De aquí salen dos reglas fundamentales:

- la *regla de la cadena*, $f_{XY}(x,y) = f_{Y|X}(y|x)\,f_X(x) = f_{X|Y}(x|y)\,f_Y(y)$, que con $M$ variables se escribe
$$f(x_1,\dots,x_M) = f(x_1)\,f(x_2|x_1)\,f(x_3|x_1,x_2)\cdots f(x_M|x_1,\dots,x_{M-1}),$$
y admite $M!$ factorizaciones, una por cada orden de las variables;
- el *teorema de Bayes*, que se obtiene igualando las dos factorizaciones:
$$f_{X|Y}(x|y) = \frac{f_{Y|X}(y|x)\,f_X(x)}{f_Y(y)}.$$

**Ejemplo 6.2.1: Una conjunta construida por partes**

Sea $X_1 \sim \mathcal{U}[2,4]$ y, dado $X_1=x_1$, $X_2 \sim \mathcal{N}(x_1,1)$. Por la regla de la cadena,

$$f(x_1,x_2) = f(x_2|x_1)\,f(x_1) = \frac{1}{2}\cdot\frac{1}{\sqrt{2\pi}}\,\mathrm{e}^{-(x_2-x_1)^2/2}, \qquad 2\le x_1\le 4,$$

y cero fuera (figura 6.9). Para muestrearla se sigue el mismo orden: se genera $x_1$ uniforme y después $x_2 = x_1 + z$ con $z\sim\mathcal{N}(0,1)$. Este *muestreo ordenado* según una factorización sirve para cualquier conjunta cuyas condicionadas se sepan muestrear.

### 6.2.2 Independencia

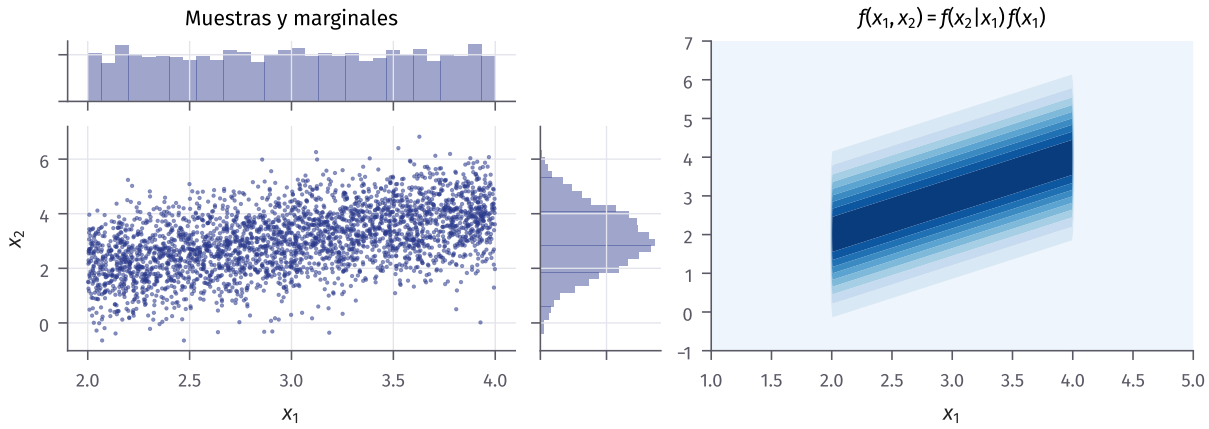



**Figura 6.9.** Muestras de la conjunta del ejemplo 6.2.1, sus histogramas marginales y la densidad conjunta.

**Definición 6.2.2: Independencia**

$X$ e $Y$ son *independientes* si $f_{XY}(x,y) = f_X(x)\, f_Y(y)$ para todo $(x,y)$, o, equivalentemente, si $f_{Y|X}(y|x) = f_Y(y)$: conocer el valor de $X$ no cambia la distribución de $Y$. $M$ variables son independientes si su conjunta es el producto de las $M$ marginales.

Si las variables son independientes, para muestrearlas basta generar cada una por separado. En el ejemplo 6.2.1, $X_1$ y $X_2$ no son independientes: la media de $X_2$ depende de $X_1$. Se dice que $X$ e $Y$ son *condicionalmente independientes* dado $Z$ si $f_{XY|Z}(x,y|z) = f_{X|Z}(x|z)\, f_{Y|Z}(y|z)$: una vez conocido $Z$, $X$ no aporta nada sobre $Y$. Es la propiedad que define las señales de Markov (sección 6.3).

### 6.2.3 Correlación y covarianza

**Definición 6.2.3: Estadísticos de un par de variables**

- *Correlación*: $R_{XY} = E\{XY\} = \iint xy\, f_{XY}(x,y)\, \mathrm{d}x\, \mathrm{d}y$.
- *Covarianza*: $C_{XY} = E\{(X-\mu_X)(Y-\mu_Y)\} = R_{XY} - \mu_X\mu_Y$. Con $Y = X$ es la varianza.
- *Coeficiente de correlación*: $\rho_{XY} = \dfrac{C_{XY}}{\sigma_X\, \sigma_Y}$.

Por la desigualdad de Cauchy-Schwarz, $-1 \leq \rho_{XY} \leq 1$, y $|\rho_{XY}| = 1$ si y solo si $Y = aX + b$, con $\rho$ del signo de $a$. El coeficiente mide, por tanto, el grado de dependencia *lineal* (figura 6.10). Se dice que $X$ e $Y$ son *incorreladas* si $C_{XY} = 0$, es decir, si $E\{XY\} = \mu_X\mu_Y$, y *ortogonales* si $R_{XY} = 0$. La varianza de una suma es

$$\mathrm{Var}\{X+Y\} = \sigma_X^2 + \sigma_Y^2 + 2C_{XY},$$

así que las varianzas se suman cuando las variables están incorreladas.

**Propiedad 6.2.4: Independencia y correlación**

Si $X$ e $Y$ son independientes, están incorreladas:

$$E\{XY\} = \iint xy\, f_X(x) f_Y(y)\, \mathrm{d}x\, \mathrm{d}y = \int x f_X(x)\, \mathrm{d}x \int y f_Y(y)\, \mathrm{d}y = \mu_X\mu_Y.$$

El recíproco es falso en general.

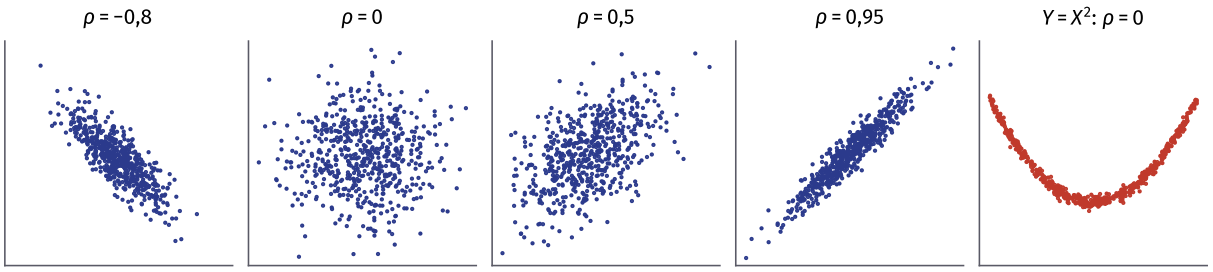



**Figura 6.10.** Pares de variables con distintos coeficientes de correlación. En el último, $Y$ depende de $X$ por completo y sin embargo $\rho = 0$.

Con $X \sim \mathcal{U}[-1,1]$ e $Y = X^2$, $C_{XY} = E\{X^3\} - \mu_X E\{X^2\} = 0 - 0 = 0$, aunque $Y$ es función de $X$. Por tanto, una correlación distinta de cero prueba que hay dependencia, pero una correlación nula no prueba la independencia. La excepción importante son las variables conjuntamente gaussianas, para las que incorrelación e independencia son equivalentes.

**Notación**

Aquí se usan $R$ para la correlación y $C$ para la covarianza, porque $\phi$ se reserva para las señales (sección 6.3) y $\psi$ es la wavelet del Tema 5.

**En código**

Estimación de medias, matriz de covarianza y coeficiente de correlación del ejemplo 6.2.1. Teóricamente $\mu_1 = \mu_2 = 3$, $\sigma_1^2 = \frac{1}{3}$, $\sigma_2^2 = 1 + \frac{1}{3}$ (la varianza de $X_1$ más la del ruido), $C_{12} = E\{(X_1 - 3)(X_1 - 3 + Z)\} = \frac{1}{3}$ y $\rho = \dfrac{1/3}{\sqrt{1/3 \cdot 4/3}} = 0{,}5$.

**Python**

```python
import numpy as np

rng = np.random.default_rng(5)
N = 100000
x1 = rng.uniform(2, 4, N)         # U[2,4]
x2 = x1 + rng.standard_normal(N) # N(x1,1)
X = np.c_[x1, x2]                 # N x 2
print(X.mean(axis=0))             # 3, 3
print(np.cov(X.T))      # [1/3 1/3; 1/3 4/3]
print(np.corrcoef(X.T)[0, 1])     # 0.5
```

**Matlab**

```matlab
N = 100000;
x1 = 2 + 2*rand(N, 1);            % U[2,4]
x2 = x1 + randn(N, 1);            % N(x1,1)
X = [x1, x2];                     % N x 2
disp(mean(X))                     % 3, 3
disp(cov(X))             % [1/3 1/3; 1/3 4/3]
c = corrcoef(X); disp(c(1, 2))    % 0.5
```

### 6.2.4 Vectores aleatorios y gaussiana multivariable

Con $M$ variables agrupadas en un vector columna $\mathbf{X} = (X_1, \ldots, X_M)^T$, los estadísticos de primer y segundo orden son el *vector de medias* y la *matriz de covarianza*:

$$\mu = E\{\mathbf{X}\}, \qquad \Sigma = E\{(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T\}, \qquad \Sigma_{ij} = C_{X_i X_j}, \quad \Sigma_{ii} = \sigma_i^2.$$

$\Sigma$ es simétrica y semidefinida positiva, y diagonal si las variables están incorreladas. Si $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$, por la linealidad de la esperanza

$$\mu_Y = \mathbf{A}\mu + \mathbf{b}, \qquad \Sigma_Y = \mathbf{A}\Sigma\mathbf{A}^T.$$

Para estimarlos a partir de $N$ muestras se colocan en una matriz de $N \times M$ (una muestra por fila) y se usan `mean`, `cov` y `corrcoef` en Matlab; `np.cov` y `np.corrcoef` esperan, por defecto, una variable por fila, de ahí la traspuesta del ejemplo anterior.

**Definición 6.2.5: Gaussiana multivariable**

$\mathbf{X} \sim \mathcal{N}(\mu, \Sigma)$ si

$$f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^M \det \Sigma}} \exp\Big( -\frac{1}{2}(\mathbf{x}-\mu)^T \Sigma^{-1} (\mathbf{x}-\mu) \Big).$$

Queda determinada por su media y su covarianza. Sus curvas de nivel son elipses (elipsoides en más dimensiones) centradas en $\mu$, con los ejes en la dirección de los autovectores de $\Sigma = \mathbf{V}\Lambda\mathbf{V}^T$ y semiejes proporcionales a $\sqrt{\lambda_k}$ (figura 6.11). Las marginales, las condicionadas y cualquier transformación lineal de una gaussiana son gaussianas. Si $\Sigma$ es diagonal, la exponencial se factoriza en un producto: incorreladas implica independientes.

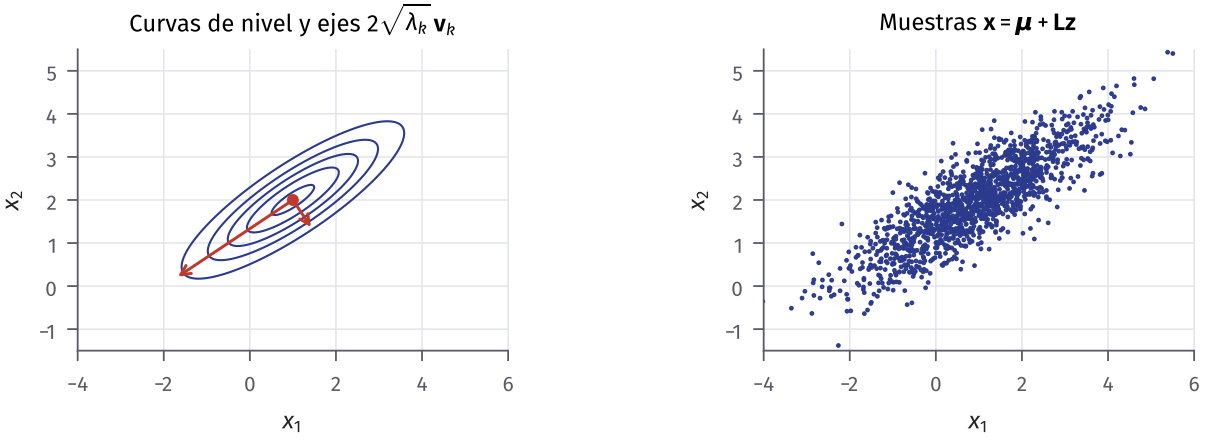



**Figura 6.11.** Gaussiana bidimensional con $\mu = (1,2)^T$ y $\Sigma = \left[\begin{smallmatrix} 2 & 1{,}2 \\ 1{,}2 & 1 \end{smallmatrix}\right]$: curvas de nivel con sus ejes, y muestras.

Para muestrearla se generan $M$ gaussianas independientes $\mathcal{N}(0,1)$, que forman un vector $\mathbf{z}$ con covarianza $\mathbf{I}$, y se transforman:

$$\mathbf{x} = \mu + \mathbf{L}\mathbf{z}, \qquad \mathbf{L}\mathbf{L}^T = \Sigma,$$

porque entonces $\Sigma_X = \mathbf{L}\mathbf{I}\mathbf{L}^T = \Sigma$. $\mathbf{L}$ puede ser el factor de Cholesky, que se calcula con `chol(S, 'lower')` en Matlab y con `np.linalg.cholesky` en Python, o $\mathbf{V}\Lambda^{1/2}$, que primero escala cada eje y después gira.

**En código**

Gaussiana bidimensional muestreada con Cholesky. Los autovalores de $\Sigma$, 0,2 y 2,8, son las varianzas a lo largo de los ejes de la elipse.

**Python**

```python
import numpy as np

rng = np.random.default_rng(6)
mu = np.array([1.0, 2.0])
S = np.array([[2.0, 1.2],
              [1.2, 1.0]])      # covarianza
L = np.linalg.cholesky(S)        # S = L L^T
Z = rng.standard_normal((2, 100000))
X = mu[:, None] + L @ Z          # 2 x N
print(X.mean(axis=1))            # ~ mu
print(np.cov(X))                 # ~ S
lam, V = np.linalg.eigh(S)       # ejes
print(lam)                       # 0.2, 2.8
```

**Matlab**

```matlab
mu = [1; 2];
S = [2.0 1.2;
     1.2 1.0];                   % covarianza
L = chol(S, 'lower');            % S = L L'
Z = randn(2, 100000);
X = mu + L*Z;                    % 2 x N
disp(mean(X, 2)')                % ~ mu
disp(cov(X'))                    % ~ S
[V, D] = eig(S);                 % ejes
disp(diag(D)')                   % 0.2, 2.8
```

### 6.2.5 Suma de variables y teorema del límite central

Sea $S = X + Y$. Por la linealidad, $\mu_S = \mu_X + \mu_Y$ siempre, y $\sigma_S^2 = \sigma_X^2 + \sigma_Y^2$ si las variables están incorreladas. Si además son independientes, la densidad de la suma es la *convolución* de las densidades:

$$f_S(s) = \int_{-\infty}^{\infty} f_X(x)\, f_Y(s-x)\, \mathrm{d}x = (f_X * f_Y)(s),$$

porque para que la suma valga $s$ con $X = x$ tiene que ser $Y = s - x$. Por ejemplo, la suma de dos $\mathcal{U}[0,1]$ independientes tiene densidad triangular en $[0,2]$, la convolución de dos pulsos.

**Propiedad 6.2.6: Teorema del límite central**

Si $X_1, \ldots, X_n$ son independientes, con la misma distribución, media $\mu$ y varianza $\sigma^2$ finita, la suma normalizada

$$Z_n = \frac{\sum_{i=1}^{n} X_i - n\mu}{\sqrt{n}\,\sigma}$$

tiende a $\mathcal{N}(0,1)$ cuando $n \to \infty$, cualquiera que sea la distribución de las $X_i$.

La convergencia es rápida (figura 6.12): la suma de doce uniformes $\mathcal{U}[0,1]$ menos 6 tiene media 0 y varianza 1, y ya es casi gaussiana. El teorema se cumple también, con condiciones, para variables no idénticas, y explica por qué el ruido que resulta de sumar muchas contribuciones independientes, como el movimiento de los electrones en una resistencia, es gaussiano.

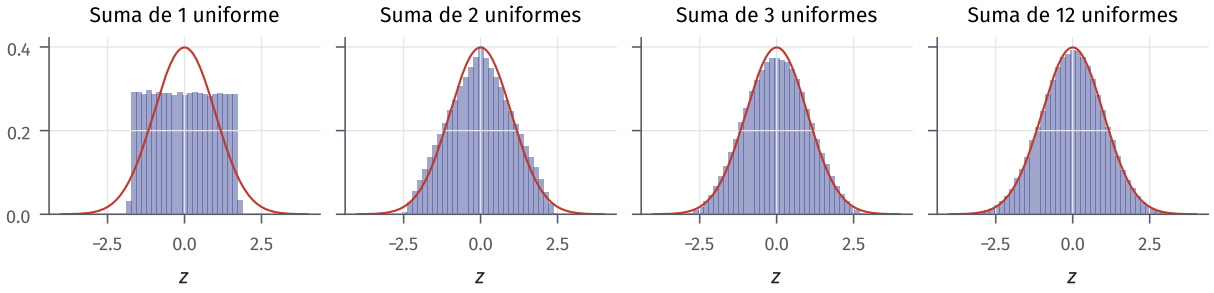



**Figura 6.12.** Histograma de la suma normalizada de $n$ uniformes y la densidad $\mathcal{N}(0,1)$.

Ejercicios 6–10.

## 6.3 Señales aleatorias

### 6.3.1 Definición y descripción

**Definición 6.3.1: Señal aleatoria**

Una *señal aleatoria* o *proceso estocástico* es una familia de variables aleatorias indexadas por el tiempo: $x[n]$, $n \in \mathbb{Z}$, en tiempo discreto, o $x(t)$, $t \in \mathbb{R}$, en tiempo continuo.

Hay dos formas de verla (figura 6.13). Fijado el resultado del experimento, se obtiene una señal determinista, una *realización*; fijado el instante $n$, $x[n]$ es una variable aleatoria. El conjunto de todas las realizaciones posibles se llama *conjunto* (*ensemble*). Siguiendo la notación del curso, escribimos la señal aleatoria con minúscula, como sus realizaciones; el contexto aclara de qué se habla.

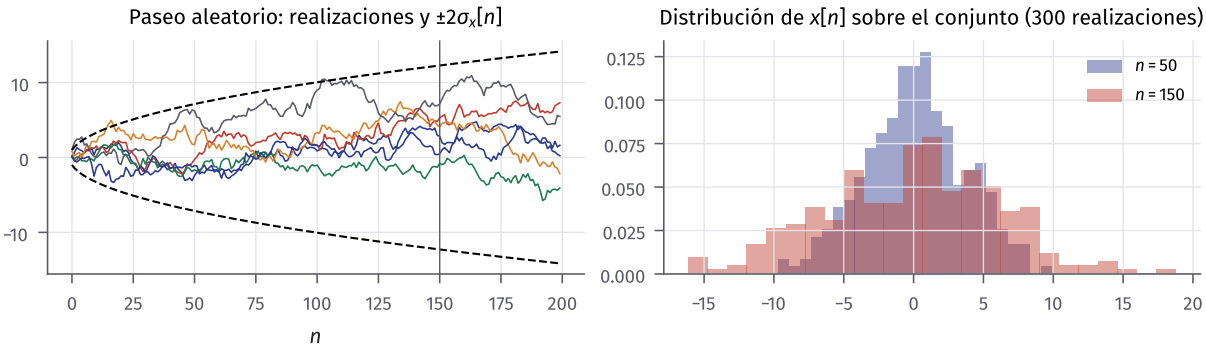



**Figura 6.13.** Seis realizaciones de un paseo aleatorio y la distribución de $x[n]$ en dos instantes, estimada con 300 realizaciones.

Una señal aleatoria queda completamente descrita por las distribuciones conjuntas de $x[n_1], \dots, x[n_k]$ para cualesquiera $k$ instantes, algo que casi nunca se conoce. En la práctica se usan los momentos de primer y segundo orden, que son los estadísticos de la sección 6.2 aplicados a los valores de la señal en uno o dos instantes:

- media: $\mu_x[n] = E\{x[n]\}$;
- valor cuadrático medio o potencia instantánea: $E\{x^2[n]\}$, y varianza $\sigma_x^2[n] = E\{x^2[n]\} - \mu_x^2[n]$;
- autocorrelación: $\phi_{xx}[n_1, n_2] = E\{x[n_1]\, x[n_2]\}$;
- autocovarianza: $\gamma_{xx}[n_1, n_2] = E\{(x[n_1] - \mu_x[n_1])(x[n_2] - \mu_x[n_2])\} = \phi_{xx}[n_1, n_2] - \mu_x[n_1]\mu_x[n_2]$;
- correlación cruzada de dos señales: $\phi_{xy}[n_1, n_2] = E\{x[n_1]\, y[n_2]\}$, y covarianza cruzada $\gamma_{xy}$.

En tiempo continuo se definen igual con $x(t_1)$ y $x(t_2)$. Si la señal es *gaussiana*, es decir, si todas sus distribuciones conjuntas son gaussianas, la media y la autocorrelación la describen por completo.

### Algunos tipos de señales.

- *I.i.d.*: las $x[n]$ son independientes e idénticamente distribuidas. Si además tienen media nula, son *ruido blanco* (el nombre se explica en la sección 6.4): $\phi_{xx}[n_1, n_2] = \sigma^2 \delta[n_1 - n_2]$.
- *De Markov* de primer orden: el futuro solo depende del pasado a través del presente,

  $$f(x[n] \,|\, x[n-1], x[n-2], \dots) = f(x[n] \,|\, x[n-1]),$$

  de modo que la conjunta se factoriza como $f(x[0], \dots, x[n]) = f(x[0]) \prod_{k=1}^{n} f(x[k] \,|\, x[k-1])$, y se muestrea generando cada valor a partir del anterior. Dos ejemplos, con $w[n]$ ruido blanco:
  - el *paseo aleatorio* $x[n] = x[n-1] + w[n]$, en el que $f(x[n]|x[n-1]) = \mathcal{N}(x[n-1], \sigma_w^2)$ si $w$ es gaussiano;
  - la señal *autorregresiva de orden 1*, AR(1): $x[n] = a\, x[n-1] + w[n]$, con $|a| < 1$, que es ruido blanco filtrado por $H(z) = 1/(1 - az^{-1})$.
- En tiempo continuo el papel del retardo lo hace la derivada. El equivalente del AR(1) es la señal que cumple $\frac{\mathrm{d}x}{\mathrm{d}t} = -\alpha x(t) + w(t)$ (proceso de Ornstein-Uhlenbeck): ruido blanco a través de un sistema de primer orden.

### 6.3.2 Estacionariedad

**Definición 6.3.2: Señal estacionaria**

Una señal es *estacionaria en sentido estricto* si sus distribuciones conjuntas no cambian al desplazar el origen de tiempos. Es *estacionaria en sentido amplio* (ESA) si su media es constante y su autocorrelación solo depende de la separación entre los instantes:

$$\mu_x[n] = \mu_x, \qquad \phi_{xx}[n+l,\, n] = \phi_{xx}[l] \quad \text{para todo } n.$$

Dos señales son *conjuntamente* ESA si cada una lo es y $\phi_{xy}[n+l, n]$ solo depende de $l$.

La estacionariedad estricta implica la amplia si los momentos existen; para señales gaussianas las dos son equivalentes. En una señal ESA también son constantes la potencia $E\{x^2[n]\} = \phi_{xx}[0]$ y la varianza. Para señales ESA usaremos, como Oppenheim y Schafer y como la función `xcorr` de Matlab,

$$\phi_{xx}[l] = E\{x[n+l]\, x[n]\}, \qquad \phi_{xy}[l] = E\{x[n+l]\, y[n]\}, \qquad \gamma_{xx}[l] = \phi_{xx}[l] - \mu_x^2,$$

y en tiempo continuo $\phi_{xx}(\tau) = E\{x(t+\tau)\, x(t)\}$.

**Ejemplo 6.3.3: ¿Cuáles son estacionarias?**

(Figura 6.14.)

1. i.i.d. $\mathcal{U}[0,1]$: estacionaria. $\mu_x = \frac{1}{2}$, $\phi_{xx}[0] = E\{x^2\} = \frac{1}{3}$ y $\phi_{xx}[l] = \mu_x^2 = \frac{1}{4}$ para $l \neq 0$, porque valores distintos son independientes.
2. Independientes $\mathcal{N}(0, \sigma^2[n])$ con $\sigma[n] = 0{,}97^n$: no estacionaria, porque la varianza depende de $n$, aunque los valores sean independientes.
3. Paseo aleatorio con $x[0] = 0$: $x[n] = \sum_{k=1}^{n} w[k]$, así que $\mu_x = 0$ pero $\mathrm{Var}\{x[n]\} = n\sigma_w^2$ crece con $n$: no estacionaria.
4. Constante aleatoria $x[n] = A$ con $A \sim \mathcal{U}[0,1]$: estacionaria, con $\mu_x = \frac{1}{2}$ y $\phi_{xx}[l] = E\{A^2\} = \frac{1}{3}$ para todo $l$.

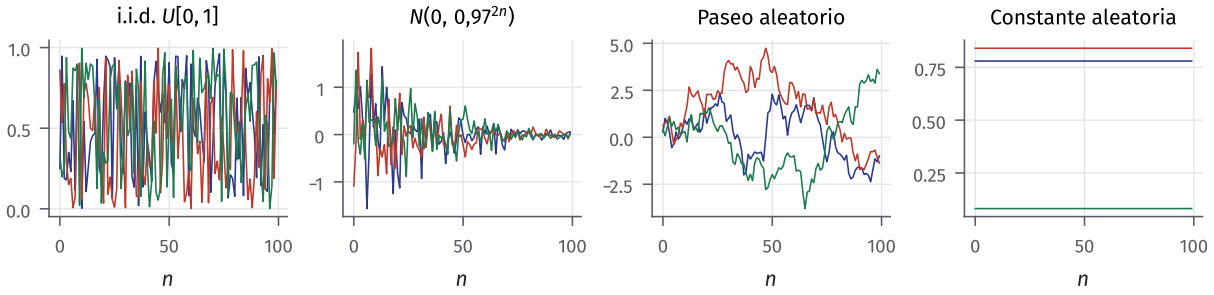



**Figura 6.14.** Tres realizaciones de las cuatro señales del ejemplo 6.3.3.

**Ejemplo 6.3.4: Sinusoide de fase aleatoria**

Sea $x[n] = A\cos(\Omega_0 n + \theta)$ con $A$ y $\Omega_0$ fijos y $\theta \sim \mathcal{U}[0, 2\pi)$. Su media es $\frac{A}{2\pi}\int_0^{2\pi} \cos(\Omega_0 n + \theta)\, d\theta = 0$, y, usando $\cos\alpha\cos\beta = \frac{1}{2}[\cos(\alpha+\beta) + \cos(\alpha-\beta)]$,

$$\phi_{xx}[n+l, n] = \frac{A^2}{2} E\{\cos(\Omega_0(2n+l) + 2\theta)\} + \frac{A^2}{2}\cos(\Omega_0 l) = \frac{A^2}{2}\cos(\Omega_0 l),$$

porque el primer término promedia un coseno sobre una fase uniforme. Es ESA, y su autocorrela-

ción es periódica y no se amortigua (figura 6.15).

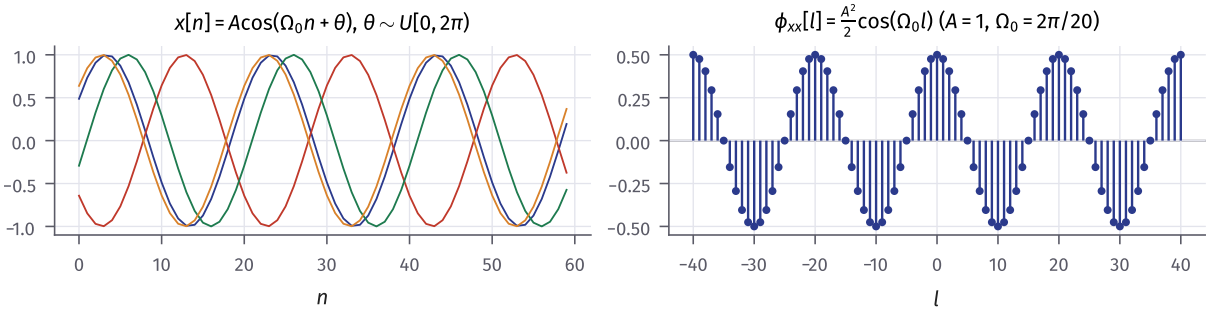



**Figura 6.15.** Realizaciones de una sinusoide de fase aleatoria y su autocorrelación.

### 6.3.3 Ergodicidad

En la práctica casi nunca se dispone de muchas realizaciones, sino de una sola grabación. La pregunta es si los estadísticos, que son promedios sobre el conjunto, se pueden obtener promediando en el tiempo una única realización.

**Definición 6.3.5: Ergodicidad**

Una señal ESA es *ergódica* (en media y en autocorrelación) si, para casi todas sus realizaciones,

$$\lim_{N\to\infty} \frac{1}{2N+1} \sum_{n=-N}^{N} x[n] = \mu_x, \qquad \lim_{N\to\infty} \frac{1}{2N+1} \sum_{n=-N}^{N} x[n+l]\, x[n] = \phi_{xx}[l].$$

En tiempo continuo, igual con $\frac{1}{2T}\int_{-T}^{T}$ y $T \to \infty$.

Intuitivamente, una realización larga recorre todos los comportamientos posibles de la señal. Las señales i.i.d., las AR(1) estacionarias y la sinusoide de fase aleatoria son ergódicas. La constante aleatoria del ejemplo 6.3.3 es estacionaria pero no ergódica: el promedio temporal de una realización es el valor $A$ que le tocó, no $\frac{1}{2}$. Una condición suficiente para la ergodicidad en media es que $\gamma_{xx}[l] \to 0$ cuando $|l| \to \infty$. Salvo que se diga lo contrario, supondremos que las señales estacionarias son ergódicas, y estimaremos sus estadísticos a partir de una realización.

### 6.3.4 Autocorrelación

**Propiedad 6.3.6: Propiedades de la autocorrelación de una señal ESA real**

- $\phi_{xx}[0] = E\{x^2[n]\} = \sigma_x^2 + \mu_x^2$ es la potencia media de la señal.
- Es par: $\phi_{xx}[-l] = E\{x[n-l]\, x[n]\} = \phi_{xx}[l]$ (basta cambiar $n$ por $n+l$).
- Tiene el máximo en el origen: $|\phi_{xx}[l]| \le \phi_{xx}[0]$, por Cauchy-Schwarz.
- Si la señal no tiene componentes periódicas ni constantes aleatorias, los valores muy separados son incorrelados: $\gamma_{xx}[l] \to 0$ y $\phi_{xx}[l] \to \mu_x^2$ cuando $|l| \to \infty$.

La última propiedad sirve para detectar componentes deterministas: si la autocorrelación no tiende a $\mu_x^2$, la señal tiene una parte determinista, y si esta es periódica, $\phi_{xx}$ oscila con el mismo periodo. La

figura 6.16 muestra cuatro casos con $T_s = 0{,}01\,\text{s}$ y 10 s de señal: ruido i.i.d. $\mathcal{N}(5,1)$, con un pico de $\sigma^2 + \mu^2 = 26$ en el origen y $\mu^2 = 25$ fuera; ruido i.i.d. $\mathcal{U}[2,6]$, con $E\{x^2\} = \frac{4+12+36}{3} = 17{,}3$ y $\mu^2 = 16$; un seno, cuya autocorrelación oscila sin amortiguarse, y un AR(1) con $a = 0{,}95$, cuya autocorrelación decae exponencialmente.

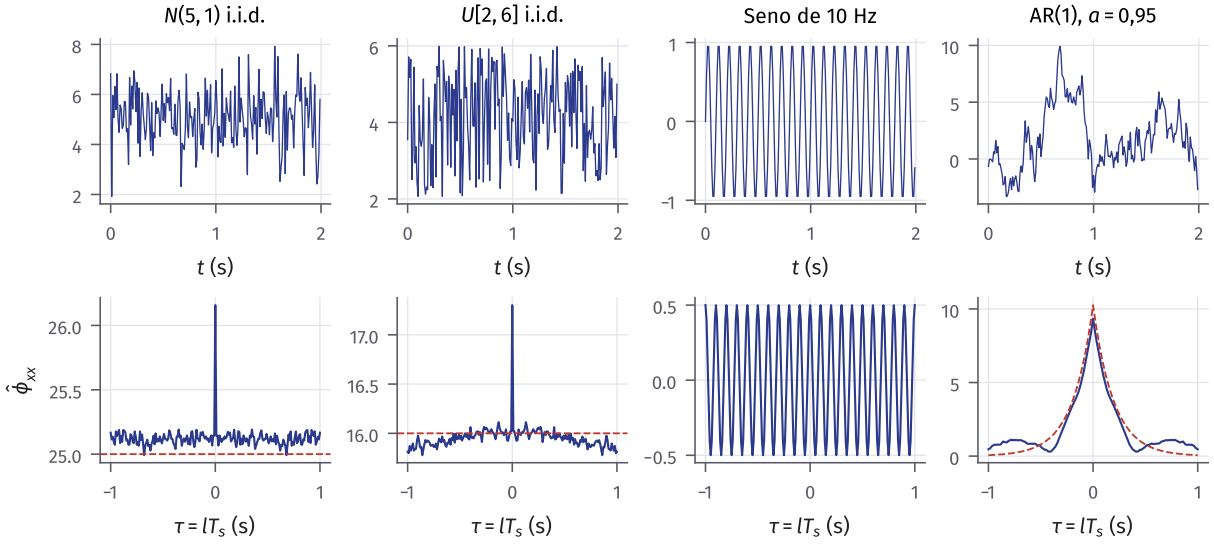



**Figura 6.16.** Cuatro señales y su autocorrelación estimada. En rojo, los valores teóricos.

**Estimación.** Con una realización de $N$ muestras $x[0], \ldots, x[N-1]$ de una señal ergódica, hay $N - |l|$ productos $x[n + |l|]\, x[n]$ disponibles para la separación $l$. Su media es el estimador *insesgado*

$$\hat{\phi}_{xx}[l] = \frac{1}{N - |l|} \sum_{n=0}^{N-1-|l|} x[n + |l|]\, x[n],$$

y si se divide por $N$ se obtiene el estimador *sesgado* $\check{\phi}_{xx}[l] = \frac{N-|l|}{N} \hat{\phi}_{xx}[l]$. El insesgado acierta en media, pero para $|l|$ cercano a $N$ promedia muy pocos productos y es muy ruidoso; el sesgado se amortigua hacia los extremos y tiene menos varianza, y su transformada de Fourier es siempre no negativa (sección 6.4). En cualquier caso, solo tiene sentido para $|l|$ bastante menor que $N$. En Matlab, `xcorr(x, L, 'unbiased')` o `'biased'` calculan los valores para $|l| \leq L$; en Python se usa `np.correlate(x, x, 'full')` y se divide por el número de productos.

**En código**

Autocorrelación estimada de ruido i.i.d. $\mathcal{N}(5,1)$ y $\mathcal{U}[2,6]$ con 1001 muestras: en el origen, unos 26 y 17,3; a 50 muestras, unos 25 y 16.

**Python**

```python
import numpy as np

def xcorr(x, y, L):
    """Media de x[n+l]y[n] (insesgada)."""
    N = len(x)
    c = np.correlate(x, y, "full")
    c = c[N-1-L:N+L]
    l = np.arange(-L, L + 1)
    return l, c/(N - abs(l))

rng = np.random.default_rng(7)
N = 1001
x = 5 + rng.standard_normal(N)  # N(5,1) iid
u = 2 + 4*rng.random(N)         # U[2,6] iid
l, px = xcorr(x, x, 100)
_, pu = xcorr(u, u, 100)
print(px[100], px[150])         # ~ 26, ~ 25
print(pu[100], pu[150])       # ~ 17.3, ~ 16
```

**Matlab**

```matlab
N = 1001;
x = 5 + randn(1, N);            % N(5,1) iid
u = 2 + 4*rand(1, N);           % U[2,6] iid
[px, l] = xcorr(x, 100, 'unbiased');
pu = xcorr(u, 100, 'unbiased');
disp([px(101), px(151)])        % ~ 26, ~ 25
disp([pu(101), pu(151)])      % ~ 17.3, ~ 16
```

**Ejemplo 6.3.7: Autocorrelación de un AR(1)**

Sea $x[n] = a\,x[n-1] + w[n]$, con $|a| < 1$ y $w$ ruido blanco de varianza $\sigma_w^2$, en régimen estacionario. Como $x[n-1]$ solo depende de $w[n-1], w[n-2], \ldots$, es independiente de $w[n]$, y

$$\sigma_x^2 = E\{(ax[n-1] + w[n])^2\} = a^2\sigma_x^2 + \sigma_w^2 \;\Rightarrow\; \sigma_x^2 = \frac{\sigma_w^2}{1-a^2}.$$

Para $l > 0$, $\phi_{xx}[l] = E\{(ax[n+l-1] + w[n+l])\,x[n]\} = a\,\phi_{xx}[l-1]$, y por ser par,

$$\phi_{xx}[l] = \frac{\sigma_w^2}{1-a^2}\,a^{|l|}.$$

Con $a$ cerca de 1 los valores están muy correlados y la señal varía despacio; con $a < 0$ la autocorrelación alterna de signo y la señal cambia de signo con frecuencia.

**En código**

Autocorrelación de un AR(1) con $a = 0{,}9$ comparada con la teórica. La potencia, $1/(1-0{,}81) = 5{,}26$, es cinco veces la del ruido de entrada.

**Python**

```python
import numpy as np
from scipy import signal

rng = np.random.default_rng(8)
a, N = 0.9, 200000
w = rng.standard_normal(N)    # blanco, s2=1
x = signal.lfilter([1], [1, -a], w)
x = x[1000:]                  # sin transitorio
for l in (0, 1, 5, 10):
    est = np.mean(x[l:]*x[:len(x)-l])
    teo = a**l/(1 - a**2)
    print(l, est, teo)
```

**Matlab**

```matlab
a = 0.9; N = 200000;
w = randn(1, N);              % blanco, s2=1
x = filter(1, [1 -a], w);
x = x(1001:end);            % sin transitorio
for l = [0 1 5 10]
  est = mean(x(1+l:end).*x(1:end-l));
  teo = a^l/(1 - a^2);
  disp([l, est, teo])
end
```

### 6.3.5 Correlación cruzada

$\phi_{xy}[l] = E\{x[n+l]\, y[n]\}$ mide cuánto se parece $x$, adelantada $l$ muestras, a $y$. A diferencia de la autocorrelación, no tiene por qué ser par ni tener el máximo en el origen.

**Propiedad 6.3.8: Propiedades de la correlación cruzada**

Para dos señales reales conjuntamente ESA:

- $\phi_{xy}[l] = \phi_{yx}[-l]$;
- $|\phi_{xy}[l]|^2 \le \phi_{xx}[0]\,\phi_{yy}[0]$ y $|\phi_{xy}[l]| \le \frac{1}{2}\big(\phi_{xx}[0] + \phi_{yy}[0]\big)$;
- si $x$ e $y$ son independientes (o incorreladas), $\phi_{xy}[l] = \mu_x \mu_y$ para todo $l$.

**Ejemplo 6.3.9: Eco**

Una señal $x$ blanca de varianza $\sigma_x^2$ se recibe con un retardo $D$, atenuada y con ruido $v$ independiente: $y[n] = \alpha\, x[n-D] + v[n]$. Entonces

$$\phi_{yx}[l] = E\{y[n+l]\, x[n]\} = \alpha\, E\{x[n+l-D]\, x[n]\} + E\{v[n+l]\}E\{x[n]\} = \alpha\sigma_x^2\, \delta[l-D]\ :$$

la correlación cruzada tiene un pico en el retardo. Si $x$ no es blanca, el pico tiene la forma de $\phi_{xx}$ centrada en $D$.

Esta es la base de muchas aplicaciones: medir distancias con radar o sonar (se correla el eco con el pulso emitido), localizar una fuente con dos micrófonos, detectar los caminos de propagación de un canal (un pico por camino) o sincronizar un receptor con una trama conocida. En la figura 6.17 se busca $x(t) = \mathrm{sinc}(t-2)$ dentro de $y(t) = \mathrm{sinc}(t-3)$ más ruido gaussiano de desviación típica 1; aunque el pulso apenas se distingue, la correlación cruzada tiene el máximo en $\tau \approx 1$ s.

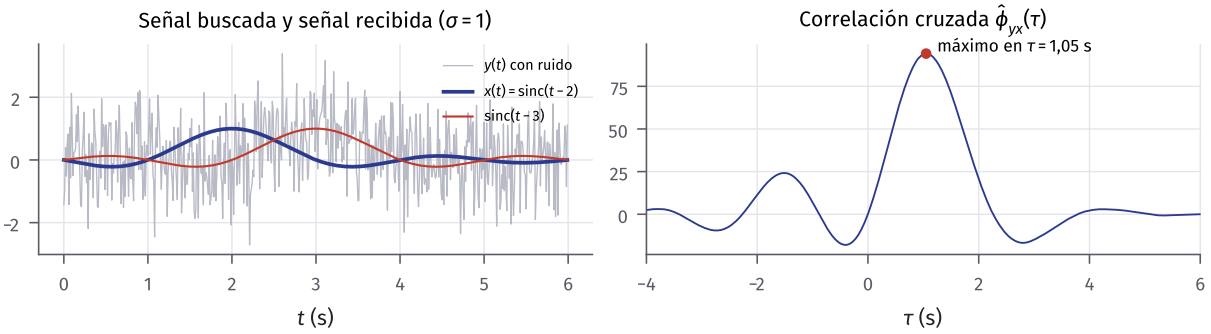



**Figura 6.17.** Estimación de un retardo con la correlación cruzada.

**En código**

Estimación del retardo. `xcorr(y, x)` y `np.correlate(y, x, 'full')` calculan $\sum_n y[n+l]\, x[n]$, es decir, $\phi_{yx}$ salvo el factor de normalización.

**Python**

```python
import numpy as np

rng = np.random.default_rng(9)
Ts = 0.01
t = np.arange(0, 6, Ts)
x = np.sinc(t - 2)                # buscada
y = np.sinc(t - 3) \
    + rng.standard_normal(t.size) # recibida
c = np.correlate(y, x, "full")  # y[n+l]x[n]
lags = np.arange(-len(x) + 1, len(y))
tau = lags[np.argmax(c)]*Ts
print(tau)                        # ~ 1 s
```

**Matlab**

```matlab
Ts = 0.01;
t = 0:Ts:6-Ts;
x = sinc(t - 2);                  % buscada
y = sinc(t - 3) ...
    + randn(size(t));             % recibida
[c, lags] = xcorr(y, x);         % y[n+l]x[n]
[~, k] = max(c);
disp(lags(k)*Ts)                  % ~ 1 s
```

**Convenio de la correlación cruzada**

Las transparencias antiguas, y otros textos, definen $\phi_{xy}(\tau) = E\{x(t)\,y(t+\tau)\}$, que es la función de estos apuntes reflejada, $\phi_{xy}(-\tau)$, o lo que es lo mismo, $\phi_{yx}(\tau)$. Con aquel convenio la relación de un sistema LTI se escribe $\Phi_{xy} = H\Phi_{xx}$; con este, $\Phi_{yx} = H\Phi_{xx}$. Seguimos el de Oppenheim y Schafer y Matlab.

Ejercicios 11–14.

## 6.4 Análisis espectral de señales aleatorias

### 6.4.1 Densidad espectral de potencia

Una realización de una señal estacionaria no decae: tiene energía infinita, y su DTFT no existe en sentido ordinario. Además, cada realización tendría una transformada distinta. Lo que sí está bien definido es la potencia media,

$$P_x = E\Big\{\lim_{N\to\infty}\frac{1}{2N+1}\sum_{n=-N}^{N}x^2[n]\Big\} = \lim_{N\to\infty}\frac{1}{2N+1}\sum_{n=-N}^{N}E\{x^2[n]\} = \phi_{xx}[0] \quad \text{(señal ESA)},$$

y, más en general, la autocorrelación, que es como una potencia que depende de la separación $l$. Su transformada de Fourier describe cómo se reparte la potencia en frecuencia.

**Definición 6.4.1: Densidad espectral de potencia**

La *densidad espectral de potencia* (DEP), o espectro de potencia, de una señal ESA es la DTFT de su autocorrelación:

$$\Phi_{xx}(e^{j\Omega}) = \sum_{l=-\infty}^{\infty}\phi_{xx}[l]\,e^{-j\Omega l}, \qquad \phi_{xx}[l] = \frac{1}{2\pi}\int_{-\pi}^{\pi}\Phi_{xx}(e^{j\Omega})\,e^{j\Omega l}\,d\Omega.$$

En tiempo continuo, $\Phi_{xx}(j\omega) = \int_{-\infty}^{\infty}\phi_{xx}(\tau)\,e^{-j\omega\tau}\,d\tau$.

**Propiedad 6.4.2: Propiedades de la DEP**

- La potencia es el área: $P_x = \phi_{xx}[0] = \dfrac{1}{2\pi}\displaystyle\int_{-\pi}^{\pi} \Phi_{xx}(\mathrm{e}^{j\Omega})\,\mathrm{d}\Omega$.
- Es real, porque $\phi_{xx}$ es par; par en $\Omega$ si $x$ es real; y no negativa.
- Teorema de Wiener-Khinchin: si $X_N(\mathrm{e}^{j\Omega})$ es la DTFT de $2N+1$ muestras de una realización,

$$\Phi_{xx}(\mathrm{e}^{j\Omega}) = \lim_{N\to\infty} \frac{1}{2N+1} E\{|X_N(\mathrm{e}^{j\Omega})|^2\}.$$

El tercer punto justifica el nombre: $\Phi_{xx}$ es el cuadrado del módulo del espectro, por unidad de tiempo y promediado sobre las realizaciones. La potencia contenida en la banda $\Omega_1 \le |\Omega| \le \Omega_2$ es $\frac{1}{2\pi}$ por la integral de $\Phi_{xx}$ en esa banda, contando las frecuencias positivas y negativas. En tiempo continuo, $P_x = \frac{1}{2\pi}\int \Phi_{xx}(j\omega)\,\mathrm{d}\omega = \int \Phi_{xx}(j2\pi f)\,\mathrm{d}f$, de modo que $\Phi_{xx}$ se mide en W/Hz si $x$ es una tensión sobre 1 Ω.

**Ejemplo 6.4.3: Densidades espectrales**

- *Ruido blanco* de varianza $\sigma^2$: $\phi_{xx}[l] = \sigma^2\delta[l]$ y $\Phi_{xx}(\mathrm{e}^{j\Omega}) = \sigma^2$. Todas las frecuencias tienen la misma potencia, como la luz blanca contiene todos los colores; de ahí el nombre.
- *AR(1)*: de $\phi_{xx}[l] = \frac{\sigma_w^2}{1-a^2}a^{|l|}$,

$$\Phi_{xx}(\mathrm{e}^{j\Omega}) = \frac{\sigma_w^2}{1-a^2}\left(\frac{1}{1-a\mathrm{e}^{-j\Omega}} + \frac{1}{1-a\mathrm{e}^{j\Omega}} - 1\right) = \frac{\sigma_w^2}{|1-a\mathrm{e}^{-j\Omega}|^2} = \frac{\sigma_w^2}{1-2a\cos\Omega+a^2}.$$

  Paso bajo si $a > 0$ y paso alto si $a < 0$ (figura 6.18).
- *Constante* $\mu$ (o componente continua): $\phi_{xx}[l] = \mu^2$ y $\Phi_{xx} = 2\pi\mu^2\delta(\Omega)$ en $[-\pi,\pi]$, repetida con periodo $2\pi$.
- *Sinusoide de fase aleatoria*: $\phi_{xx}[l] = \frac{A^2}{2}\cos\Omega_0 l$ y $\Phi_{xx} = \frac{\pi A^2}{2}\big[\delta(\Omega-\Omega_0)+\delta(\Omega+\Omega_0)\big]$: toda la potencia, $A^2/2$, en $\pm\Omega_0$.

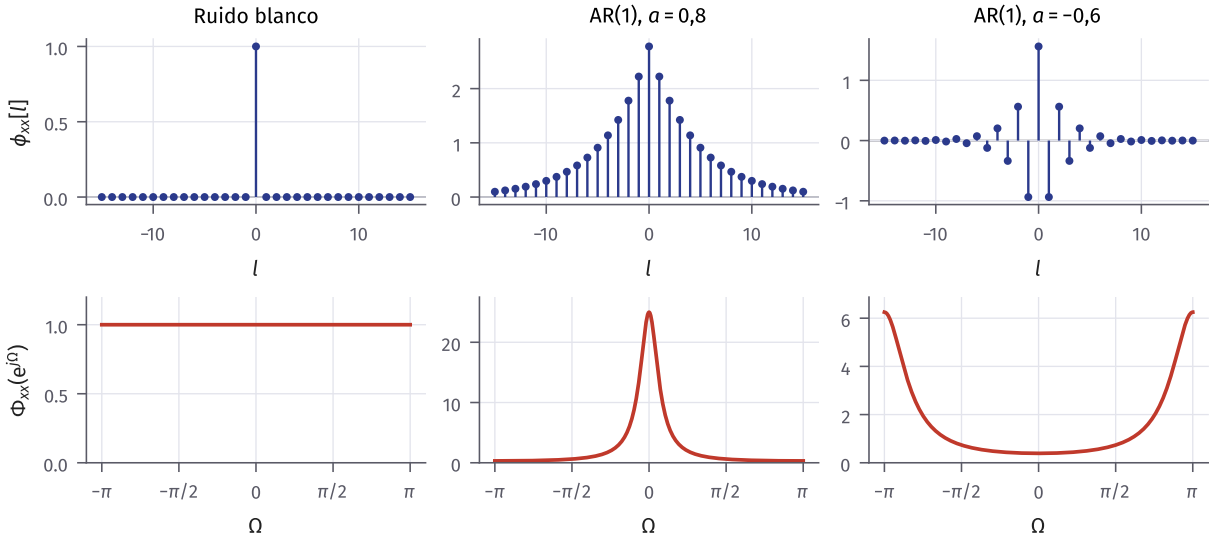



**Figura 6.18.** Autocorrelación y densidad espectral de potencia de ruido blanco y de dos AR(1), con $\sigma_w^2 = 1$.

Según la forma de su DEP, las señales se clasifican en *blancas* (DEP plana), *coloreadas* (cualquier otra),

de *banda ancha* o de *banda estrecha*, *paso bajo* o *paso banda*. Como en cualquier par transformado, la anchura de la autocorrelación es inversa al ancho de banda: una señal de banda estrecha tiene valores correlados a gran distancia.

**Ruido blanco en tiempo continuo**

El ruido blanco continuo tiene $\phi_{xx}(\tau) = \frac{N_0}{2}\delta(\tau)$ y $\Phi_{xx}(j\omega) = \frac{N_0}{2}$. Su potencia, $\phi_{xx}(0)$, es infinita, así que no existe físicamente: es un modelo de un ruido cuya DEP es plana en toda la banda que interesa. El ruido térmico de una resistencia $R$ a temperatura $T$ tiene $N_0/2 = 2kTR\,\mathrm{V}^2/\mathrm{Hz}$ hasta frecuencias de teraherzios. Tras un paso bajo ideal de $B\,\mathrm{Hz}$ la potencia es $\frac{N_0}{2}\cdot 2B = N_0 B$, y si esa señal se muestrea a $f_s = 2B$ resulta ruido blanco discreto de varianza $N_0 B$.

**Densidad espectral cruzada.** Es la DTFT de la correlación cruzada:

$$\Phi_{xy}(\mathrm{e}^{j\Omega}) = \sum_{l=-\infty}^{\infty} \phi_{xy}[l]\,\mathrm{e}^{-j\Omega l}.$$

En general es compleja, porque $\phi_{xy}$ no es par; como $\phi_{yx}[l] = \phi_{xy}[-l]$, para señales reales $\Phi_{yx} = \Phi_{xy}^*$. Su módulo indica en qué frecuencias tienen $x$ e $y$ potencia en común, y su fase, el desfase entre ellas. La *coherencia*

$$C_{xy}(\mathrm{e}^{j\Omega}) = \frac{|\Phi_{xy}(\mathrm{e}^{j\Omega})|^2}{\Phi_{xx}(\mathrm{e}^{j\Omega})\,\Phi_{yy}(\mathrm{e}^{j\Omega})} \in [0,1]$$

vale 1 en las frecuencias en las que $y$ es una versión filtrada de $x$ sin ruido y 0 donde no tienen nada en común.

### 6.4.2 Respuesta de un sistema LTI

Sea $y[n] = \sum_k h[k]\,x[n-k]$ la salida de un sistema LTI estable, con $h$ real y determinista, a una entrada $x$ ESA. Toda la aleatoriedad está en la señal. Por la linealidad de la esperanza:

- *Media*:

$$\mu_y = E\Big\{\sum_k h[k]\,x[n-k]\Big\} = \sum_k h[k]\,E\{x[n-k]\} = \mu_x \sum_k h[k] = H(\mathrm{e}^{j0})\,\mu_x.$$

- *Correlación cruzada salida-entrada*:

$$\phi_{yx}[l] = E\{y[n+l]\,x[n]\} = \sum_k h[k]\,E\{x[n+l-k]\,x[n]\} = \sum_k h[k]\,\phi_{xx}[l-k] = (h * \phi_{xx})[l].$$

- *Autocorrelación de la salida*:

$$\phi_{yy}[l] = E\Big\{\sum_k h[k]x[n+l-k]\sum_r h[r]x[n-r]\Big\} = \sum_k\sum_r h[k]\,h[r]\,\phi_{xx}[l-k+r].$$

  Con $c[m] = \sum_r h[r]\,h[r+m] = (h * \tilde{h})[m]$, $\tilde{h}[n] = h[-n]$, y el cambio $m = k - r$, queda $\phi_{yy}[l] = \sum_m c[m]\,\phi_{xx}[l-m] = (h * \tilde{h} * \phi_{xx})[l]$.

Ninguno depende de $n$: la salida es ESA, y conjuntamente ESA con la entrada. En frecuencia las convoluciones se convierten en productos, y como la DTFT de $\tilde{h}$ es $H(\mathrm{e}^{-j\Omega}) = H^*(\mathrm{e}^{j\Omega})$:

**Propiedad 6.4.4: Señal aleatoria a través de un sistema LTI**

$$\mu_y = H(e^{j0})\,\mu_x, \qquad \Phi_{yx}(e^{j\Omega}) = H(e^{j\Omega})\,\Phi_{xx}(e^{j\Omega}), \qquad \Phi_{yy}(e^{j\Omega}) = |H(e^{j\Omega})|^2\,\Phi_{xx}(e^{j\Omega}).$$

La potencia de salida es $P_y = \frac{1}{2\pi}\int_{-\pi}^{\pi}|H(e^{j\Omega})|^2\,\Phi_{xx}(e^{j\Omega})\,d\Omega$; con entrada blanca de varianza $\sigma_x^2$, $P_y = \sigma_x^2\sum_k h^2[k]$.

En tiempo continuo se cumple lo mismo con $H(j\omega)$. Un filtro no crea potencia en frecuencias donde no la había: solo multiplica la de cada frecuencia por $|H|^2$. Al revés, filtrando ruido blanco con el $H$ adecuado se obtiene ruido con la DEP que se quiera, $\Phi_{yy} = \sigma_x^2|H|^2$ (figura 6.19): es como se simulan perturbaciones realistas, y el AR(1) no es más que ruido blanco tras $1/(1-az^{-1})$. Un filtro *blanqueador* $1/H$, si es estable, convierte ese ruido coloreado otra vez en blanco.

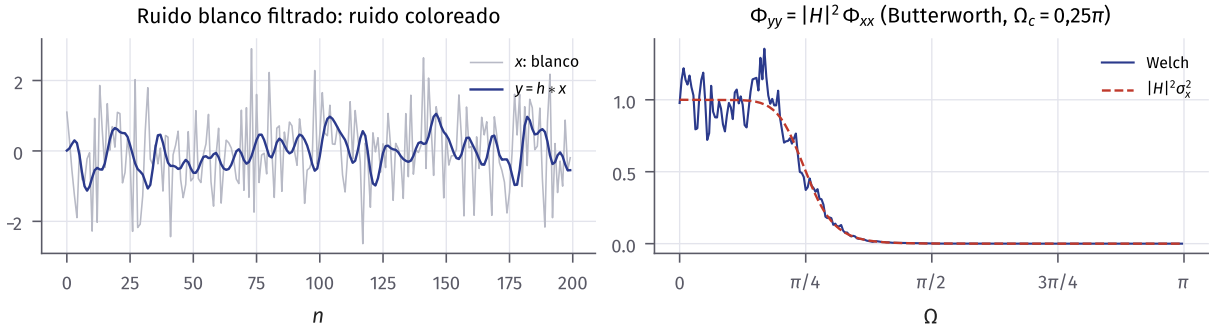



**Figura 6.19.** Ruido blanco a través de un Butterworth discreto de orden 4: realización y DEP de la salida estimada con el método de Welch.

**Identificación de sistemas.** Las relaciones anteriores permiten medir la respuesta en frecuencia de un sistema desconocido: se le aplica una entrada aleatoria y se miden entrada y salida. De $\Phi_{yx} = H\Phi_{xx}$,

$$\hat{H}(e^{j\Omega}) = \frac{\Phi_{yx}(e^{j\Omega})}{\Phi_{xx}(e^{j\Omega})},$$

que da el módulo y la fase. Si la salida se mide con un ruido $v$ incorrelado con la entrada, $\Phi_{yx}$ no cambia, porque $\phi_{vx} = 0$; en cambio $\Phi_{yy}$ aumenta en $\Phi_{vv}$, así que $|H|^2 = \Phi_{yy}/\Phi_{xx}$ daría un resultado sesgado, y además sin fase. Con entrada blanca, $\phi_{yx}[l] = \sigma_x^2 h[l]$: la correlación cruzada es la respuesta al impulso. La figura 6.20 muestra el resultado para un sistema de segundo orden.

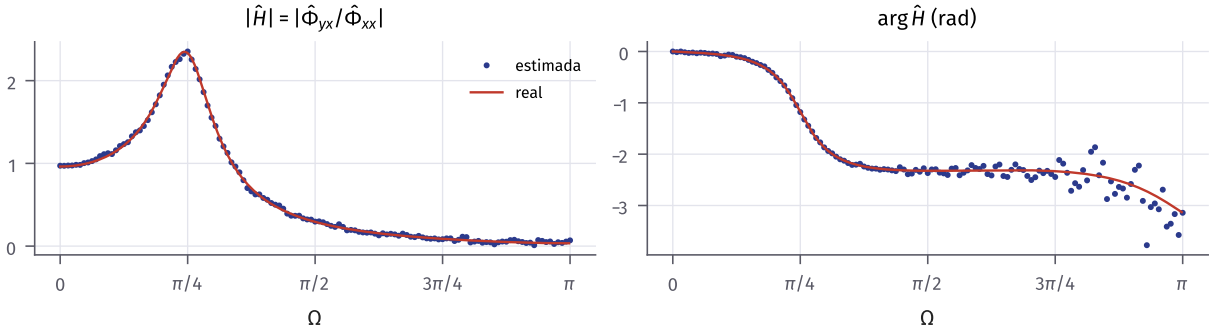



**Figura 6.20.** Respuesta en frecuencia de $H(z) = (0{,}2 + 0{,}3z^{-1})/(1 - 1{,}2z^{-1} + 0{,}72z^{-2})$ estimada con 20 000 muestras de ruido blanco y ruido de medida de desviación típica 0,3.

**En código**

Identificación con `tfestimate` (Matlab) y con `csd`/`welch` (Python). `scipy.signal.csd(x, y)` promedia $X^*Y$, que con nuestro convenio estima $\Phi_{yx}$.

**Python**

```python
import numpy as np
from scipy import signal

rng = np.random.default_rng(11)
b, a = [0.2, 0.3], [1, -1.2, 0.72]
x = rng.standard_normal(50000)   # blanco
y = signal.lfilter(b, a, x)      # sistema
y += 0.3*rng.standard_normal(x.size)
f, Pxx = signal.welch(x, nperseg=256,
                      detrend=False)
_, Pxy = signal.csd(x, y, nperseg=256,
                    detrend=False)
H = Pxy/Pxx                  # Phi_yx/Phi_xx
_, Hr = signal.freqz(b, a, 2*np.pi*f)
k = [0, 32, 64]              # W=0,pi/4,pi/2
print(abs(H[k]), abs(Hr[k]))
```

**Matlab**

```matlab
b = [0.2 0.3]; a = [1 -1.2 0.72];
x = randn(1, 50000);              % blanco
y = filter(b, a, x);              % sistema
y = y + 0.3*randn(size(x));
w = hanning(256);
[H, f] = tfestimate(x, y, w, [], 256, 1);
Hr = freqz(b, a, 2*pi*f);
k = [1 33 65];                % W=0,pi/4,pi/2
disp([abs(H(k)), abs(Hr(k))])
```

### 6.4.3 Estimación de la densidad espectral

**Periodograma.** Con $N$ muestras de una realización, la estimación inmediata, sugerida por el teorema de Wiener-Khinchin, es el *periodograma*

$$I_N(e^{j\Omega}) = \frac{1}{N}\left|\sum_{n=0}^{N-1} x[n]\, e^{-j\Omega n}\right|^2,$$

que se calcula en las frecuencias $\Omega_k = 2\pi k/N$ con la FFT. Se puede comprobar que es la DTFT del estimador sesgado de la autocorrelación. Es asintóticamente insesgado: su media tiende a $\Phi_{xx}$ cuando $N$ crece. Pero su varianza no disminuye con $N$, sino que es del orden de $\Phi_{xx}^2$: con más muestras se obtienen más frecuencias, cada una tan ruidosa como antes (figura 6.21). Además, al truncar la señal, su media es la convolución de $\Phi_{xx}$ con el espectro de la ventana rectangular, con las fugas del Tema 3.

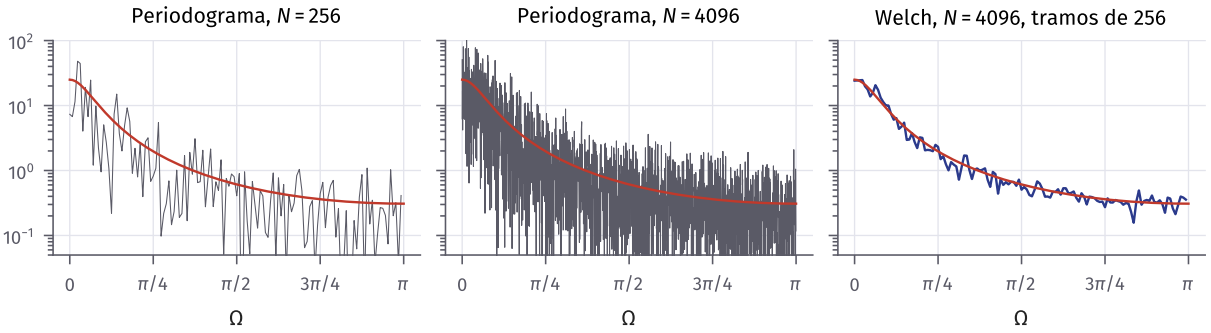



**Figura 6.21.** Periodogramas de un AR(1) con $a = 0{,}8$ y $N = 256$ y 4096 muestras, y estimación de Welch. En rojo, la DEP teórica.

**Métodos de Bartlett y Welch.** Para reducir la varianza se promedia. El método de *Bartlett* divide la señal en $K$ tramos de $L$ muestras, calcula el periodograma de cada uno y los promedia; si los tramos son aproximadamente independientes, la varianza baja en un factor $K$, a cambio de una resolución en frecuencia

de $\approx 2\pi/L$ en lugar de $2\pi/N$. El de *Welch* enventana cada tramo (Hann, Hamming), para reducir las fugas, y los solapa, típicamente un 50 %, para aprovechar mejor los datos. Es el mismo compromiso entre resolución y estabilidad que el del espectrograma del Tema 5. Las densidades cruzadas se estiman igual promediando $X_m^*(e^{j\Omega})Y_m(e^{j\Omega})$ de cada tramo.

En Matlab: `periodogram`, `pwelch`, `cpsd`, `tfestimate` y `mscohere`; en Python, las funciones de mismo nombre de `scipy.signal`: `periodogram`, `welch`, `csd` y `coherence`. Por defecto devuelven la DEP *unilateral* (el doble de $\Phi_{xx}$ entre 0 y $f_s/2$, para que la integral en ese intervalo dé la potencia) y por unidad de frecuencia: por hercio si se da $f_s$, y en Matlab por rad/muestra si no se da. Con la opción bilateral (`'twosided'`, `return_onesided=False`) y $f_s = 1$, el resultado es directamente $\Phi_{xx}(e^{j2\pi f})$ con $f$ en ciclos por muestra. Además, `scipy.signal.welch` resta por defecto la media de cada tramo (`detrend='constant'`), que elimina la componente continua; con `detrend=False` se conserva. El solapamiento se indica en muestras en Matlab y como fracción en Octave; dejándolo vacío (`[]`) ambos usan el 50 %.

**En código**

DEP de un AR(1) con $a = 0{,}8$ estimada con Welch y comparada con la teórica: $\Phi_{xx}(e^{j0}) = 1/(1-a)^2 = 25$, $\Phi_{xx}(e^{j\pi/2}) = 1/(1+a^2) = 0{,}61$ y potencia $1/(1-a^2) = 2{,}78$. El valor en $\Omega = 0$, donde hay un pico estrecho, es el que peor se estima.

**Python**

```python
import numpy as np
from scipy import signal

rng = np.random.default_rng(10)
a, N = 0.8, 2**14
x = signal.lfilter([1], [1, -a],
                   rng.standard_normal(N))
f, P = signal.welch(x, fs=1, nperseg=256,
                    return_onesided=False,
                    detrend=False)  # Phi
W = 2*np.pi*f                   # f en ciclos
teo = 1/abs(1 - a*np.exp(-1j*W))**2
print(P[0], teo[0])             # ~ 25
print(P[64], teo[64])         # W=pi/2: 0.61
print(np.mean(P), x.var())      # potencia
```

**Matlab**

```matlab
a = 0.8; N = 2^14;
x = filter(1, [1 -a], randn(1, N));
w = hanning(256);
[P, f] = pwelch(x, w, [], 256, 1, ...
                'twosided');     % Phi
W = 2*pi*f;                     % f en ciclos
teo = 1./abs(1 - a*exp(-1j*W)).^2;
disp([P(1), teo(1)])            % ~ 25
disp([P(65), teo(65)])        % W=pi/2: 0.61
disp([mean(P), var(x)])         % potencia
```

### 6.4.4 Aplicaciones

- *Medida de ruido*: la DEP del ruido de un sistema y su relación señal a ruido en cada banda.
- *Detección de tonos* débiles: al promediar, la varianza del fondo de ruido disminuye y un tono que estaba oculto en un periodograma aislado destaca.
- *Identificación* de sistemas en funcionamiento, añadiendo una pequeña señal aleatoria a la entrada.
- *Filtro de Wiener*: si una señal $s$ se observa con un ruido aditivo $v$ incorrelado, $x = s + v$, el filtro lineal que minimiza el error cuadrático medio de la estimación de $s$ es

$$H(e^{j\Omega}) = \frac{\Phi_{ss}(e^{j\Omega})}{\Phi_{ss}(e^{j\Omega}) + \Phi_{vv}(e^{j\Omega})},$$

que deja pasar las frecuencias en las que domina la señal y atenúa aquellas en las que domina el ruido. Es la idea de los reductores de ruido y de la restauración de imágenes del Tema 5.

Ejercicios 15–18.

## Bibliografía


- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Apéndice A (señales aleatorias) y capítulo 10 (estimación espectral). De aquí procede el convenio de la correlación cruzada.
- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos de señales aleatorias y de estimación del espectro de potencia.
- A. Papoulis y S. U. Pillai, *Probability, Random Variables and Stochastic Processes*, 4.ª ed. McGraw-Hill, 2002. Referencia completa de probabilidad y procesos estocásticos.
- A. Gelb (ed.), *Applied Optimal Estimation*. MIT Press, 1974. Capítulo de repaso de probabilidad y procesos aleatorios, con la notación $\phi$ y $\Phi$ de las versiones anteriores de este tema.
- C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. Capítulos 2 (distribuciones) y 11 (métodos de muestreo: inversa, rechazo, Monte Carlo).


## 6.5 Ejercicios

Los ejercicios 1–5 corresponden a la parte 1 del tema (variables aleatorias y muestreo), los ejercicios 6–10 a la parte 2 (varias variables), los ejercicios 11–14 a la parte 3 (señales aleatorias) y los ejercicios 15–18 a la parte 4 (análisis espectral). Se usa la notación de los apuntes: $\phi_{xx}[l] = E\{x[n+l]x[n]\}$, $\phi_{yx}[l] = E\{y[n+l]x[n]\}$ y $\Phi_{xx}(e^{j\Omega})$ para la densidad espectral de potencia. Los resultados se pueden comprobar con los ejemplos de `teoría/T6/codigo/`.

### Variables aleatorias

**Ejercicio 1** (un dado cargado) Un dado está cargado de forma que el seis sale la mitad de las veces y las otras caras son equiprobables.

a) Escribe su función de probabilidad $p_X(x)$ y dibuja su función de distribución $F_X(x)$.

b) Calcula la media, el valor cuadrático medio y la varianza. Compáralos con los de un dado equilibrado.

c) ¿Cuánto vale $P(X \geq 5)$?

d) Con el método de la ruleta, ¿qué valor se obtiene para $u = 0{,}37$? ¿Y para $u = 0{,}62$?

**Solución**

a) $p_X(x) = 0{,}1$ para $x = 1, \ldots, 5$ y $p_X(6) = 0{,}5$. $F_X$ es una escalera con saltos de 0,1 en 1, 2, 3, 4 y 5 y un salto de 0,5 en 6: vale 0,1, 0,2, 0,3, 0,4, 0,5 y 1.

b) $\mu_X = 0{,}1(1+2+3+4+5) + 0{,}5 \cdot 6 = 4{,}5$; $E\{X^2\} = 0{,}1(1+4+9+16+25) + 0{,}5 \cdot 36 = 23{,}5$; $\sigma_X^2 = 23{,}5 - 4{,}5^2 = 3{,}25$. El dado equilibrado tiene $\mu = 3{,}5$, $E\{X^2\} = 91/6 \approx 15{,}2$ y $\sigma^2 =$

$35/12 \approx 2{,}92$.

c) $P(X \geq 5) = 0{,}1 + 0{,}5 = 0{,}6$.

d) Se busca el tramo $F_X(x_{i-1}) \leq u < F_X(x_i)$. Para $u = 0{,}37$, $0{,}3 \leq 0{,}37 < 0{,}4$ y sale un 4. Para $u = 0{,}62$, $0{,}5 \leq 0{,}62 < 1$ y sale un 6.

**Ejercicio 2** (variable uniforme)

a) Demuestra que si $X \sim \mathcal{U}[a, b]$, entonces $\mu_X = \frac{a+b}{2}$, $E\{X^2\} = \frac{a^2+ab+b^2}{3}$ y $\sigma_X^2 = \frac{(b-a)^2}{12}$.

b) Para $X \sim \mathcal{U}[2, 6]$, calcula $\mu_X$, $E\{X^2\}$, $\sigma_X^2$ y $P(X > 5)$.

c) El error de cuantificación de un conversor con escalón $\Delta$ se modela como $\mathcal{U}[-\Delta/2, \Delta/2]$. ¿Qué potencia tiene?

**Solución**

a) Integrando,

$$\mu_X = \frac{1}{b-a}\int_a^b x\,\mathrm{d}x = \frac{b^2-a^2}{2(b-a)} = \frac{a+b}{2}, \qquad E\{X^2\} = \frac{b^3-a^3}{3(b-a)} = \frac{a^2+ab+b^2}{3},$$

y restando, $\sigma_X^2 = \frac{4a^2+4ab+4b^2-3a^2-6ab-3b^2}{12} = \frac{(b-a)^2}{12}$.

b) $\mu_X = 4$, $\sigma_X^2 = 16/12 = 4/3$, $E\{X^2\} = 4/3 + 16 = 52/3 \approx 17{,}3$ y $P(X > 5) = 1/4$.

c) Media nula, así que la potencia es la varianza: $\Delta^2/12$, el resultado del Tema 2.

**Ejercicio 3** (variable gaussiana) Sea $X \sim \mathcal{N}(5, 4)$ (media 5 y varianza 4). Usa $\Phi(1) = 0{,}8413$ y $\Phi(2) = 0{,}9772$.

a) Calcula $P(3 < X < 9)$.

b) ¿Qué distribución tiene $Y = 3X - 2$?

c) ¿Entre qué valores cae $X$ el 95,4 % de las veces?

**Solución**

a) Con $\sigma = 2$, $Z = (X-5)/2 \sim \mathcal{N}(0, 1)$ y $P(3 < X < 9) = P(-1 < Z < 2) = \Phi(2) - \Phi(-1) = 0{,}9772 - (1 - 0{,}8413) = 0{,}8186$.

b) Una transformación lineal de una gaussiana es gaussiana: $Y \sim \mathcal{N}(3\cdot 5 - 2,\ 9\cdot 4) = \mathcal{N}(13, 36)$.

c) En $\mu \pm 2\sigma$, es decir, entre 1 y 9.

**Ejercicio 4** (generar muestras)

a) La distribución de Rayleigh tiene $F_X(x) = 1 - \mathrm{e}^{-x^2/(2\sigma^2)}$ para $x \geq 0$. Obtén la fórmula para generarla con el método de la inversa a partir de $U \sim \mathcal{U}[0, 1]$.

b) Se quiere muestrear $f_X(x) = 6x(1-x)$ en $[0, 1]$ por aceptación y rechazo con una propuesta uniforme. ¿Cuál es la menor constante $M$ válida? ¿Qué fracción de candidatos se acepta?

c) ¿Cuántos números aleatorios uniformes se gastan, en media, por cada muestra aceptada?

**Solución**

a) Se resuelve $u = 1 - \mathrm{e}^{-x^2/(2\sigma^2)}$: $x = \sigma\sqrt{-2\ln(1-u)}$. Como $1-U$ también es uniforme, vale igual $x = \sigma\sqrt{-2\ln u}$, que es el radio de Box–Muller.

b) Hace falta $Mq(x) \geq f_X(x)$ con $q = 1$. El máximo de $6x(1-x)$ está en $x = 1/2$ y vale 1,5, así que $M = 1{,}5$. Se acepta una fracción $1/M = 2/3$.

c) Cada candidato gasta dos uniformes (uno para $x$ y otro para decidir), y hacen falta en media $M = 1{,}5$ candidatos por muestra: tres uniformes por muestra aceptada.

**Ejercicio 5** (estimadores y Monte Carlo)

a) Con $N$ muestras independientes de una variable de desviación típica $\sigma = 1$, ¿cuántas hacen falta para que la media estimada esté a menos de $\pm 0{,}01$ de la verdadera con un 95 % de probabilidad?

b) ¿Por qué el estimador insesgado de la varianza divide entre $N-1$? ¿Qué dividen `var` de Matlab y `np.var` de Python?

c) Un histograma de 10 000 muestras con intervalos de anchura $\Delta = 0{,}5$ tiene 1 200 muestras en un intervalo. ¿Qué altura tiene la barra normalizada como densidad?

**Solución**

a) La media estimada tiene desviación típica $\sigma/\sqrt{N}$ y, por el teorema del límite central, es aproximadamente gaussiana. El 95 % cae en $\pm 1{,}96\sigma/\sqrt{N}$, así que $1{,}96/\sqrt{N} \leq 0{,}01$ y $N \geq 38\,416$: unas 40 000 muestras. Cada cifra decimal más multiplica $N$ por cien.

b) Las desviaciones se miden respecto a $\hat{\mu}$, que está más cerca de las muestras que la media verdadera, y la suma de cuadrados sale en media $(N-1)\sigma^2$. `var(x)` divide entre $N-1$; `np.var(x)` divide entre $N$ salvo que se pida `ddof=1`.

c) $\hat{f} = \dfrac{n_j}{N\Delta} = \dfrac{1200}{10000 \cdot 0{,}5} = 0{,}24$.

## Varias variables aleatorias

**Ejercicio 6** (una conjunta discreta) Las variables $X \in \{0, 1\}$ e $Y \in \{0, 1, 2\}$ tienen la probabilidad conjunta del cuadro 6.1.

| | $Y=0$ | $Y=1$ | $Y=2$ |
|---|---|---|---|
| $X=0$ | 0,1 | 0,2 | 0,1 |
| $X=1$ | 0,2 | 0,1 | 0,3 |

**Cuadro 6.1.** Probabilidad conjunta $p_{XY}(x, y)$.

a) Calcula las marginales. ¿Son independientes?

b) Calcula $p_{Y|X}(y \mid 1)$.

c) Calcula $E\{X\}$, $E\{Y\}$, $R_{XY}$, $C_{XY}$ y $\rho_{XY}$.

**Solución**

a) Sumando filas y columnas, $p_X = (0{,}4;\ 0{,}6)$ y $p_Y = (0{,}3;\ 0{,}3;\ 0{,}4)$. No son independientes: $p_{XY}(0,0) = 0{,}1 \neq p_X(0)p_Y(0) = 0{,}12$.

b) Se divide la segunda fila entre $p_X(1) = 0{,}6$: $p_{Y|X}(\cdot\,|\,1) = (1/3;\ 1/6;\ 1/2)$.

c) $E\{X\} = 0{,}6$; $E\{Y\} = 0{,}3 + 2\cdot 0{,}4 = 1{,}1$; $R_{XY} = E\{XY\} = 1\cdot 0{,}1 + 2\cdot 0{,}3 = 0{,}7$; $C_{XY} = 0{,}7 - 0{,}6\cdot 1{,}1 = 0{,}04$. Con $\sigma_X^2 = 0{,}6 - 0{,}36 = 0{,}24$ y $\sigma_Y^2 = 1{,}9 - 1{,}21 = 0{,}69$, $\rho_{XY} = 0{,}04/\sqrt{0{,}24\cdot 0{,}69} = 0{,}098$: casi incorreladas, aunque dependientes.

**Ejercicio 7** (una conjunta construida por partes) Sea $X_1 \sim \mathcal{U}[2,4]$ y, dado $X_1 = x_1$, $X_2 \sim \mathcal{N}(x_1, 1)$ (el ejemplo de los apuntes).

a) Escribe $X_2 = X_1 + Z$. ¿Qué distribución tiene $Z$ y es independiente de $X_1$?

b) Calcula $\mu_2$, $\sigma_2^2$, $C_{12}$ y $\rho_{12}$.

c) ¿Es $X_2$ gaussiana?

**Solución**

a) $Z = X_2 - X_1 \sim \mathcal{N}(0,1)$ para cualquier valor de $X_1$; como su distribución condicionada no depende de $x_1$, $Z$ es independiente de $X_1$.

b) $\mu_2 = \mu_1 + 0 = 3$. Por independencia, $\sigma_2^2 = \sigma_1^2 + 1 = \frac{4}{12} + 1 = \frac{4}{3}$. $C_{12} = E\{(X_1 - 3)(X_1 - 3 + Z)\} = \sigma_1^2 = \frac{1}{3}$. $\rho_{12} = \frac{1/3}{\sqrt{(1/3)(4/3)}} = \frac{1}{2}$.

c) No: su densidad es la convolución de la uniforme con la gaussiana, una gaussiana «ensanchada» con la parte central plana.

**Ejercicio 8** (incorreladas pero dependientes) Sea $X \sim \mathcal{U}[-1,1]$ e $Y = X^2$.

a) Calcula $C_{XY}$.

b) ¿Son independientes? Razónalo con $P(Y > 1/4\,|\,X = 0)$.

c) ¿Puede pasar lo mismo con dos variables conjuntamente gaussianas?

**Solución**

a) $C_{XY} = E\{X^3\} - E\{X\}E\{X^2\} = 0 - 0 = 0$, porque la densidad de $X$ es par.

b) No: $P(Y > 1/4) = P(|X| > 1/2) = 1/2$, pero $P(Y > 1/4\,|\,X = 0) = 0$. Conocer $X$ fija $Y$ por completo. La correlación solo mide dependencia lineal.

c) No. Si son conjuntamente gaussianas e incorreladas, $\Sigma$ es diagonal, la densidad conjunta se factoriza y son independientes.

**Ejercicio 9** (sumas y teorema del límite central)

a) Calcula la densidad de $S = X_1 + X_2$ con $X_1, X_2 \sim \mathcal{U}[0,1]$ independientes.

b) Se suman 12 uniformes $\mathcal{U}[0,1]$ independientes y se resta 6. ¿Qué media y varianza tiene el resultado? ¿Por qué es un generador aproximado de $\mathcal{N}(0,1)$?

c) ¿Qué le falta a esa aproximación en las colas?

**Solución**

a) La convolución de dos pulsos de anchura 1 es un triángulo: $f_S(s) = s$ en $[0,1]$, $2-s$ en $[1,2]$ y cero fuera.

b) Media $12 \cdot \frac{1}{2} - 6 = 0$ y varianza $12 \cdot \frac{1}{12} = 1$. Por el teorema del límite central, la suma de 12 variables independientes es ya casi gaussiana (figura de los apuntes).

c) El resultado está acotado en $[-6,6]$, así que $P(|Z| > 6) = 0$, mientras que la gaussiana da unos $2 \cdot 10^{-9}$. Para simular sucesos muy raros no sirve.

**Ejercicio 10** (gaussiana bidimensional) Un vector gaussiano tiene media nula y covarianza $\Sigma = \begin{pmatrix} 2 & 1{,}2 \\ 1{,}2 & 1 \end{pmatrix}$.

a) Calcula el coeficiente de correlación.

b) Calcula los autovalores de $\Sigma$. ¿Qué forma tienen las curvas de nivel?

c) Calcula el factor de Cholesky $\mathbf{L}$ y explica cómo generar muestras a partir de dos $\mathcal{N}(0,1)$ independientes.

**Solución**

a) $\rho = 1{,}2/\sqrt{2 \cdot 1} = 0{,}849$.

b) $\lambda^2 - 3\lambda + 0{,}56 = 0$, de donde $\lambda = 2{,}8$ y $\lambda = 0{,}2$. Las curvas de nivel son elipses con los ejes en la dirección de los autovectores y semiejes proporcionales a $\sqrt{2{,}8} = 1{,}67$ y $\sqrt{0{,}2} = 0{,}45$: muy alargadas, inclinadas hacia la diagonal.

c) $L_{11} = \sqrt{2} = 1{,}414$, $L_{21} = 1{,}2/\sqrt{2} = 0{,}849$, $L_{22} = \sqrt{1 - 0{,}72} = 0{,}529$. Con $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $\mathbf{x} = \mathbf{L}\mathbf{z}$ tiene covarianza $\mathbf{L}\mathbf{L}^T = \Sigma$: $x_1 = 1{,}414 z_1$ y $x_2 = 0{,}849 z_1 + 0{,}529 z_2$.

## Señales aleatorias

**Ejercicio 11** (estacionariedad y ergodicidad) Sea $w[n]$ ruido blanco gaussiano de media nula y varianza $\sigma^2$, $A \sim \mathcal{N}(0,1)$ y $\theta \sim \mathcal{U}[0, 2\pi)$, independientes. Para cada señal, di si es estacionaria en sentido amplio y si es ergódica en media:

a) $x[n] = A\cos(\Omega_0 n)$.

b) $x[n] = \cos(\Omega_0 n + \theta)$, $0 < \Omega_0 < \pi$.

c) $x[n] = A$.

d) $x[n] = w[n] + w[n-1]$.

**Solución**

a) Media nula, pero $E\{x[n+l]x[n]\} = \cos(\Omega_0(n+l))\cos(\Omega_0 n)$ depende de $n$: no es ESA. Cada realización es un coseno de amplitud fija.

b) Es ESA: media nula y $\phi_{xx}[l] = \frac{1}{2}\cos(\Omega_0 l)$ (sinusoide de fase aleatoria). Es ergódica en media: el promedio temporal de cualquier coseno es cero.

c) ESA (media 0, $\phi_{xx}[l] = 1$) pero no ergódica: cada realización es una constante distinta y su promedio temporal es $A$, no 0.

d) ESA, con media nula, $\phi_{xx}[0] = 2\sigma^2$, $\phi_{xx}[\pm 1] = \sigma^2$ y cero para $|l| \geq 2$. Es ergódica: muestras separadas más de una posición son independientes.

**Ejercicio 12** (paseo aleatorio) Sea $x[n] = \sum_{k=1}^{n} w[k]$ para $n \geq 1$, con $w[k]$ i.i.d. de media $\mu$ y varianza $\sigma^2$.

a) Calcula la media y la varianza de $x[n]$.

b) Con $\mu = 0$, demuestra que $\phi_{xx}[n_1, n_2] = \sigma^2 \min(n_1, n_2)$.

c) ¿Es estacionaria? ¿Es un proceso de Markov?

**Solución**

a) $E\{x[n]\} = n\mu$ y, por independencia, $\sigma_x^2[n] = n\sigma^2$: la dispersión crece como $\sqrt{n}$.

b) $E\{x[n_1]x[n_2]\} = \sum_{k=1}^{n_1} \sum_{m=1}^{n_2} E\{w[k]w[m]\}$. Solo sobreviven los términos con $k = m$, que hay $\min(n_1, n_2)$, y cada uno vale $\sigma^2$.

c) No es estacionaria: la varianza depende de $n$ y la autocorrelación no depende solo de $n_1 - n_2$. Sí es de Markov: $x[n] = x[n-1] + w[n]$ y $w[n]$ es independiente del pasado.

**Ejercicio 13** (proceso AR(1)) Sea $x[n] = a\,x[n-1] + w[n]$, con $|a| < 1$ y $w[n]$ ruido blanco de media $\mu_w$ y varianza $\sigma_w^2$, en régimen estacionario.

a) Calcula la media de $x[n]$.

b) Con $\mu_w = 0$, calcula la varianza y la autocorrelación $\phi_{xx}[l]$.

c) Para $a = 0{,}9$ y $\sigma_w^2 = 1$, ¿cuánto valen $\phi_{xx}[0]$ y $\phi_{xx}[10]$? ¿Qué cambia con $a = -0{,}9$?

**Solución**

a) Tomando esperanzas, $\mu_x = a\mu_x + \mu_w$, de donde $\mu_x = \mu_w/(1-a)$, que es $H(e^{j0})\mu_w$ con $H(z) = 1/(1 - az^{-1})$.

b) $\sigma_x^2 = a^2\sigma_x^2 + \sigma_w^2$ porque $x[n-1]$ y $w[n]$ son independientes: $\sigma_x^2 = \sigma_w^2/(1-a^2)$. Para $l > 0$, multiplicando por $x[n]$ y tomando esperanzas, $\phi_{xx}[l] = a\phi_{xx}[l-1]$, y por paridad $\phi_{xx}[l] = \sigma_x^2 a^{|l|}$.

c) $\phi_{xx}[0] = 1/0{,}19 = 5{,}26$ y $\phi_{xx}[10] = 5{,}26 \cdot 0{,}9^{10} = 1{,}83$. Con $a = -0{,}9$ la potencia es la misma, pero $\phi_{xx}[l]$ alterna de signo: la señal cambia de signo casi en cada muestra.

**Ejercicio 14** (eco y retardo) Un sensor recibe $y[n] = \alpha x[n - D] + v[n]$, donde $x[n]$ es ruido blanco de varianza $\sigma^2$ y $v[n]$ es ruido independiente de varianza $\sigma_v^2$.

a) Calcula la correlación cruzada $\phi_{yx}[l]$.

b) Comprueba la cota $|\phi_{yx}[l]|^2 \leq \phi_{xx}[0]\phi_{yy}[0]$.

c) ¿Cómo se estima $D$ con una sola realización? ¿Qué pasaría si $x[n]$ fuese un AR(1) con $a = 0{,}95$?

**Solución**

a) $\phi_{yx}[l] = E\{(\alpha x[n+l-D] + v[n+l])x[n]\} = \alpha\phi_{xx}[l-D] = \alpha\sigma^2\delta[l-D]$.

b) $\phi_{yy}[0] = \alpha^2\sigma^2 + \sigma_v^2$, así que $\phi_{xx}[0]\phi_{yy}[0] = \alpha^2\sigma^4 + \sigma^2\sigma_v^2 \geq \alpha^2\sigma^4 = |\phi_{yx}[D]|^2$. La igualdad solo se da sin ruido.

c) Se estima $\hat{\phi}_{yx}[l]$ con `xcorr(y, x)` o `np.correlate(y, x, 'full')` y se busca la posición del máximo. Con un AR(1), $\phi_{yx}[l] = \alpha\sigma_x^2 0{,}95^{|l-D|}$: el máximo sigue en $D$, pero el pico es mucho más ancho y con ruido cuesta más localizarlo. Por eso conviene que la señal de prueba

sea de banda ancha.

## Análisis espectral

**Ejercicio 15** (DEP de un AR(1)) Para el AR(1) $x[n] = 0{,}8\,x[n-1] + w[n]$ con $\sigma_w^2 = 1$:

a) Calcula $\Phi_{xx}(e^{j\Omega})$ como DTFT de $\phi_{xx}[l]$ y como $|H(e^{j\Omega})|^2\sigma_w^2$. Comprueba que coinciden.

b) Calcula $\Phi_{xx}$ en $\Omega = 0$ y $\Omega = \pi$. ¿Es paso bajo o paso alto?

c) Comprueba que $\frac{1}{2\pi}\int_{-\pi}^{\pi}\Phi_{xx}\,d\Omega = \phi_{xx}[0]$.

**Solución**

a) Con $H = 1/(1-ae^{-j\Omega})$, $|H|^2 = \frac{1}{1-2a\cos\Omega+a^2}$. Por la otra vía, sumando las dos series geométricas, $\sigma_x^2\left(\frac{1}{1-ae^{-j\Omega}} + \frac{1}{1-ae^{j\Omega}} - 1\right) = \sigma_x^2\frac{1-a^2}{1-2a\cos\Omega+a^2}$, y con $\sigma_x^2 = 1/(1-a^2)$ es lo mismo.

b) $\Phi_{xx}(e^{j0}) = 1/0{,}2^2 = 25$ y $\Phi_{xx}(e^{j\pi}) = 1/1{,}8^2 = 0{,}309$: paso bajo, con 80 veces más densidad en continua que en $\pi$.

c) La integral de $\frac{1}{1+a^2-2a\cos\Omega}$ en un periodo vale $\frac{2\pi}{1-a^2}$, así que la potencia es $1/0{,}36 = 2{,}78 = \phi_{xx}[0]$.

**Ejercicio 16** (media móvil de ruido blanco) Se filtra ruido blanco de varianza $\sigma^2 = 2$ con la media móvil $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$.

a) Calcula $\phi_{yy}[l]$ y la potencia de la salida.

b) Calcula $\Phi_{yy}(e^{j\Omega})$. ¿En qué frecuencias se anula?

c) Calcula $\phi_{yx}[l]$.

**Solución**

a) $\phi_{yy} = \sigma^2(h * \tilde{h})[l]$ con $h = \frac{1}{3}(1,1,1)$: $\phi_{yy}[0] = \frac{6}{9} = \frac{2}{3}$, $\phi_{yy}[\pm1] = \frac{4}{9}$, $\phi_{yy}[\pm2] = \frac{2}{9}$ y cero fuera. La potencia, $2/3$, es un tercio de la de entrada.

b) $H(e^{j\Omega}) = \frac{1}{3}e^{-j\Omega}(1+2\cos\Omega)$, así que $\Phi_{yy} = \frac{2}{9}(1+2\cos\Omega)^2$. Se anula en $\cos\Omega = -\frac{1}{2}$, es decir, en $\Omega = \pm 2\pi/3$.

c) $\phi_{yx}[l] = \sigma^2 h[l]$: vale $\frac{2}{3}$ en $l = 0,1,2$ y cero en el resto.

**Ejercicio 17** (identificación de un sistema) Se excita un sistema LTI desconocido con ruido blanco $x[n]$ de varianza $\sigma^2$ y se mide $y[n] = (h * x)[n] + v[n]$, con $v[n]$ ruido de medida independiente de $x$.

a) Demuestra que $\phi_{yx}[l] = \sigma^2 h[l]$.

b) ¿Por qué $\hat{H} = \Phi_{yx}/\Phi_{xx}$ no está sesgado por el ruido de medida y $\sqrt{\Phi_{yy}/\Phi_{xx}}$ sí?

c) ¿Qué información da $\Phi_{yx}$ que no da $\Phi_{yy}$?

**Solución**

a) $\phi_{yx}[l] = E\{\sum_k h[k]x[n+l-k]x[n]\} + E\{v[n+l]x[n]\} = \sum_k h[k]\sigma^2\delta[l-k] + 0 = \sigma^2 h[l]$.

b) $v$ es independiente de $x$ y no contribuye a $\phi_{yx}$, así que $\Phi_{yx} = H\Phi_{xx}$ exactamente. En cambio $\Phi_{yy} = |H|^2\Phi_{xx} + \Phi_{vv}$: el ruido se suma y $|H|$ sale por exceso, sobre todo donde $|H|$ es pequeño.

c) La fase: $\Phi_{yx}$ es compleja y su argumento es el de $H$, mientras que $\Phi_{yy}$ solo da $|H|^2$.

**Ejercicio 18** (estimación de la DEP y ruido térmico)

a) Con $N = 16\,384$ muestras, segmentos de $L = 256$ y solapamiento del 50 %, ¿cuántos segmentos promedia Welch? ¿En qué factor reduce la varianza respecto al periodograma, aproximadamente?

b) ¿Qué resolución en frecuencia tiene esa estimación?

c) Un ruido blanco en tiempo continuo con $N_0/2 = 10^{-9}$ W/Hz pasa por un filtro antialiasing ideal de $B = 10\,\text{kHz}$ y se muestrea a $f_s = 20\,\text{kHz}$. ¿Qué potencia tiene y qué varianza tienen las muestras?

**Solución**

a) $K = \lfloor (16384 - 256)/128 \rfloor + 1 = 127$ segmentos. Si fuesen independientes, la varianza se dividiría por 127; con el solapamiento del 50 % y Hann la reducción real es algo menor, del orden de 100.

b) La de la ventana: el lóbulo principal de Hann mide $8\pi/L$ entre ceros, una resolución del orden de $4\pi/L \approx 0{,}05$ rad/muestra (frente a $2\pi/N$ del periodograma completo).

c) La potencia es el área de la DEP bilateral en $[-B, B]$: $10^{-9} \cdot 2 \cdot 10^4 = 2 \cdot 10^{-5}$ W. Como $f_s = 2B$, las muestras son incorreladas, es decir, ruido blanco discreto de varianza $2 \cdot 10^{-5}$.